

Стюарт Рассел, Питер Норвиг

Искусственный интеллект: современный подход, 2-е изд.

Часть VI. Обучение

Содержание

Часть VI. Обучение	1
Глава 18. Обучение на основе наблюдений	1
Глава 19. Применение знаний в обучении	41
Глава 20. Статистические методы обучения	85
Глава 21. Обучение с подкреплением	146

Часть VI. Обучение

Обучение Обучение на основе наблюдений 864 Применение знаний в обучении 902
Статистические методы обучения 945 Обучение с подкреплением 1010

Глава 18. Обучение на основе наблюдений

18 Обучение на ОСНОВЕ НАБЛЮДЕНИЙ В данной главе рассматриваются агенты, способные усовершенствовать свое поведение благодаря тщательному изучению собственного опыта.

В основе обучения лежит представление о том, что результаты восприятия должны использоваться не только для осуществления действий, но и для повышения способности агента действовать в будущем. Обучение происходит по мере того, как агент наблюдает за своим взаимодействием с миром и собственными процессами принятия решений. Обучение может охватывать широкий спектр действий, начиная от тривиального накопления в памяти результатов полученного опыта, как было показано на примере агента для мира вампуса в главе 10, и заканчивая созданием целых научных теорий, что было успешно продемонстрировано Альбертом Эйнштейном. В этой главе рассматривается индуктивное обучение на основе наблюдений.

В частности, в ней описано, как в процессе обучения формируются простые теории в терминах пропозициональной логики. В ней также приведены результаты теоретического анализа, позволяющие понять принципы индуктивного обучения.

18.1. Формы обучения В главе 2 было показано, что проект обучающегося агента может рассматриваться как состоящий из производительного элемента, определяющего, какие действия должны быть выполнены, и обучающего элемента, который модифицирует производительный элемент для того, чтобы он вырабатывал лучшие решения (см. рис. 2.7).

Исследователи, работающие в области машинного обучения, предложили целый ряд типов обучающих элементов. Для того чтобы разобраться в их работе, целесообразно рассмотреть, как влияет на их проект тот контекст, в котором они должны функционировать. На проект обучающего элемента влияют три описанных ниже аспекта.

- Компоненты производительного элемента, подлежащие обучению.
- Обратные связи, которые могут применяться для обучения этих компонентов.
- Способы представления, используемые для компонентов.

Проведем анализ каждого из этих аспектов по очереди. В данной книге уже было показано, что существует много способов построения производительного элемента для агента. В главе 2 было описано несколько проектов агентов (см. рис. 2.3-2.6).

Ниже перечислены компоненты этих агентов. 1. Средства прямого отображения условий (распространяющихся на текущее состояние) в действия.

2. Средства логического вывода релевантных свойств мира из последовательности результатов восприятия.

3. Информация о том, как развивается мир и какие результаты возможных действий могут быть получены агентом.

4. Информация о полезности, которая показывает, насколько желательными являются те или иные состояния мира.

5. Информация о ценности действий, показывающая желательность действий.

6. Цели, описывающие классы состояний, достижение которых максимизирует полезность для агента.

Обучение каждого из этих компонентов может осуществляться с помощью соответствующей обратной связи. Рассмотрим, например, агента, который учится вождению, чтобы стать таксистом. Каждый раз, когда инструктор кричит "Тормози!", агент должен усвоить очередное правило "условие—действие", позволяющее определить, когда следует тормозить (компонент 1). Рассматривая множество видеоизображений, на которых, как ему сказано, имеются автобусы, он может научиться распознавать автобусы (компонент 2). Осуществляя попытки выполнения действий и наблюдая за их результатами (например, проводя жесткое торможение на мокрой дороге), он может определить путем обучения, каковы результаты его действий (компонент 3). В дальнейшем, перестав получать чаевые от пассажиров, которые почувствовали себя полностью разбитыми во время утомительной поездки, агент может обучить полезный компонент своей общей функции полезности (компонент 4), который будет подсказывать, что пассажиров надо беречь.

Тип обратной связи, доступной для обучения, обычно является наиболее важным фактором, определяющим характер задачи обучения, с которой сталкивается агент.

В области машинного обучения, как правило, различаются три случая: контролируемое обучение, неконтролируемое обучение и обучение с подкреплением.

В задаче $\wedge$ контролируемого обучения предусматривается изучение некоторой функции на примерах ее входных и выходных данных. Все приведенные выше случаи, касающиеся компонентов 1, 2 и 3, представляют собой примеры задач контролируемого обучения. В случае, который относится к компоненту 1, агент изучает правило "условие—действие", касающееся торможения, т.е. функцию, которая связывает входные состояния с булевым выходом (тормозить или не тормозить). В случае компонента 2 агент изучает функцию, преобразующую входные изображения в булев выход (который показывает, содержит ли изображение автобус). В случае компонента 3 сведения о торможении выражаются в виде функции, которая связывает

состояния и действия по торможению, скажем, с длиной тормозного пути в футах.

Обратите внимание на то, что в случаях 1 и 2 учитель указывает в примерах правильное выходное значение, а в случае 3 выходное значение должно быть получено непосредственно из результатов восприятия агента. В полностью наблюдаемых вариантах среды всегда соблюдается такое условие, что агент может наблюдать за результатами своих действий и поэтому использовать методы контролируемого обучения, чтобы научиться предсказывать эти результаты. В частично наблюдаемых вариантах среды задача обучения становится более сложной, поскольку непосредственные результаты могут оказаться недоступными для восприятия.

Задача $\wedge$ неконтролируемого обучения касается выявления определенных закономерностей во входных данных в тех условиях, когда не задаются конкретные выходные значения. Например, агент-водитель такси может постепенно развить представление о "днях с хорошими условиями дорожного движения" и "днях с плохими условиями дорожного движения", даже не получая обозначенных соответствующими заголовками примеров тех или других дней. Обучающийся агент, полностью остающийся без контроля, не может узнать в процессе обучения, что делать, поскольку он не имеет информации о том, каковым является определение понятия правильного действия или желательного состояния. Мы будем изучать проблему неконтролируемого обучения в основном в контексте систем формирования вероятностных задач $\wedge$ обучения с подкреплением, которая будет рассматриваться в главе 21, является наиболее общей из этих трех категорий. Агент, проходящий обучение с подкреплением, не получает от учителя указаний о том, что

делать, а должен обучаться на основе ¹ подкрепления. Например, отсутствие чаевых в конце поездки (или большой штраф за столкновение с идущим впереди автомобилем) будет служить для агента определенным показателем того, что его поведение нежелательно.

Задача обучения с подкреплением обычно включает подзадачу обучения тому, как функционирует среда, в которой существует агент.

В определении того, как должен действовать алгоритм обучения, важную роль играет также применяемое представление той информации, которая должна быть освоена в процессе обучения. Любой из компонентов агента можно представить с использованием любой из схем представления, описанных в этой книге. Выше уже было приведено несколько примеров: в программах ведения игр для представления функций полезности применяются полиномы с линейными весами; высказывания в пропозициональной логике и логике первого порядка применимы для представления всех компонентов логического агента; а вероятностные описания, такие как байесовские сети, применяются в компонентах агента, действующего на основе теории решений, которые предназначены для обеспечения вероятностного вывода. Для всех этих средств представления разработаны эффективные алгоритмы обучения.

В данной главе рассматриваются методы, относящиеся к пропозициональной логике, в главе 19 описаны методы для логики первого порядка, а в главе 20 — методы для байесовских и нейронных сетей (которые включают линейные полиномы в качестве частного случая).

¹ Термин подкрепление является синонимом термина вознаграждение, который использовался в главе 17.

Последним важным аспектом проектирования обучающихся систем является наличие априорных знаний. Большинство исследований проблем обучения, проводимых в области искусственного интеллекта, компьютерных наук и психологии, касались того случая, в котором агент приступает к обучению, вообще не имея никаких знаний о том, что он пытается изучить. Он имеет доступ только к примерам, полученным на основе собственного опыта. Хотя этот случай представляет собой важный частный случай, его не следует рассматривать как общий случай. Основная часть человеческого обучения проходит в контексте, связанном с наличием большого объема фоновых знаний. Некоторые психологи и лингвисты утверждают, что даже новорожденные младенцы демонстрируют наличие у них знаний о мире. Но, независимо от того, являются ли эти утверждения истинными, нет никакого сомнения в том, что априорные знания могут оказать огромную помощь в обучении. Физик, рассматривающий стопку фотографий, полученных с помощью пузырьковой камеры, может оказаться способным создать

теорию, доказывающую существование новой частицы с определенной массой и зарядом, а если та же стопка фотографий будет показана без дополнительных пояснений искусствоведу, то он сможет лишь утверждать, будто этот "художник" относится к какой-то школе абстракционистов или экспрессионистов. В главе 19 будет показано несколько способов, позволяющих упростить обучение с использованием существующих знаний; кроме того, в ней показано, как можно компилировать знания для ускорения процесса принятия решений. В главе 20 описаны способы применения априорных знаний для упрощения изучения вероятностных теорий.

18.2. Индуктивное обучение Любой алгоритм детерминированного контролируемого обучения получает в качестве исходной информации правильные значения неизвестной функции, соответствующие конкретным входным данным, и должен предпринять попытку восстановить эту неизвестную функцию или сформировать какую-то другую функцию, близкую к ней. Более формально можно определить, что $\hat{f}$ пример представляет собой пару $(x, \hat{f}(x))$, где x — входное, а $\hat{f}(x)$ — выходное значение функции, применяемой к x . Основная задача $\hat{f}$ чисто индуктивного логического вывода (или просто индукции) указана ниже.

На основании совокупности примеров входных и выходных данных функции f получить функцию $\hat{f}$, которая аппроксимирует f .

Функция $\hat{f}$ называется $\hat{f}$ гипотезой. С концептуальной точки зрения та причина, по которой задача обучения является трудной, состоит в том, что обычно нелегко определить, действительно ли какая-то конкретная функция $\hat{f}$ является хорошей аппроксимацией для f . Качественная гипотеза должна обеспечивать приемлемое $\hat{f}$ обобщение, т.е. должна правильно предсказывать появление еще не полученных примеров. В этом состоит фундаментальная $\hat{f}$ проблема индукции.

Эта проблема изучалась в течение многих столетий; в разделе 18.5 приведено ее частичное решение.

На рис. 18.1 приведен известный пример: подгонка функции от одной переменной к некоторым точкам из набора данных. Примеры представляют собой

пары $(x, f(x))$, где x и $f(x)$ — действительные числа. Выберем в качестве $\hat{f}$ пространства гипотез H (множества гипотез, которые мы будем рассматривать в качестве потенциально приемлемых) множество полиномов, имеющих степень не больше k , таких как $3x^2 + 2$, $x^{17} - 4x^3$ и т.д. На рис. 18.1, а показаны данные, которые соответствуют некоторой прямой (полиному первой степени). Эта прямая называется $\hat{f}$ совместимой с гипотезой, поскольку она согласуется со всеми данными. На рис. 18.1, б показан полином более высокой степени, который также согласуется с этими данными. Данный случай может служить иллюстрацией к наиболее важной проблеме в индуктивном обучении: $\hat{f}$ как осуществлять выбор

среди многочисленных согласованных гипотез? Ответ состоит в использовании принципа бритвы Оккама², согласно которому предпочтение следует отдавать наиболее простой гипотезе, согласующейся с данными. Интуитивно ясно, что такой подход имеет смысл, поскольку гипотезы не позволяют извлекать из данных какую-либо информацию, если они не проще самих данных.

Определить, какая гипотеза проще, а какая сложнее, обычно нелегко, но, повидимому, вполне резонным является утверждение, что полином первой степени проще по сравнению с полиномом двенадцатой степени. № № № № i ^x а) б) в) г) Рис. 18.1. Типичные задачи обучения: пример пар $(x, f(x))$ и совместимой линейной гипотезы (а); совместимая гипотеза для того же набора данных в виде полинома седьмой степени (б); другой набор данных, который точно соответствует полиному шестой степени или приблизительно соответствует прямой линии (в); простая синусоидальная функция, которая точно соответствует тому же набору данных (г) На рис. 18.1, в показан второй набор данных. С этим набором данных нельзя совместить прямую линию; в действительности для обеспечения точного согласования с ним требуется полином шестой степени (с семью параметрами). Количество точек равно только семи, поэтому полином должен иметь столько же параметров, сколько имеется точек данных; таким образом, создается впечатление, что этот полином не позволяет найти в данных какие-либо повторяющиеся шаблоны, и поэтому не следует ожидать, что с его помощью будет получено хорошее обобщение. Может оказаться, что лучше согласовать этот набор данных с прямой линией, которая не будет точно совместимой, но позволит получать вполне обоснованные предсказания.

Принятие данного решения равносильно признанию такой возможности, что истинная функция не является детерминированной (или, что примерно эквивалентно ² Этот принцип получил название в честь английского философа XIV века Уильяма из Окхема, но в его обозначении название города, в котором работал философ, записывают как "Оккам", в соответствии с французской транскрипцией, "Guillaume d'Occam" — Гийом из Оккама.

этому утверждению, истинные входные данные не являются полностью наблюдаемыми). cf= При наличии недетерминированных функций неизбежно приходится искать компромисс между сложностью гипотезы и степенью ее согласования с данными. В главе 20 показано, как достичь этого компромисса с помощью теории вероятностей.

Следует всегда учитывать, что возможность или невозможность найти простую, согласованную гипотезу зависит главным образом от выбранного пространства гипотез. На рис. 18.1, г показано, что данные, приведенные на рис. 18.1, в, могут быть точно согласованы с простой функцией в форме $ax+b+c\sin x$. Этот пример подчеркивает важность выбора пространства гипотез. Пространство гипотез,

состоящее из полиномов конечной степени, не позволяет точно представить синусоидальные функции, поэтому ученик, использующий такое пространство гипотез, не сможет осуществить обучение с использованием синусоидальных данных. Принято считать, что задача обучения является $\wedge$ реализуемой, если пространство гипотез содержит подходящую функцию; в противном случае она является $\wedge$ нереализуемой. К сожалению, в любой ситуации невозможно сразу же определить, относится ли данная конкретная задача обучения к категории реализуемых, поскольку не известна истинная функция. Один из способов, позволяющих преодолеть этот барьер, состоит в использовании априорных знаний для логического вывода пространства гипотез, в котором, как известно, должна находиться истинная функция. Эта тема рассматривается более подробно в главе 19.

Еще один подход состоит в применении наибольшего возможного пространства гипотез. Например, почему бы не использовать в качестве H класс всех машин Тьюринга? В конечном итоге любая вычислимая функция может быть представлена с помощью некоторой машины Тьюринга, и это — наилучший способ представления, который только может применяться. Но при реализации этой идеи возникает проблема, связанная с тем, что в ней не учитывается вычислительная сложность обучения. $s^{\wedge} =$ Необходимо найти компромисс между выразительностью пространства гипотез и сложностью поиска простой, совместимой гипотезы в этом пространстве.

Например, подгонка к данным прямых линий осуществляется очень просто; подбор полиномов высокой степени становится сложнее, а создание соответствующих машин Тьюринга действительно представляет собой очень сложную задачу, поскольку неразрешима в общем виде даже проблема определения того, является ли конкретная машина Тьюринга совместимой с данными. Второй причиной, по которой следует предпочесть простые пространства гипотез, является то, что результирующие гипотезы могут оказаться более простыми в использовании, т.е. вычисление $h(x)$, если h — линейная функция, будет осуществляться быстрее, чем при использовании программы, моделирующей произвольную машину Тьюринга.

По этим причинам основной объем исследований в области обучения был сосредоточен на относительно простых способах представления. В данной главе в основном рассматриваются пропозициональная логика и связанные с ней языки. В главе 19 рассматриваются теории обучения в логике первого порядка. Ниже будет показано, что компромисс между выразительностью и сложностью найти не так просто, как кажется на первый взгляд, поскольку, как было описано в главе 8, выразительный язык позволяет создать простую теорию, согласующуюся с данными, а ограничение выразительности языка приводит к тому, что любая согласованная теория должна стать очень сложной. Например, правила шахмат могут быть записаны в

логике первого порядка на одной или двух страницах, но потребуют тысячи страниц при их формулировке в пропозициональной логике. Кроме того, в подобных случаях при использовании более выразительного языка намного ускоряется обучение.

18.3. Формирование деревьев решений на основе обучения Индуктивный логический вывод деревьев решений представляет собой одну из простейших и вместе с тем наиболее успешных форм алгоритмов обучения. Эта тема может служить хорошим введением в проблематику индуктивного обучения, а алгоритмы, разработанные в ее рамках, легко поддаются реализации. В настоящем разделе вначале описан производительный элемент проекта агента, а затем показано, как обеспечить его обучение. В ходе этого изложения представлены основные понятия, которые применяются во всех областях индуктивного обучения.

Деревья решений, рассматриваемые как производительные элементы $\wedge$ Дерево решений принимает в качестве входных данных объект или ситуацию, описанную с помощью множества $\wedge$ атрибутов, и возвращает "решение" — предсказанное выходное значение, соответствующее входным данным. Входные атрибуты могут быть дискретными или непрерывными. На данный момент подразумевается, что входные данные являются дискретными. Выходные значения также могут быть дискретными или непрерывными; процесс формирования в ходе обучения функции с дискретными значениями называется обучением $\wedge$ классификации; формирование в ходе обучения непрерывной функции называется обучением $\wedge$ регрессии. Вначале мы сосредоточимся на булевой классификации, согласно которой каждый пример обозначается как истинный ($\wedge$ положительный) или ложный ($\wedge$ отрицательный).

Дерево решений позволяет перейти к содержащемуся в нем решению путем выполнения последовательности проверок. Каждый внутренний узел в дереве соответствует проверке значения одного из свойств, а ветви, исходящие из этого узла, обозначены возможными значениями результатов проверки. Каждый листовой узел в дереве задает значение, возвращаемое после достижения этого листа. Повидимому, представление в виде дерева решений кажется людям вполне естественным; в действительности многие инструктивные руководства (например, по ремонту автомобилей) полностью оформлены в виде одного дерева решений, разбросанного по нескольким сотням страниц.

Несколько более простой пример может быть основан на применении методов обучения к задаче, в которой клиент ждет, пока освободится место за столиком в ресторане. Цель состоит в том, чтобы изучить определение для $\wedge$ целевого предиката WillWait (Следует ли ждать). Подготавливая данный пример для использования в качестве задачи обучения, необходимо вначале определить, какие атрибуты доступны для описания примеров ситуаций в данной проблемной области. В главе 19 будет показано, как автоматизировать выполнение этого этапа, а на

данный момент предположим, что решено использовать приведенный ниже список атрибутов.

1. Al terna te (Альтернативный вариант). Есть ли поблизости подходящий ресторан такого же класса.
2. Bar (Бар). Имеется ли в ресторане уютный бар, в котором можно подождать.
3. Fri /Sat (Уик-энд). Принимает истинное значение по пятницам и субботам.
4. Hungry (Чувство голода). Испытывает ли посетитель чувство голода.
5. Patrons (Посетители). Сколько людей находится в ресторане; значениями этого атрибута являются None (Ресторан пуст), Some (В ресторане есть посетители) и Full (Ресторан заполнен).
6. Price (Цены). Ценовая категория ресторана (*, **, ***).
7. Raining (Дождь). Идет ли дождь на улице.
8. Reservation (Бронирование). Забронировано ли место за посетителем.
9. Type (Тип). Тип ресторана (ресторан с французской (French), итальянской (Italian), тайской (Thai) кухней или ресторан-закусочная (Burger)).
10. WaitEstimate (Оценка продолжительности ожидания).

Продолжительность ожидания, оценка которой сделана метрдотелем @-10, 10-30, 30-60, >60 минут).

Дерево решений, обычно используемое одним из авторов (Стюартом Расселом), для данной проблемной области, показано на рис. 18.2. Обратите внимание на то, что в этом дереве не используются атрибуты Price и Type; по сути это означает, что лицо, принимающее решение, рассматривает их как малозначимые. Обработка примеров ситуаций с помощью этого дерева начинается от корня и проходит по соответствующей ветви до тех пор, пока не будет достигнут какой-то лист. В частности, пример ситуации, в которой Patrons=Full и WaitEstimate=0-10 будет рассматриваться как положительный (да, следует дождаться освобождения столика).

Patrons? Рис. 18.2. Дерево решений для определения того, следует ли подождать, пока освободится столик

Выразительность деревьев решений С точки зрения логики любая конкретная гипотеза из дерева решений для целевого предиката WillWait может рассматриваться как утверждение в следующей форме:

$\forall s \text{ WillWait}(s) \leftrightarrow (P_1(s) \vee P_2(s) \vee \dots \vee P_n(s))$ где каждое условие $P_i(s)$ представляет собой конъюнкцию проверок, соответствующих пути от корня дерева к листу с положительным результатом. Хотя это выражение с виду напоминает высказывание в логике первого порядка, оно в определенном смысле является

пропозициональным, поскольку содержит только одну переменную, а все предикаты являются унарными. Это дерево решений фактически описывает связь между предикатом $WillWait$ и некоторой логической комбинацией значений атрибутов. Деревья решений не могут использоваться для представления проверок, относящихся к двум или нескольким разным объектам, например, таких проверок, как показано ниже ("Есть ли поблизости более дешевый ресторан?").

$3r2$ Nearby $[r2f\ r]$ л $Price\{r,p\}$ л $Price\ (r2,p2)$ л $Cheaper\ (p2 ,p)$ Очевидно, что можно было бы ввести еще один булев атрибут с именем $CheaperRestaurantNearby$ (Наличие поблизости более дешевого ресторана), но задача введения всех подобных атрибутов является трудно осуществимой. В главе 19 приведены дополнительные сведения о задаче правильного обучения в логике первого порядка.

Деревья решений в рамках класса пропозициональных языков являются полностью выразительными; это означает, что в виде дерева решений может быть оформлена любая булева функция. Такую задачу можно выполнить тривиально, предположив, что каждая строка в истинностной таблице для данной функции соответствует определенному пути в дереве. Но подобный подход приводит к получению представления в виде дерева решений с экспоненциальными размерами, поскольку количество строк в истинностной таблице увеличивается экспоненциально, тогда как очевидно, что деревья решений позволяют представить многие функции с помощью гораздо меньших деревьев.

Но для некоторых видов функций такая проблема становится вполне реальной.

Например, если рассматриваемая функция является $\wedge$ функцией определения четности, которая возвращает 1 тогда и только тогда, когда на входах задано четное количество единиц, то потребуется дерево решений, имеющее экспоненциальную величину. Кроме того, сложной является задача использования дерева решений для представления $\wedge$ мажоритарной функции, которая возвращает 1, если больше чем на половине ее входов имеется 1.

Иными словами, деревья решений вполне подходят для функций одних типов и не подходят для других. Существует ли представление какого-либо типа, являющееся эффективным для функций всех типов? К сожалению, ответ на этот вопрос отрицателен. Это утверждение можно доказать с помощью такого общего способа.

Рассмотрим множество всех булевых функций от p атрибутов. Каково общее количество различных функций в этом множестве? Оно определяется количеством различных истинностных таблиц, которые могут быть составлены на этих атрибутах, поскольку функция определяется своей истинностной таблицей. Истинностная таблица имеет 2^p строк, так как для описания вариантов входных данных применяется p атрибутов.

Столбец "ответов" такой таблицы может рассматриваться как число, состоящее из 2П битов, которое определяет функцию. Независимо от того, какое представление используется для функций, некоторые из функций (а фактически почти все) должны потребовать для своего представления именно такое количество битов.

Если для определения функции требуется 2П битов, то существует 22 различных функций от п атрибутов. Это число является весьма обескураживающим.

Например, при наличии лишь шести булевых атрибутов существует $2^6 = 64$ различных функций, из числа которых может быть сделан выбор. Для поиска совместимых гипотез в таком большом пространстве потребуется некоторые остроумные алгоритмы.

Индуктивный вывод деревьев решений на основе примеров. Пример для булева дерева решений состоит из вектора входных атрибутов X и одного булева выходного значения y. Множество примеров $\{X_1fy_1\}, \dots, \{X_nfy_n\}$ показано в табл. 18.1. Положительными примерами являются те, в которых цель WillWait имеет истинное значение (X₁, X₃,...), а отрицательными — те, в которых эта цель имеет ложное значение (X₂, X₄,...). Полное множество примеров называется обучающим множеством.

Таблица 18.1. Примеры для проблемной области задачи с рестораном

Пример	Alt	Bar	Fri	Hun	Pat	Атрибуты	Price	Rain	Res	Type	Est	Цель	WillWait	Xi	Yes	No	No	Yes	Some																								
***	No	Yes	French	0-10	Yes	X ₂	Yes	No	No	Yes	Full	*	No	No	Thai	3	0-60	No	Xi	No	Yes	No																					
No	Some	*	No	No	Burger	0-10	Yes	X ₄	Yes	No	Yes	Yes	Full	*	Yes	No	Thai	10-3	0	Yes	X ₅																						
Yes	No	Yes	No	Full	***	No	Yes	French	7	60	No	X ₆	No	Yes	No	Yes	Some	**	Yes	Yes	Italian	0-10	Yes	X _n	No	Yes	No	No	None	*	Yes	No	Burger	0-10	No	X ₈	No	No	No	Yes	Some	**	Yes
Yes	Thai	0-10	Yes	X ₉	No	Yes	Yes	No	Full	*	Yes	No	Burger	7	60	No	X ₁₀	Yes	Yes	Yes	Yes	Full	***	No	Yes	Italian	10-3	0	No	X _n	No	No	No	None	*	No	No	Thai	0-10	No	X ₁₂	Yes	Yes
Yes	Yes	Full	*	No	No	Burger	30-60	Yes																																			

Проблема поиска дерева решений, которое согласуется с обучающим множеством, на первый взгляд может показаться сложной, но фактически имеет простейшее решение. Можно просто сформировать дерево решений, в котором имеется по одному пути к листовому узлу для каждого примера, где в пути проверяется каждый атрибут по очереди и происходит переход к значению для данного примера, а листовой узел содержит классификацию для данного примера. После повторного поступления того же примера³ дерево решений обеспечивает правильную классификацию. Но, к сожалению, оно не позволяет учесть какие-либо иные случаи!

³ При этом нужно различать тот же пример или пример с тем же описанием; это различие очень важно, и мы вернемся к нему в главе 19.

Недостатком такого простейшего дерева решений является то, что в нем просто запоминаются результаты наблюдений. Оно не позволяет извлечь какие-либо закономерности из примеров, поэтому нельзя надеяться на то, что оно даст возможность экстраполировать примеры, которые еще не встречались. Применяя принцип бритвы Оккама, необходимо вместо этого найти наименьшее дерево решений, совместимое с рассматриваемым примером. К сожалению, применительно к любому обоснованному определению понятия "наименьший" задача поиска наименьшего дерева является трудноразрешимой. Однако с помощью некоторой простой эвристики можно многого достичь в направлении поиска "меньшего" дерева. Основная идея, лежащая в основе алгоритма Decision-Tree-Learning, состоит в том, что в первую очередь следует проверять наиболее важный атрибут. Под "наиболее важным" атрибутом подразумевается такой атрибут, который вносит наибольшее различие в классификацию примера. Таким образом, можно рассчитывать на обеспечение правильной классификации с помощью небольшого количества проверок, а это означает, что все пути в дереве будут короткими, а само дерево в целом окажется небольшим.

На рис. 18.3 показано, как начинается работа алгоритма. Ему предъявляются 12 обучающих примеров, которые классифицированы на множества положительных и отрицательных примеров. После этого принимается решение о том, какой атрибут должен использоваться для первой проверки дерева. На рис. 18.3, а показано, что Туре — неподходящий атрибут, поскольку после его проверки остаются четыре возможных результата, каждый из которых содержит одинаковое количество положительных и отрицательных примеров. С другой стороны, на рис. 18.3, б можно видеть, что Patrons — довольно важный атрибут, поскольку если его значение равно None или Some, то остаются такие множества примеров, для которых можно сразу же получить определенный ответ (No и Yes соответственно). А если значением является Full, то остается смешанное множество примеров. Вообще говоря, после того, как проверка первого атрибута разбивает множество примеров на подмножества, каждый результат сам становится определением новой задачи обучения дерева решений с меньшим количеством примеров и количеством атрибутов, сократившимся на единицу. Ниже описаны четыре случая, которые следует рассматривать при анализе подобных рекурсивных задач.

1. Если имеются некоторые положительные и отрицательные примеры, то следует выбрать наилучший атрибут для выполнения по нему разбиения. Как показано на рис. 18.3,б, для разбиения оставшихся примеров используется атрибут Hungry.
2. Если все оставшиеся примеры являются положительными (или отрицательными), то работа закончена и можно дать ответ Yes или No. На рис. 18.3, б показаны примеры достижения этой цели в случаях None и Some.

3. Если не осталось ни одного примера, это означает, что соответствующие примеры не наблюдались, поэтому следует вернуть применяемое по умолчанию значение, вычисленное на основании мажоритарной классификации в родительском узле данного узла.

4. Если не осталось ни одного атрибута, но имеются и положительные и отрицательные примеры, то налицо проблема. Такая ситуация означает, что данные примеры имеют полностью одинаковое описание, но классифицированы по-

разному. Такое происходит, если некоторые данные определены неправильно; принято говорить, что в данных присутствует $\wedge$ шум. Кроме того, такое происходит, либо если атрибуты не предоставляют достаточной информации для полного описания ситуации, либо если данная проблемная область действительно является недетерминированной. Одним из простых способов преодоления такой проблемы является использование мажоритарного голосования.

French Д 3 4 % 8 12 а в а а ш ш Italian/ Щ ш Type \Thai т. ш вш 1 3. 46 8 12 в н в в жд Burger 3 12 BBI а) б) В В в ш Рис. 18.3. Разбиение множества примеров на подмножества путем проверки атрибутов: в результате разбиения по атрибуту Type не удастся приблизиться к решению задачи проведения различий между положительными и отрицательными примерами (а); разбиение по атрибуту Patrons позволяет многого добиться в части разделения положительных и отрицательных примеров (б). После разбиения по атрибуту Patrons довольно неплохой второй проверкой становится проверка по атрибуту Hungry Алгоритм Decision-Tree-Learning показан в листинге 18.1. Подробные сведения о методе Choose-Attribute приведены в следующем подразделе.

Листинг 18.1. Алгоритм обучения дерева решений function Decision-Tree-Learning(examples,attrs,default) returns дерево решений inputs: examples, множество примеров attrs, множество атрибутов default, заданное по умолчанию значение для целевого предиката if множество examples пусто then return default else if все примеры имеют одну и ту же классификацию then return классификация else if множество attrs пусто then return Majority-Value(examples) else best $\leftarrow$ Choose-Attribute(attrs, examples) tree $\leftarrow$ новое дерево решений с результатом best проверки корневого узла т $\leftarrow$ Majority-Value(examples) for each значение vx результата best do

examples_i $\leftarrow$ {элементы множества examples со значениями best=V_i} subtree $\leftarrow$ Decision-Tree-Learning (examples_i, attrs-best, default) добавить к дереву решений tree ветвь с меткой V_i и поддеревом subtree return tree Окончательное дерево решений, полученное с помощью этого алгоритма после применения к набору данных с 12 примерами, показано на рис. 18.4. Очевидно, что указанное дерево отличается от исходного дерева, показанного на рис. 18.2, несмотря на тот факт, что обучающие

данные в действительности были сформированы агентом, использовавшим исходное дерево. Можно было бы прийти к заключению, что этот обучающий алгоритм не очень хорошо справляется с задачей изучения правильной функции. Однако такое заключение далеко от истины. В обучающем алгоритме рассматриваются примеры, а не правильная функция, и на самом деле сформированная в нем гипотеза (см. рис. 18.4) не только согласуется со всеми примерами, но и значительно проще по сравнению с первоначальным деревом. В процессе работы обучающего алгоритма не было обнаружено причин для включения проверок значений атрибутов Raining и Reservation, поскольку алгоритм мог классифицировать все примеры без них. Кроме того, алгоритм обнаружил интересный и ранее не наблюдавшийся шаблон поведения: первый автор, указанный на обложке данной книги, готов ждать, чтобы поест тайской пищи по уик-эндам.

None 1ш9 Patrons? Some^ | Yes j Рис. 18.4. Дерево решений, сформированное на основании обучающего множества с 12 примерами Безусловно, если бы было собрано больше примеров, то вполне возможно, что сформированное дерево в большей степени напоминало бы исходное. К тому же дерево, приведенное на рис. 18.4, способно выработать ошибочное решение, например, при его формировании никогда не встречался случай, в котором требовалось ждать от 0 до 10 минут, а ресторан был полон. На ту ситуацию, в которой атрибут Hungry имеет ложное значение, дерево сообщает, что ждать не следует, тогда как сам автор (Стюарт Рассел), безусловно, стал бы ждать. Исходя из этого, возникает очевидный вопрос: если алгоритм выводит на основании примеров согласованное, но неправильное дерево, то насколько неправильным может вообще оказаться это

дерево? Ниже, после рассмотрения подробных сведений об этапе выбора атрибутов, будет показано, как проводить экспериментальный анализ этого вопроса.

Выбор проверок атрибутов Подход к выбору атрибутов, используемый в обучении дерева решений, предназначен для минимизации глубины окончательно сформированного дерева. Идея этого подхода состоит в том, что следует выбирать в первую очередь такой атрибут, который позволяет сразу же выполнить максимально возможный объем работы по обеспечению правильной классификации примеров. Идеальный атрибут позволяет разделить множество примеров на подмножества, целиком состоящие из положительных или отрицательных примеров. Атрибут Patrons не идеален, но является достаточно приемлемым. С другой стороны, действительно бесполезный атрибут, такой как Туре, создает подмножества примеров приблизительно с такими же соотношениями положительных и отрицательных примеров, как и в первоначальном множестве.

Таким образом, нам достаточно лишь определить формальный критерий оценки понятий "достаточно приемлемый" и "действительно бесполезный", после чего

появится возможность реализовать функцию Choose-Attribute, применяемую в алгоритме, который приведен в листинге 18.1. Этот критерий должен принимать максимальное значение, когда атрибут является идеальным, и минимальное значение, когда атрибут полностью бесполезен. Одним из подходящих критериев является ожидаемый объем информации, предоставляемый атрибутом; в данном определении термин "информация" используется в математическом смысле, впервые определенном Шенноном и Уивером [1394]. Чтобы понять суть концепции информации, достаточно представить себе, что она является ответом на вопрос, например, о том, упадет ли подброшенная монета орлом вверх. Объем информации, содержащийся в ответе на этот вопрос, зависит от наших априорных знаний. Чем меньше мы знаем, тем больше получим информации. В теории информации информационное содержание ответа измеряется в битах. Один бит информации является достаточным для ответа "да" или "нет" на вопрос о том, что полностью неизвестно, например, каков результат подбрасывания подлинной монеты. Вообще говоря, если возможные ответы v_i имеют вероятности $P(v_i)$, то информационное содержание J фактического ответа определяется с помощью следующего уравнения: $J = -\sum_{i=1}^V P(v_i) \log_2 P(v_i)$. Для проверки этого уравнения рассмотрим случай с подбрасыванием подлинной монеты. При этом будет получено следующее: $J = -1 \log_2 1 - 1 \log_2 1 = 0$. Если центр тяжести монеты смещен таким образом, что в 99% выпадает орел, то будет получено $J = -1/100 \log_2 1/100 - 99/100 \log_2 99/100 = 0.08$ битов, а поскольку вероятность выпадения орла приближается к 1, информационное содержание фактического ответа приближается к 0.

Что касается обучения деревьев решений, то необходимо найти ответ на вопрос — какова правильная классификация для данного примера? Правильное дерево решений должно позволить найти ответ на этот вопрос. Оценка вероятностей возможных ответов, полученная перед тем, как будет выполнена проверка какого-либо из этих атрибутов, определяется соотношениями положительных и отрицательных примеров в обучающем множестве. Предположим, что обучающее множество включает p положительных примеров и n отрицательных. В таком случае оценка информации, содержащейся в правильном ответе, равна: $J = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$. Обучающее множество для задачи с рестораном, приведенное в табл. 18.1, включает количество примеров $p=n=6$, поэтому требуется 1 бит информации.

Теперь отметим, что обычно проверка по одному атрибуту A не позволит нам получать такой объем информации, но по меньшей мере предоставит часть этой информации. Мы можем точно измерить, какова эта часть, определяя, сколько еще информации нам потребуется после проверки этого атрибута. Любой атрибут A делит обучающее множество E на подмножества $E_1, \dots, E_n$ в соответствии со

значениями атрибута A, если A может иметь v различных значений. Каждое подмножество E_i включает p_± положительных примеров и n_± отрицательных примеров, поэтому после перехода по ветви этого атрибута нам потребуется дополнительно -ПРЛУ (P_i+n_i) ,n_j (P_i+n_{ii})) битов информации, чтобы ответить на вопрос.

Случайно выбранный пример из обучающего множества содержит i-е значение рассматриваемого атрибута с вероятностью {p_±+n_±} / {p+n}, поэтому в среднем после проверки атрибута A для классификации данного примера потребуется показанное ниже количество бит информации.

Remainder {A} = J]—* I i=I p+n (\ P_i n_i P_i + n_i P_i + n_i 9k Приращение информации, полученное в результате проверки атрибута, представляет собой разность между первоначальной потребностью в информации и новой потребностью:

Gain(A) = I ———, | - Remainder(A) {p + n p + n; Эвристика, используемая в функции Choose-Attribute, состоит в том, что следует выбирать атрибут с наибольшим приращением. Возвращаясь к атрибутам, показанным на рис. 18.3, получаем следующее:

Gain(Patrons) = 1 - Gain{Type} = 1 2 4 6 2 4 _mi, + -K_{i,o}) + -iW 0.541 бит 12 2 2 12 2 2 12 4 4 12 4 4

Эти соотношения подтверждают интуитивное предположение о том, что Patrons— наилучший атрибут, по которому следует выполнять разбиение. В действительности атрибут Patrons позволяет получить наибольшее приращение информации по сравнению с любыми другими атрибутами и должен быть выбран алгоритмом обучения дерева решений в качестве корневого.

Оценка производительности обучающего алгоритма Обучающий алгоритм является приемлемым, если он вырабатывает гипотезы, обеспечивающие качественные предсказания в отношении того, к какому классу будут относиться еще не встречавшиеся примеры. В разделе 18.5 будет показано, как можно заранее оценить качество предсказания. А в данном разделе рассматривается методология оценки качества предсказания после его фактического получения.

Очевидно, что предсказание является качественным, если оказалось, что оно — истинное, поэтому можно оценить качество гипотезы, сверяя ее предсказания с правильной классификацией после того, как она становится известной. Такая задача осуществляется с помощью множества примеров, которые принято называть ^ проверочным множеством. Дело в том, что если обучение проведено по всем доступным примерам, то приходится возвращаться к действительности и заниматься подбором дополнительных примеров для проведения по ним проверки, поэтому чаще более удобно использовать описанную ниже методологию.

1. Собрать множество примеров большого объема. 2. Разделить его на два непересекающихся подмножества: обучающее множество и проверочное множество.
3. Применить обучающий алгоритм к обучающему множеству для формирования гипотезы h .
4. Определить, какой процент примеров в проверочном множестве правильно классифицируется с помощью гипотезы h .
5. Повторять этапы 2-4 для различных размеров обучающих множеств и различных случайно выбранных обучающих множеств каждого размера.

Результатом применения этой процедуры становится набор данных, которые можно обрабатывать для получения среднего качества предсказаний, которое выражается в виде некоторой функциональной зависимости от размера обучающего множества. Эта функция может быть изображена в виде графика, представляющего собой так называемую $\wedge$ кривую обучения для данного алгоритма в данной конкретной проблемной области. Кривая обучения для алгоритма Decision-Tree Learning, полученная с использованием примеров из задачи с рестораном, показана на рис. 18.5. Обратите внимание на то, что качество предсказания повышается по мере увеличения размера обучающего множества (по этой причине подобные кривые часто называют счастливыми графиками). Это — хороший признак, который показывает, что в данных действительно есть какие-то повторяющиеся шаблоны (закономерности) и обучающий алгоритм их выделяет.

Безусловно, что обучающему алгоритму не должно быть разрешено "касаться" проверочных данных перед тем, как по ним будет проверена изученная гипотеза.

К сожалению, часто очень легко можно попасть в ловушку, связанную с тем, что алгоритм $\wedge$ компрометирует проверочные данные. Компрометация обычно происходит следующим образом: в обучающем алгоритме могут быть предусмотрены всевозможные "регуляторы", предназначенные для настройки его поведения, например различные и разнотипные критерии выбора следующего атрибута в процессе обучения дерева решений. Итак, формируются гипотезы для всевозможных различных установок этих регуляторов, измеряется их производительность на проверочном множестве и формируется отчет о производительности предсказания наилучшей гипотезы. К сожалению, при этом происходит компрометация! Причина этого состоит в том, что гипотеза была выбрана по результатам измерения ее производительности на проверочном множестве, поэтому информация о проверочном множестве проникла в обучающий алгоритм. Мораль этой истории состоит в том, что в любом процессе, предусматривающем сравнение производительностей гипотез на проверочном

множестве, необходимо использовать новое проверочное множество для измерения производительности окончательно выбранной гипотезы. Но на практике такой подход осуществить слишком сложно, поэтому исследователи по-прежнему продолжают выполнять эксперименты на бывших в употреблении множествах примеров.

0,4 -1 | . . . » 0 20 40 60 80 100 Размер обучающего множества Рис. 18.5. Кривая обучения для алгоритма обучения дерева решений на 100 случайно сформированных примерах в проблемной области задачи с рестораном. В этом графике подытожены результаты 20 попыток Шум и чрезмерно тщательная подгонка Как было показано выше, если имеются два или несколько примеров с одинаковым описанием (с точки зрения атрибутов), но с разными классификациями, то работа алгоритма Decision-Tree-Learning обязательно закончится неудачей, поскольку невозможно будет найти дерево решений, совместимое со всеми примерами. Кроме того, уже упоминалось, что приемлемый способ решения этой проблемы может предусматривать либо применение в каждом листовом узле мажоритарной классификации для относящегося к нему множества примеров (если требуется детерминированная гипотеза), либо формирование оценок вероятностей каждой классификации с использованием относительных частот. К сожалению, описанные выше ситуации не исчерпывают перечень всех возможных нарушений в процессе фор-

мирования дерева решений. Вполне возможна такая ситуация (которая действительно часто встречается на практике), что алгоритм обучения деревьев решений формирует дерево решений, совместимое со всеми примерами, даже несмотря на то, что эти примеры не содержат крайне важной информации для данной задачи классификации. Это связано с тем, что в рассматриваемом алгоритме могут использоваться не относящиеся к делу атрибуты (если они имеются), в результате чего проводятся несуществующие различия между примерами.

Рассмотрим задачу, в которой осуществляются попытки предсказать результаты броска игральной кости. Предположим, что эксперименты проводятся в течение продолжительного периода времени с различными игральными костями и что атрибуты, описывающие каждый обучающий пример, являются следующими:

1. Day (День недели). День, в который был выполнен бросок игральной кости (Mon (Понедельник), Tue (Вторник), Wed (Среда), Thi (Четверг)).
2. Month (Месяц). Месяц, в который был выполнен бросок игральной кости (Jan (Январь) или Feb (Февраль)).
3. Color (Цвет). Цвет игральной кости (Red (Красный) или Blue (Синий)).

При условии, что не существует двух примеров с одинаковыми описаниями и разными классификациями, алгоритм Decision-Tree-Learning позволяет найти точную гипотезу. Чем больше количество используемых атрибутов, тем выше вероятность, что будет найдена точная гипотеза. Но все такие гипотезы будут полностью не связанными с действительностью, поскольку рассматриваемые атрибуты не влияют на выпадение игральной кости. Требуется лишь то, чтобы алгоритм Decision-Tree-Learning возвратил единственный листовой узел с вероятностями, близкими к $1/6$, для каждого результата выпадения очков на игральной кости, как только будет получено достаточное количество примеров.

Каждый раз, когда приходится сталкиваться с множеством возможных гипотез, имеющим большой объем, необходимо тщательно следить за тем, чтобы возникающая при этом свобода выбора не использовалась для поиска бессмысленных "закономерностей" в данных. Эта проблема называется ^ чрезмерно тщательной подгонкой. На практике очень часто встречается такой феномен, что чрезмерно тщательная подгонка происходит, даже если целевая функция вообще не является случайной. Указанный недостаток возникает в обучающих алгоритмах любого типа, а не только в алгоритмах обучения деревьев решений.

Полная математическая трактовка проблемы чрезмерно тщательной подгонки выходит за рамки данной книги. Но в этом разделе представлен простой метод, называемый ^ отсечением ветвей дерева решений, позволяющий в определенной степени справиться с указанной проблемой. Метод отсечения ветвей действует по принципу предотвращения рекурсивного разбиения по атрибутам, релевантность которых не является очевидной, даже если в соответствующем узле дерева имеются данные, не классифицированные на подмножества с равным количеством положительных и отрицательных примеров. Вопрос состоит в том, как обнаружить нерелевантный атрибут.

Предположим, что осуществляется разбиение множества примеров с использованием какого-то нерелевантного атрибута. Вообще говоря, следует полагать, что результирующие подмножества в этом случае будут иметь приблизительно такие же

соотношения количества примеров из каждого класса, как и первоначальное множество. Это означает, что приращение информации будет близким к нулю⁴. Таким образом, хорошим показателем релевантности атрибута является приращение информации. В таком случае возникает вопрос, насколько большим должно быть это приращение для того, чтобы имело смысл осуществлять разбиение по какому-то конкретному атрибуту?

На этот вопрос можно ответить с использованием статистической χ^2 проверки значимости. Такая проверка начинается с принятия предположения о том, что в данных нет никаких скрытых закономерностей (это предположение называется H_0 нульгипотезой). После этого проводится анализ фактических данных для определения того, в какой степени они отличаются от данных, характеризующихся абсолютным отсутствием закономерностей. Если полученная степень отклонения является статистически маловероятной (обычно принято считать, что вероятность отклонения составляет 5% или меньше), то такие данные рассматриваются как надежное свидетельство наличия значимых закономерностей в данных. Вероятности вычисляются на основании стандартных распределений величины отклонения, которые можно надеяться обнаружить в случайно сформированных выборках.

В данном случае нуль-гипотеза состоит в том, что атрибут является нерелевантным, а следовательно, приращение информации для бесконечно большого образца будет равно нулю. Необходимо вычислить вероятность того, что после принятия нуль-гипотезы образец с размером n будет показывать наблюдаемое отклонение от ожидаемого распределения положительных и отрицательных примеров. Такое отклонение можно измерить, сравнивая фактические количества положительных и отрицательных примеров в каждом подмножестве, p_L и $p_{\pm}$, с ожидаемыми количествами, P_i и p_i при том условии, что принято предположение об истинной нерелевантности атрибута:

$$P_i + m - p_i + n_i \chi^2 = \sum_{i=1}^m \frac{(p_i - P_i)^2}{P_i} + \sum_{i=1}^n \frac{(p_i - p_i)^2}{p_i} = \sum_{i=1}^m \frac{(p_i - P_i)^2}{P_i} + \sum_{i=1}^n \frac{(p_i - p_i)^2}{p_i}$$

$$P_i + m - p_i + n_i \chi^2 = \sum_{i=1}^m \frac{(p_i - P_i)^2}{P_i} + \sum_{i=1}^n \frac{(p_i - p_i)^2}{p_i} = \sum_{i=1}^m \frac{(p_i - P_i)^2}{P_i} + \sum_{i=1}^n \frac{(p_i - p_i)^2}{p_i}$$
Удобный критерий суммарного отклонения определяется следующей формулой: $\chi^2 = \sum_{i=1}^m \frac{(p_i - P_i)^2}{P_i} + \sum_{i=1}^n \frac{(p_i - p_i)^2}{p_i}$ Согласно нуль-гипотезе, значение D распределяется в соответствии с распределением χ^2 (хи-квадрат) с $v-1$ степенями свободы. Вероятность того, что атрибут действительно является нерелевантным, можно рассчитать с помощью стандартных таблиц χ^2 или с применением статистического программного обеспечения. В упр.

18.11 предлагается внести соответствующие изменения в алгоритм DecisionTree-Learning для реализации этой формы отсека, которая известна под названием χ^2 отсечение %2.

Отсечение позволяет также справиться с шумом: ошибки классификации приводят к линейному увеличению ошибки предсказания, а ошибки в описаниях примеров оказывают асимптотическое влияние, которое становится все более ярко выра4 В действительности это приращение всегда будет положительным, за исключением тех случаев, когда все эти соотношения будут абсолютно одинаковыми (см. упр. 18.10).

женным по мере того, как дерево все больше сокращается до меньших множеств.

Деревья, сформированные с помощью отсечения, показывают гораздо лучшую производительность, чем деревья, сформированные без отсечения, в том случае, если данные содержат большой объем шума. Деревья с отсеченными ветвями часто бывают намного меньше, поэтому их структуру проще понять.

Еще одним методом, позволяющим уменьшить влияние чрезмерно тщательной подгонки, является $\wedge$ перекрестная проверка. Этот метод применим к любому алгоритму обучения, а не только к алгоритму обучения деревьев решений. Основная идея этого метода состоит в том, что можно оценить, насколько хорошо каждая гипотеза строит предсказание по данным, не встречавшимся ранее. Этот метод осуществляется на практике путем резервирования определенной части известных данных и дальнейшего использования их для проверки производительности предсказания той гипотезы, которая выведена из оставшихся данных. В методе k -кратной перекрестной проверки предусматривается проведение k экспериментов с резервированием каждый раз другой части данных для проведения проверки, объем которой равен $1/k$ от первоначальных данных, с последующим усреднением результатов. В качестве значения k часто используются 5 и 10. В пределе в данном методе может применяться $k=p$; такой вариант метода называют также перекрестной проверкой с исключением одного примера (leave-one-out cross-validation). Перекрестная проверка может использоваться в сочетании с любым методом формирования дерева (включая метод отсечения ветвей) в целях создания дерева, характеризующегося высокой производительностью предсказания. В дальнейшем, чтобы исключить возможность компрометации проверочных данных, необходимо провести измерение производительности полученного дерева с помощью нового проверочного множества.

Расширение области применения деревьев решений Для того чтобы распространить методы индуктивного вывода деревьев решений на более широкий круг задач, необходимо решить целый ряд проблем. В данном разделе кратко описана каждая из этих проблем, но более полного их понимания можно добиться, выполнив указанные здесь упражнения.

- Недостаточные данные. Во многих проблемных областях не все значения атрибутов могут быть определены для каждого примера, в связи с тем, что такие значения могут оказаться незарегистрированными или задача их получения является слишком дорогостоящей. Такая ситуация приводит к возникновению двух проблем. Во-первых, если дано полное дерево решений, то как следует классифицировать некоторый объект, для которого не задан один из проверяемых атрибутов? Во-вторых, как следует модифицировать формулу приращения информации, если в некоторых примерах неизвестны значения данного атрибута? Эти вопросы рассматриваются в упр. 18.12.
- Многозначные атрибуты. Если атрибут имеет много возможных значений, то критерий приращения информации придает оценке

полезности атрибута не соответствующую ей значимость. В крайнем случае может встретиться такой атрибут, который имеет в каждом примере другое значение, скажем RestaurantName (Название ресторана). В таком случае каждое подмножество примеров становится одноэлементным подмножеством с уникальной клас-

сификацией, поэтому критерий приращения информации для соответствующего атрибута принимает наивысшее значение. Тем не менее этот атрибут может оказаться нерелевантным или бесполезным. Одним из решений данной проблемы является использование χ^2 коэффициента приращения (упр. 18.13).

- Непрерывные и целочисленные входные атрибуты. Непрерывные или целочисленные атрибуты, такие как Height (Рост) и Weight (Вес), имеют бесконечное множество возможных значений. Но вместо формирования бесконечно большого количества ветвей алгоритмы обучения деревьев решений, как правило, находят χ^2 точку разбиения, позволяющую получить наивысшее приращение информации. Например, в каком-то конкретном узле дерева может оказаться, что наибольший объем информации позволяет получить проверка по условию $Weight > 160$. Разработаны эффективные методы динамического программирования для поиска приемлемых точек разбиения, но они все еще представляют собой тот компонент реальных приложений в области обучения деревьев решений, который требует намного больше затрат по сравнению с другими компонентами.
- Выходные атрибуты с непрерывными значениями. Если предпринимается попытка предсказать некоторое числовое значение, такое как оценка произведения искусства, а не провести дискретную классификацию, то необходимо получить χ^2 дерево регрессии. В каждом листовом узле такого дерева задана линейная функция от некоторого подмножества числовых атрибутов, а не единственное значение. Например, ветвь, которая относится к гравюрам, раскрашенным вручную, может оканчиваться линейной функцией от площади, возраста работы и количества цветов. Обучающий алгоритм должен выработать решение о том, когда следует прекратить разбиение и приступить к применению метода линейной регрессии с использованием оставшихся атрибутов (или некоторого их подмножества).

Система обучения деревьев решений для реальных приложений должна быть способной решать все эти проблемы. Особенно важной является обработка переменных с непрерывными числовыми значениями, поскольку числовые данные применяются, например, в физических и финансовых процессах. Было разработано несколько коммерческих пакетов, соответствующих этим критериям, которые использовались для создания нескольких сотен проблемно-ориентированных систем. Во многих областях промышленности и торговли деревья решений обычно становятся первым методом, к которому пытаются прибегнуть, когда из некоторого набора данных необходимо извлечь соответствующий ему метод классификации. Одним из важных свойств деревьев решений является то, что вывод обучающего

алгоритма доступен для понимания людей (а в действительности в этом также состоит законодательное требование к финансовым решениям, на которые распространяются законы против дискриминации).

Таким свойством не обладают нейронные сети (см. главу 20).

18.4. Обучение ансамбля До сих пор в этой главе рассматривались методы обучения, в которых для получения предсказаний использовалась отдельная гипотеза, выбранная из пространства $\mathcal{H}$. В отличие от этого, идея методов $\wedge$ обучения ансамбля состоит в том, что из

пространства гипотез следует выбрать целую коллекцию, или так называемый ансамбль гипотез, и в дальнейшем комбинировать предсказания, полученные с помощью гипотез этого ансамбля. Например, может быть сформировано сто разных деревьев решений из одного и того же обучающего множества, после чего проведено голосование для определения наилучшей классификации нового примера.

В основе стремления использовать обучение ансамбля лежит простая причина.

Рассмотрим ансамбль из $m=5$ гипотез и предположим, что их предсказания комбинируются с использованием несложного мажоритарного голосования. Для того чтобы этот ансамбль неправильно классифицировал новый пример, его должны неправильно классифицировать по меньшей мере три из пяти гипотез. Однако вполне можно рассчитывать на то, что данная ситуация является гораздо менее вероятной по сравнению с ошибочной классификацией при использовании единственной гипотезы. Допустим, что предполагается, будто каждая гипотеза h_i в ансамбле допускает ошибку с вероятностью p . Иными словами, вероятность того, что случайно выбранный пример будет неправильно классифицирован гипотезой h_i равна p . Кроме того, допустим, что предполагается, будто ошибки, допущенные с применением каждой гипотезы, являются независимыми. В таком случае, если вероятность p мала, то вероятность одновременного появления большого количества ошибок классификации становится микроскопической. Например, простой расчет (упр. 18.14) показывает, что использование ансамбля из пяти гипотез позволяет сократить частоту ошибок от величины $1/10$ до величины меньше чем $1/100$. Тем не менее очевидно, что предположение о независимости гипотез неоправданно, поскольку во всех гипотезах, скорее всего, будут возникать одинаковые искажения, вызванные одними и теми же искажающими их аспектами одинаковых обучающих данных. Но если гипотезы хоть немного отличаются друг от друга, что приводит к уменьшению корреляции между их ошибками, то обучение ансамбля может оказаться очень полезным.

Еще один способ трактовки идеи ансамбля состоит в том, что ансамбль — это универсальный метод расширения пространства гипотез. Это означает, что сам

ансамбль может рассматриваться как гипотеза, а новое пространство гипотез — как множество всех возможных ансамблей, которые могут быть сформированы из гипотез первоначального пространства. Как показано на рис. 18.6, такой подход может привести к созданию более выразительного пространства гипотез. Если первоначальное пространство гипотез допускает возможность использовать простой и эффективный алгоритм обучения, то метод формирования ансамбля предоставляет возможность формировать в процессе обучения гораздо более выразительный класс гипотез без значительного дополнительного увеличения вычислительной или алгоритмической сложности.

Наиболее широко используемый метод формирования ансамбля называется $\wedge$ усилением. Для того чтобы понять, как он работает, необходимо вначале ознакомиться с идеей $\wedge$ взвешенного обучающего множества. В таком обучающем множестве с каждым примером связан вес $W_j > 0$. Чем больше вес примера, тем выше важность, присвоенная ему в процессе изучения какой-то гипотезы. Рассматриваемые до сих пор в этой главе алгоритмы обучения несложно модифицировать для работы со взвешенными обучающими множествами⁵.

5 Имеются также такие обучающие алгоритмы, которые не предоставляют подобной возможности.

Но для них вместо этого можно создать тиражированное обучающее множество, в котором i -й пример появляется W_j раз, а для учета нецелочисленных значений весов используется рандомизация.

Л // Jj++ + f\ \ Рис. 18.6. Схема, показывающая, что обучение ансамбля позволяет добиться повышения выразительной мощности гипотез.

Здесь представлены три линейные пороговые гипотезы, каждая из которых формирует положительную классификацию на незаштрихованной стороне, а в целом как положительные классифицируются все примеры, которые являются положительными согласно всем трем гипотезам. Полученная в итоге треугольная область представляет собой гипотезу, которая не может быть выражена в первоначальном пространстве гипотез. Процедура усиления начинается с задания $W_j = 1$ для всех примеров (т.е. с обычного обучающего множества). На основании этого множества вырабатывается первая гипотеза h_1 , которая классифицирует одни обучающие примеры правильно, а другие — неправильно. Желательно, чтобы следующая гипотеза лучше справлялась с неправильно классифицированными примерами, поэтому веса последних увеличиваются, а веса правильно классифицированных примеров уменьшаются. По этому обучающему множеству со вновь назначенными весами вырабатывается гипотеза h_2 .

Описанный процесс продолжается таким же образом до тех пор, пока не будет выработано m гипотез, где остановится входом для алгоритма усиления.

Окончательная гипотеза-ансамбль представляет собой взвешенную мажоритарную комбинацию из всех m гипотез, каждой из которых назначен вес, соответствующий тому, насколько высокую производительность она показала при обработке обучающего множества.

На рис. 18.7 показана концептуальная иллюстрация работы алгоритма. Эта основная идея усиления имеет много вариантов, в которых применяются различные способы корректировки весов и комбинирования гипотез. В листинге 18.2 показан один из конкретных алгоритмов, называемый AdaBoost. Хотя подробные сведения о том, как происходит корректировка весов, не имеет столь важного значения, алгоритм AdaBoost обладает очень важным свойством: если входной обучающий алгоритм L является $\wedge$ слабым обучающим алгоритмом (а это означает, что L всегда возвращает гипотезу со взвешенной ошибкой на обучающем множестве, которая лишь ненамного лучше по сравнению со случайным угадыванием, например, равным 50% при булевой классификации), то алгоритм AdaBoost при достаточно большом значении m возвращает гипотезу, идеально классифицирующую обучающие данные. Таким

образом, этот алгоритм значительно повышает точность первоначального обучающего алгоритма применительно к обучающим данным. Такой результат остается в силе независимо от того, насколько невыразительным является первоначальное пространство гипотез, а также от того, насколько сложна изучаемая функция.

Рис. 18.7. Иллюстрация работы алгоритма усиления.

Каждый затененный прямоугольник соответствует некоторому примеру; высота прямоугольника соответствует весу.

Галочки и крестики показывают, был ли данный пример классифицирован правильно с помощью текущей гипотезы.

Размер дерева решений показывает вес этой гипотезы в окончательном ансамбле Листинг 18.2. Один из вариантов метода усиления для обучения ансамбля, представленный в виде алгоритма AdaBoost. Этот алгоритм вырабатывает гипотезу, последовательно корректируя веса обучающих примеров. Функция `Weighted-Majority` вырабатывает гипотезу, которая возвращает выходное значение с наивысшими результатами голосования из числа гипотез, относящихся к вектору h , где результаты голосования взвешиваются с помощью вектора z

`function AdaBoost(examples, L, M) returns взвешенная мажоритарная комбинация гипотез`
`inputs: examples, множество из  $iV$  размеченных примеров  $(x_i, y_i), \dots, (x_N, y_N)$   $L$ , обучающий алгоритм  $M$ , количество гипотез в ансамбле`
`local variables:  $w$ , вектор из`

N весов примеров, первоначально равных $1/N$, вектор из M гипотез z , вектор из M весов гипотез

```

for  $\tau = 1$  to  $M$  do
   $h[m] \leftarrow \arg\min_m \sum_{j=1}^N \text{error}(h[m](x_j), y_j)$ 
  for  $j = 1$  to  $N$  do
    if  $h[m](x_j) \neq y_j$  then  $\text{error} \leftarrow \text{error} + w[j]$ 
  for  $j = 1$  to  $N$  do
    if  $h[m](x_j) = y_j$  then  $w[j] \leftarrow w[j] - \text{error} / (A - \text{error})$ 
   $w \leftarrow \text{Normalize}(w)$ 
   $z[m] \leftarrow \log(A - \text{error}) / \text{error}$ 
return  $\text{Weighted-Majority}(h, z)$ 

```

Рассмотрим, насколько хорошо метод усиления действует применительно к данным о ресторане. Выберем в качестве первоначального пространства гипотез класс $\wedge$ одноузловых деревьев решений, представляющих собой деревья решений только с одной проверкой, в корневом узле. Нижняя кривая, приведенная на рис. 18.8, а, показывает, что неусиленные одноузловые деревья решений не очень эффективно действуют применительно к этому набору данных, достигая производительности предсказания, составляющей только 81% в расчете на 100 обучающих примеров.

А после применения метода усиления (при $m=5$) производительность становится выше и достигает 93% после обработки 100 примеров.

Одноузловое дерево решений - S M 0,95 2Ц 0,9 II 0,85 !§ 0,8 O (V) = « 0 7 о о. н с 0,65 0,6 Ошибки на обучающем множестве Ошибки на проверочном множестве 0 20 40 60 80 Объем обучающего множества а) 50 100 150 Количество гипотез M б) Рис. 18.8. Анализ производительности алгоритмов: график, показывающий, как изменяется производительность усиленных одноузловых деревьев решений при $K=5$ по сравнению с неусиленными одноузловыми деревьями решений на примере данных о ресторане (а); доля правильных ответов, полученных на обучающем множестве и проверочном множестве, как функция от M (от количества гипотез в ансамбле) (б). Обратите внимание на то, что точность распознавания примеров из проверочного множества немного повышается даже после того, как точность распознавания примеров из обучающего множества достигает 1, т.е. после того, как ансамбль гипотез полностью согласуется с данными. По мере увеличения размера ансамбля m обнаруживается интересное явление.

На рис. 18.8, б показана производительность обучающего множества (на 100 примерах) как функция от m . Обратите внимание на то, что ошибка достигает нуля (как и следует из определения метода усиления), когда M остановится равным 20; это означает, что взвешенная мажоритарная комбинация из 20 одноузловых деревьев решений вполне позволяет определить точное соответствие для 100 примеров. По мере введения в ансамбль дополнительных одноузловых деревьев решений ошибка остается равной нулю. Этот график также показывает, что $\leq$ производительность обработки проверочного множества продолжает возрастать в течение долгого времени после того, как ошибка на обучающем множестве достигает нуля. При $M=20$ производи-

тельность на проверочном множестве равна 0,95 (что соответствует 0,05 ошибки) и после чего увеличивается до 0,98 при таком большом значении, как $n=137$, прежде чем постепенно уменьшиться до 0,95.

Эта особенность, которая неизменно проявляется в самых разных наборах данных и пространствах гипотез, после ее обнаружения впервые показалась исследователям весьма неожиданной. Согласно принципу бритвы Оккама, не следует создавать гипотезы, более сложные, чем необходимо, а этот график говорит нам о том, что по мере усложнения гипотезы-ансамбля предсказания улучшаются! Для объяснения этого феномена было предложено несколько трактовок. Один из подходов к анализу такого явления состоит в том, что в процессе усиления аппроксимируется байесовское обучение (см. главу 20), притом что можно доказать, что байесовский алгоритм является оптимальным обучающим алгоритмом, а аппроксимация улучшается по мере введения дополнительных гипотез. Еще одно возможное объяснение состоит в том, что введение дополнительных гипотез позволяет добиться того, что ансамбль проводит все более определенное различие между положительными и отрицательными примерами, а это свойство способствует лучшей классификации новых примеров.

18.5. Принципы функционирования алгоритмов обучения: теория вычислительного обучения Один из важных вопросов, поставленных в разделе 18.2, на который не был получен ответ, состоял в следующем: как можно убедиться в том, что в результате применения разработанного кем-то обучающего алгоритма была создана теория, позволяющая правильно предсказывать будущее? Формально этот вопрос можно переформулировать следующим образом: как определить, насколько гипотеза h близка к целевой функции f , если неизвестно, каковой является сама функция f ?

Подобные вопросы были предметом размышлений ученых в течение нескольких столетий.

До тех пор, пока на них не будут получены ответы, машинное обучение в лучшем случае может рассматриваться лишь как научная область, причины успешных достижений которой остаются необъяснимыми.

Подход, принятый в данном разделе, основан на теории вычислительного обучения — научной области, которая находится на стыке искусственного интеллекта, статистики и теоретических компьютерных наук. Принцип, лежащий в ее основе, состоит в следующем: любая гипотеза, которая содержит серьезные ошибки, почти наверняка будет "открыта" с большой вероятностью после обработки небольшого количества примеров, поскольку она дает неправильные предсказания. Поэтому любая гипотеза, согласованная с достаточно большим множеством обучающих примеров, с меньшей вероятностью будет содержать серьезные

ошибки; это означает, что она обязательно будет & приблизительно правильной с определенной вероятностью.

Любой обучающий алгоритм, вырабатывающий гипотезы, которые с определенной вероятностью являются приблизительно правильными (Probably Approximately Correct — PAC), называется алгоритмом $\wedge$ PAC-обучения.

При анализе приведенных выше доводов необходимо учитывать некоторые нюансы.

Основной вопрос состоит в том, какова связь между обучающими и проверочными примерами; в конечном итоге желательно, чтобы гипотеза была приблизительно правильной

применительно к проверочному множеству, а не только к обучающему множеству.

Основное предположение состоит в том, что и обучающее, и проверочное множества примеров извлекаются случайно и независимо друг от друга из одной и той же популяции примеров с одним и тем же распределением вероятностей. Это предположение называется предположением о $\wedge$ стационарности. Если не принято предположение о стационарности, то теория вычислительного обучения не позволяет формулировать вообще какие-либо утверждения о будущем, поскольку не определена необходимая связь между будущим и прошлым. Предположение о стационарности равносильно тому предположению, что процесс, в котором осуществляется отбор примеров, не подвержен неблагоприятному влиянию. Очевидно, что если обучающее множество состоит только из надуманных примеров (например, фотографий двухголовых собак), то обучающий алгоритм не сможет сделать ничего иного, кроме как предложить безуспешные обобщения, касающиеся того, как распознавать обычных собак.

Оценка количества необходимых примеров Для того чтобы перевести эти предположения на практическую почву, необходимо ввести некоторые описанные ниже обозначения. • Обозначим символом X множество всех возможных примеров. • Обозначим символом D распределение, из которого извлекаются примеры. • Обозначим символом H множество возможных гипотез. • Обозначим символом N количество примеров в обучающем множестве.

Первоначально будем предполагать, что истинная функция f является элементом множества H . Теперь можно определить $\wedge$ ошибку гипотезы h применительно к истинной функции f , если дано распределение вероятностей D по примерам, описывающее вероятность того, что гипотеза h отлична от функции f на некотором примере: $error(h) = P\{f(x) \neq h(x) \mid x \text{ извлечен из } D\}$ Это — такое же количество, которое измерялось экспериментально с помощью кривых обучения, описанных выше в данной главе.

Гипотеза h называется приблизительно правильной, если $\text{error}(h) < \epsilon$, где ϵ — небольшая константа. Примем к действию план решения этой проблемы, который состоит в том, чтобы доказать, что после просмотра N примеров все совместимые гипотезы с высокой вероятностью станут приблизительно правильными.

Приблизительно правильная гипотеза может рассматриваться как "близкая" к истинной функции в пространстве гипотез: она находится внутри так называемого ϵ -шара, который окружает истинную функцию f . На рис. 18.9 показано множество всех гипотез h , которое подразделяется на ϵ -шар, окружающий функцию f , и все остальные гипотезы, принадлежащие к множеству, которое мы будем называть H_{bad} .

Вероятность того, что гипотеза $h \in H_{\text{bad}}$, содержащая "серьезную ошибку", будет согласована с первыми N примерами, можно вычислить следующим образом.

Известно, что $\text{error}(h) > \epsilon$. В таком случае вероятность того, что эта гипотеза согласуется с заданным примером, равна по меньшей мере $1 - \epsilon$. Граничное значение этой вероятности для N примеров равно:

$$P(h \text{ согласуется с } N \text{ примерами}) < (1 - \epsilon)^N$$

Над Рис. 18.9. Схематическое изображение пространства гипотез, на котором показан ϵ -шар, окружающий истинную функцию f . Вероятность того, что множество H_{bad} содержит по меньшей мере одну совместимую гипотезу, ограничивается суммой отдельных вероятностей: $P(H_{\text{bad}} \text{ содержит совместимую гипотезу}) < |H_{\text{bad}}| (1 - \epsilon)^N < |H| (1 - \epsilon)^N$ где учитывался тот факт, что $|H_{\text{bad}}| < |H|$. Желательно уменьшить вероятность этого события так, чтобы она не превышала некоторого небольшого числа δ : $|H| (1 - \epsilon)^N < \delta$. С учетом того, что $1 - \epsilon < e^{-\epsilon}$, такой цели можно добиться, предоставив алгоритму возможность обработать следующее количество примеров:

$$N > \frac{1}{\epsilon} (\ln |H| + \ln \frac{1}{\delta})$$
 (A8.1) Таким образом, если обучающий алгоритм возвратит гипотезу, совместимую с этим или большим количеством примеров, то с вероятностью, по меньшей мере равной $1 - \delta$, он будет иметь, самое большее, ошибку ϵ . Иными словами, полученная гипотеза будет по всей вероятности приблизительно правильной. Количество требуемых примеров, зависящее от ϵ и δ , называется ϵ -выборочной сложностью (sample complexity) пространства гипотез.

Поэтому создается впечатление, что основной вопрос сводится к определению размера пространства гипотез. Как было показано выше, если h — множество всех булевых функций от p атрибутов, то $|h| = 2^p$. Таким образом, выборочная сложность пространства растет в зависимости от 2^p . Поскольку количество возможных примеров также равно 2^p , на этом основании можно утверждать, что любой обучающий алгоритм для пространства всех булевых функций не может функционировать лучше, чем поисковая таблица, если он просто возвращает

гипотезу, совместимую со всеми известными примерами. Еще один способ убедиться в справедливости этого утверждения может быть основан на том наблюдении, что для любого не встречавшегося ранее примера пространство гипотез будет содержать столько же со-

вместимых гипотез, которые предсказывают положительный результат, сколько имеется гипотез, предсказывающих отрицательный результат.

Поэтому дилемма, с которой мы сталкиваемся, состоит в следующем: если пространство функций, которые могут рассматриваться алгоритмом, останется неограниченным, то алгоритм не будет способен к обучению, но, с другой стороны, ограничение пространства может привести к полному удалению истинной функции.

Существуют два способа "разрешения" этой дилеммы. Первый способ состоит в соблюдении требования, чтобы алгоритм возвращал не просто любую совместимую гипотезу, а преимущественно наиболее простую гипотезу (что и предусмотрено в обучении деревьев решений). Теоретический анализ подобных алгоритмов выходит за рамки данной книги, но в тех случаях, когда задача поиска простых совместимых гипотез является легко разрешимой, результаты определения выборочной сложности обычно бывают лучше по сравнению с результатами анализа, основанными только на определении совместимости. Второй способ разрешения дилеммы, который рассматривается ниже, состоит в том, что нужно сосредоточиться на тех подмножествах всего множества булевых функций, которые доступны для обучения.

Идея этого способа состоит в том, что в большинстве случаев не требуется полная выразительная мощь булевых функций и можно ограничиться использованием более простых языков. В следующем разделе один из таких ограниченных языков рассматривается более подробно.

Обучение списков решений ^ Список решений — это логическое выражение с ограниченной формой. Он состоит из ряда проверок, каждая из которых представляет собой конъюнкцию литералов. Если проверка, применяемая к описанию примера, завершается успешно, то список решений задает возвращаемое значение. Если же проверка оканчивается неудачей, то обработка продолжается со следующей проверки в списке. Списки решений напоминают деревья решений, но их общая структура проще. С другой стороны, отдельные проверки в них намного сложнее. На рис. 18.10 показан список решений, который представляет следующую гипотезу:

$Vx \text{ WillWait}(x) \iff \text{Patrons}(x, \text{Some}) \vee \{ \text{Patrons}(x, \text{Full}) \wedge \text{Fri/Sat}(x) \}$ Если допускается применение проверок с произвольными размерами, то списки решений становятся способными представить любую булеву функцию (упр. 18.15).

С другой стороны, если размер каждой проверки ограничен, самое большее, j с литералами, то для обучающего алгоритма появляется возможность успешно создавать обобщения на основе небольшого количества примеров. Мы будем называть такой язык списков решений с k литералами языком k -DL (DL— Decision List).

Пример, приведенный на рис. 18.10, относится к языку 2-DL. Можно легко показать (упр. 18.15), что язык k -DL включает в качестве подмножества язык k -DT (DT — Decision Tree), который представляет собой множество всех деревьев решений с глубиной, не превышающей $7k$. Важно помнить, что конкретный язык, к которому применяется обозначение k -DL, зависит от атрибутов, используемых для описания примеров. Мы будем использовать запись k -DL(n) для обозначения языка k -DL, в котором применяется n булевых атрибутов.

6 Поэтому список решений идентичен по своей структуре оператору COND в языке Lisp.

D / PQtronS(A, oufnc/ Yes ШШ No Patrons(x, Full) a Fri/Sat(x) IYes ym No , '1 Рис. 18.10. Список решений для задачи с рестораном Первая задача состоит в том, чтобы показать, что язык k -DL доступен для изучения, т.е. что любая функция, заданная в языке k -DL, может быть точно аппроксимирована после обучения на приемлемом количестве примеров. Для этого необходимо рассчитать количество гипотез, которые могут быть представлены на этом языке. Допустим, что язык проверок (конъюнкций из самое большее k литералов, в которых используется n атрибутов) обозначается как $\text{Conj}(n, k)$. Поскольку любой список решений состоит из проверок, а каждая проверка может быть связана с результатом Yes или No или может отсутствовать в списке решений, то количество различных множеств проверок, из которых могут состоять гипотезы, измеряется величиной $3^{|\text{Conj}(n, k)|}$. Каждое из этих множеств проверок может быть задано в любом порядке, поэтому справедливо следующее соотношение: $|\text{DL}(n)| < 3^{|\text{Conj}(n, k)|}$. Количество конъюнкций из k литералов с n атрибутами может быть определено таким образом:

$|\text{Conj}(n, k)| = \sum_{i=0}^k \binom{n}{i} = 2^n - 1$ Поэтому после проведения некоторых преобразований можно получить следующее: $|\text{DL}(n)| = 2^{2^n - 1}$ Это соотношение можно подставить в уравнение 18.1, чтобы показать, что количество примеров, необходимое для PAC-обучения некоторой функции k -DL, определяется полиномиальной зависимостью от n :

$n > \frac{1}{\epsilon} \ln \frac{1}{\delta} + O(nk \log_2(nk))$ Поэтому любой алгоритм, возвращающий согласованный список решений, будет обеспечивать PAC-обучение некоторой функции k -DL с помощью приемлемого количества примеров, если значение k невелико.

Следующая задача состоит в том, чтобы найти эффективный алгоритм, который возвращает совместимый список решений. Мы будем использовать жадный алгоритм, называемый Decision-List-Learning, который повторно отыскивает проверку, точно согласующуюся с некоторым подмножеством обучающего множества. Найдя такую проверку, алгоритм добавляет ее к создаваемому списку решений и удаляет соответствующие примеры. Затем алгоритм формирует только оставшуюся часть списка решений, используя лишь оставшиеся примеры. Такая операция повторяется до тех пор, пока не останется ни одного примера. Этот алгоритм показан в листинге 18.3.

Листинг 18.3. Алгоритм обучения списков решений function Decision-List-Learning(examples) или индикатор неудачи failure returns список решений if множество examples пусто then return тривиальный список решений No t $\leftarrow$ проверка, которая согласуется с непустым подмножеством examplest множества examples, таким что все элементы examplest являются либо положительными, либо отрицательными примерами if такая проверка t отсутствует then return failure if примеры в подмножестве examplest являются положительными then o $\leftarrow$ Yes else o $\leftarrow$ No return список решений с начальной проверкой t, результатом o и оставшимися проверками, определяемыми выражением Decision-List-Learning(examples-examplest) В этом алгоритме не определен метод выбора следующей проверки для добавления к списку решений. Хотя формально приведенные выше результаты не зависят от такого метода выбора, представляется резонным подход, позволяющий отдавать предпочтение небольшим проверкам, которые согласуются с большими множествами однородно классифицируемых примеров, для того чтобы общий список решений был как можно более компактным. Простейшая стратегия достижения этой цели состоит в том, что нужно отыскивать наименьшую проверку t, которая согласуется с любым однородно классифицируемым подмножеством, независимо от размера этого подмножества. Как показано на рис. 18.11, даже такой примитивный подход действует вполне успешно.

1 1 0,9 \ 0,8 0,7] S & 0,6 с со ' t* S 0,5 0,4 20 40 60 80 Объем обучающего множества
Рис. 18.11. Кривая обучения по данным о ресторане при использовании алгоритма Decision-List-Learning.

Для сравнения показана кривая, соответствующая алгоритму Decision-Tree-Learning
Обсуждение полученных результатов В рамках теории вычислительного обучения был создан новый способ трактовки проблемы обучения. В начале 1960-х годов разработки в области теории обучения

сосредоточивались в основном на проблеме успешной $\wedge$ идентификации в пределе.

В соответствии с этим понятием любой алгоритм идентификации должен возвращать гипотезу, которая точно совпадает с истинной функцией. Один из способов решения этой задачи состоит в следующем: вначале упорядочить все гипотезы в множестве H в соответствии с некоторым критерием простоты, затем выбрать простейшую гипотезу, совместимую со всеми полученными до сих пор примерами.

По мере получения новых примеров в этом методе предусмотрен отказ от более простой гипотезы, которая стала недействительной, и принятие вместо нее более сложной гипотезы. Но после того как достигается истинная функция, от нее никогда не происходит отказ. К сожалению, в больших пространствах гипотез количество примеров и продолжительность вычислений, необходимая для достижения истинной функции, становятся колоссальными. Поэтому теория вычислительного обучения не определяет безусловного требования, чтобы обучающийся агент определил "единственный истинный закон", руководящий его средой, а вместо этого допускает, чтобы он нашел гипотезу с определенной степенью прогностической точности.

Кроме того, в теории вычислительного обучения большое внимание уделяется выявлению компромисса между выразительностью языка гипотез и сложностью обучения, поэтому развитие этой теории непосредственно привело к созданию важного класса обучающих алгоритмов, получившего название машин поддерживающих векторов (support vector machine).

Приведенные в данном разделе результаты РАС-обучения представляют собой результаты определения сложности для наихудшего случая и не обязательно отражают выборочную сложность для среднего случая, которая измеряется с помощью приведенных здесь кривых обучения. При анализе среднего случая необходимо также принять предположения о распределении вероятностей примеров и распределении вероятностей истинных функций, которые должны быть изучены с помощью данного алгоритма. По мере того как достигается все лучшее понимание этих проблем, теория вычислительного обучения предоставляет все более ценную помощь исследователям в области машинного обучения, для которых важно предсказать или модифицировать обучающую способность создаваемых ими алгоритмов. Кроме списков решений, важные результаты были получены почти для всех известных индуктивного обучения, в которых агент приступает к работе без априорных знаний о целевой функции, обычно являются очень трудными. Как будет показано в главе 19, использование априорных знаний для управления индуктивным обучением дает возможность изучать весьма крупные множества высказываний на основе приемлемого количества примеров, даже если используется столь выразительный язык, как логика первого порядка.

18.6. Резюме Настоящая глава главным образом посвящена изложению темы индуктивного обучения детерминированных функций на основе примеров. Основные вопросы, рассматриваемые в этой главе, перечислены ниже.

Обучение принимает много разных форм в зависимости от характера производительного элемента, компонента, подлежащего усовершенствованию, и доступной обратной связи.

Если доступна обратная связь либо от учителя, либо от среды, позволяющая получать правильные значения, относящиеся к примерам, то задача обучения относится к типу контролируемого обучения. Такая задача, называемая также индуктивным обучением, сводится к изучению некоторой функции на примерах ее входных и выходных данных. Изучение функции с дискретными значениями называется классификацией; изучение непрерывной функции называется регрессией.

Индуктивное обучение сводится к поиску совместимой гипотезы, которая согласуется с примерами. При этом должен соблюдаться принцип бритвы Оккама, согласно которому следует всегда выбирать наиболее простую совместимую гипотезу. Сложность решения этой задачи зависит от выбранного способа представления.

Деревья решений позволяют представить любые булевы функции. Эвристика, определяющая приращение информации, может стать основой эффективного метода поиска простого, совместимого дерева решений.

Производительность обучающего алгоритма измеряется кривой обучения, которая показывает прогностическую точность применения алгоритма к проверочному множеству как функцию от размера обучающего множества.

Методы обучения ансамбля деревьев решений, такие как усиление, часто показывают более высокую производительность по сравнению с методами обучения отдельных деревьев решений.

Для анализа выборочной и вычислительной сложности индуктивного обучения применяется теория вычислительного обучения. Эта теория позволяет найти компромисс между выразительностью языка гипотез и легкостью обучения.

Библиографические и исторические заметки История философских исследований в области индуктивного обучения описана в главе 1. Уильям из Окхема (1280-1349), наиболее влиятельный философ своей эпохи, внесший в средние века наибольший вклад в развитие эпистемологии, логики и метафизики, заслужил признание потомков за то, что выдвинул важный принцип, называемый "бритвой Оккама", который на латыни выражается словами "Entia non sunt multiplicanda praeter necessitatem", а в переводе звучит так: "Не следует множить сущности без

необходимости". К сожалению, это замечательное изречение не удалось найти в его трудах точно в такой формулировке.

Одной из первых систем, в которой использовались деревья решений (или, как они в ней именовались, различительные сети (discrimination nets)), была система EPAM (Elementary Perceiver And Memorizer) Фейгенбаума [457]. Система EPAM предназначалась для применения в качестве модели имитации познания в процессе обучения человека новым понятиям. В системе CLS [707] для формирования деревьев решений использовался эвристический прогностический метод. В системе ID3

[1256] была дополнительно реализована крайне важная идея о том, что для обеспечения функционирования эвристической функции может применяться информационное содержание. Сама теория информации была разработана Клодом Шенноном для использования в качестве средства исследования процессов связи [1394].

Важный вклад, сделанный Шенноном, состоит также в том, что он разработал первые образцы систем, действующих по принципу машинного обучения, в частности механическую мышь, получившую имя Theseus (Тезей), которая обучалась прохождению через лабиринт по методу проб и ошибок. Метод %2 отсечения ветвей деревьев был описан Куинланом [1257]. Описание пакета C4.5 производственного назначения, который основан на использовании деревьев решений, можно найти в [1259].

В литературе по статистике существует отдельное направление, посвященное исследованиям в области деревьев решений. Одним из основных источников информации по этой теме является книга Classification and Regression Trees [180], известная под названием книги "CART".

Предпринимались также попытки применить многие другие алгоритмические подходы к обучению. Подход, основанный на использовании текущей наилучшей гипотезы, предусматривает сопровождение единственной гипотезы, а также ее уточнение, если обнаруживается, что она слишком широка, и обобщение, если она оказывается слишком узкой. Такой принцип проведения исследований уже давно применяется в философии [1049]. Кроме того, даже ранние работы в области когнитивной психологии показали, что в этом состоит естественная форма изучения концепций людьми [199]. В искусственном интеллекте такой подход наиболее тесно связан с работой Патрика Уинстона, в докторской диссертации которого [1602] рассматривается проблема изучения описаний сложных объектов. В методе пространства версий [1061], [1062] принят другой подход, в котором сопровождается множество всех совместимых гипотез и удаляются оказавшиеся несовместимыми с новыми примерами. Такой подход был реализован в экспертной

системе Meta-Dendral, применяемой в химии [202], а затем — системе Митчелла Lex [1066], которая обучалась решению задач исчисления. Третье влиятельное направление возникло под влиянием работы Михальского и его коллег над рядом алгоритмов AQ, применявшихся для изучения множеств логических правил [1038], [1041].

Метод обучения ансамбля деревьев решений все чаще применяется для повышения производительности обучающих алгоритмов. Первый эффективный метод такого типа, называемый $\wedge$ созданием мультимножеств [179], предусматривает формирование комбинаций гипотез, изученных на основе многочисленных наборов данных начальной загрузки, каждый из которых формируется путем выборки подмножества из первоначального множества данных. Метод усиления, описанный в данной главе, впервые был изложен в теоретической работе Шейпира [1361].

Алгоритм AdaBoost был разработан Фрейндом и Шейпиром [501], а результаты его теоретического анализа приведены в [1360]. В [505] принципы работы метода усиления описаны с точки зрения специалиста по статистике.

Теоретический анализ обучающих алгоритмов начался с работы Голда [569] по проблеме идентификации в пределе. Стимулом к развитию этого подхода отчасти явились модели научного открытия, разработанные в рамках философии науки [1229], но этот подход в основном применялся к задаче изучения грамматик на примерах предложений [1162].

Подход, основанный на изучении "идентификации в пределе", в основном посвящен достигаемой в конечном итоге сходимости, а исследования $\wedge$ колмогоровской сложности, или алгоритмической сложности, проведенные независимо Соломоновым [1445] и Колмогоровым [828], представляют собой попытку дать формальное определение понятия простоты, используемого в принципе бритвы Оккама. Чтобы исключить проблему, связанную с тем, что простота зависит от способа представления информации, было предложено измерять простоту как длину кратчайшей программы для универсальной машины Тьюринга, которая правильно воспроизводит наблюдаемые данные.

Хотя существует много возможных универсальных машин Тьюринга и поэтому много возможных "кратчайших" программ, было обнаружено, что эти программы отличаются по длине не больше чем на некоторую константу, независимую от объема данных. Это — великолепное открытие, которое по сути показывает, что любое искажение в первоначальном представлении данных в конечном итоге должно быть преодолено с помощью самих же данных. Его широкому применению препятствует лишь то, что задача вычисления длины кратчайшей программы является неразрешимой. Однако вместо точного значения длины могут применяться

такие приближенные критерии, как $\hat{L}$ минимальная длина описания, или MDL (Minimum Description Length) [1291], которые позволяют добиться на практике превосходных результатов. Наилучшим источником сведений по проблеме колмогоровской сложности является книга Ли и Витаньи [927].

Теория вычислительного обучения (т.е. теория PAC-обучения) была выдвинута Лесли Валиантом [1528]. В работе Валианта подчеркивается важность вычислительной и выборочной сложности. Совместно с Майклом Кернсом [783] Валиант показал, что задача PAC-обучения некоторых классов концепций является трудноразрешимой, даже если в примерах присутствует достаточный объем информации.

Некоторые положительные результаты были получены для таких классов, как списки решений [1293].

В статистике сложилось независимое направление анализа выборочной сложности, начиная с одной важной работы Вапника и Червоненкиса по $\hat{L}$ теории равномерной сходимости [1538]. Понятие так называемой $\hat{L}$ VC-размерности (VC — сокращение от Vapnik—Chervonenkis) позволяет получить критерий, приблизительно аналогичный, но более общий, чем критерий $\ln |n|$, полученный на основе анализа PAC-обучения. В частности, критерий VC-размерности может применяться к непрерывным классам функций, на которые не распространяется стандартный анализ PAC-обучения. Теория PAC-обучения и теория VC-размерности были впервые связаны в единое целое группой ученых, называемых "четырьмя немцами" (хотя ни один из них фактически не является немцем), — Блумером, Эренфойхтом, Хаусслером и Вармутом [141]. Дальнейшие исследования в области теории VC-размерности привели к открытию понятия машины поддерживающих векторов, или SVM (Support Vector Machine) [156], [1537], которая будет описана в главе 20.

Большое количество важных статей по машинному обучению было собрано в книге Readings in Machine Learning [1399]. Кроме того, много важных статей, а также огромные библиографические справочники содержатся в двух томах, Machine Learning 1 [1039] и Machine Learning 2 [1040]. В [1565] дано широкое введение в область методов изучения функций, применяемых в машинном обучении, статистике и нейронных сетях. Исследования, намного превосходящие по своей широте все прочие работы в области сравнения производительности обучающих алгоритмов,

были проведены в рамках проекта Statlog [1047]. Результаты продуктивных современных исследований по машинному обучению публикуются в ежегодном сборнике трудов конференций International Conference on Machine Learning и Conference on Neural Information Processing Systems, в журналах Machine Learning и

Journal of Machine Learning Research, а также в основных журналах по искусственному интеллекту.

Кроме того, труды в области теории вычислительного обучения публикуются в материалах ежегодного семинара LCM Workshop on Computational Learning Theory (COLT), а само это направление представлено в книгах [34] и [786].

Упражнения 18.1. Рассмотрим проблему, стоящую перед младенцем, который учится говорить и понимать язык. Объясните, как этот процесс вписывается в общую модель обучения, указывая по мере необходимости каждый из компонентов этой модели.

18.2. Повторите упр. 18.1 для случая обучения игре в теннис (или обучения какому-то другому виду спорта, с которым вы знакомы). Относится ли оно по своему типу к контролируемому обучению или обучению с подкреплением?

18.3. Составьте дерево решений для задачи принятия решения о том, следует ли двигаться вперед на уличном перекрестке в той ситуации, когда на светофоре только что загорелся зеленый свет.

18.4. Мы никогда не проверяем дважды один и тот же атрибут вдоль одного пути в дереве решений. Почему этого не делается?

18.5. Предположим, что на основе дерева решений сформировано обучающее множество, а затем к этому обучающему множеству применен алгоритм обучения дерева решений. Можно ли рассчитывать на то, что этот обучающий алгоритм в конечном итоге возвратит правильное дерево, по мере того как размер обучающего множества будет стремиться к бесконечности? Почему да или почему нет?

18.6. Хороший обучающий алгоритм со "спарринг-партнером" состоит в следующем. Вначале создать таблицу из всех обучающих примеров и выяснить, какие выходные результаты встречаются наиболее часто среди всех обучающих примеров; назовем их d . Затем, после получения входных данных, которые отсутствуют в таблице, возвращать просто данные из множества d , а для входных данных, которые имеются в таблице, возвращать связанные с ними выходные данные (если же количество выходных данных больше единицы, возвращать наиболее часто встречающиеся выходные данные). Реализуйте этот алгоритм и определите, насколько хорошо он действует в проблемной области задачи с рестораном. Это позволит вам получить представление о точке отсчета для этой проблемной области — о минимальной производительности, которую должен быть способен достичь любой алгоритм.

18.7. Предположим, что вы проводите эксперименты по обучению с помощью нового алгоритма. У вас имеется набор данных, состоящий из 25 примеров каждого из двух классов. Вы планируете использовать перекрестную проверку с

исключением одного примера и для определения точки отсчета выполняете прогон вашей экспериментальной системы на простом мажоритарном классификаторе. (Мажоритарный классификатор получает на входе множество обучающих данных, а затем всегда выводит тот класс, который имеет преимущественное распространение в обучающем множестве, независимо от входных данных.) Вы предполагаете, что мажоритарный классификатор в среднем должен позволить получить оценку около 50% при перекрестной проверке с исключением одного примера, но, к вашему удивлению, он выдает оценку нуль. Можете ли вы объяснить причины, по которым получен такой результат?

18.8. При рекурсивном формировании деревьев решений иногда происходит так, что в листовом узле остается смешанное множество положительных и отрицательных примеров, даже несмотря на то, что были использованы все атрибуты.

Предположим, что количество положительных примеров равно p , а количество отрицательных примеров — n . а) Покажите, что решение, используемое в алгоритме Decision-Tree Learning, в котором выбирается мажоритарная классификация, минимизирует абсолютную ошибку на множестве примеров в листовом узле. б) Покажите, что $p/(p+n)$ минимизирует сумму квадратичных ошибок.

18.9. Предположим, что в обучающем алгоритме предпринимается попытка найти совместимую гипотезу, притом что фактически классификации примеров являются случайными. Имеется p булевых атрибутов, а примеры извлекаются с помощью равномерного распределения из множества 2^p возможных примеров. Рассчитайте количество примеров, которые требуется получить, прежде чем вероятность обнаружения противоречия в данных достигнет 0,5.

18.10. Предположим, что некоторый атрибут разбивает множество примеров на подмножества E_L и E_R и что каждое подмножество включает p_L положительных примеров и n_L отрицательных примеров. Покажите, что этот атрибут строго обеспечивает положительное приращение информации, если отношение $p_i / (p_i + n_i)$ не является одинаковым для всех i .

18.11. Модифицируйте алгоритм Decision-Tree Learning, для того чтобы в нем применялось 2-отсечение. Для ознакомления с подробными сведениями по этой теме вам может потребоваться обратиться к [1257].

18.12. (И) В стандартном алгоритме Decision-Tree Learning, описанном в данной главе, не учитываются такие случаи, в которых отдельные примеры имеют отсутствующие значения атрибутов. а) Прежде всего необходимо найти способ классификации таких примеров, если дано дерево решений, включающее проверки

по атрибутам, для которых могут отсутствовать значения. Предположим, что в некотором примере X отсутствует значение для атрибута A и что в дереве решений выполняется проверка атрибута A в узле, достигнутом примером X . Один из способов справиться с такой ситуацией состоит в том, чтобы предположить, что в этом примере заданы все возможные значения для данного атрибута, но взвесить каждое значение с учетом его частоты среди всех примеров, достигших этого узла в дереве решений. Алгоритм классификации должен проследовать по всем ветвям в каждом узле, для которого

отсутствует некоторое значение, и умножить полученные значения на веса вдоль каждого пути. Напишите модифицированный алгоритм классификации для деревьев решений, который действует по такому принципу. б) Теперь необходимо внести изменения в расчеты приращения информации таким образом, чтобы в любой конкретной коллекции примеров S в любом конкретном узле дерева в процессе его формирования в качестве значений любого из оставшихся атрибутов присваивались "предполагаемые" величины, соответствующие частотам этих значений в множестве S .

18.13. В данной главе было отмечено, что при использовании атрибутов со многими различными возможными значениями могут возникать проблемы, связанные с применением критерия приращения информации. Такие атрибуты, как правило, разбивают примеры на многочисленные небольшие классы или даже одноэлементные классы, из-за чего они начинают казаться в высшей степени релевантными в соответствии с критерием приращения информации.

Критерий коэффициента приращения позволяет выбирать атрибуты с учетом отношения между соответствующим им приростом и свойственным им информационным содержанием, т.е. количеством информации, содержащемся в ответе на вопрос: "Каково значение этого атрибута?" Поэтому при использовании критерия коэффициента приращения предпринимается попытка измерить, насколько полно этот атрибут предоставляет информацию о правильной классификации некоторого примера. Запишите математическое выражение для информационного содержания атрибута и реализуйте критерий коэффициента приращения в алгоритме Decision-Tree-Learning.

18.14. Рассмотрите алгоритм обучения ансамбля деревьев решений, в котором используется простое мажоритарное голосование среди m изучаемых гипотез.

Предположим, что каждая гипотеза имеет ошибку e и что ошибки, допущенные в каждой гипотезе, не зависят от ошибок в других гипотезах. Составьте формулу для ошибки в алгоритме обучения ансамбля в терминах m и e и проведите по ней вычисления для тех случаев, когда $m=5, 10$ и 20 , а $e=0.1, 0.2$ и 0.4 . Возможно ли, что

ошибка алгоритма обучения ансамбля будет хуже, чем е, если предположение о независимости будет исключено?

18.15. Это упражнение касается вопроса о выразительности списков решений (раздел 18.5). а) Покажите, что списки решений позволяют представить любую булеву функцию, если размер проверок не ограничен. б) Покажите, что если проверки могут включать самое большое k литералов каждая, то списки решений позволяют представить любую функцию, которая может быть представлена с помощью дерева решений с глубиной k .

Глава 19. Применение знаний в обучении

19 Применение В этой главе рассматривается задача обучения в тех условиях, когда уже кое-что известно.

Во всех подходах к обучению, описанных в предыдущих трех главах, основная идея состоит в том, что должна быть сформирована функция, входные и выходные данные которой соответствуют закономерностям, наблюдаемым в данных. В каждом подобном случае методы обучения можно рассматривать как поиск в пространстве гипотез для обнаружения подходящей функции, начиная только с очень простых предположений о форме этой функции, таких как "полином второго порядка" или "дерево решений", и руководствуясь такими критериями, как "чем проще, тем лучше". Применение такого подхода равносильно требованию, что перед началом изучения чего-либо нового необходимо в первую очередь забыть (почти) все, что вы знаете. А в данной главе рассматриваются методы обучения, позволяющие воспользоваться [^] априорными знаниями о мире. В большинстве случаев априорные знания представлены в виде общих логических теорий первого порядка, поэтому вначале в этой главе необходимо систематизировать сведения о представлении знаний и обучении.

19.1. Логическая формулировка задачи обучения поиска гипотезы, которая согласуется с наблюдаемыми примерами. В данной главе это определение уточняется и распространяется на тот случай, когда гипотеза представлена в виде множества логических высказываний. Описания примеров и определения классов также заданы в виде логических высказываний, а классификация нового примера может быть выполнена путем логического вывода классификационного высказывания из гипотезы и описания примера. Такой подход обеспечивает инкрементное формирование гипотез путем последовательного добавления каждый раз по одному предложению. Он также дает возможность использовать априорные знания, поскольку уже известные высказывания могут помочь при классификации новых примеров. На первый взгляд может показаться, что логическая формулировка

задачи обучения требует выполнения вначале большого объема дополнительной работы, но, как оказалось, она позволяет прояснить многие нерешенные проблемы обучения. Указанный подход дает возможность намного превзойти простые методы обучения, описанные в главе 18, поставив на службу обучению всю мощь логического вывода.

Примеры и гипотезы Вернемся к описанной в главе 18 задаче обучения с рестораном, в которой нужно было изучить правило принятия решения о том, при каких условиях следует ждать освобождения столика. В этой главе для описания примеров применялись атрибуты, такие как Alternate, Bar, Fri/ Sat и т.д. В логической формулировке задачи любой пример представляет собой объект, для описания которого используется логическое высказывание, а атрибуты становятся унарными предикатами. Примем общее обозначение X_i для i -го примера. В частности, первый пример, приведенный в табл.

18.1, может быть описан с помощью таких высказываний:

Alternate (X_i) л -iBar(X_i) л -.Fri / Sat (X_i) л Hungry (X_i) л ...

Обозначение D_i (X_i) будет использоваться для ссылки на описание X_i ? где D_i может представлять собой любое логическое выражение, принимающее один параметр.

Классификация объекта определяется примерно таким высказыванием:

WillWait(X_i)! Общее обозначение $Q(X_i)$ будет применяться, если пример является положительным, а $\neg Q(X_i)$ — если отрицательным. В таком случае полное обучающее множество представляет собой конъюнкцию всех описательных и классификационных высказываний.

Целью индуктивного обучения в логической постановке задачи является поиск эквивалентного логического выражения для целевого предиката Q , который может использоваться для правильной классификации примеров. Подобное выражение, которое мы будем называть $\wedge$ потенциальным определением целевого предиката, предлагается в каждой гипотезе. Используя S_i для обозначения потенциального определения, можно утверждать, что каждая гипотеза ϕ представляет собой высказывание в форме $\forall x Q(x) \leq S_i(x)$. В частности, дерево решений представляет собой утверждение, что целевой предикат принимает истинное значение по отношению к какому-то объекту тогда и только тогда, когда выполняются условия в одной из ветвей, ведущих к листовому узлу со значением true. Таким образом, на рис. 18.4 в графической форме выражено следующее логическое определение (которому мы присвоим обозначение γ для использования его в будущем):

$\forall r \text{ WillWait}(r) \iff \text{Patrons}\{r, \text{Some}\} \vee \text{Patrons}\{r, \text{Full}\} \wedge \text{Hungry}(r) \wedge \text{Type}(r, \text{French}) \vee \text{Patrons}(r, \text{Full}) \wedge \text{Hungry}(r) \wedge \text{Type}(r, \text{Thai}) \wedge \text{Fri/Sat}(r) \vee \text{Patrons}[r, \text{Full}] \wedge \text{Hungry}(r) \wedge \text{Type}(r, \text{Burger})$ A9.1) Каждая гипотеза предсказывает, что некоторое множество примеров

(а именно множество тех примеров, которые соответствуют ее потенциальному определению) будет представлять собой примеры целевого предиката. Такое множество называется

$\wedge$ расширением предиката. Это означает, что две гипотезы с разными расширениями являются логически несовместимыми друг с другом, поскольку они не согласуются в своих предсказаниях по меньшей мере в одном примере. А если две гипотезы имеют одно и то же расширение, они логически эквивалентны.

Пространство гипотез алгоритма обучения представляет собой множество всех гипотез $\{H_1, \dots, H_n\}$, для поддержки которых предназначен данный конкретный обучающий алгоритм. Например, алгоритм Decision-Tree-Learning способен поддерживать любую гипотезу в дереве решений, определенную в терминах заранее предусмотренных атрибутов, поэтому пространство гипотез этого алгоритма состоит из всех таких деревьев решений. Предполагается, что алгоритм обучения позволяет с полной уверенностью утверждать, что одна из гипотез является правильной; это означает, что данный алгоритм выражает определенную степень уверенности в истинности следующего высказывания:

$H_1 \vee H_2 \vee H_3 \vee \dots \vee H_n$ A9.2) По мере поступления новых примеров появляется возможность исключать гипотезы, не совместимые с этими примерами. Рассмотрим более внимательно затронутое здесь понятие совместимости. Очевидно, что если гипотеза H_i совместима со всем обучающим множеством, то она должна быть совместимой с каждым примером. А что повлекла бы за собой его несовместимость с одним из примером? Такая ситуация может проявиться в одном из двух описанных ниже вариантов. • Некоторый пример может оказаться $\wedge$ ложно отрицательным для данной гипотезы, если гипотеза утверждает, что он должен быть отрицательным, но фактически этот пример положителен. В частности, новый пример X_{13} , описанный выражением $\text{Patrons}\{X_{13}, \text{Full}\} \wedge \text{Wait}\{X_{13}, 0-10\} \wedge \neg \text{Hungry}(X_{13}) \wedge \dots \wedge \text{WillWait}(X_{13})$ был бы ложно отрицателен для гипотезы H_g , приведенной выше. Из гипотезы H_g и описания этого можем вывести и выражение $\text{WillWait}\{X_{13}\}$, которое как раз является утверждением данного примера, и выражение $\neg \text{WillWait}(X_{13})$, которое представляет собой предсказание самой гипотезы. Поэтому в данном случае гипотеза и пример логически несовместимы. • Пример может быть $\wedge$ ложно положительным для данной гипотезы, если в гипотезе утверждается, что он должен быть положительным, но фактически он является отрицательным¹.

Если некоторый пример является ложно положительным или ложно отрицательным применительно к некоторой гипотезе, то данный пример и данная гипотеза являются логически несовместимыми друг с другом. При условии, что рассматриваемый пример представляет собой правильное наблюдение факта, такая ситуация позволяет исключить данную гипотезу. С точки зрения логики соответствующая операция исключения гипотезы полностью аналогична операции

применения пра1 Термины "ложно положительный" и "ложно отрицательный" используются также в медицине для описания ошибочных результатов лабораторных исследований. Результат исследования является ложно положительным, если он указывает, что у пациента имеется диагностируемое заболевание, тогда как в действительности у пациента этого заболевания нет.

вила резолюции в логическом выводе (см. главу 9); в этой аналогии дизъюнкция гипотез соответствует выражению, а пример соответствует литералу, который взаимно уничтожается с одним из литералов выражения. Поэтому в принципе может быть обеспечено обучение на примерах обычной системы логического вывода путем удаления одной или нескольких гипотез. В частности, предположим, что пример оформлен в виде высказывания x_1 ? а пространство гипотез представляет собой высказывание $H_1 \vee H_2 \vee H_3 \vee H_4$. В таком случае, если высказывание H_1 несовместимо с выражениями $\neg H_2$ и $\neg H_3$, то система логического вывода может сформировать новое высказывание H_4 , соответствующее уточненному пространству гипотез.

Таким образом индуктивное обучение в логической постановке задачи можно охарактеризовать как процесс постепенного устранения гипотез, несовместимых с примерами, и сужения пространства возможных гипотез. Но поскольку пространство гипотез обычно является колоссальным (или даже бесконечным, в случае логики первого порядка), не рекомендуется даже пытаться создать систему обучения с использованием доказательства теорем на основе резолюции и полного перебора пространства гипотез.

Вместо этого в этой главе будут описаны два подхода, которые позволяют находить логически совместимые гипотезы с гораздо меньшими усилиями.

Поиск текущей наилучшей гипотезы В основе метода поиска ^текущей наилучшей гипотезы лежит подход, предусматривающий сопровождение единственной гипотезы и ее корректировку по мере поступления новых примеров в целях поддержки совместимости. Основной алгоритм этого метода был впервые описан Джоном Стюартом Миллом [1049], но вполне мог быть изобретен еще раньше.

Предположим, что в нашем распоряжении имеется определенная гипотеза, скажем H_1 , которая нас полностью устраивает. До тех пор пока каждый новый пример остается с ней совместимым, не нужно ничего делать. Наконец, вслед за ними поступает ложно отрицательный пример, x_{13} . Что теперь следует делать? На рис. 19.1, а гипотеза H_1 показана схематически в виде области; все, что находится внутри прямоугольника, входит в состав расширения H_1 . Примеры, которые фактически встретились до сих пор, показаны как "+" или "-", и рисунок наглядно демонстрирует, что гипотеза H_1 правильно классифицирует все примеры как положительные или отрицательные примеры предиката WillWait. Показанный на

рис. 19.1, б новый пример (обозначенный кружком) является ложно отрицательным — в гипотезе утверждается, что он должен быть отрицательным, но фактически этот пример положителен.

Расширение гипотезы должно быть увеличено с целью его включения. Такая операция называется $\wedge$ обобщением; одно из возможных обобщений показано на рис. 19.1, в. После этого на рис. 19.1, г показан ложно положительный пример — в гипотезе утверждается, что новый пример (обозначенный кружком) должен быть положительным, но фактически этот пример отрицателен. Расширение гипотезы должно быть уменьшено в целях исключения данного примера. Такая операция называется $\vee$ уточнением; на рис. 19.1, д показано одно из возможных уточнений данной гипотезы. Отношения "более общий чем" и "более конкретный чем" между гипотезами позволяют создать пространство гипотез с такой логической структурой, которая дает возможность осуществлять эффективный поиск.

; + \ + + +1 ш \ +1 1 + 1 а) б) в) г) д) Рис. 19.1. Операции уточнения и обобщения: совместимая гипотеза (а); ложно отрицательный пример (б); гипотеза обобщена (в); ложно положительный пример (г); гипотеза уточнена (д) Теперь можно определить алгоритм Current-Best-Learning, приведенный в листинге 19.1. Обратите внимание на то, что каждый раз при рассмотрении возможности обобщения или уточнения гипотезы необходимо проверять согласованность этих операций с другими примерами, поскольку произвольное увеличение/уменьшение расширения может привести к включению/исключению рассматривавшихся ранее отрицательных/положительных примеров.

Листинг 19.1. Алгоритм обучения по методу поиска текущей наилучшей гипотезы. Он осуществляет поиск совместимой гипотезы и выполняет возврат, если не удастся найти какое-либо совместимое уточнение/обобщение

```
function Current-Best-Learning(examples) returns гипотеза Я
  Я <— любая гипотеза, совместимая с первым
  примером в множестве примеров examples
  for each из оставшихся примеров в
  множестве examples do
    if пример е является ложно положительным применительно
    к Я then Я <— выбрать уточнение Я, совместимое с примерами
    examples
    else if
    пример е является ложно положительным применительно к Я then Я <—
    выбрать обобщение Я, совместимое с примерами examples
  if не удастся найти какое-либо
  совместимое уточнение/обобщение then
    неудачное завершение поиска
  return Я
```

Выше в данном разделе обобщение и уточнение были определены как операции, модифицирующие расширение гипотезы. Теперь необходимо точно определить, как эти операции могут быть реализованы в виде синтаксических операций, которые вносят изменения в потенциальное определение, связанное с гипотезой, чтобы эти операции можно было выполнять с помощью какой-то программы. Прежде чем приступить к решению этой задачи, необходимо отметить, что обобщение и уточнение также представляют собой логические связи между гипотезами. Если

гипотеза $n \pm$ с определением d является обобщением гипотезы $n2$ с определением $c2$, то должно иметь место следующее выражение:

$$\forall x C2(x) \Rightarrow C1(x)$$

Поэтому, чтобы сформировать обобщение гипотезы $n2$, необходимо просто найти определение $C1$? которое логически следует из $C2$. Такая задача решается довольно легко.

Например, если $C2(x)$ представляет собой высказывание $Al\ ternate(x) \wedge APatrons(x, Some)$, то одно из возможных обобщений задается в виде $d(x) = Patrons(x, Some)$.

Такая форма операции обобщения называется $\wedge$ удалением условий. Интуитивно ясно, что в результате такой операции формируется более слабое определение и поэтому гипотеза допускает использование более крупного множества положительных примеров.

Может быть также предусмотрен целый ряд других операций обобщения, в зависимости от языка, в котором выражаются эти операции. Аналогичным образом, уточнение гипотезы может осуществляться путем введения дополнительных условий в ее потенциальное определение или путем удаления дизъюнктов из какого-то дизъюнктивного определения. Рассмотрим, как могут быть выполнены подобные операции в задаче с рестораном, используя данные, приведенные в табл. 18.1. • Первый пример, $x17$ является положительным. Выражение $Alternate(X \wedge$ имеет истинное значение, поэтому допустим, что начальная гипотеза имеет такой вид:

$H1: \forall x WillWait(x) \Leftrightarrow Alternate(x)$ • Второй пример, $x2$, отрицателен. Гипотеза $H \pm$ предсказывает, что он должен быть положительным, поэтому данный пример — ложно положителен. Это означает, что необходимо уточнить гипотезу $n1$. Это можно сделать, введя дополнительное условие, позволяющее исключить пример $x2$. Один из возможных вариантов состоит в следующем:

$H2: \forall x WillWait(x) \Leftrightarrow Alternate(x) \wedge \neg Patrons(x, Some)$ • Третий пример, $x3$, является положительным. Гипотеза $n2$ предсказывает, что он должен быть отрицательным, поэтому данный пример — ложно отрицательный. Это означает, что необходимо обобщить гипотезу $H2$. Удалим условие $Al\ ternate$ и получим следующее:

$H3: \forall x WillWait(x) \Leftrightarrow Patrons(x, Some)$ • Четвертый пример, $x4$, также положителен. Гипотеза $H3$ предсказывает, что он должен быть отрицательным, поэтому данный пример — ложно отрицательный. Это означает, что необходимо обобщить гипотезу $y3$. Условие $Patrons$ удалить нельзя, поскольку такая операция приведет к созданию всеохватывающей гипотезы, которая будет несовместимой с примером $x2$. Одна из возможностей состоит в добавлении дизъюнкта:

H4: $Vx \text{ WillWait}(x) \Leftrightarrow \text{Patrons}(x, \text{Some}) \vee \{\text{Patrons}\{x, \text{Full}\} \wedge \text{Fri/Sat}(x)\}$) И так, гипотеза уже начинает выглядеть как приемлемая. Очевидно, что есть и другие варианты, совместимые с первыми четырьмя примерами; ниже приведены два из них.

H4*: $Vx \text{ WillWait}(x) \Leftrightarrow \neg \text{WaitEstimate}(x, 30-60)$) H4'': $Vx \text{ WillWait}(x) \Leftrightarrow \text{Patrons}(x, \text{Some}) \wedge \text{Patrons}(x, \text{Full}) \wedge \text{WaitEstimate}(x, 10-30)$)

Алгоритм Current-Best-Learning описан здесь в недетерминированной форме, поскольку в любой момент может существовать возможность применить несколько вариантов уточнения или обобщения. Выбранный вариант не всегда обеспечивает получение простейшей гипотезы и может привести также к безвыходной ситуации, в которой ни одна простая модификация гипотезы не будет совместимой со всеми данными. В подобных случаях программа должна выполнить возврат к предыдущей точке выбора.

Алгоритм Current-Best-Learning и его варианты использовались во многих системах машинного обучения, начиная с программы "обучения распознаванию арок"

Патрика Уинстона [1602]. Однако при большом количестве экземпляров примеров и большом пространстве гипотез возникают некоторые описанные ниже трудности.

1. В алгоритме снова и снова проводится проверка всех предыдущих экземпляров при каждой модификации, а это требует больших затрат.
2. Процесс поиска может быть связан с очень интенсивным перебором с возвратами. Как было показано в главе 18, размеры пространства гипотез могут определяться двойной экспоненциальной зависимостью.

Поиск на основе оценки наименьшего вклада Перебор с возвратами возникает из-за того, что в подходе, предусматривающем поиск текущей наилучшей гипотезы, приходится выбирать конкретную гипотезу как наилучшее текущее предположение, даже несмотря на то, что еще отсутствует достаточный объем данных, позволяющий убедиться в правильности этого выбора.

Вместо этого можно было бы просто "держать под рукой" те и только те гипотезы, которые являются совместимыми со всеми поступившими до сих пор данными. В таком случае каждый новый экземпляр данных либо не оказывает никакого влияния на состав гипотез, либо позволяет избавиться от некоторых гипотез. Напомним, что первоначальное пространство гипотез может рассматриваться как следующее дизъюнктивное высказывание:

$H_1 \vee H_2 \vee H_3 \vee \dots \vee H_n$ По мере обнаружения того, что различные гипотезы являются несовместимыми с примерами, эта дизъюнкция сужается и остаются только те гипотезы, которые еще не исключены. Это означает, что при условии, что первоначальное пространство гипотез действительно содержит правильный ответ, этот правильный ответ должен также содержаться в сокращенной дизъюнкции,

поскольку были удалены только неправильные гипотезы. Множество оставшихся гипотез называется $\wedge$ пространством версий, а соответствующий алгоритм обучения (приведенный в листинге 19.2) называется алгоритмом обучения в пространстве версий (а также алгоритмом $\wedge$ удаления потенциальных гипотез).

Листинг 19.2. Алгоритм обучения в пространстве версий. Он находит подмножество гипотез V , совместимое с примерами `examples` function `Version-Space-Learning(examples)` returns пространство версий `local variables: V`, пространство версий - множество всех гипотез $V < \text{множество всех гипотез}$

for each из примеров e в множестве примеров `examples` do if множество V не пусто then $V \leftarrow \text{Version-Space-Update}(V, e)$ return V function `Version-Space-Update(V, e)` returns обновленное пространство версий $V \leftarrow \{h \in V: h \text{ совместима с } e\}$ Одним важным свойством этого алгоритма является то, что он инкрементный, — при его использовании никогда не приходится возвращаться и повторно исследовать старые примеры, поскольку в любом случае гарантируется, что оставшиеся гипотезы остаются совместимыми с ними. Этот алгоритм также представляет собой алгоритм поиска с наименьшим вкладом, поскольку в нем не используются произвольные варианты выбора (сравните его с алгоритмом планирования с частичным упорядочением, который приведен в главе 11). Но при использовании этого алгоритма возникает одна очевидная проблема. Как уже было сказано, пространство гипотез часто имеет огромные размеры; как же в таком случае можно записать соответствующую ему огромную дизъюнкцию?

Для анализа этой ситуации можно применить следующую очень полезную простую аналогию. Как представить все действительные числа в интервале между 1 и 2?

В конце концов, ведь количество этих чисел бесконечно велико! Ответ на этот вопрос состоит в том, что должно использоваться интервальное представление, в котором задаются границы этого множества, $[1, 2]$. Это представление применимо в связи с тем, что во множестве действительных чисел задано упорядочение.

В пространстве версий также существует отношение упорядочения, а именно отношение обобщения/уточнения. Но это — отношение частичного упорядочения, поэтому каждая граница пространства представляет собой не точку, а скорее множество гипотез, называемое $\wedge$ граничным множеством. Превосходной особенностью подхода, основанного на понятии граничного множества, является то, что он позволяет представить все пространство версий с использованием только двух граничных множеств — наиболее общего граничного множества ($\wedge G$ -множества, где G — сокращение от *general*) и наиболее конкретного граничного множества ($\wedge S$ -множества, где S — сокращение от *specific*). При этом гарантируется совместимость с примерами всех версий, которые находятся в пределах между этими двумя множествами. Прежде чем перейти к доказательству этого

утверждения, подытожим сказанное выше в настоящей главе. • Текущее пространство версий представляет собой множество гипотез, совместимых со всеми примерами, встретившимися до сих пор. Это пространство представлено с помощью S_i G -множества, каждое из которых представляет собой множество гипотез. • Каждый элемент S -множества является совместимым со всеми полученными до сих пор результатами наблюдений, и нет ни одной совместимой гипотезы, которая была бы более конкретной, чем этот элемент. • Каждый элемент G -множества является совместимым со всеми полученными до сих пор результатами наблюдений, и нет ни одной совместимой гипотезы, которая была бы более общей, чем этот элемент.

Необходимо, чтобы первоначальное пространство версий (существовавшее до того, как встретились какие-либо примеры) содержало все возможные гипотезы.

Для этого достаточно задать G -множество таким образом, чтобы оно содержало гипотезу True (гипотезу, которая включает все возможные случаи), а S -множество — так, чтобы оно содержало гипотезу False (гипотезу, расширение которой пусто).

На рис. 19.2 показана общая структура представления пространства версий с помощью граничных множеств. Для того чтобы показать, что это представление является достаточным, необходимо определить два описанных ниже свойства.

Все гипотезы из этой области несовместимы с примерами $G \setminus G_i \subset W_i$!

Более общие А Более конкретные Все гипотезы из этой области несовместимы с примерами Рис. 19.2. Схематическая иллюстрация того, что пространство версий содержит все гипотезы, совместимые с примерами Каждая совместимая гипотеза (отличная от гипотез граничных множеств) является более конкретной, чем некоторый элемент G -множества, и более общей, чем некоторый элемент S -множества (это означает, что за рамками пространства версий не остается ни одной "забытой гипотезы"). Такое свойство следует непосредственно из определений S_i G -множеств. Если бы существовала какая-то не охваченная гипотеза h , то она была бы не более конкретной, чем любой элемент G -множества, поскольку в таком случае она принадлежала бы к G -множеству, и не более общей, любой элемент S -множества, поскольку в таком случае она принадлежала бы к S -множеству.

Любая гипотеза, более конкретная, чем некоторый элемент G -множества, и более общая, чем некоторый элемент S -множества, является совместимой гипотезой. (Это означает, что между границами нет "пустот", в которых находились бы гипотезы, не охваченные отношениями обобщения/уточнения.) Любая гипотеза h , лежащая между S_i G -множествами, должна отвергать все отрицательные примеры, отвергнутые каждым элементом G -множества (поскольку она является более конкретной), и должна принимать все поло-

жительные примеры, принятые любым элементом S -множества (поскольку она является более общей). Таким образом, гипотеза h должна согласовываться со всеми примерами и поэтому не может быть несовместимой.

Соответствующая ситуация показана на рис. 19.3 — не существует каких-либо известных примеров вне S -множества, но внутри G -множества, поэтому любая гипотеза в промежутке между ними должна быть совместимой. $\sim G, + + Is + + + SI + + + + 1$ Рис. 19.3. Расширения G_i S -множества, показанные в виде отдельных элементов. Между двумя множествами границ не находится ни один известный пример. Таким образом, было показано, что если будет обеспечено правильное сопровождение S_i G -множества в соответствии с их определениями, то эти множества обеспечат удовлетворительное представление пространства версий. Остается единственная проблема — как обновлять S_i G -множество с учетом нового примера (выполнение этой операции возложено на функцию Version-Space-Update). На первый взгляд эта задача может показаться довольно затруднительной, но на основе определений множеств и с помощью рис. 19.2 не слишком сложно составить необходимый для этого алгоритм.

Необходимо прежде всего обеспечить использование элементов S_i и d из S_i G -множества соответственно. Применительно к каждому из таких элементов каждый новый экземпляр примера может быть ложно положительным или ложно отрицательным. Возникающие при этом варианты описаны ниже.

1. Ложно положительный пример для S^+ . Появление такого примера означает, что гипотеза S_i является слишком общей, но (по определению) совместимое уточнение для гипотезы S_i отсутствует, поэтому ее необходимо исключить из S -множества.

2. Ложно отрицательный пример для S_i . Появление такого примера означает, что гипотеза S_i является слишком конкретной, поэтому она заменяется всеми своими непосредственными обобщениями, при соблюдении условия, что они остаются более конкретными, чем любой элемент G -множества.

3. Ложно положительный пример для G^+ . Появление такого примера означает, что гипотеза G_i является слишком общей, поэтому она заменяется всеми

своими непосредственными уточнениями, при соблюдении условия, что они остаются более общими, чем любой элемент S -множества.

4. Ложно отрицательный пример для G^+ . Появление такого примера означает, что гипотеза G_i является слишком конкретной, но (по определению) совместимое обобщение для гипотезы G_i отсутствует, поэтому ее необходимо исключить из G -множества.

Указанные выше операции продолжают применительно к каждому новому экземпляру примера до тех пор, пока не произойдет одно из перечисленных ниже событий.

1. В пространстве версий останется одно и только одно определение некоторой концепции, и в этом случае оно будет возвращено как уникальная гипотеза.
2. Пространство версий свернется (либо S-, либо G-множество станет пустым), и это будет означать, что для данного обучающего множества не существуют совместимые гипотезы. Такая ситуация аналогична неудачному завершению поиска, которое возникает в простом варианте алгоритма формирования дерева решений.
3. Примеры закончатся, а в пространстве версий останется несколько гипотез.

Это означает, что пространство версий представляет собой дизъюнкцию гипотез. Если при поступлении любого нового примера все дизъюнкты этой дизъюнкции согласуются, то можно возратить определяемую ими классификацию данного примера. Если же они не согласуются, один из вариантов состоит в использовании мажоритарного голосования.

Рекомендуем читателю в качестве упражнения попытаться применить алгоритм Version-Space-Learning к данным о ресторане.

Описанный здесь подход к использованию пространства версий характеризуется описанными ниже принципиальными недостатками. • Если проблемная область характеризуется наличием шума или недостаточного количества атрибутов для точной классификации, то пространство версий всегда свертывается. • Если в пространстве гипотез допускается наличие неограниченного количества дизъюнкций, то S-множество всегда будет содержать единственную наиболее конкретную гипотезу, а именно дизъюнкцию описаний положительных примеров, встретившихся до сих пор. Аналогичным образом, G-множество будет содержать просто отрицание дизъюнкции описаний отрицательных примеров. • Для некоторых пространств гипотез количество элементов в S или в G-множестве может увеличиваться в экспоненциальной зависимости от количества атрибутов, что не позволяет решить задачу обучения, даже несмотря на то, что для этих пространств гипотез существуют эффективные алгоритмы обучения.

Полностью эффективное решение проблемы шума не найдено до сих пор. Проблемы дизъюнкции можно решить, допуская использование ограниченных форм дизъюнкции или включая $\wedge$ иерархию обобщения более общих предикатов. Например, вместо применения дизъюнкции $\text{WaitEstimate}(x, 30-60) \vee \text{WaitEstimate}(x, >60)$ можно

вести в действие единственный литерал $\text{LongWait}(x)$. Для реализации такого подхода можно легко дополнить множество операций обобщения и уточнения.

Рассматриваемый в этом разделе чистый алгоритм сопровождения пространства версий был впервые применен в системе Meta-Dendral, которая была предназначена для изучения правил предсказания того, как молекулы разделяются на фрагменты в массовом спектрометре [202]. Система Meta-Dendral показала свою способность вырабатывать правила, которые оказались достаточно новаторскими, чтобы гарантировать их публикацию в журнале аналитической химии; это были первые реальные научные знания, полученные с помощью компьютерной программы. Такой алгоритм использовался также в изящной системе Lex [1066], которая проявила способность к обучению методам решения задач символического интегрирования, анализируя свои собственные успехи и неудачи. Хотя методы сопровождения пространства версий, по-видимому, не могут найти практического применения в большинстве реальных задач обучения, в основном из-за проблемы шума, они позволяют многое понять в отношении того, какова логическая структура пространства гипотез.

19.2. Применение знаний в обучении В предыдущем разделе описывалась простейшая постановка задачи индуктивного обучения. А в этом разделе речь пойдет о логических связях между гипотезами, описаниями примеров и классификациями, что позволяет понять роль априорных знаний. Допустим, что Descriptions обозначает конъюнкцию всех описаний примеров в обучающем множестве, а Classifications — конъюнкцию всех классификаций примеров. В таком случае гипотеза, которая позволяет "объяснить результаты наблюдений", должна удовлетворять следующему свойству (напомним, что $1=$ означает "влечет за собой логически"):

Hypothesis л Descriptions И Classifications A9.3) Мы будем называть связь такого рода $\wedge$ ограничением логического следствия, в котором Hypothesis представляет собой "неизвестное". Чисто индуктивное обучение сводится к разрешению этого ограничения, притом что гипотеза Hypothesis извлекается из некоторого заранее определенного пространства гипотез. Например, если дерево решений рассматривается как логическая формула (см. уравнение 19.1), то дерево решений, совместимое со всеми примерами, будет удовлетворять уравнению 19.3. Если же на логическую форму гипотезы не налагаются никакие ограничения, то, разумеется, условие Hypothesis=Classifications также удовлетворяет этому ограничению. Согласно принципу бритвы Оккама, следует отдавать предпочтение небольшим, совместимым гипотезам, поэтому мы должны попытаться найти лучшее решение, чем просто запоминание примеров.

Такой простой подход к индуктивному обучению, в котором не предусматривалось использование знаний, господствовал в искусственном интеллекте до начала 1980-х годов. А современный подход состоит в том, что должны проектироваться агенты, которые с[^]= уже кое-что знают и пытаются освоить в процессе обучения

некоторую дополнительную информацию. На первый взгляд открытие, приведшее к такой смене подходов, не кажется чрезвычайно глубоким, но оно стало причиной весьма существенного изменения способа проектирования агентов. Кроме того, это открытие может иметь определенное отношение к рассматриваемым в данной книге теориям о том, как развивается сама наука. Общая идея этого открытия показана на рис. 19.4.

Наблюдения Предсказания Рис. 19.4. Схема, которая показывает, как в процессе обучения с накоплением знаний используется и со временем обогащается запас фоновых знаний Если должен быть создан автономный обучающийся агент, который использует фоновые знания, то в этом агенте необходимо предусмотреть некоторый метод получения в первую очередь фоновых знаний, для того, чтобы их можно было использовать в новых эпизодах обучения. Применяемый при этом метод сам должен представлять собой процесс обучения. Поэтому история существования агента должна быть таковой, чтобы ее можно было охарактеризовать как поступательное, или инкрементное, развитие. Разумеется, что агент может начать свое существование без какого-либо запаса знаний, осуществляя индуктивные выводы в условиях отсутствия знаний, как и добрая старая программа чисто индуктивного вывода. Но как только агент съест плод с Древа познания, он не будет больше обязан заниматься подобными примитивными рассуждениями, а должен использовать свои фоновые знания для все более эффективного обучения. Таким образом, вопрос заключается в том, как фактически следует использовать знания в обучении.

Некоторые простые примеры Рассмотрим некоторые общеизвестные примеры обучения на основе фоновых знаний. При этом необходимо учитывать, что во многих формах поведения, требующих логического вывода релевантных гипотез в связи с поступлением новых результатов наблюдения, не применимы даже самые элементарные принципы чистой индукции. • Иногда возникают ситуации, в которых люди сразу же приходят к общим заключениям после одного лишь наблюдения. Перу Гэри Ларсона принадлежит юмористический рисунок, на котором вооруженный очками пещерный человек Зог поджаривает на огне ящерицу, насаженную на заостренную палочку. За ним наблюдает восхищенная толпа его менее интеллектуальных соплеменников, которые до сих пор всегда разогревали свою пищу над огнем, держа ее голыми руками. Этот наглядный пример является достаточным для того, чтобы ознакомить зрителей с общим принципом безболезненного приготовления пищи. • В качестве еще одного примера рассмотрим случай, в котором путешественник, прибывший в Бразилию, встречает первого в своей жизни бразильца. Ус-

лышав от него речь на португальском языке, путешественник сразу же приходит к выводу, что бразильцы говорят на португальском, но узнав, что его собеседника

зовут Фернандо, он не делает вывод, что все бразильцы носят имя Фернандо. Аналогичные примеры можно найти в любой научной области.

Например, когда начинающий студент-физик измеряет плотность и проводимость медного образца при определенной температуре, он без колебаний обобщает эти данные на все предметы из меди. Но измерив затем массу этого образца, студент даже не рассматривает такую гипотезу, что все предметы из меди имеют такую же массу. С другой стороны, он может вполне резонно сделать такое обобщение относительно всех монет одинакового достоинства. • Наконец, рассмотрим случай, в котором незнакомый с фармакологией, но глубоко освоивший специальность диагностики студент-медик присутствует на консультации, которую дает пациенту опытный терапевт. После ряда вопросов и ответов специалист сообщает пациенту, чтобы он прошел курс лечения каким-то конкретным антибиотиком. На основании увиденного студент-медик выводит общее правило, что данный конкретный антибиотик является эффективным средством лечения при данном конкретном типе инфекции.

Все эти случаи относятся к такому типу, в котором $s^{\wedge}$ = использование фоновых знаний обеспечивает гораздо более быстрое обучение по сравнению с тем, чего можно ожидать при использовании программы чисто индуктивного обучения.

Некоторые общие схемы В каждом из предыдущих примеров имеется возможность обратиться к априорным знаниям, чтобы попытаться обосновать выбранный способ обобщения.

В данном разделе рассмотрим, какого рода ограничения логического следствия применяются в каждом из этих случаев. В таких ограничениях, кроме гипотезы Hypothesis, описаний наблюдаемых примеров Descriptions и классификации Classifications, применяются фоновые знания Background.

В случае поджаривания ящерицы пещерный человек проводит обобщение, объясняя своим соплеменникам успех, достигнутый с помощью заостренной палочки, — он держит ящерицу над огнем и вместе с тем не приближает руки к огню.

Из этого объяснения его соплеменники могут вывести общее правило, что для поджаривания небольших мясистых съедобных объектов можно использовать любой длинный, твердый, заостренный предмет. Процесс обобщения такого рода получил название $\wedge$ обучения на основе объяснения, или сокращенно EBL (Explanation-Based Learning). Обратите внимание на то, что общее правило следует логически из фоновых знаний, которыми обладают рассматриваемые в данном примере пещерные люди. Поэтому ограничения логического следствия, которые удовлетворяются в процессе EBL, являются таковыми:

Hypothesis л Descriptions N Classifications Background \= Hypothesis Поскольку в процессе EBL используется уравнение 19.3, на первых порах он рассматривался как наилучший способ обучения на примерах. Но в связи с тем, что для обучения на основе объяснения требуется, чтобы фоновые знания были достаточными для объяснения гипотезы Hypothesis, которая в свою очередь объясняет ре-

зультаты наблюдений, $c^{\wedge}$ = в действительности агент из полученного экземпляра примера не извлекает никаких принципиально новых знаний. Дело в том, что агент мог бы просто вывести логическим путем предъявленный ему пример из того, что он уже знает, хотя для этого, возможно, потребовался бы неоправданно большой объем вычислений. В настоящее время обучение на основе объяснения рассматривается как метод преобразования теорий, складывающихся из основных принципов, в полезные знания специального назначения. Алгоритмы обучения на основе объяснения рассматриваются в разделе 19.3.

В следующем примере, когда речь шла о путешественнике в Бразилию, ситуация совсем иная, поскольку не каждый путешественник знает о том, почему встретившийся ему бразилец говорит именно на португальском языке, если этот путешественник никогда не слышал о соответствующих буллах, изданных папой Римским. Более того, к аналогичному обобщению мог бы прийти путешественник, полностью незнакомый с историей португальских колоний. Соответствующие априорные знания состоят в том, что в каждой конкретной стране большинство людей, как правило, говорят на одном и тот же языке; с другой стороны, не предполагается, что имя Фернандо носят все бразильцы, поскольку указанная закономерность, касающаяся общего языка для всей страны, не распространяется на имена. Аналогичным образом, начинающий студент-физик вряд ли мог бы объяснить, почему измеренные им значения проводимости и плотности меди являются именно таковыми. Но он знает, что проводимость объекта зависит и от материала, из которого состоит объект, и от его температуры.

В данном случае априорные знания Background касаются $\wedge$ релевантности множества характеристик по отношению к целевому предикату. Эти знания, наряду с результатами наблюдений, позволяют агенту вывести новое, общее правило, которое объясняет результаты наблюдений, следующим образом:

Hypothesis л Descriptions \= Classifications Background л Descriptions л Classifications \= Hypothesis A9.4) Мы будем называть обобщение такого рода $\wedge$ обучением с учетом релевантности, или сокращенно RBL (Relevance-Based Learning), хотя такое название указанного метода обучения не является общепринятым. Обратите внимание на то, что в обучении с учетом релевантности используются результаты наблюдений, но не вырабатываются гипотезы, выходящие за пределы логического содержания фоновых знаний и наблюдений. Оно представляет собой дедуктивную

форму обучения и поэтому не может само по себе рассматриваться как создание новых знаний с пустого места.

А в том случае, когда студент-медик наблюдает за работой опытного врача, предполагается, что априорных знаний студента достаточно, чтобы прийти к заключению на основании симптомов о том, каково заболевание D этого пациента. Но этого не достаточно, чтобы объяснить тот факт, что врач прописал конкретное лекарство M. Студент должен выдвинуть гипотезу, касающуюся еще одного правила, а именно, что лекарство M обычно является эффективным средством против заболевания D.

С учетом этого правила и априорных знаний студент может теперь объяснить, почему в данном конкретном случае опытный врач прописал лекарство M. Этот пример можно обобщить так: указать, что он основан на приведенном ниже ограничении логического следствия.

Background л Hypothesis л Descriptions N Classifications A9.5)

Это означает, что $s^{\wedge}$ для объяснения примеров объединяются фоновые знания и новая гипотеза. Как и при чисто индуктивном обучении, алгоритм обучения должен выдвигать гипотезы, которые являются как можно более простыми и совместимыми с данным ограничением. Алгоритмы, удовлетворяющие ограничению 19.5, называются алгоритмами $s^{\wedge}$ индуктивного обучения на основе знаний, или сокращенно KBIL (Knowledge-Based Inductive Learning).

Алгоритмы KBIL, которые подробно описаны в разделе 19.5, в основном исследовались в области $s^{\wedge}$ индуктивного логического программирования, или сокращенно ILP (Inductive Logic Programming). В системах ILP априорные знания выполняют две описанные ниже ключевые функции при решении задачи уменьшения сложности обучения.

1. Поскольку любая сформированная гипотеза должна быть совместимой с априорными знаниями, а также с новыми наблюдениями, эффективный размер пространства гипотез сокращается таким образом, чтобы в него были включены только теории, согласованные с тем, что уже известно.
2. Для любого конкретного множества наблюдений размер гипотезы, требуемый для формирования объяснения полученных результатов наблюдений, может быть намного сокращен, поскольку доступны априорные знания, на которые могут опираться новые правила при формировании объяснений результатов наблюдений. А чем короче гипотеза, тем проще ее сформулировать.

Системы ILP не только позволяют использовать в процессе индукции априорные знания, но и дают возможность формулировать гипотезы на языке общей логики первого порядка, а не на ограниченном языке, основанном на атрибутах, который

рассматривается в главе 18. Это означает, что подобные системы могут осуществлять обучение в таких вариантах среды, которые остаются недоступными для понимания при использовании более простых систем.

19.3. Обучение на основе объяснения Как было описано во введении в данной главе, обучение на основе объяснения представляет собой метод извлечения общих правил из отдельных результатов наблюдений.

В качестве примера рассмотрим задачу дифференцирования и упрощения алгебраических выражений (упр. 9.15). После выполнения дифференцирования по X такого выражения, как X^2 , будет получено выражение $2X$. (Обратите внимание на то, что мы используем прописную букву для обозначения арифметического неизвестного X , чтобы отличить его от логической переменной x .) В системе формирования логических рассуждений такая цель может быть выражена как $\text{Ask}(\text{Derivative}(X^2, X)=d, KB)$ с предполагаемым решением $d=2X$.

Любой, кто знает дифференциальное исчисление, может найти данное решение, лишь "ознакомившись с условиями задачи", благодаря приобретенным ранее навыкам решения подобных задач. А перед студентом, столкнувшимся с подобными задачами впервые, или перед программой, не накопившей опыта в решении этих задач, возникают гораздо более существенные затруднения. Применение стандартных правил дифференцирования в конечном итоге приводит к получению выражения

$1 \times Vx(x^2 - 1)$), а это выражение в дальнейшем упрощается до $2X$. В логической программной реализации, подготовленной авторами данной книги, для решения такой задачи потребовалось 136 шагов доказательства, причем 99 из этих шагов относились к тупиковым ветвям доказательства. Тот, кто накопил такой отрицательный опыт, вполне естественно, стремится к тому, чтобы та же программа решала аналогичную задачу намного более быстро, встретившись с ней в следующий раз.

В компьютерных науках для ускорения работы программ уже в течение долгого времени используется метод $\wedge$ запоминания (memoization), который предусматривает сохранение в памяти результатов вычислений. Фундаментальная идея, лежащая в основе запоминающих функций, состоит в том, что должна накапливаться база данных, состоящая из входных/выходных пар; после вызова функции в первую очередь выполняется проверка базы данных для определения того, можно ли обойтись без решения рассматриваемой задачи с самого начала. Обучение на основе объяснения представляет собой значительный шаг вперед, поскольку в нем предусматривается создание общих правил, охватывающих целый класс вариантов. В случае дифференцирования метод запоминания позволяет записать в память информацию о том, что производная от X^2 по X равна $2X$, но

потребуется от агента, чтобы он вычислял производную Z_2 по z с самого начала. Однако было бы желательно иметь возможность извлечь общее правило², что для любого арифметического неизвестного i производная от i^2 по i равна $2i$. В терминах логики такая мысль может быть выражена с помощью следующего правила:

$\text{ArithmeticUnknown}(u) \Rightarrow \text{Derivative}(i^2, i) = 2i$ Если база знаний содержит такое правило, то для любого нового случая, который является экземпляром этого правила, может быть немедленно найдено решение.

Безусловно, это лишь тривиальный пример очень общего феномена. После того как достигнуто понимание какой-то идеи, она может быть обобщена и повторно использована в других обстоятельствах. Освоенная идея становится "очевидным" этапом мышления, после чего может использоваться в качестве строительного блока при решении еще более сложных задач. Альфреду Норту Уайтхеду [1583], в соавторстве с которым Бертран Рассел написал свою знаменитую книгу *Principia Mathematica*, принадлежат такие слова: $\leq$ "Цивилизация развивается, увеличивая количество важных операций, которые ее представители могут выполнять, не задумываясь". По-видимому, сам Уайтхед применял обучение на основе объяснения, чтобы истолковать полученные им результаты, по аналогии с тем, как Зог объяснял соплеменникам свое открытие. Если вы поняли основную идею примера с дифференцированием, то ваш мозг уже занимается извлечением из него основных принципов обучения на основе объяснения. Обратите внимание на то, что вам не требовалось заранее изобретать принципы обучения на основе объяснения, прежде чем вы ознакомились с этим примером. Как и пещерным людям, наблюдающим за манипуляциями Зога, вам (и авторам данной книги) нужен был пример для того, чтобы прийти к пониманию основных принципов. Именно поэтому часто гораздо проще сначала показать, почему так хороша какая-то идея, чем сразу же ее изложить.

² Безусловно, может быть также сформировано общее правило для u^n , но текущего примера достаточно, чтобы проиллюстрировать общий принцип.

Извлечение общих правил из примеров Обучение на основе объяснения базируется на том важном принципе, что вначале необходимо составить объяснение данного наблюдения с использованием априорных знаний, а затем сформировать определение класса случаев, для которого может использоваться такая же структура объяснения. Подобное определение может лечь в основу правила, охватывающего все случаи данного класса. Само "объяснение" может представлять собой логическое доказательство, но в более общем случае объяснением может служить любой процесс формирования рассуждений или решения задачи, этапы которого полностью определены. Ключом к успешному формированию объяснения является

определение необходимых условий для того, чтобы такие же этапы процесса можно было применить к другому случаю.

Мы будем использовать в качестве рассматриваемой здесь системы формирования логических рассуждений простую систему доказательства теорем по методу обратного логического вывода, которая описана в главе 9. Дерево доказательства для случая $\text{Derivative } \{X^2, X\} = 2X$ слишком велико для того, чтобы его можно было использовать в качестве примера, поэтому для иллюстрации применяемого метода обобщения рассмотрим более простую задачу. Предположим, что задача состоит в том, чтобы упростить выражение $1x @ + X$. База знаний включает следующие правила:

Rewrite (и, v) л Simplify {v, w} => Simplify(u,w) Primitive (и) => Simplify {и, и)
 ArithmeticUnknown(u) => Primitive(u) Number (и) => Primitive {и) Rewrite(lxu,u)
 Rewrite@+u,u) Доказательство того, что ответом является X, приведено на рис. 19.5, сверху. В методе обучения на основе объяснения фактически одновременно формируются два дерева доказательства. Во втором дереве доказательства используется цель, описанная с помощью переменных, в которой константы первоначальной цели заменены переменными. По мере развития первоначального доказательства так же поэтапно развивается доказательство с помощью переменных, в котором применяются точно такие же приложения правил. Использование некоторых правил может вызвать конкретизацию определенных переменных. Например, для того чтобы можно было применить правило Rewrite(lxu, и), необходимо вначале связать с 1 переменную x в подцели Rewrite (xx{y+z} , v). Аналогичным образом, переменная y должна быть связана с 0 в подцели Rewrite(y+z,v*), для того чтобы можно было использовать правило Rewrite(0 + uf и). После получения обобщенного дерева доказательства остается только взять листовые узлы (с необходимыми связываниями) и сформулировать общее правило для целевого предиката, следующим образом:

Rewrite A x @ + z, 0 + z) л Rewrite{ 0 + z, z) л ArithmeticUnknown(z) => Simplify{l x{0 + z) , z) Обратите внимание на то, что первые два условия в левой части правила являются истинными независимо от значения z. Поэтому их можно удалить из данного правила, что приводит к получению такого соотношения:

ArithmeticUnknown(z) => Simplify(lx@ + z) , z)

1 Simplify<{z,w} 1 {w/z} J Primitive{z} \ Arithmetic Unknownjz) \ Yes,{y/0,V/z} Yes, { } Рис. 19.5. Деревья доказательства для рассматриваемой задачи упрощения. Первое дерево демонстрирует доказательство для первоначального экземпляра задачи, на основании которого можно вывести следующее:

ArithmeticUnknown (z) => Simplify (lx@+z), z) Второе дерево демонстрирует доказательство для экземпляра задачи, в котором все константы заменены

переменными, что позволяет вывести целый ряд других правил. Вообще говоря, из окончательного правила можно удалять такие условия, которые не налагают ограничений на переменные в правой части правила, поскольку при этом результирующее правило остается истинным и становится более эффективным. Обратите внимание на то, что условие $\text{ArithmeticUnknown}(z)$ удалять нельзя, поскольку не все возможные значения z являются арифметическими неизвестными. Для значений, отличных от арифметических неизвестных, могут потребоваться другие формы упрощения; например, если переменная z равна $2x3$, то правильным упрощением выражения $1x @ + Vx3$)) должно быть 6, а не $2x3$.

Подводя итог, можно отметить, что основной процесс обучения на основе объяснения действует, как описано ниже.

1. Используя рассматриваемый пример, составить доказательство того, что целевой предикат является применимым к данному примеру, с использованием доступных фоновых знаний.
2. Одновременно с этим сформировать обобщенное дерево доказательства для цели, описанной с помощью переменных, в котором используются такие же этапы логического вывода, как и в первоначальном дереве доказательства.
3. Сформулировать новое правило, левая часть которого состоит из листовых узлов дерева доказательства, а правая часть представляет собой цель, описанную с помощью переменных (после применения соответствующих связываний из обобщенного доказательства).
4. Удалить все условия, которые являются истинными независимо от значений переменных в цели.

Повышение эффективности правила. Обобщенное дерево доказательства, показанное на рис. 19.5, фактически приводит к получению больше чем одного обобщенного правила. Например, если будет прекращено наращивание правой ветви в дереве доказательства после того, как одна ветвь достигнет этапа Primitive , т.е. если будет выполнено отсечение этой ветви, то получим следующее правило:

$\text{Primitive}(z) \Rightarrow \text{Simplify}(lx@ + z), z$ Это правило является таким же действительным, как и правило, в котором используется предикат ArithmeticUnknown , но более общим, поскольку оно охватывает также случаи, в которых z является числом. Из дерева доказательства можно извлечь еще более общее правило, выполняя отсечение ветви после этапа $\text{Simplify}(y+z, w)$, что приводит к получению такого правила:

$\text{Simplify}(y+zfw) \Rightarrow \text{Simplify}(lx(y+z),w)$ Вообще говоря, из любого частичного поддерева обобщенного дерева доказательства можно извлечь какое-то правило.

Это означает, что имеет место еще одна проблема: какое из этих правил следует выбрать?

Выбор правила, подлежащего выработке, сводится к решению вопроса об эффективности выбранного правила. Ниже описаны три фактора, которые касаются анализа прироста эффективности, достигнутого в результате обучения на основе объяснения.

1. Введение большого количества правил влечет за собой замедление процесса формирования рассуждений, поскольку механизм логического вывода так или иначе должен проверять эти правила, даже в тех случаях, когда они не приводят к решению. Иными словами, увеличение количества правил приводит к увеличению коэффициента ветвления в пространстве поиска.

2. Для того чтобы компенсировать замедление процесса формирования рассуждений, производные правила должны обеспечивать существенное увеличение скорости для тех случаев, которые они охватывают. Такое увеличение достигается в основном благодаря тому, что производные правила позволяют избегать тупиков, которые могли бы возникнуть без их использования, а также в связи с тем, что они сокращают само доказательство.

3. Производные правила должны быть как можно более общими, для того чтобы они применялись к максимально возможному множеству случаев.

Общепринятый подход к обеспечению эффективности таких производных правил состоит в том, чтобы следить за соблюдением требования, касающегося $\wedge$ операционной пригодности каждой подцели в правиле. Подцель является операционно пригодной, если ее можно достаточно "легко" решить. Например, подцель

$\text{Primitiv}(z)$ решить легко, и для этого требуется самое большее два этапа вывода, а для достижения подцели $\text{Simplify}(y+z, w)$ может потребоваться произвольный объем логического вывода в зависимости от значений y и z . Если проверка на операционную пригодность выполняется на каждом этапе формирования обобщенного доказательства, то появляется возможность отсечь оставшуюся ветвь, как только будет найдена операционно пригодная подцель, и оставить в качестве конъюнкта нового правила лишь эту операционно пригодную подцель.

К сожалению, обычно приходится искать компромисс между операционной пригодностью и общностью. Для более конкретных подцелей обычно бывает проще найти решение, но они охватывают меньше случаев. Кроме того, сам критерий операционной пригодности в определенной степени субъективен: можно уверенно сказать, что подцель, для достижения которой требуется один или два этапа, является операционно пригодной, но можно ли это утверждать, если количество

этапов приближается к 10 или 100? Наконец, стоимость поиска решения для данной конкретной подцели зависит от того, какие еще правила имеются в базе знаний. По мере введения дополнительных правил эта стоимость может и увеличиваться, и уменьшаться. Таким образом, при эксплуатации систем EBL фактически приходится сталкиваться с очень сложными проблемами оптимизации, пытаясь добиться максимальной эффективности заданной начальной базы знаний. Иногда существует возможность вывести математическую модель влияния операции добавления какого-то конкретного правила на общую эффективность и использовать эту модель для определения наилучшего правила, подлежащего добавлению. Но такой анализ может стать очень сложным, особенно когда приходится сталкиваться с рекурсивными правилами. Один из перспективных подходов состоит в эмпирическом решении проблемы эффективности путем добавления нескольких правил и определения того, какое из них является полезным и действительно позволяет ускорить работу.

Эмпирический анализ эффективности фактически становится основой системы EBL. То, что в предыдущем абзаце неформально было названо "эффективностью заданной базы знаний", в действительности представляет собой усредненную по рассматриваемым случаям сложность решения задач, предъявляемых системе и описываемых некоторым распределением вероятностей. c^* Система EBL позволяет проводить обобщение по результатам решения примеров задач, встретившихся в прошлом, поэтому способствует повышению эффективности базы знаний применительно к тем типам задач, которые, скорее всего, должны встретиться в будущем. Такая организация работы способствует достижению успеха, при условии, что распределение вероятностей прошлых примеров приблизительно соответствует распределению вероятностей будущих примеров; это — такое же предположение, которое лежало в основе метода PAC-обучения, описанного в разделе 18.5. Если проведено тщательное и продуманное проектирование системы EBL, то появляется возможность достичь существенного ускорения работы. Например, очень крупная система обработки естественного языка на основе Prolog, предназначенная для прямого и обратного перевода речи со шведского на английский, позволила добиться приемлемой производительности работы в реальном времени только благодаря применению системы EBL в процессе синтаксического анализа [1351].

19.4. Обучение с использованием информации о релевантности Путешественник, приехавший в Бразилию, о котором говорилось раньше, повидимому, сумел прийти к достоверному обобщению, касающемуся языка, на котором говорят остальные бразильцы. Полученный этим путешественником логический вывод был основан на его фоновых знаниях, а именно на знаниях о том, что население данной конкретной страны (обычно) говорит на одном и тот же языке.

Это утверждение можно выразить в логике первого порядка следующим образом³:

$Nationality(x, n) \wedge Nationality(y, n) \wedge Language(x, 1) \Rightarrow Language(y, 1)$ A9.6) Это высказывание буквально означает: "Если x и y имеют одинаковое гражданство и x говорит на языке 1, то y также говорит на этом языке". Нетрудно показать, что на основании этого высказывания и следующего наблюдения:

$Nationality(Fernando, Brazil) \wedge Language(Fernando, Portuguese)$ можно прийти к такому заключению (см. упр. 19.1):

$Nationality(x, Brazil) \Rightarrow Language(x, Portuguese)$ Высказывания, подобные приведенному в уравнении 19.6, выражают строгую форму релевантности: если известно гражданство человека, то полностью определен язык, на котором он говорит (эту мысль можно выразить иначе — язык определяется функциональной зависимостью от гражданства). Такие высказывания называются $\wedge$ функциональными зависимостями, или $\wedge$ определениями, и встречаются настолько часто в приложениях определенных типов (например, в спецификациях проектов баз данных), что для их записи используется специальный синтаксис. В этой главе мы будем придерживаться системы обозначений, предложенной Дэвисом [329]:

$Nationality(x, n) \cup Language(x, 1)$ Как обычно, это обозначение представляет собой синтаксическое упрощение, но оно подчеркивает тот факт, что определение в действительности показывает связь между предикатами; в данном случае гражданство определяет язык. Аналогичным образом могут быть представлены релевантные свойства, определяющие проводимость и плотность:

$Material(x, t) \wedge Temperature(x, t) \cup Conductance(x, p) \wedge Material(x, t) \wedge Temperature(x, t) \cup Density(x, d)$ Соответствующие обобщения следуют логически из определений и наблюдений.

Определение пространства гипотез Хотя определения позволяют обосновать общие заключения, касающиеся всех бразильцев или всех образцов меди, исследуемых при заданной температуре, они, безусловно, не позволяют создать на основании одного примера общую прогностику³ Для упрощения в этой главе предполагается, что каждый человек говорит только на одном языке. Безусловно, в это правило потребовалось бы ввести поправку, если бы дело касалось таких многоязычных стран, как Швейцария или Индия.

ческую теорию, которая распространялась бы на граждан всех государств или на любые материалы, исследуемые при любых температурах. По существу, основной эффект применения определений можно рассматривать как ограничение пространства гипотез, которые должен рассматривать обучающийся агент. В частности, согласно определению, при прогнозировании проводимости достаточно учитывать только материал и температуру, но игнорировать массу, права

собственности на рассматриваемый образец, день недели, фамилию главы правительства и т.д. Но гипотезы, касающиеся проводимости, безусловно, могут включать термы, которые в свою очередь зависят от материалы и температуры, в том числе учитывающие молекулярную структуру, тепловую энергию или плотность свободных электронов. $\Phi^{\wedge}$ Определения задают базовый словарь, достаточный для построения гипотез, касающихся целевого предиката. Это утверждение можно доказать, продемонстрировав, что данное конкретное определение логически эквивалентно утверждению о том, что правильная формулировка целевого предиката является одной из множества всех формулировок, которые могут быть представлены с помощью предикатов из левой части определения.

Интуитивно ясно, что сокращение размера пространства гипотез должно способствовать упрощению задачи изучения целевого предиката. Возможный при этом прирост эффективности можно оценить с использованием основных результатов теории вычислительного обучения (см. раздел 18.5). Вначале напомним, что для булевых функций требуется $10d(|n|)$ примеров, чтобы свести все пространство гипотез к одной приемлемой гипотезе, где $|n|$ — размер пространства гипотез. Если в распоряжении ученика имеется p булевых характеристик, для которых должны быть построены гипотезы, то в отсутствии дополнительных ограничений $|n|=2^p$, поэтому необходимое количество примеров составляет $O(2^p d)$. Если же в левой части определения находятся d предикатов, то для ученика потребуется только $O(d)$ примеров, что соответствует сокращению в 2^{p-d} раз. Для смещенного пространства гипотез, такого как конъюнктивно смещенное пространство, это сокращение будет не столь радикальным, но все еще достаточно существенным.

Обучение и использование информации о релевантности Как было указано во введении к данной главе, априорные знания являются полезными при обучении, но они также должны быть освоены в процессе обучения.

Поэтому для обеспечения целостного подхода к обучению с учетом релевантности необходимо предусмотреть алгоритм обучения, относящийся к определениям.

Алгоритм обучения, приведенный в этом разделе, представляет собой прямолинейную попытку найти простейшее определение, совместимое с рассматриваемыми результатами наблюдений. В частности, определение $P^{\wedge}Q$ говорит о том, что если какиелибо примеры согласуются по выражению P , то они должны также согласовываться по выражению Q . Поэтому определение является совместимым с множеством примеров, если каждая пара, которая согласуется по предикатам в левой части определения, согласуется также по целевому предикату, т.е. входит в одну и ту же классификацию. Например, предположим, что имеются примеры результатов измерения проводимости образцов материалов, приведенные в табл. 19.1.

Минимальным согласованным определением является MaterialATemperatureyConductance. Существует также одно не минимальное, но совместимое определение, а именно MassASizeATemperatureyConductance. Оно совместимо с примерами, поскольку масса и размер определяют плотность, а в рассматриваемом наборе данных отсутствуют два разных материала с одной и той же плотностью. Как обычно, для устранения гипотез, которые только внешне кажутся правильными, необходимо иметь более крупное множество примеров.

Таблица 19.1. Примеры результатов измерения проводимости образцов материалов

Образец	S1	S1	S2	S3	S3	S4	Масса	Температура	Материал	Медь	Медь	Медь	Свинец
Свинец	Свинец	Свинец	Размер	Проводимость	0,59	0,57	0,59	0,05	0,04	0,05	Для поиска	минимальных совместимых определений можно применить несколько подходящих для этого алгоритмов. Наиболее очевидный подход состоит в проведении поиска в пространстве определений и проверке всех определений с одним предикатом, двумя предикатами и т.д. до тех пор, пока не будет найдено совместимое определение. Предполагается, что используется простое представление на основе атрибутов, аналогичное тому, которое использовалось при обучении деревьев решений в главе 18. Определение d будет представлено с помощью множества атрибутов в левой части, поскольку предполагается, что целевой предикат остается постоянным. Основной алгоритм, соответствующий этому подходу, приведен в листинге 19.3.	

Листинг 19.3. Алгоритм поиска минимального совместимого определения

```
function Minimal-Consistent-Det(E, A) returns множество атрибутов
inputs: E, множество примеров A, множество атрибутов с количеством элементов n
for i < 0, ..., n do
  for each подмножество A1 множества A с количеством элементов i do
    if Consistent-Det?(A1, E) then return A1
function Consistent-Det?(A, E) returns истинностное значение
inputs: A, множество атрибутов E, множество примеров
local variables: Я, хэш-таблица
for each пример e in E do
  if некоторый пример в таблице Я имеет такое же значение множества атрибутов A, что и пример e, но имеет другую классификацию then return false
  сохранить обозначение класса примера e в таблице Я под индексом, соответствующим значениям атрибутов A примера e
return true
```

Временная сложность этого алгоритма зависит от размера наименьшего совместимого определения. Предположим, что это определение включает p атрибутов из общего количества, равного n атрибутам. В таком случае алгоритм не отыщет данное определение до тех пор, пока не выполнит поиск во всех подмножествах A с размером p . Количество таких подмножеств измеряется значением $\binom{n}{p} = O(n^p)$, P поэтому сложность алгоритма зависит экспоненциально от размера минимального определения. Как оказалось, данная задача является NP-полной, поэтому не следует рассчитывать на то, что в общем случае будет достигнута более высокая производительность. Тем не менее во многих проблемных областях

существует локальная структура (понятие локально структурированных проблемных областей рассматривается в главе 14), благодаря которой p становится небольшим.

Получив в свое распоряжение алгоритм изучения определений, обучающийся агент приобретает способность создавать минимальную гипотезу, с помощью которой он может изучать целевой предикат. Например, можно скомбинировать алгоритм MinimalConsistent-Det с алгоритмом Decision-Tree-Learning. Это приведет к созданию алгоритма обучения дерева решений с учетом релевантности, RBDTL (Relevance-Based Decision-Tree Learning), в котором вначале определяется минимальное множество релевантных атрибутов, а затем это множество передается алгоритму формирования дерева решений для обучения. В отличие от алгоритма Decision-Tree-Learning, алгоритм RBDTL одновременно обеспечивает и обучение, и использование информации о релевантности для минимизации своего пространства гипотез. Можно рассчитывать на то, что алгоритм RBDTL даст возможность проводить обучение быстрее по сравнению с алгоритмом Decision-Tree-Learning, и действительно так и происходит. На рис. 19.6 показана производительность обучения для этих двух алгоритмов на основе случайно сгенерированных данных для функции, которая зависит только от 5 из 16 атрибутов. Ясно, что в тех случаях, когда все доступные атрибуты являются релевантными, алгоритм RBDTL не показывает каких-либо преимуществ.

20 40 60 80 100 120 Объем обучающего множества

Рис. 19.6. Сравнение производительности алгоритмов RBDTL и Decision-Tree-Learning при обработке случайно сгенерированных данных применительно к целевой функции, которая зависит только от 5 из 16 атрибутов

В данном разделе приведены лишь самые основные сведения из области исследования $\wedge$ декларативного смещения, задача которой состоит в изучении того, как могут использоваться априорные знания для выявления приемлемого пространства гипотез, в котором должен осуществляться поиск правильного целевого определения. В нем не даны ответы на многие вопросы, например, на те, которые приведены ниже.

- Как могут быть дополнены эти алгоритмы для того, чтобы в них учитывался шум?
- Можно ли обеспечить обработку переменных с непрерывными значениями?
- Можно ли использовать другие виды априорных знаний, кроме определений?
- Как можно обобщить эти алгоритмы для охвата любой теории первого порядка, а не только представлений на основе атрибутов?

Некоторые из этих вопросов рассматриваются в следующем разделе.

19.5. Индуктивное логическое программирование В индуктивном логическом программировании (Inductive Logic Programming — ILP) объединяются индуктивные методы с мощными представлениями в логике первого порядка, и оно в основном сосредоточивается на представлении теорий в виде логических программ⁴. Это

научное направление получило широкую известность по трем причинам. Во-первых, индуктивное логическое программирование лежит в основе строгого подхода к решению общих задач индуктивного обучения на основе базы знаний. Во-вторых, в рамках этого направления созданы полные алгоритмы логического вывода общих теорий в логике первого порядка с использованием примеров; это означает, что данные алгоритмы могут успешно применяться для обучения в тех проблемных областях, в которых использование алгоритмов на основе атрибутов связано с определенными затруднениями. В качестве соответствующего примера можно привести применение методов ILP в изучении того, как сворачиваются белковые структуры (рис. 19.7). Трехмерную конфигурацию молекулы белка невозможно представить с помощью какого-либо приемлемого способа в виде множества атрибутов, поскольку конфигурация молекулы неявно обусловлена связями между объектами, а не атрибутами отдельного объекта. Одним из языков, подходящих для описания таких связей, является логика первого порядка. В-третьих, с помощью методов индуктивного логического программирования могут вырабатываться гипотезы, которые являются (относительно) простыми для восприятия людьми. Например, перевод на естественный язык правила, приведенного в подписи к рис. 19.7, может уточняться и подвергаться критике биологами, которые работают в данной области. Это означает, что системы индуктивного логического программирования могут участвовать в научном цикле экспериментирования, выработки гипотез, обсуждения и опровержения. Такое участие невозможно при использовании таких систем, которые вырабатывают классификаторы в виде "черных ящиков", например нейронных сетей.

4 На данном этапе читателю может потребоваться обратиться к главе 9 для повторного ознакомления с некоторыми из основных понятий, включая хорновские выражения, конъюнктивную нормальную форму, унификацию и резолюцию.

2mhr - восходящая и нисходящая связка , , _ , lomd - EP из четырех спиралей а) б) Рис. 19.7. Пример решения задачи обучения: положительный и отрицательный примеры применения понятия "восходящей и нисходящей связки из четырех спиралей" в проблемной области сворачивания белка (а), (б). Пример каждой структуры кодируется в виде логического выражения, состоящего примерно из 100 конъюнктов, такого как TotalLength (D2mhr, 118) л NumberHelices (D2mhr, 6)л.... На основании этих описаний и таких классификаций, как Fold fFour-Helical-Up-And-Down-Bundle,D2mhr^ система индуктивного логического программирования Progol [1098] сформировала по методу обучения следующее правило:

Fold fFour-Helical-Up-And-Down-Bundle,p; 4= Helix fp/hij л Length (hi,High) л Position (p,hi,n) л $A < n < 3$ л Adjacent (q ,\11\12) л Helix fp/h^ Правило такого рода невозможно выработать путем обучения или даже представить с помощью любых механизмов, основанных на атрибутах, наподобие тех, которые рассматривались в

предыдущих главах. Это правило можно перевести на естественный язык, как показано ниже.

Белок Р относится к классу методов сворачивания "связка из четырех восходящих и нисходящих спиралей", если он включает длинную спираль h_1 во второй структурной позиции между позициями 1 и 3 и если спираль h_2 находится рядом со второй спиралью. Практический пример. Как было указано выше, из уравнения 19.5 следует, что общая задача индукции на основе знаний состоит в "решении" ограничения логического следствия для неизвестной гипотезы Hypothesis, если даны фоновые знания Background и примеры, представленные с помощью описаний Descriptions и классификаций Classifications:

Background $\wedge$ Hypothesis $\wedge$ Descriptions $\wedge$ Classifications. Для иллюстрации этого процесса рассмотрим задачу изучения семейных связей на примерах. Описания будут состоять из развернутого генеалогического дерева, описанного в терминах отношений Mother (Мать), Father (Отец) и Married (Состоит в браке), а также свойств Male (Мужчина) и Female (Женщина). В качестве примера будет использоваться генеалогическое дерево, рассматриваемое в упр.

8.11, которое показано на рис. 19.8. Соответствующие описания приведены ниже.

George M Mum Spencer M Kydd Elizabeth N Philip Diana M Charles Anne M Mark Andrew M Sarah Edward William Harry Peter Zara Beatrice Eugenie. Рис. 19.8. Типичное генеалогическое дерево. Father {Philip, Charles} Father(Philip, Anne) Mother {Mum, Margaret} Mother{Mum, Elizabeth} Married{Diana, Charles} Married{Elizabeth, Philip} Male{Philip} Male{Charles} Female{Beatrice} Female{Margaret}. Высказывания, входящие в состав классификаций Classifications, зависят от изучаемого целевого понятия. Например, может потребоваться изучить понятия Grandparent (Дедушка или бабушка), BrotherInLaw (Двоюродный брат) или Ancestor (Предок). Что касается понятия Grandparent, то полное множество Classifications содержит $20 \times 20 = 400$ конъюнктов в следующей форме:

Grandparent {Mum, Charles} Grandparent{Elizabeth, Beatrice} $\rightarrow$ Grandparent (Mum, Harry) $\wedge$ Grandparent{Spencer, Peter}. Безусловно, обучение может быть проведено на основе какого-то подмножества этого полного множества.

Задачей применения любой программы индуктивного обучения является выработка множества высказываний, соответствующих гипотезе Hypothesis, такого, что в нем выполняется заданное ограничение логического следствия. На данный момент предположим, что у агента отсутствуют фоновые знания: отношение Background является пустым. В таком случае одно из возможных решений для Hypothesis состоит в следующем:

$\text{Grandparent}(x,y) \Leftrightarrow [\exists z \text{ Mother}(x,z) \wedge \text{Mother}(z,y)] \vee [\exists z \text{ Mother}(x,z) \wedge \text{Father}(z,y)] \vee [\exists z \text{ Father}(x,z) \wedge \text{Mother}(z,y)] \vee [\exists z \text{ Father}(x,z) \wedge \text{Father}(z,y)]$ Следует отметить, что алгоритм обучения на основе атрибутов, такой как Decision-Tree-Learning, в попытке решить эту задачу бесконечно углубится в дерево, но не достигнет успеха. Для того чтобы выразить отношение Grandparent в виде атрибута (т.е. унарного предиката), необходимо будет преобразовывать пары людей в объекты:

$\text{Grandparent}(\{ \text{Mum}, \text{Charles} \})$... Затем мы окажемся в тупике, пытаясь представить описания примеров.

Единственно возможными атрибутами станут такие ужасно неуклюжие конструкции, как следующая:

$\text{FirstElementIsMotherOfElizabeth}(\{ \text{Mum}, \text{Charles} \})$ Определением отношения Grandparent в терминах этих атрибутов становится большая дизъюнкция описаний конкретных случаев, которая вообще не может быть обобщена с учетом новых примеров. <3° Алгоритмы обучения на основе атрибутов не способны обеспечить изучение реляционных предикатов. Таким образом, одним из принципиальных преимуществ алгоритмов ILP является их применимость для решения гораздо более широкого диапазона задач, включая реляционные задачи.

Читатель должен был, безусловно, заметить, что для представления определения Grandparent мог бы помочь небольшой фрагмент фоновых знаний. Например, если в состав фоновых знаний Background входит следующее высказывание:

$\text{Parent}(x,y) \Leftrightarrow [\text{Mother}(x,y) \vee \text{Father}(x,y)]$ то определение отношения Grandparent можно свести к следующему:

$\text{Grandparent}(x,y) \Leftrightarrow [\exists z \text{ Parent}(x,z) \wedge \text{Parent}(z,y)]$ Этот пример показывает, насколько существенно фоновые знания позволяют сократить размер гипотез, требуемых для объяснения результатов наблюдений.

Алгоритмы ILP позволяют также создавать новые предикаты для упрощения способов выражения объяснительных гипотез. Применительно к примеру данных, приведенному выше, было бы вполне резонно, чтобы программа ILP предложила дополнительный предикат, который можно было бы назвать "Parent" (Родитель), для упрощения определений целевых предикатов. Алгоритмы, которые способны вырабатывать новые предикаты, называются алгоритмами $\wedge$ конструктивной индукции. Очевидно, что конструктивная индукция является необходимой частью подхода к обучению на основе накопления знаний, который был кратко описан во введении. Задача накопления знаний всегда была одной из сложнейших задач машинного обучения, но некоторые методы ILP предоставляют эффективные механизмы ее решения.

В оставшейся части данной главы исследуются два принципиальных подхода к индуктивному логическому программированию. В первом из них используется обобщение методов на основе деревьев решений, а во втором используются методы, основанные на инвертировании доказательства с помощью резолюции.

Нисходящие методы индуктивного обучения. Описанный здесь первый подход к индуктивному логическому программированию начинается с получения очень общего правила и его постепенного уточнения для достижения соответствия с данными. По сути такой же процесс происходит при обучении деревьев решений, когда дерево решений постепенно наращивается до тех пор, пока не становится совместимым с результатами наблюдений. Но для осуществления индуктивного логического программирования вместо атрибутов используются литералы первого порядка, а вместо дерева решений рассматриваются гипотезы, представляющие собой множества выражений. В этом разделе рассматривается программа Foil [1258], одна из первых программ ILP.

Предположим, что предпринимается попытка изучить определение предиката *Grandfather* (x, y) с использованием тех же данных о семейных отношениях, как и перед этим. По такому же принципу, как и при обучении деревьев решений, эти примеры можно разделить на положительные и отрицательные. Положительными примерами являются следующие:

(George, Anne), (Philip, Peter), (Spencer, Harry), ... а отрицательными — такие примеры: (George, Elizabeth), (Harry, Zara), (Charles, Philip), ...

Обратите внимание на то, что каждый пример представляет собой пару объектов, поскольку *Grandfather* — это бинарный предикат. В целом в данном генеалогическом дереве имеется 12 положительных примеров и 388 отрицательных (все прочие пары людей).

Программа Foil строит множества выражений, в каждом из которых *Grandfather* (x, y) является головой предиката. Эти выражения должны классифицировать 12 положительных примеров как экземпляры отношения *Grandfather*{ x, y } и вместе с тем исключать 388 отрицательных примеров. Рассматриваемые выражения являются хорновскими выражениями, расширенными с помощью отрицаемых литералов, в которых используется отрицание как недостижение цели, по аналогии с языком Prolog.

Первоначальное выражение имеет такое пустое тело: $\Rightarrow \text{Grandfather}(x, y)$ Это выражение классифицирует каждый пример как положительный, поэтому требует уточнения. Такую операцию можно выполнить, каждый раз вводя по одному литералу в левую часть выражения. Ниже перечислены три возможных дополнения.

$\text{Father}(x,y) \Rightarrow \text{Grandfather}(x,y)$ $\text{Parent}(x, z) \Rightarrow \text{Grandfather}(x,y)$ $\text{Father}(x, z) \Rightarrow \text{Grandfather}(x, y)$ (Обратите внимание на то, что здесь предполагается, будто выражение, определяющее предикат Parent , уже является частью фоновых знаний.)
Первое из этих

трех выражений неправильно классифицирует все 12 положительных примеров как отрицательные и поэтому может быть проигнорировано. Второе и третье выражения согласуются со всеми положительными примерами, но второе является неправильным применительно к большей части отрицательных примеров (этих примеров вдвое больше, чем в первом случае, поскольку в них допускается произвольная подстановка данных и о матерях, и об отцах). Поэтому предпочтительным является третье выражение.

Теперь необходимо дополнительно уточнить это выражение, чтобы исключить те случаи, в которых x является отцом некоторого z , но z не является родителем y .

Добавление единственного литерала $\text{Parent}(z,y)$ позволяет получить следующее выражение, которое правильно классифицирует все примеры:

$\text{Father}(x,z) \wedge \text{Parent}(z,y) \Rightarrow \text{Grandfather}(x,y)$ Программа Foil способна найти и выбрать этот литерал, решая тем самым задачу обучения. Вообще говоря, программе Foil обычно приходится выполнять поиск среди многих неподходящих выражений, прежде чем она сможет найти правильное решение.

Этот пример может служить очень простой иллюстрацией того, как действует программа Foil. набросок общего алгоритма этой программы приведен в листинге 19.4. По существу, алгоритм повторно составляет выражение, вводя в него один литерал за другим до тех пор, пока это выражение не начинает согласовываться с некоторым подмножеством положительных примеров и при этом не согласуется ни с одним отрицательным примером. Затем положительные примеры, охваченные данным выражением, удаляются из обучающего множества и процесс продолжается до тех пор, пока не остается ни одного положительного примера. Двумя основными процедурами, требующими пояснения, являются *New-Literals*, которая создает все возможные новые литералы для добавления к выражению, и *Choose-Literal*, которая выбирает литерал для добавления.

Листинг 19.4. набросок алгоритма Foil, применяемого для изучения множеств хорновских выражений первого порядка на основе примеров. Функции *New-Literals* и *Choose-Literal* описаны в тексте `function Foil(examples, target) returns множество хорновских выражений inputs: examples, множество примеров target, литерал для целевого предиката local variables: clauses, множество выражений, первоначально пустое while множество examples содержит положительные примеры do clause $\leftarrow$ New-Clause(examples, target) удалить примеры, охваченные выражением clause, из множества examples добавить выражение clause к множеству clauses return clauses`

function New-Clause(examples, target) returns хорновское выражение local variables: clause, выражение, головой которого является target, а тело пусто 1, литерал, который должен быть добавлен к выражению clause extended_examples, множество примеров со значениями для новых переменных

extended_examples $\leftarrow$ examples while extended_examples содержат отрицательные примеры do 1 $\leftarrow$ Choose-Literal(New-Literals(clause), extended_examples) добавить 1 к телу выражения clause extended_examples $\leftarrow$ множество примеров, созданных путем применения функции Extend-Example к каждому примеру в множестве extended_examples return clause function Extend-Example(example, literal) returns множество примеров if пример example согласуется с литералом literal then return множество примеров, созданное путем дополнения примера example каждым возможным значением константы для каждой новой переменной в литерале literal else return пустое множество Процедура New-Literals выбирает некоторое выражение и составляет все возможные "полезные" литералы, которые могут быть добавлены к этому выражению.

Рассмотрим в качестве примера следующее выражение:

Father(x, z) $\Rightarrow$ Grandfather(x,y) К нему могут быть добавлены три описанные ниже разновидности литералов.

1. Литералы, в которых используются предикаты. Литерал может быть отрицаемым или неотрицаемым, в нем может применяться любой существующий предикат (включая целевой предикат), а все параметры должны быть переменными. В качестве любого параметра предиката может использоваться любая переменная, с учетом одного ограничения: каждый литерал должен включать по меньшей мере одну переменную из предыдущего литерала или из головы выражения. Такие литералы, как Mother(z, и), Married(z, y), ¬Kale (y) и Grandfather (v, x), являются допустимыми, а Married(u,v) — нет. Обратите внимание на то, что благодаря имеющейся возможности применять предикат из головы выражения программа Foil может использоваться для изучения рекурсивных определений.

2. Литералы равенства и неравенства. Эти литералы связывают переменные, уже присутствующие в данном выражении. Например, можно ввести литерал z^x .

Данные литералы могут также включать константы, заданные пользователем.

Для изучения арифметических выражений могут использоваться константы 0 и 1, а для изучения функций, заданных на списках, — пустой список [].

3. Арифметические сравнения. При обработке функций непрерывных переменных могут быть добавлены такие литералы, как $x > y$ и $y < z$. По аналогии с обучением деревьев решений могут быть выбраны пороговые значения для максимизации различительной мощности проверки.

Возникающий в результате коэффициент ветвления в этом пространстве поиска очень велик (см. упр. 19.6), но в программе Foil для его уменьшения может также использоваться информация о типе. Например, если проблемная область включает данные не только о числах, но и о людях, то ограничения типов позволят исключить возможность выработки в процедуре New-Literals таких литералов, как $\text{Parent}(x, p)$, где x — человек, а p — число.

В процедуре Choose-Literal используется эвристика, немного напоминающая эвристику, основанную на приросте информации (см. с. 878), для определения того, какой литерал должен быть выбран для добавления. Точные сведения о том, как это осуществляется, не имеет в контексте данной главы слишком большого значения, к тому же было опробовано много различных вариантов таких эвристик. Одно интересное дополнительное средство программы Foil состоит в использовании принципа бритвы Оккама для устранения некоторых гипотез. Если некоторое выражение становится длиннее (согласно определенной метрике), чем общая длина положительных примеров, объясняемых с помощью этого выражения, оно больше не рассматривается в качестве потенциальной гипотезы. Такой метод предоставляет удобный способ предотвращения создания чрезмерно сложных выражений, которые обусловлены наличием шума в данных. Объяснение связи между шумом и длиной выражений приведено на с. 949.

Программа Foil и аналогичные ей программы использовались для изучения широкого ряда определений. Одна из наиболее впечатляющих демонстраций возможностей таких программ [1260] касалась решения длинной последовательности упражнений по функциям обработки списков из учебника по языку Prolog, написанного Братко [174]. В каждом случае программа показала свою способность найти в процессе обучения правильное определение функции на основе небольшого множества примеров, используя в качестве фоновых знаний ранее изученные функции.

Индуктивное обучение с помощью обратной дедукции Второй важный подход к индуктивному логическому программированию предусматривает обращение обычного процесса дедуктивного доказательства. Метод $\wedge$ обратной резолюции основан на том наблюдении, что если классификация Classifications примера следует из выражения $\text{Background} \wedge \text{Hypothesis} \wedge \text{Descriptions}$, то должна существовать возможность доказать этот факт с помощью метода резолюции (поскольку метод резолюции является полным). А если есть возможность "провести доказательство в обратном направлении", то можно найти такую гипотезу Hypothesis, через которую проходит это доказательство. Поэтому задача состоит в том, чтобы найти способ обращения процесса резолюции.

В этом разделе будет показан процесс обратного доказательства для инверсного правила резолюции, который состоит из отдельных обратных этапов. Напомним, что в обычном этапе резолюции берутся два выражения, d и $C2$, и к ним применяется операция уничтожения взаимно противоположных литералов (операция резолюции) для получения резольвенты C . С другой стороны, на этапе инверсной резолюции берется резольвента C и формируются два выражения, Cg и $C2$, такие, что C является результатом применения операции уничтожения взаимно противоположных литералов в выражениях Cx и $C2$. Еще один вариант состоит в том, что можно взять резольвенту C и выражение $C\pm$ и сформировать выражение $C2$, такое, что C является результатом применения операции резолюции к $C\pm$ и $C2$.

Начальные этапы процесса обратной резолюции показаны на рис. 19.9; на этих этапах вся работа сосредоточивается на положительном примере Grandparent

{George, Anne). Процесс начинается с конца доказательства (который обозначен квадратом в нижней части рисунка). Предполагается, что резольвента C представляет собой пустое выражение (т.е. противоречие), а в качестве $C2$ берется выражение $\neg \text{Grandparent}\{\text{George}, \text{Anne}\}$, которое является отрицанием целевого примера.

В первом обратном этапе используются выражения C и $C2$ и формируется выражение $\text{Grandparent}\{\text{George}, \text{Anne}\}$, соответствующее Cg . На следующем этапе это выражение берется в качестве C , выражение $\text{Parent}\{\text{Elizabeth}, \text{Anne}\}$ используется в качестве $c2$, а в качестве cg формируется следующее выражение: $\neg \text{Parent}\{\text{Elizabeth}, y\} \vee \text{Grandparent}\{\text{George}, y\}$. На последнем этапе данное выражение рассматривается как резольвента. Если в качестве $C2$ берется выражение $\text{Parent}\{\text{George}, \text{Elizabeth}\}$, то одним из возможных выражений Cg становится гипотеза:

$\text{Parent}\{x, z\} \wedge \text{Parent}\{z, y\} \Rightarrow \text{Grandparent}\{x, y\}$ Теперь в нашем распоряжении имеется доказательство с помощью резолюции, что из данной гипотезы, описаний и фоновых знаний следует классификация $\text{Grandparent}\{\text{George}, \text{Anne}\}$. » $\text{Grandparent}\{x, y\} \wedge \text{Parent}\{x, z\} \wedge \text{Parent}\{z, y\} \Rightarrow \text{Grandparent}\{x, y\}$ $\text{Parent}\{\text{George}, \text{Elizabeth}\} * \text{Parent}\{\text{Elizabeth}, \text{Anne}\} \vee \text{Grandparent}\{\text{George}, \text{Anne}\}$ $\text{Parent}\{\text{George}, \text{Elizabeth}\} * \text{Parent}\{\text{Elizabeth}, \text{Anne}\} \vee \text{Grandparent}\{\text{George}, \text{Anne}\}$ Рис. 19.9. Начальные этапы в процессе обратной резолюции. Затененные выражения сформированы на этапах обратной резолюции из выражений, находящихся справа и внизу. Незатененные выражения взяты из множеств Descriptions и Classifications. Очевидно, что в процессе обратной резолюции требуется поиск. Каждый этап обратной резолюции является недетерминированным, поскольку для любого выражения C может существовать большое или даже бесконечное количество выражений cg и $c2$, которые в результате операции резолюции преобразуются в C . Например, вместо применения выражения $\neg \text{Parent}\{\text{Elizabeth}, y\} \vee \text{Grandparent}\{\text{George}, y\}$ в качестве Cg на последнем этапе, показанном на рис. 19.9, на этом этапе обратной резолюции

можно было выбрать любое из следующих высказываний (см. упр. 19.4 и 19.5): — $iParent\{Elizabeth, Anne\} \vee Grandparent\{George, Anne\}$ — $iParent\{z, Anne\} \vee Grandparent\{George, Anne\}$ — $iParent\{z, y\} \vee Grandparent\{George, y\}$

Более того, выражения, которые участвуют на каждом этапе, могут быть выбраны из фоновых знаний Background, из описаний примеров Descriptions, из отрицаемых выражений в множестве Classifications или из выражений, выдвинутых в качестве гипотезы, которые уже были сформированы в дереве обратной резолюции. Такое большое количество возможностей означает, что без дополнительного управления возникает большой коэффициент ветвления (и поэтому поиск становится неэффективным). В реализованных на практике системах ILP был опробован целый ряд подходов к управлению поиском, в том числе подходы, описанные ниже.

1. Могут быть удалены избыточные варианты выбора, например, путем формирования только наиболее конкретных гипотез из всех возможных и соблюдения требования о том, чтобы все выражения, принятые в качестве гипотез, были совместимыми и друг с другом, и с результатами наблюдений.

Применение последнего критерия позволило бы исключить выражение $iParent(z, y) \vee Grandparent(George, y)$, приведенное выше.

2. Могут быть наложены ограничения на стратегию доказательства. Например, в главе 9 было показано, что линейная резолюция — это полная, ограниченная стратегия, которая позволяет создавать только такие деревья доказательства, которые имеют линейную структуру ветвления (как на рис. 19.9).

3. Могут быть наложены ограничения на язык представления, например, путем устранения функциональных символов или применения только хорновских выражений. Например, язык Progol оперирует с хорновскими выражениями с использованием $\wedge$ обратного логического следствия. Идея этого метода состоит в том, что ограничение логического следствия:

Background $\wedge$ Hypothesis $\wedge$ Descriptions $\wedge$ Classifications должно быть преобразовано в такую логически эквивалентную форму:

Background $\wedge$ Descriptions $\wedge$ $\neg$ Classifications $\wedge$ $\neg$ Hypothesis Исходя из этой формы представления, для вывода гипотезы Hypothesis можно использовать процесс, аналогичный обычной дедукции Prolog с помощью хорновских выражений, в котором применяется отрицание как недостижение цели. Поскольку данный метод ограничивается хорновскими выражениями, он является неполным, но может оказаться более эффективным, чем полная резолюция. Кроме того, при обратном логическом следствии появляется возможность использовать полный логический вывод [717].

4. Логический вывод может осуществляться с помощью проверки по модели, а не с помощью доказательства теоремы. В системе Progol [1098] для ограничения поиска используется определенная форма проверки по модели. Это означает, что в этой системе, как и в программировании множества ответов, вырабатываются все возможные значения логических переменных, которые затем проверяются на совместимость.

5. Логический вывод может осуществляться с помощью базовых пропозициональных выражений, а не выражений в логике первого порядка. Система Linus [897] действует по принципу преобразования теорий логики первого порядка в выражения пропозициональной логики, поиска для этих выражений решений с помощью пропозициональной обучающейся системы, а затем

обратного преобразования. Как было показано на примере алгоритма SATplan в главе 11, при решении некоторых задач может оказаться более эффективной организация работы с помощью пропозициональных формул.

Совершение открытий с помощью индуктивного логического программирования
Любая процедура обратной резолюции, в которой стратегия полной резолюции действует в противоположном направлении, в принципе представляет собой полный алгоритм изучения теорий первого порядка. Это означает, что если на основе некоторой неизвестной гипотезы Hypothesis вырабатывается множество примеров, то процедура обратной резолюции может выработать гипотезу Hypothesis из этих примеров. Такое замечание подсказывает любопытную возможность: предположим, что имеющиеся примеры включают данные о разнообразных траекториях падающих тел. Существует ли такая теоретическая возможность, что программа обратной резолюции позволит вывести логическим путем закон гравитации? Очевидно, что ответ на этот вопрос является положительным, поскольку именно закон гравитации позволяет объяснять все примеры траекторий при использовании подходящего вспомогательного математического аппарата. Аналогичным образом, вполне можно представить себе, что в пределах возможностей программ ILP находится открытие законов электромагнитного поля, квантовой механики и теории относительности.

Разумеется, открытие этих законов находится и в пределах возможностей такой экспериментальной установки, в которой обезьяну посадили за пишущую машинку; нужны лишь более качественные эвристики и новые способы структуризации пространства поиска.

Но не углубляясь в эти крайности, можно отметить, что системы обратной резолюции действительно позволяют совершать небольшие, но важные открытия — создавать новые предикаты. Такую их способность часто рассматривают как нечто магическое, поскольку компьютеры принято считать устройствами, "просто

работающими с тем, что им задано". Но в действительности новые предикаты непосредственно следуют из этапов обратной резолюции. Простейшим случаем создания таких предикатов является преобразование в гипотезы двух новых выражений C_1 и C_2 на основе выражения C . Применение операции резолюции к выражениям C_1 и C_2 приводит к удалению в этих двух выражениях общего литерала, поэтому вполне возможно, что удаленный литерал содержал предикат, который не присутствовал в C .

Таким образом, при проведении доказательства в обратном направлении одна из возможностей состоит в формировании нового предиката, из которого реконструируется недостающий литерал.

На рис. 19.10 показан пример, в котором новый предикат p формируется в процессе изучения определения для отношения *Ancestor*. После его формирования предикат p может использоваться на дальнейших этапах обратной резолюции.

Например, на одном из дальнейших этапов может быть выдвинута гипотеза, что $Mother(x,y) \Rightarrow P(x,y)$. Таким образом, новый предикат p имеет смысл, определяемый процессом выработки гипотез, в которых он участвует. А обработка еще одного примера может привести к созданию ограничения $Father(x,y) \Rightarrow P(x,y)$.

Иными словами, предикат p представляет собой описание того, что обычно рассматривается как отношение *Parent*. Как было указано выше, введение новых пре-

дикатов позволяет существенно уменьшить размер определения целевого предиката.

Это означает, что системы обратной резолюции часто позволяют решать задачи обучения, неразрешимые с помощью других методов, поскольку предоставляют возможность изобретать новые предикаты. —1 $Father(George,y) \vee Ancestor(George,y)$
Рис. 19.10. Этап обратной резолюции, на котором вырабатывается новый предикат P Некоторые из самых радикальных революций в науке стали следствием изобретения новых предикатов и функций; в качестве примера можно привести открытие Галилеем принципа ускорения или уточнение Джоулем понятия тепловой энергии.

Как только в распоряжении ученых оказываются эти новые определения, открытие новых законов становится относительно простым. При этом труднее всего понять, что какая-то новая сущность, имеющая конкретную связь с уже известными сущностями, позволяет объяснить целый класс результатов наблюдений с помощью гораздо более простой и изящной теории по сравнению с теми теориями, которые применялись раньше.

До сих пор с помощью систем ILP еще не были сделаны открытия на уровне Галилея или Джоуля, но даже небольшие открытия, сделанные с их помощью, оказались достойными публикации в научной литературе. Например, в *Journal of Molecular Biology* были описаны результаты автоматизированного открытия правил сворачивания белка с помощью программы Progol, действующей по принципу индуктивного логического программирования [1517]. Многие правила, открытые программой Progol, и раньше можно было вывести из известных принципов, но большинство из них еще не было до сих пор опубликовано в составе какой-либо стандартной биологической базы данных (один из примеров такого правила приведен на рис. 19.7).

В одной из работ, относящихся к близкой теме, рассматривалась проблема открытия правил определения мутагенной способности нитроароматических веществ, обусловленной их молекулярной структурой [1454]. Такие вещества были обнаружены в выхлопных газах автомобилей. Для 80% этих веществ, данные о которых содержались в стандартной базе данных, удалось выявить четыре важные характеристики, причем применение метода линейной регрессии для анализа данных характеристик показало, что этот метод превышает по своей производительности метод ILP. А для оставшихся 20% веществ такие характеристики, отдельно взятые, не позволяют получить надежный прогноз, но система ILP выявила отношения, которые позволили ей превзойти по производительности системы с линейной регрессией, нейронными сетями и деревьями решений. В [798] показано, как обеспечить прогнозирование терапевтической эффективности различных лекарств, исходя из их молекулярной структуры. Анализ этих практических примеров позволяет прийти к выводу, что достижению высокой производительности систем ILP способствует то, что они позволяют представлять новые отношения и использовать фоновые знания. Тот факт, что правила, найденные системами ILP, могут интерпретироваться людьми, стал

причиной того, что эти методы были взяты на вооружение не только специалистами в области компьютерных наук, но и биологами.

Применение методов ILP способствует развитию не только биологии, но и других наук. Одним из наиболее важных таких направлений является обработка естественного языка, в котором системы ILP используются для извлечения сложной реляционной информации из текста. Итоговые результаты, достигнутые в этом научном направлении, описаны в главе 23.

19.6. Резюме В данной главе рассматривались различные способы, при использовании которых априорные знания могут помочь агенту обучаться на основе новых опытных данных. Поскольку основная часть априорных знаний выражена в терминах реляционных моделей, а не моделей, основанных на атрибутах, в настоящей главе описаны также системы, которые позволяют изучать

реляционные модели. Наиболее важные идеи, изложенные в этой главе, перечислены ниже.

- Успешное применение априорных знаний в обучении показывает, что еще более перспективным является кумулятивное обучение, в котором обучающиеся агенты повышают свою способность к обучению по мере того, как приобретают все больше и больше знаний.
- Априорные знания способствуют обучению, поскольку позволяют устранять ошибочные гипотезы, которые в ином случае считались бы совместимыми, а также "дополнять" объяснения примерами, что приводит к созданию более коротких гипотез. Такие возможности часто позволяют добиться ускоренного обучения с помощью меньшего количества примеров.
- Априорные знания могут выполнять различные функции в логическом выводе, а описания этих функций, выраженные в форме ограничений логического следствия, позволяют определить целый ряд разнообразных методов обучения.
- В методе обучения на основе объяснения (Explanation-Based Learning — EBL) предусматривается извлечение общих правил из отдельных примеров путем формирования объяснений для этих примеров и обобщения объяснений.

Метод EBL представляет собой дедуктивный метод, с помощью которого знания основных принципов преобразуются в полезные, эффективные экспертные знания специального назначения.

- В методе обучения с учетом релевантности (Relevance-Based Learning — RBL) используются априорные знания в форме определений для выявления релевантных атрибутов, что приводит к созданию уменьшенного пространства гипотез и ускорению обучения. Метод RBL обеспечивает также создание дедуктивных обобщений на основе отдельных примеров.
- В методе индуктивного обучения на основе знаний (Knowledge-Based Inductive Learning — KBIL) предусматривается поиск индуктивных гипотез, позволяющих объяснять множества результатов наблюдений с помощью фоновых знаний.
- В методах индуктивного логического программирования (Inductive Logic Programming — ILP) метод KBIL применяется к знаниям, выраженным в ло-

гике первого порядка. Методы ILP позволяют получать в процессе обучения такие реляционные знания, которое не могут быть выражены в системах на основе атрибутов.

- Индуктивное логическое программирование может осуществляться в рамках нисходящего подхода, в котором осуществляется уточнение очень общего правила, или восходящего подхода, обратного по отношению к процессу дедуктивного логического вывода.
- С помощью методов ILP естественным путем вырабатываются новые предикаты, с помощью которых могут быть выражены новые емкие теории, поэтому они открывают возможность создания систем формирования научных теорий общего назначения.

Библиографические и исторические заметки Хотя вполне очевидно, что одной из главных тем философии науки должно быть использование априорных знаний в

обучении, до последнего времени в этой области был проведен лишь небольшой объем работы. В книге *Fact, Fiction, and Forecast*, написанной философом Нельсоном Гудманом [578], была опровергнута применявшаяся ранее гипотеза о том, что индукция сводится просто к изучению достаточного количества примеров некоторого высказывания с квантором всеобщности, а затем принятия его в качестве гипотезы. Рассмотрим, например, гипотезу "Цвета всех изумрудов изменчивы", где под $\wedge$ изменчивым подразумевается "зеленый, если наблюдения проводятся до времени t , но голубой, если наблюдения проводятся после этого". В любое время, предшествующее t , человеком могли бы рассматриваться миллионы примеров, подтверждающих правило, что цвета изумрудов изменчивы, и ни одного опровергающего примера, и тем не менее не была бы выражена готовность принять это правило. Такое свойство человеческой психики можно объяснить только с помощью анализа той роли, которую играют в процессе индукции релевантные априорные знания. Гудман предложил рассматривать различные виды априорных знаний, которое могли бы оказаться применимыми в логическом выводе, включая одну из версий способа формулировки определений, называемую определением сверхгипотез. К сожалению, идеи Гудмана никогда не применялись в исследованиях по машинному обучению.

Истоками подхода к обучению на основе объяснения явились методы, используемые в планировщике *Strips* [465]. После формирования какого-то плана его обобщенная версия сохраняется в библиотеке планов и используется в дальнейшем планировании в качестве макрооператора. Аналогичные идеи просматриваются в архитектуре АСТ* Андерсона, где они выступают под названием компиляции знаний [31], и в архитектуре *Soar* под названием формирование фрагментов [881].

Непосредственными причинами быстрого повышения интереса к методам обучения на основе объяснения стало появление методов приобретения схемы [379], аналитического обобщения [1062] и обобщения на основе ограничений [1056], которые были подытожены в статьях [381] и [1065]. Хирш [659] предложил алгоритм EBL, описанный в данной книге, и показал, что этот алгоритм можно непосредственно включить в систему логического программирования. Ван Хармелен и Банди [1531] показали, что метод

обучения на основе объяснения представляет собой один из вариантов метода частичной оценки, используемого в системах анализа программ [742].

Проведенный в последнее время строгий анализ привел к лучшему пониманию потенциальных затрат и выгод, связанных с применением метода EBL, с точки зрения его влияния на скорость решения задач. Минтон [1057] показал, что использование метода EBL без значительных дополнительных усилий со стороны пользователя вполне может привести к существенному замедлению работы любой программы. Тамб и др. [1488] обнаружили, что аналогичная проблема возникает при

формировании фрагментов знаний, предназначенных для запоминания, и предложили способ уменьшения выразительной мощи языка правил, позволяющий свести к минимуму стоимость операции согласования правил с содержимым рабочей памяти. В этой работе обнаружилось явные аналогии с полученными недавно результатами анализа сложности логического вывода в ограниченных версиях логики первого порядка (см. главу 9). Формальный вероятностный анализ ожидаемых преимуществ использования метода EBL можно найти в [595] и [1471]. Превосходный обзор этого метода приведен в [396].

Вместо использования примеров в качестве материала для обобщения их можно применять непосредственно для решения новых задач в процессе, называемом $\wedge$ формированием рассуждений по аналогии. Такой метод формирования рассуждений осуществляется в нескольких вариантах, во-первых, в виде формирования приемлемых рассуждений с учетом степени подобия [540], во-вторых, в виде дедуктивного логического вывода, основанного на определениях, но требующего привлечения примеров [330], и, в-третьих, в виде метода EBL с "отложенным принятием решений", который позволяет корректировать направление обобщения старого примера с учетом потребностей решения новой задачи. Последняя разновидность процесса формирования рассуждений по аналогии наиболее часто встречается в системах формирования рассуждения на основе конкретных случаев [830] и системах создания деривационной аналогии [1540].

Подход к использованию информации о релевантности в форме функциональных зависимостей был впервые разработан в сообществе пользователей баз данных, где он служит для структуризации больших множеств атрибутов на более легко управляемые подмножества. Функциональные зависимости применялись для формирования рассуждений по аналогии Карбонеллом и Коллинзом [222], а их трактовка, более подходящая для использования в логике, была предложена Бобровым и Рафаэлем [144]. Зависимости были повторно открыты независимо и подверглись всестороннему логическому анализу в работах Дэвиса и Рассела [329], [330]. Кроме того, зависимости использовались в методе декларативного смещения Расселом и Грософом [1327]. Эквивалентность таких понятий, как определения и пространства гипотез с ограниченным словарем, была доказана в [1323]. Возможность применения алгоритмов обучения к определениям и достижения повышенной производительности с помощью метода RBDTL впервые была показана на примере алгоритма Focus в [20]. Тадепалли [1485] описал остроумный алгоритм обучения на основе определений, который демонстрирует существенное увеличение скорости обучения.

Идею о том, что индуктивное обучение может осуществляться по методу обратной дедукции, можно проследить до работы В.С. Джевонса [736], который писал:

"Изучение формальной логики и теории вероятностей привело меня к тому, что я стал приверженцем мнения, что не существует такой вещи, как отдельный метод индукции, противопоставляемый дедукции, но индукция является просто обратным воплощением дедукции". Исследования в области вычислительных алгоритмов начались с замечательных тезисов докторской диссертации Гордона Плоткина [1216], опубликованных в Эдинбурге. Хотя Плоткин разработал множество теорем и методов, которые в настоящее время находят применение в индуктивном логическом программировании, он был разочарован некоторыми результатами, показывающими неразрешимость определенных подзадач методами индукции. В системе MIS [1396] снова была предпринята попытка решения задачи изучения логических программ, но самым автором эта работа в основном рассматривалась как вклад в теорию автоматизированной отладки. Работа в области индуктивного вывода правил, подобная проведенной при создании систем ЮЗ [1257] и CN2 [263], привела к появлению системы Foil [1258], которая впервые позволила осуществлять на практике методы индуктивного вывода реляционных правил. Исследования в области реляционного обучения получили новый стимул после публикации работы Магглтона и Бантайна [1100], в которой была описана программа Cigol, реализующая немного неполную версию метода обратной резолюции, но способную вырабатывать новые предикаты⁵. Задачи изобретения предикатов рассматриваются также в [1604].

Следующей важной системой стала Golem [1102], в которой используется алгоритм формирования покрытия, основанный на выдвинутом Плоткиным понятии относительного наименьшего общего обобщения. Тогда как система Foil была нисходящей, системы Cigol и Golem действовали в восходящем направлении. К примерам других систем, появившихся в тот период, относятся Itou [1312] и Clint [352]. В разработанной в дальнейшем системе Progol [1098] был принят гибридный (нисходящий и восходящий) подход к формированию обратного логического следствия; эта система применялась для решения целого ряда практических задач, особенно в области биологии и обработки естественного языка. В [1099] описано расширение системы Progol, позволяющее учитывать неопределенность, в форме стохастических логических программ.

Формальный анализ методов ILP приведен в [1096], большая подборка статей представлена в [1097], а описания методов и приложений собраны в [897]. В [1163] можно найти более современный обзор истории развития этой области и описание проблем, которые должны быть решены в будущем. Первые результаты анализа сложности, опубликованные в [632], свидетельствовали о том, что задача изучения высказываний в логике первого порядка является безнадежно сложной. Однако благодаря лучшему пониманию важности синтаксических ограничений различного рода, налагаемых на выражения, удалось получить положительные результаты даже

для выражений с рекурсией [426]. Результаты изучения возможности обучения для методов ILP изложены в [279] и [794].

Несмотря на то что в настоящее время индуктивное логическое программирование, по-видимому, является доминирующим подходом к решению задач конструктивной индукции, это — не единственный опробованный подход. Разработаны так называемые $\wedge$ системы открытий, которые направлены на моделирование процесса научного открытия новых понятий, обычно путем прямого поиска в пространстве определений понятий. В системе AM, или Automated Mathematician (автоматизированный математик) Метод обратной резолюции опубликован также в [1322], где в одной из сносок приведен простой алгоритм.

тик), Дуга Лената [337] использовались эвристики открытий, выраженные в виде правил экспертной системы для управления осуществляемым в ней поиском понятий и научных предположений в области элементарной теории чисел. В отличие от большинства систем, предназначенных для формирования математических рассуждений, в системе AM отсутствовало понятие доказательства, поэтому она могла выдвигать только предположения. Эта система повторно открыла предположение Гольдбаха и теорему уникального разложения на простые множители. Архитектура системы AM была обобщена в системе Eurisko [907] в результате введения механизма, способного перезаписывать собственные эвристики открытия этой системы. Система Eurisko применялась во многих областях, отличных от области поиска математических открытий, хотя и с меньшим успехом, чем AM. Но методология систем AM и Eurisko оказалась противоречивой [909], [1292].

Еще один класс систем открытия предназначен для обработки реальных научных данных в целях поиска новых законов. Системы Dalton, Glauber и Stahl [885] представляют собой системы на основе правил, предназначенные для поиска количественных связей в экспериментальных данных, полученных из физических систем; в каждом случае системы показали свою способность снова выдвинуть широко известное открытие из истории науки. Системы открытия, основанные на вероятностных методах (в частности, алгоритмы кластеризации, позволяющие обнаруживать новые категории), рассматриваются в главе 20.

Упражнения 19.1. Преобразовав выражение в конъюнктивную нормальную форму и применив метод резолюции, покажите, что приведенное на с. 923 заключение, касающееся бразильцев, является обоснованным.

19.2. Для каждого из следующих определений запишите логическое представление и объясните, почему это определение является истинным (если его действительно можно считать таковым). а) Название штата (в США) можно определить по почтовому индексу. б) Масса монеты зависит от ее проекта и номинала. б)

Применительно к конкретной программе выходные данные определяются входными данными. в) Изменение массы тела в большую или меньшую сторону происходит под влиянием климата, рациона питания, физической нагрузки и интенсивности обмена веществ. г) Наличие лысины у мужчины определяется тем, была ли лысина у деда со стороны матери.

19.3. Могла бы найти применение вероятностная версия определения? Предложите приемлемую трактовку такого понятия.

19.4. Подставьте недостающие значения выражения C_1 или C_2 (или обоих выражений) в приведенный ниже ряд выражений, при условии, что C — резольвента C_1 и C_2 . а) $C = \text{True} \Rightarrow P(A,B), C_1 = P(x,y) \Rightarrow Q(x,y), C_2 = ??$. б) $C = \text{True} \Rightarrow P(A,B), C_1 = ??, C_2 = ??$.

в) $C = P(x,y) \Rightarrow P(x, f(y)), C_1 = ??, C_2 = ??$. • Если имеется больше одного возможного решения, приведите по одному примеру каждого из различных решений.

19.5. ё) Предположим, что разрабатывается логическая программа, которая осуществляет один из этапов логического вывода по методу резолюции. Это означает, что такая программа, $\text{Resolve}(clf\ c_1\ c_2\ c)$, завершается успешно, если c — результат применения операции резолюции к выражениям c_1 и c_2 . При обычных обстоятельствах программа Resolve используется в составе системы автоматического доказательства теорем, в которой она вызывается с параметрами c_1 и c_2 , конкретизированными значениями некоторых выражений, и возвращает значение резольвенты c . А теперь предположим, что вместо этого данная программа вызывается с конкретизированным значением c и неконкретизированными значениями c_1 и c_2 . Приведет ли это к успешному получению приемлемых результатов на этапе обратной резолюции?

Потребуется ли внести какие-либо специальные изменения в рассматриваемую систему логического программирования для того, чтобы можно было применять программу в такой форме?

19.6. Предположим, что в системе Foil рассматривается задача введения литерала в выражение с использованием бинарного предиката p и что предыдущие литералы (включая голову выражения) содержат пять разных переменных. а) Какое количество функционально различных литералов может быть сформировано? Два литерала являются функционально идентичными, если они отличаются только по именам содержащихся в них новых переменных. б) Можете ли вы найти общую формулу для количества различных литералов с предикатом, имеющим n -арность p , если ранее использовались k переменных? в) Почему в системе Foil не допускается введение литералов, которые не содержат ранее использованных переменных?

19.7. Используя данные из генеалогического дерева, приведенного на рис. 19.8, или подмножество этих данных, примените алгоритм Foil для изучения определения предиката Ancestor.

Глава 20. Статистические методы обучения

20 Статистические В данной главе рассматривается обучение как способ формирования рассуждений в условиях неопределенности на основании результатов наблюдений.

В главах части V было указано, что в реальных вариантах среды ситуации преимущественно являются неопределенными. Агенты могут справиться с неопределенностью, используя методы теории вероятностей и теории решений, но вначале должны сформировать в процессе обучения на основании полученного опыта свои вероятностные теории о мире. В этой главе показано, как может быть достигнута эта цель. В ней описано, как сформулировать саму задачу обучения в виде процесса вероятностного вывода (раздел 20.1). Материал, изложенный в данной главе, свидетельствует о том, что байесовский подход к обучению является чрезвычайно мощным и предоставляет общие решения проблем шума, чрезмерно тщательной подгонки и оптимального предсказания. В настоящей главе учитывается также тот факт, что агент, не являющийся полностью всезнающим, не будет никогда уверен в том, что та или иная теория о мире действительно является правильной, но все равно обязан принимать решения, используя некоторую теорию о мире.

В разделах 20.2 и 20.3 описаны методы обучения вероятностных моделей (в основном байесовских сетей). В разделе 20.4 рассматриваются методы обучения, предусматривающие сохранение и извлечение из памяти конкретных экземпляров примеров. В разделе 20.5 описано обучение нейронных сетей, а в разделе 20.6 даны вводные сведения о ядерных машинах. Часть материала этой главы имеет богатое математическое содержание (и для ее освоения требуются элементарные знания в области многомерного исчисления), хотя основные приведенные здесь научные результаты можно понять, не углубляясь в детали. При изучении этого материала читателю может потребоваться еще раз просмотреть главы 13 и 14 и ознакомиться с математическими сведениями, приведенными в приложении А.

20.1. Статистическое обучение Основными понятиями в данной главе, как и в главе 18, являются данные и гипотезы. Но в этой главе данные рассматриваются как свидетельства, т.е. конкретизации некоторых или всех случайных переменных, описывающих проблемную область, а гипотезы представляют собой вероятностные теории того, как функционирует проблемная область, включающие логические теории в качестве частного случая.

Рассмотрим очень простой пример. Наши любимые леденцы "Сюрприз" выпускаются в двух разновидностях: вишневые (сладкие) и лимонные (кислые). У

изготовителя леденцов особое чувство юмора, поэтому он заворачивает каждую конфету в одинаковую непрозрачную бумагу, независимо от разновидности. Леденцы продаются в очень больших пакетах (также внешне не различимых), о которых известно, что они относятся к пяти следующим типам: h_1 : 100% вишневых леденцов h_2 : 75% вишневых + 25 % лимонных леденцов h_3 : 50% вишневых + 50 % лимонных леденцов h_4 : 25% вишневых + 75 % лимонных леденцов h_5 : 100 % лимонных леденцов. Получив новый пакет леденцов, любитель конфет пытается угадать, к какому типу он относится, и обозначает тип пакета случайной переменной Y (сокращение от hypothesis — гипотеза), которая имеет возможные значения от h_1 до h_5 . Безусловно, значение переменной Y невозможно определить с помощью непосредственного наблюдения. По мере развертывания и осмотра конфет регистрируются данные о них, $? > 1$, $? > 2$, ..., A^* , где каждый элемент данных, d , представляет собой случайную переменную с возможными значениями cherry (вишневый леденец) и lime (лимонный леденец). Основная задача, стоящая перед агентом, состоит в том, что он должен предсказать, к какой разновидности относится следующая конфета¹.

Несмотря на кажущуюся простоту, постановка этой задачи позволяет ознакомиться с многими важными темами. В действительности агент должен вывести логическим путем теорию о мире, в котором он существует, хотя и очень простую.

В $\wedge$ байесовском обучении исходя из полученных данных просто вычисляется вероятность каждой гипотезы и на этой основе делаются предсказания. Это означает, что предсказания составляются с использованием всех гипотез, взвешенных по их вероятностям, а не с применением только одной "наилучшей" гипотезы. Таким образом, обучение сводится к вероятностному выводу. Допустим, что переменная D представляет все данные, с наблюдаемым значением d ; в таком случае вероятность каждой гипотезы может быть определена с помощью правила Байеса:

$P(h_i|d) = \frac{1}{Z} P(d|h_i) P(h_i)$ B0.1) Теперь предположим, что необходимо сделать предсказание в отношении неизвестного количества X . В таком случае применяется следующее уравнение:

¹ Читатели, искушенные в статистике, узнают в этом сценарии один из вариантов постановки задачи с урнами и шарами. Но авторы считают урны и шары менее привлекательными, чем леденцы; более того, с самими леденцами можно также связать и другие задачи, например задачу принятия решения о том, есть ли смысл перепродать какой-то пакет конфет своему другу (см. упр. 20.3).

$-E P(x|d) = \sum_x P(x|d, h_i) P(h_i|d) = \sum_x P(x|h_i) P(h_i|d)$ B0.2) где предполагается, что каждая гипотеза определяет распределение вероятностей по x . Это уравнение показывает, что предсказания представляют собой взвешенные средние по предсказаниям

наблюдении обнаруживается лимонный леденец (a); байесовские предсказания $P(d_{N+1}=l \mid d_1, \dots, d_N)$, полученные из уравнения 20.2 (б) Этот пример показывает, что c^* истинная гипотеза в конечном итоге будет доминировать над байесовским предсказанием. В этом состоит характерная особенность байесовского обучения. При любом заданном распределении априорных вероятностей, которое не исключает с самого начала истинную гипотезу, апостериорная вероятность любой ложной гипотезы в конечном итоге полностью исчезает просто потому, что вероятность неопределенно долгого формирования "нехарактерных" данных исчезающе мала (сравните это замечание с аналогичным замечанием, сделанным при обсуждении PAC-обучения в главе 18). Еще более важно то, что байесовское предсказание является оптимальным, независимо от того, применяется ли большой или малый набор данных. При наличии распределения априорных вероятностей гипотезы все другие предсказания будут правильными менее часто.

Но за оптимальность байесовского обучения, безусловно, приходится платить.

В реальных задачах обучения пространство гипотез обычно является очень большим или бесконечным, как было показано в главе 18. В некоторых случаях операция вычисления суммы в уравнении 20.2 (или, в непрерывном случае, операция интегрирования) может быть выполнена успешно, но в большинстве случаев приходится прибегать к приближенным или упрощенным методам.

Один из широко распространенных приближенных подходов (из числа тех, которые обычно применяются в научных исследованиях) состоит в том, чтобы делать предсказания на основе единственной наиболее вероятной гипотезы, т.е. той гипотезы h_i , которая максимизирует значение $P(h_i \mid d)$. Такую гипотезу часто называют $\hat{h}$ максимальной апостериорной гипотезой, или сокращенно MAP (Maximum A Posteriori; произносится "эм-эй-пи"). Предсказания $\hat{h}$, сделанные на основе MAP-гипотезы, являются приближенно байесовскими до такой степени, что $P(x \mid d) = P(x \mid \text{LMAP}) \cdot P(\text{LMAP} \mid d)$. В рассматриваемом примере с конфетами $\text{LMAP} = h_5$ после обнаружения трех лимонных леденцов подряд, поэтому агент, обучающийся с помощью MAP-гипотезы, после этого предсказывает, что четвертая конфета представляет собой лимонный леденец, с вероятностью 1.0, а это — гораздо более радикальное предсказание, чем байесовское предсказание вероятности 0.8, приведенное на рис. 20.1.

По мере поступления дополнительных данных предсказания с помощью MAP-гипотезы и байесовские предсказания сближаются, поскольку появление гипотез, конкурирующих с MAP-гипотезой, становится все менее и менее вероятным. Хотя в рассматриваемом примере это не показано, поиск MAP-гипотез часто бывает намного проще по сравнению с байесовским обучением, поскольку требует решения задачи оптимизации, а не задачи вычисления большой суммы (или

интегрирования). Примеры, подтверждающие это замечание, будут приведены ниже в данной главе.

И в байесовском обучении, и в обучении с помощью MAP-гипотез важную роль играет распределение априорных вероятностей гипотезы $P(h_i)$. Как было показано в главе 18, если пространство гипотез является слишком выразительным, в том смысле, что содержит много гипотез, хорошо согласующихся с набором данных, то может происходить чрезмерно тщательная подгонка. С другой стороны, байесовские методы обучения и методы обучения на основе MAP-гипотез не налагают произвольный предел на количество подлежащих рассмотрению гипотез, а позволяют использовать распределение априорных вероятностей для наложения штрафа за сложность. Как правило, более сложные гипотезы имеют более низкую априорную вероятность, отчасти потому, что сложных гипотез обычно бывает намного больше, чем простых. С другой стороны, более сложные гипотезы имеют большую способность согласовываться с данными (в крайнем случае какая-то поисковая таблица может оказаться способной точно воспроизводить данные с вероятностью 1).

Поэтому в распределении априорных вероятностей гипотезы воплощен компромисс между сложностью гипотезы и степенью ее согласования с данными.

Влияние такого компромисса можно наблюдать наиболее наглядно в случае использования логических гипотез, когда переменная Y содержит только детерминированные гипотезы. В таком случае значение $P(d|h_i)$ равно 1, если гипотеза h_i согласуется с данными, и 0 — в противном случае. Рассматривая уравнение 20.1, можно определить, что ЪШР в таких условиях представляет собой $\hat{c}$ = простейшую логическую теорию, согласованную с данными. Поэтому обучение с помощью максимальной апостериорной гипотезы представляет собой естественное воплощение принципа бритвы Оккама.

Еще один способ анализа компромисса между сложностью и степенью согласованности состоит в том, что можно исследовать уравнение 20.1, взяв его логарифм.

Применение значения h^p для максимизации выражения $P(d|h_i) P(h_i)$ эквивалентно минимизации следующего выражения: $-\log_2 P(d|h_i) - \log_2 P(h_i)$. Используя связь между информационным содержанием и вероятностью, которая была описана в главе 18, можно определить, что терм $-\log_2 P(h_i)$ определяет количество битов, требуемых для задания гипотезы h_i . Кроме того, терм $-\log_2 P(d|h_i)$ представляет собой дополнительное количество битов, требуемых для задания данных, если дана рассматриваемая гипотеза (чтобы убедиться в этом, достаточно отметить, что если гипотеза точно предсказывает данные, как в случае гипотезы h_5 и сплошного ряда конфет с лимонными леденцами, не требуется ни одного бита, поскольку $1 \log_2 1 = 0$).

Таким образом, обучение с помощью MAP-гипотезы равносильно выбору гипотезы, которая обеспечивает максимальное сжатие данных.

Такую же задачу можно решить более прямо с помощью метода обучения на основе $\wedge$ минимальной длины описания, или сокращенно MDL (Minimum Description

Length), в котором вместо манипуляций с вероятностями предпринимаются попытки минимизировать размер гипотезы и закодированного представления данных.

Окончательное упрощение может быть достигнуто путем принятия предположения о равномерном распределении априорных вероятностей по пространству гипотез. В этом случае обучение с помощью MAP-гипотезы сводится к выбору гипотезы h_i которая максимизирует значение $p(d | H_i)$. Такая гипотеза называется гипотезой с $\wedge$ максимальным правдоподобием (Maximum Likelihood — ML) и сокращенно обозначается hML. Обучение на основе гипотезы с максимальным правдоподобием очень широко применяется в статистике, поскольку в этой научной области многие исследователи не доверяют распределениям априорных вероятностей гипотезы, считая, что они имеют субъективный характер. Это — приемлемый подход, применяемый в тех обстоятельствах, когда нет оснований априорно отдавать предпочтение одной гипотезе перед другой, например, в тех условиях, когда все гипотезы являются в равной степени сложными. Такой метод обучения становится хорошей аппроксимацией байесовского обучения и обучения с помощью MAP-гипотезы, когда набор данных имеет большие размеры, поскольку данные сами исправляют распределение априорных вероятностей по гипотезам, но связан с возникновением определенных проблем (как будет показано ниже) при использовании небольших наборов данных.

20.2. Обучение с помощью полных данных Начнем разработку методов статистического обучения с простейшей задачи — $\wedge$ обучение параметрам с помощью $\wedge$ полных данных. Задача обучения параметрам сводится к поиску числовых параметров для вероятностной модели, имеющей фиксированную структуру. Например, может потребоваться определить в процессе обучения условные вероятности в байесовской сети с заданной структурой. Данные называются полными, если каждая точка данных содержит значения для каждой переменной в вероятностной модели, применяемой при обучении. При наличии полных данных задача определения в процессе обучения параметров сложной модели значительно упрощается. Кроме того, в данном разделе кратко рассматривается задача изучения структуры.

Обучение параметрам с помощью метода максимального правдоподобия: дискретные модели Допустим, что мы покупаем пакет конфет с лимонными и вишневыми леденцами, выпущенный новым изготовителем, соотношение

лимонных и вишневых леденцов в продукции которого полностью неизвестно; это означает, что доля тех и других леденцов может измеряться любым значением от 0 до 1. В данном случае приходится рассматривать континуум гипотез. Кроме того, в этом случае параметром, который будет обозначаться как θ , является доля вишневых леденцов, а гипотезой является h_0 (доля лимонных леденцов выражается как $1-\theta$). Если принято предположение, что все возможные значения долевого состава априорно являются равновероятными, то становится обоснованным подход на основе гипотезы с максимальным правдоподобием. Если мы промоделируем эту ситуацию с помо-

щью байесовской сети, то потребуется только одна случайная переменная, Flavor (разновидность конфеты, случайно выбранной из пакета). Эта переменная принимает значения cherry и lime, где вероятность cherry равна θ (рис. 20.2, а). Теперь предположим, что развернуто iV конфет, из которых s оказались вишневыми леденцами, а $n-s$ были лимонными леденцами. Согласно уравнению 20.3, правдоподобие этого конкретного набора данных выражается следующей формулой:

$$P(d|h) = \prod_{j=1}^N P(d_j|h) = \theta^s (1-\theta)^{n-s}$$
 Гипотеза с максимальным правдоподобием задается значением θ , которое максимизирует это выражение. Такое же значение может быть получено путем максимизации значения $\log$ логарифмического правдоподобия:

$$L(d|h) = \log P(d|h) = \sum_{j=1}^N \log P(d_j|h) = \sum_{j=1}^N \log \theta^{d_j} (1-\theta)^{1-d_j} = s \log \theta + (n-s) \log (1-\theta)$$
 (Взяв логарифмы, мы преобразовали произведение в сумму по данным, которую обычно легче максимизировать.) Чтобы найти значение максимального правдоподобия θ , дифференцируем L по θ и приравняем полученное выражение к нулю следующим образом: $\frac{dL}{d\theta} = \frac{s}{\theta} - \frac{n-s}{1-\theta} = 0$ $\Rightarrow \theta = \frac{s}{n}$ Таким образом, если описать это выражение на естественном языке, то гипотеза с максимальным правдоподобием h_{ML} утверждает, что фактическая доля вишневых леденцов в пакете равна наблюдаемой доле этих леденцов в конфетах, развернутых до сих пор!

На первый взгляд создается впечатление, что мы проделали большой объем работы лишь для того, чтобы открыть этот очевидный факт. Но в действительности описанным выше путем был создан один из стандартных методов обучения параметрам с максимальным правдоподобием, который описан ниже.

1. Записать выражение для правдоподобия данных как функции от параметра (параметров).
2. Найти производную логарифмического правдоподобия по отношению к каждому параметру.
3. Найти такие значения параметров, чтобы производные стали равными нулю.

Обычно самым сложным является последний из этих этапов. В рассматриваемом примере он был простейшим, но ниже будет показано, что во многих случаях приходится прибегать к использованию итерационных алгоритмов поиска решений или других числовых методов оптимизации, как было описано в главе 4. Кроме того, этот пример иллюстрирует важную проблему, которая в целом характерна для обучения с учетом максимального правдоподобия: $s^i =$ если набор данных достаточно мал и поэтому некоторые события еще не наблюдались (например, не было обнаружено ни одной конфеты с вишневым леденцом), то гипотеза с максимальным правдоподобием

одной конфеты с вишневым леденцом), то гипотеза с максимальным правдоподобием присваивает этим событиям нулевую вероятность. Для предотвращения возникновения этой проблемы использовались различные приемы, такие как инициализация счетчиков для каждого события значением 1, а не 0.

Рассмотрим еще один пример. Предположим, что этот новый изготовитель конфет хочет дать потребителю небольшую подсказку и использует для конфет обертки красного и зеленого цветов. Значение переменной *Wrapper*, соответствующей цвету обертки для каждой конфеты, выбирается по вероятностным законам, в соответствии с некоторым неизвестным условным распределением, но в зависимости от разновидностей конфет. Соответствующая вероятностная модель показана на рис. 20.2, б. Обратите внимание на то, что она имеет три параметра: 0, 0X и 02. С использованием этих параметров правдоподобие события, связанного, скажем, с обнаружением вишневого леденца в зеленой обертке, можно определить на основе стандартной семантики для байесовских сетей (с. 664):

$P(\text{Flavor}^{\wedge}\text{cherry}, \text{Wrapper}=\text{green} \mid h_0 \text{ie } e_2) = P\{\text{Flavor}=\text{cherry} \mid h_0, 0 \text{ e } \} P(\text{Wrapper}=\text{green} \mid \text{Flavor}=\text{cherry}, h_0, e_2) = e^{-(1-G_1)} P(F=\text{cherry}) P(F=\text{cherry} \mid \text{Flavor}=\text{lime}) P(W=\text{red} \mid F=\text{cherry})$ а) б) Рис. 20.2. Обучение параметром с помощью байесовской сети: модель в виде байесовской сети для случая, в котором доля вишневых и лимонных леденцов в пакете неизвестна (а); модель для того случая, когда цвета обертки связаны (вероятностной) зависимостью с разновидностями конфет (б) Теперь допустим, что развернуто N конфет, из которых c оказались вишневыми леденцами, а ? — лимонными, а количество оберток оказалось таковым: gc вишневых леденцов имели красные обертки, а dc — зеленые, тогда как g(лимонных леденцов имели красные обертки, а d(— зеленые. Правдоподобие этих данных выражается следующим образом: $p(d \mid h, e, e_2) = e^{c(A-e)} \cdot e^{g^*:(i-e_i)gc} \cdot e^{i(i-e_2)9}$ На первый взгляд это соотношение кажется весьма сложным, но его можно упростить, взяв логарифмы, следующим образом:

Глава 29. Статистические методы обучения $L = [clogB + ПодА-9)] + [rclog9i + gclog(l-9i)] + [гДод92 + дДодА-92)]$ Преимущество взятия логарифмов является очевидным

— логарифмическое правдоподобие представляет собой сумму трех термов, каждый из которых содержит единственный параметр. После взятия производных по каждому параметру и приравнивания их к нулю будет получено три независимых уравнения, каждое из которых содержит только один параметр: $\frac{\partial}{\partial \theta} \log P(\mathcal{D}) = 0$. Решение для θ остается таким же, как и прежде. Решение для θ_1 вероятности того, что вишневый леденец имеет красную обертку, представляет собой наблюдаемую долю вишневых леденцов в красных обертках, и аналогичным образом определяется решение для θ_2 .

Эти результаты являются очень удобными, и легко показать, что их можно распространить на любую байесовскую сеть, условные вероятности в которой представлены в виде таблицы. Наиболее важный вывод состоит в том, что $\hat{\theta}$ при наличии полных данных задана обучения параметрам с максимальным правдоподобием для байесовской сети декомпонуется на отдельные задачи обучения, по одной для каждого параметра. Еще один важный вывод состоит в том, что значения параметра для любой переменной при наличии ее родительских значений представляют собой наблюдаемые частоты значений переменных для каждого набора родительских значений. Как и прежде, необходимо внимательно следить за предотвращением появления нулевых значений, если набор данных является небольшим.

Наивные байесовские модели По-видимому, к числу моделей на основе байесовской сети, которые наиболее широко используются в машинном обучении, относятся наивные байесовские модели. В таких моделях переменная "класса" C (значение которой должно быть предсказано) задана в корневом узле, а переменные "атрибутов" X_i заданы в листовых узлах. Такие модели называются "наивными", поскольку в них предполагается, что атрибуты являются условно независимыми друг от друга, если определен рассматриваемый класс (модель, приведенная на рис. 29.2, б, представляет собой наивную байесовскую модель только с одним атрибутом). При условии, что переменные являются булевыми, рассматриваемые параметры принимают такой вид:

$\theta = P(\text{true})$, $\theta_c = P(X_i = \text{true} | C = \text{true})$, $\theta_{i2} = P(X_i = \text{true} | C = \text{false})$ Значения параметров с максимальным правдоподобием можно найти с помощью точно такого же способа, который применялся в сети на рис. 29.2, б. Сразу после 3 Случай, в котором отсутствуют таблицы условных вероятностей и каждый параметр влияет на несколько условных вероятностей, рассматривается в упр. 20.7.

обучения данной модели с помощью такого способа она может использоваться для классификации новых примеров, в которых переменная класса C является ненаблюдаемой. При наличии значений наблюдаемых атрибутов $x_1, \dots, x_p$ вероятность каждого класса определяется следующим соотношением:

$P(C|x_i, x_p) = a P(C) + P(x_i|C)$ Детерминистическое предсказание может быть получено путем выбора наиболее вероятного класса. На рис. 20.3 показана кривая обучения для этого метода, соответствующая примеру его применения к задаче с рестораном, описанной в главе 18.

Обучение с помощью этого метода происходит довольно успешно, но не так хорошо, как при обучении деревьев решений; следует полагать, это связано с тем, что истинная гипотеза (представляющая собой дерево решений) не является точно представимой с помощью наивной байесовской модели. Как оказалось, метод наивного байесовского обучения действует удивительно успешно в самых разнообразных приложениях, а его усиленная версия (упр. 20.5) является одним из наиболее эффективных алгоритмов обучения общего назначения. Метод наивного байесовского обучения хорошо масштабируется на очень большие задачи: при наличии p булевых атрибутов имеется только $2p+1$ параметров и s^p для обнаружения наивной байесовской гипотезы с максимальным правдоподобием, h_{ML} , не требуется поиск. Наконец, метод наивного байесовского обучения не сталкивается с затруднениями при обработке зашумленных данных и может предоставить вероятностные предсказания, когда это необходимо.

0,9 0,8 0,7 -1 S. & 0,6 \ 0,5 0,4 Дерево решений | Наивная байесовская сеть | 0 20 40 60 80 100 Объем обучающего множества Рис. 20.3. Кривая обучения для случая применения метода наивного байесовского обучения к задаче с рестораном из главы 18; для сравнения показана кривая обучения для случая применения метода обучения дерева решений Обучение параметрам с максимальным правдоподобием: непрерывные модели Непрерывные вероятностные модели, такие как линейная гауссова модель, описывались в разделе 14.3. Поскольку в реальных приложениях в основном используются непрерывные переменные, важно знать, как должно осуществляться обучение

непрерывных моделей на основе данных. Принципы обучения с максимальным правдоподобием идентичны применяемым в дискретном случае.

Начнем с очень простого случая: обучение параметрам гауссовой функции плотности от одной переменной. Это означает, что данные вырабатываются следующим образом:

$(X, Y) \sim P(X, Y)$ Параметрами этой модели являются математическое ожидание μ и среднеквадратичное отклонение σ (обратите внимание на то, что нормализующая "константа" зависит от σ , поэтому ее нельзя игнорировать). Допустим, что наблюдаемыми значениями являются $x_1, \dots, x_N$. В таком случае логарифмическое правдоподобие определяется следующим образом:

$N \log \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2\right) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2$ Приравняв, как обычно, производные к нулю, получим такие уравнения:

$(x_j - \mu) = 0 \Rightarrow \sigma^2 = (X_j - \mu)^2 / N$ (В.4) Таким образом, значение максимального правдоподобия среднего представляет собой среднее по выборкам, а значение максимального правдоподобия среднеквадратического отклонения выражается квадратным корнем от дисперсии выборки. И в данном случае получены удобные результаты, которые подтверждают обоснованность практических методов, созданных на основе "здравого смысла".

Теперь рассмотрим линейную гауссову модель с одним непрерывным родительским значением X и непрерывным дочерним значением Y . Как было описано на стр. 672, значение Y имеет гауссово распределение, математическое ожидание которого линейно зависит от значения x , а среднеквадратическое отклонение является постоянным. Чтобы определить в результате обучения распределение условных вероятностей $P(Y|X)$, можно максимизировать условное правдоподобие следующим образом: $P(y|x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(y - (\theta_0 + \theta_1 x))^2}{2\sigma^2}}$ (В.5) где параметрами являются θ_0 и θ_1 . Данные представляют собой множество пар (X_j, y_j) , как показано на рис. 20.4. Используя обычные методы (упр. 20.6), можно

найти значения параметров с максимальным правдоподобием. Но в этом контексте нужно сделать еще одно замечание. Если рассматриваются только параметры θ_0 и θ_1 , которые определяют линейную связь между x и y , то становится очевидно, что максимизация логарифмического правдоподобия по отношению к этим параметрам равносильна минимизации числителя в экспоненте уравнения 20.5:

$E = \sum_{j=1}^n (y_j - (\theta_0 + \theta_1 x_j))^2$ (В.6) Рис. 20.4. Примеры применения линейной гауссовой модели: линейная гауссова модель, описанная как $y = \theta_0 + \theta_1 x$, к которой добавляется гауссов шум с постоянной дисперсией (σ^2); множество из 50 точек данных, сформированных с помощью этой модели (б) Величина $\{y_j - (\theta_0 + \theta_1 x_j)\}$ представляет собой ошибку для (x_j, y_j) , т.е. разность между фактическим значением y_j и прогнозируемым значением $\theta_0 + \theta_1 x_j$, поэтому E представляет собой хорошо известную сумму квадратичных ошибок.

Она является величиной, которую можно минимизировать с помощью стандартной процедуры линейной регрессии. Теперь можно понять, с чем это связано: минимизация суммы квадратичных ошибок позволяет получить линейную модель с максимальным правдоподобием, при условии, что данные вырабатывались с гауссовым шумом, имеющим постоянную дисперсию.

Обучение байесовским параметрам Метод обучения с максимальным правдоподобием может стать основой некоторых очень простых процедур, но обнаруживает определенные серьезные недостатки при работе с небольшими наборами данных. Например, после обнаружения одного вишневого леденца в этом методе вырабатывается гипотеза с максимальным правдоподобием, что данный пакет на 100% состоит из вишневых леденцов (т.е. $\theta = 1$).

Но если только принятое распределение априорных вероятностей гипотезы не сводится к тому, что в пакетах должны находиться лишь одни вишневые леденцы либо исключительно лимонные леденцы, то такое заключение является необоснованным.

В байесовском подходе к обучению параметрам распределению априорных вероятностей гипотезы дается предпочтение над возможными значениями параметров, а само распределение обновляется по мере поступления данных.

В примере с конфетами, приведенном на рис. 20.2, я, имеется только один параметр, θ , — вероятность того, что случайно выбранная конфета относится к разновидности вишневых леденцов. С точки зрения байесовского подхода θ представляет собой (неизвестное) значение случайной переменной θ ; распределение априорных вероятностей гипотезы представляет собой распределение априорных вероятностей $P(\theta)$. Таким образом, $P(\theta)$ — это априорная вероятность того, что в пакете имеется доля θ вишневых леденцов.

Если параметр θ может иметь любое значение от 0 до 1, то $P(\theta)$ должно представлять собой непрерывное распределение, которое является ненулевым только между 0 и 1 и интеграл которого равен 1. Одним из потенциальных распределений, пригодных для этой роли, является распределение с равномерной плотностью $P(\theta) = \frac{1}{b-a}$ (θ) (см. главу 13). Как оказалось, распределение с равномерной плотностью является членом семейства β -распределений. Каждое бетараспределение определяется двумя β -гиперпараметрами a и b , такими, что справедливо следующее соотношение для θ в диапазоне значений $[0, 1]$:
$$P(\theta) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1}$$
 Константа нормализации a зависит от a и b (см. упр. 20.8). На рис. 20.5 показано, как выглядит это распределение при различных значениях a и b . Среднее значение этого распределения равно $a/(a+b)$, поэтому большие значения a показывают обоснованность убеждения, что θ ближе к 1, чем к 0. При больших значениях $a+b$ распределение становится более заостренным, что выражает большую уверенность в правильности значения θ . Таким образом, семейство β -распределений предоставляет удобный ряд возможностей выбора распределений априорных вероятностей гипотезы.

2, $b \in [2, 1.5]$ II СУ $\theta \in [0, 1]$ $\bullet$ $\theta^a (1-\theta)^b$ $\bullet$ $\Gamma(a+b) / (\Gamma(a)\Gamma(b))$ $\bullet$ $a, b \in [0.2, 0.4, 0.6]$ Параметр θ а) 0,8 Рис. 20.5. Примеры распределения $\beta(a, b)$ для различных значений $[a, b]$ 4 Они называются гиперпараметрами, поскольку параметризуют распределение по θ , который сам является параметром.

Кроме такой гибкости, семейство β -распределений обладает еще одним замечательным свойством: если переменная θ имеет распределение априорных вероятностей $\beta(a, b)$, то после наблюдения в любой точке данных распределение апостериорных вероятностей для θ также становится β -распределением.

Семейство бета-распределений называется сопряженным распределением априорных вероятностей для семейства распределений, относящихся к некоторой булевой переменной⁵. Рассмотрим, как применяется это свойство. Предположим, что наблюдается появление вишневого леденца; в таком случае имеет место следующее соотношение:

$P(D_i = \text{cherry}) = \frac{a}{a+b} P(D_i = \text{cherry} | \theta)$ $P(\theta) = a^a \cdot b^b \cdot \text{beta}[a,b](\theta) = a^a \cdot b^b \cdot \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1}$ Таким образом, после обнаружения вишневого леденца наращивается параметр a для получения нового распределения апостериорных вероятностей; аналогичным образом, после обнаружения лимонного леденца наращивается параметр b . Поэтому гиперпараметры a и b можно рассматривать как виртуальные счетчики, в том смысле, что распределение априорных вероятностей $\text{beta}[a, b]$ ведет себя точно так же, как если бы обучение начиналось с равномерного распределения априорных вероятностей $\text{beta}[1,1]$, после чего было фактически обнаружено $a-1$ вишневых леденцов и $b-1$ лимонных.

Изучая последовательность бета-распределений, соответствующих возрастающим значениям a и b , и поддерживая постоянные пропорции, можно наглядно продемонстрировать, как изменяется распределение апостериорных вероятностей по параметру θ по мере поступления новых данных. Например, предположим, что пакет с конфетами в действительности содержит 75% вишневых леденцов. На рис. 20.5,6 показана последовательность распределений $\text{beta}[3,1]$, $\text{beta}[6,2]$, $\text{beta}[30,10]$. Очевидно, что эта последовательность сходится к узкому пику вокруг истинного значения θ . Поэтому при наличии больших наборов данных процесс байесовского обучения постепенно сходится (по меньшей мере в данном случае) и позволяет получить такие же результаты, как и обучение с учетом максимального правдоподобия.

Сеть, показанная на рис. 20.2, б, имеет три параметра, θ_1 , θ_2 и θ_3 , где θ_1 — вероятность наличия красной обертки на вишневом леденце, а θ_2 — вероятность наличия красной обертки на лимонном леденце. Распределение априорных вероятностей байесовской гипотезы должно охватывать все три параметра; это означает, что необходимо задать распределение $P(\theta_1, \theta_2, \theta_3)$. Обычно предполагается, что соблюдается свойство независимости параметров, как показано ниже.

$P(\theta_1, \theta_2, \theta_3) = P(\theta_1)P(\theta_2)P(\theta_3)$ Согласно этому предположению, каждый параметр может иметь свое собственное бета-распределение, которое обновляется отдельно по мере поступления данных.

После того как была сформулирована идея, что неизвестные параметры могут быть представлены случайными переменными, такими как θ , из нее можно вывести естественное заключение, что эти параметры можно включить в саму байесовскую

5 К другим сопряженным распределениям априорных вероятностей относятся семейство распределений Дирихле для параметров дискретного многомерного распределения и семейство распределений нормальных-Висхарта для параметров гауссова распределения [109].

сеть. Для этого также потребуется сделать копии переменных, описывающих каждый экземпляр. Например, если проверены три леденца, то для их описания потребуются переменные $Flavor^1, Flavor^2, Flavor^3$, а также $Wrapper^1, Wrapper^2, Wrapper^3$. Параметрическая переменная θ_i определяет вероятность каждой переменной $Flavors$ $P(Flavor_i = cherry | \theta_i = 9) = 9$. Аналогичным образом, вероятности оберток зависят от θ_1 и θ_2 , например:

$P(Wrapper_i = red | Flavors = cherry, \theta_i = 9) = 9$ Теперь весь байесовский процесс обучения можно сформулировать как задачу вероятностного вывода в байесовской сети, имеющей соответствующую структуру (рис. 20.6). Предсказание, касающееся нового экземпляра примера, можно получить, добавляя к сети новые переменные экземпляра, с тем условием, что значения некоторых из них можно будет определять с помощью запросов. Такая формулировка процессов обучения и предсказания наглядно показывает, что для байесовского обучения не требуется задавать дополнительные "принципы обучения". Кроме того, это означает, что $\mathcal{L}$ в действительности существует лишь единственный алгоритм обучения, т.е. алгоритм вероятностного вывода для байесовских сетей.

Рис. 20.6. Байесовская сеть, которая соответствует байесовскому процессу обучения. Распределения апостериорных вероятностей для параметрических переменных θ_1, θ_2 и θ_3 можно определить путем вероятностного вывода на основании распределений апостериорных вероятностей этих параметрических переменных и свидетельств, касающихся переменных $Wrapper^1$ и $Flavor^1$. Определение путем обучения структур байесовских сетей До сих пор предполагалось, что структура байесовской сети задана, и мы просто пытаемся определить в процессе обучения ее параметры, тогда как структура самой

сети представляет основные причинные знания о проблемной области, которые часто может без особых затруднений сформулировать не только специалист, но даже неопытный пользователь. Но в некоторых случаях причинная модель может оказаться недоступной или стать предметом спора (например, некоторые корпорации долгое время утверждали, что курение не является причиной рака), поэтому важно понять, как может быть определена путем обучения структура байесовской сети на основе данных. В настоящее время алгоритмы структурного обучения находятся на начальном этапе развития, поэтому в данном разделе будет приведен лишь краткий обзор основных идей.

Наиболее очевидным подходом к решению этой задачи является поиск качественной модели. Эту работу можно начать с модели, не содержащей связей, и приступить к введению родительских узлов для каждого узла, согласуя параметры с помощью только что описанных методов и измеряя точность результирующей модели.

Еще один вариант состоит в том, что можно начать с исходного предположения о структуре и использовать поиск с восхождением к вершине или с эмуляцией отжига для внесения модификаций, возвращая параметры после каждого изменения в структуре. Модификации могут включать обращение, добавление или удаление дуг.

В этом процессе следует избегать появления циклов, поскольку во многих алгоритмах принято предположение, что для переменных задано упорядочение и что узел может иметь родительские узлы только среди тех узлов, которые присутствуют перед ним в этом упорядочении (точно так же, как и в процессе создания сети, описанном в главе 14). Для достижения полной общности необходимо также обеспечить поиск среди возможных упорядочений.

Существуют два альтернативных метода принятия решения о том, нужно ли прекратить поиск, поскольку обнаружена приемлемая структура. Первый из них предусматривает проверку того, действительно ли в данных удовлетворяются предположения об условной независимости, неявно заданные в этой структуре. Например, сам факт использования наивной байесовской модели для задачи с рестораном равносителен предположению о том, что справедливо приведенное ниже соотношение, поэтому можно проверить по самим данным, соблюдается ли это соотношение применительно к соответствующим условным частотам.

$P(\text{Fri/Sat, Bar} \setminus \text{willWait}) = P(\text{Fri/Sat} \setminus \text{WillWait}) P(\text{Bar} \setminus \text{willWait})$ Итак, даже если полученная структура описывает истинный причинный характер проблемной области, наличие статистических флуктуации в наборе данных означает, что это уравнение никогда не будет выполняться точно, поэтому необходимо выполнить подходящую статистическую проверку для определения того, есть ли достаточно весомые свидетельства нарушения гипотезы о независимости. Сложность результирующей сети будет зависеть от пороговых значений, используемых для этой проверки — чем строже проверка на независимость, тем больше связей будет введено в сеть и тем выше опасность чрезмерно тщательной подгонки.

Подход, более совместимый с идеями, изложенными в этой главе, состоит в определении того, до какой степени предложенная модель объясняет данные (в вероятностном смысле). Но необходимо соблюдать осторожность при выборе способа измерения этой степени. Если будет просто предпринята попытка найти гипотезу с максимальным правдоподобием, то в конечном итоге будет получена

полносвязная сеть, поскольку введение дополнительных родительских узлов для некоторого узла

не позволяет повысить его правдоподобие (см. упр. 20.9). Поэтому приходится каким-то образом вводить штраф за сложность модели. В подходе MAP (или MDL) просто вычитаются штрафные оценки из значений правдоподобия каждой структуры (после настройки параметров) до сравнения различных структур, а в байесовском подходе налагается совместное распределение априорных вероятностей на структуры и параметры. Но обычно количество структур, по которым должно быть выполнено суммирование, слишком велико (оно определяется суперэкспоненциальной зависимостью от количества переменных), поэтому большинство практиков используют алгоритм MCMC для формирования выборок по структурам.

Применение способа штрафования за сложность (с помощью либо методов MAP, либо байесовских методов) влечет за собой появление важной связи между оптимальной структурой и характером представления для распределений условных вероятностей в сети. При использовании табличных распределений значения штрафов за сложность для распределения вероятностей узла растут экспоненциально в зависимости от количества родительских узлов, а при использовании, скажем, распределений зашумленного ИЛИ они растут только линейно. Это означает, что обучение с помощью моделей зашумленного ИЛИ (или других компактно параметризованных моделей), как правило, приводит к созданию в результате обучения таких структур, в которых имеется больше родительских узлов, чем при обучении с помощью табличных распределений.

20.3. Обучение с помощью скрытых переменных: алгоритм EM Предыдущий раздел относился к полностью наблюдаемому случаю. Но многие реальные задачи характеризуются наличием скрытых переменных (иногда называемых ^Λлатентными переменными), которые не наблюдаются в данных, доступных для обучения. Например, истории болезни часто включают описания наблюдаемых симптомов, применяемого лечения и, возможно, результата лечения, но редко содержат сведения о непосредственном наблюдении за развитием самого заболевания! Очевидно, напрашивается вопрос: "Если ход развития заболевания не наблюдается, то почему бы не построить модель без него?" Ответ на этот вопрос можно найти на рис. 20.7, где показана небольшая фиктивная диагностическая модель для сердечного заболевания. Существуют три наблюдаемых фактора, способствующих или препятствующих развитию заболевания, и три наблюдаемых симптома (которые имеют слишком сложные медицинские названия, поэтому им присвоены условные обозначения). Предполагается, что каждая переменная имеет три возможных значения (например, none (отсутствующий), moderate (умеренный) и severe (серьезный)). Удаление скрытой переменной из сети, приведенной на рис. 20.7, а,

влечет за собой создание сети, показанной на рис. 20.7, б\ при этом общее количество параметров увеличивается с 78 до 708. Таким образом, $\wedge$ г скрытые переменные б В некоторых историях болезни содержатся сведения о диагнозе, предложенном лечащим врачом, но такой диагноз является причинным следствием симптомов, причиной которых, в свою очередь, стало заболевание.

позволяют резко уменьшить количество параметров, требуемых для определения байесовский сети. А такой факт, в свою очередь, позволяет резко уменьшить объем данных, необходимых для определения в процессе обучения рассматриваемых параметров. а) б) Рис. 20.7. Иллюстрация необходимости использования скрытых переменных: простая диагностическая сеть для сердечного заболевания, в которой предполагается наличие скрытой переменной; каждая переменная имеет три возможных значения и соответствующий ей узел обозначен количеством независимых параметров в распределении ее условной вероятности; общее количество параметров равно 78 (а); эквивалентная сеть с удаленным узлом HeartDisease. Обратите внимание на то, что переменные симптомов больше не являются условно независимыми, если даны значения их родительских переменных. Для этой сети требуется 708 параметров (б) Скрытые переменные важны, но их введение приводит к усложнению задачи обучения. Например, на рис. 20.7, а не показано наглядно, как найти в процессе обучения распределение условных вероятностей для переменной HeartDisease, если заданы ее родительские переменные, поскольку значение переменной HeartDisease в каждом случае не известно; такая же проблема возникает при поиске в процессе обучения распределений вероятностей для симптомов. В этом разделе описан алгоритм, называемый $\wedge$ ожиданием—максимизацией, или сокращенно EM (Expectation—Maximization), который позволяет решить эту проблему очень общим способом. Мы рассмотрим три примера, а затем приведем общее описание.

На первых порах складывается впечатление, что этот алгоритм действует как по волшебству, но как только будет достигнуто его интуитивное понимание, читатель сможет найти приложения для алгоритма EM в огромном спектре задач обучения.

Неконтролируемая кластеризация: определение в процессе обучения смешанных гауссовых распределений $\wedge$ Неконтролируемой кластеризацией называется задача выделения многочисленных категорий в коллекции объектов. Эта проблема — неконтролируемая, поскольку категориям не назначены метки. Например, предположим, что регистрируются спектры сотен тысяч звезд; звезды подразделяются на типы, которые можно определить по спектру, а если так оно и есть, то сколько таких типов и каковы их характеристики? Мы все знакомы с терминами наподобие "красный гигант" и "белый карлик", но звезды не носят эти надписи на своих шляпах, поэтому астрономы должны

выполнять неконтролируемую кластеризацию для выявления их категорий. К другим примерам относится выявление видов, родов, отрядов и других категорий в таксономии живых организмов, установленной Линнеем, а также создание естественных разновидностей для распределения по категориям обычных объектов (см. главу 10).

Неконтролируемая кластеризация начинается с данных. На рис. 20.8, а показано 500 точек данных, каждая из которых задает значения двух непрерывных атрибутов.

Точки данных могут соответствовать звездам, а атрибуты — интенсивностям спектра на двух определенных частотах. Затем необходимо понять, какого рода распределение вероятностей могло быть сформировано этими данными. Сама возможность кластеризации равносильна предположению, что данные сформированы с помощью $\wedge$ смешанного распределения P . Такое распределение имеет k $\wedge$ компонентов, каждый из которых представляет собой отдельное распределение. Точка данных формируется путем первоначального выбора компонента, а затем формирования выборки из этого компонента. Допустим, что этот компонент определяет случайная переменная C со значениями $1, \dots, k$; в таком случае смешанное распределение задается следующим выражением:

$P(x) = \sum_{i=1}^k P(C=i) P(x|C=i)$ где x относится к значениям атрибутов для точки данных. В случае непрерывных данных естественным вариантом выбора для распределений компонентов является многомерное гауссово распределение, что приводит к созданию семейства распределений, представляющего собой так называемое $\wedge$ смешанное гауссово распределение. Параметрами смешанного гауссова распределения являются $W_i = P(C=i)$ (вес каждого компонента), μ_i (математическое ожидание каждого компонента) и S_i (ковариация каждого компонента). На рис. 20.8, б показано смешанное гауссово распределение, состоящее из трех распределений; фактически это смешанное распределение было источником данных, приведенных на рис. 20.8, а.

0,8 0,6 0,4 0,2 0 0,2 0,4 0,6 0,8 а) 0,8 if 0,6 0,4 0,2 0 0,2 0,4 0,6 0,8 б) 0,8 4 / 0,6 \ 0,4 0,2]
OL.:© 0 0,2 0,4 0,6 0,8 1 в) Рис. 20.8. Смешанное гауссово распределение: 500 двумерных точек данных, показывающих наличие трех кластеров (а); модель смешанного гауссова распределения с тремя компонентами; веса компонентов (слева направо) равны 0.2, 0.3 и 0.5. Данные, приведенные на рис. 20.8, а, были сформированы с помощью этой модели (б); модель, реконструированная из данных, приведенных на рис. 20.8, а, с помощью алгоритма EM (в)

Таким образом, задача неконтролируемой кластеризации состоит в восстановлении модели смешанного распределения, подобной приведенной на рис. 20.8, в, из исходных данных, таких как показано на рис. 20.8, а. Очевидно, что если бы мы знали, с помощью какого компонента сформирована каждая точка данных, то было бы

легко восстановить гауссово распределение для этого компонента, поскольку можно было просто выбрать все точки данных, соответствующие такому компоненту, а затем применить уравнение 20.4 (вернее, его многомерную версию) для согласования параметров гауссова распределения с множеством данных. С другой стороны, если бы были известны параметры каждого компонента, то можно было бы, по меньшей мере в вероятностном смысле, присвоить каждую точку данных компоненту.

Но проблема состоит в том, что не известны ни присваивания, ни параметры.

Основная идея алгоритма EM, рассматриваемого в данном контексте, состоит в том, что необходимо принять допущение, будто нам известны параметры этой модели, а затем вычислить вероятность того, что каждая точка данных принадлежит к тому или иному компоненту. После этого нужно снова согласовать компоненты с данными таким образом, чтобы каждый компонент согласовывался со всем набором данных, каждой точке которого назначен вес, соответствующий вероятности того, что она принадлежит к данному компоненту. Такой процесс продолжается итерационно до достижения сходимости. По сути при этом происходит "дополнение" данных путем вычисления распределений вероятностей по скрытым переменным (определяющим, какому компоненту принадлежит каждая точка данных) на основании текущей модели. При использовании смешанного гауссова распределения модель смешанного распределения инициализируется произвольными значениями параметров, а затем осуществляются итерации по описанным ниже двум шагам.

1. E-шаг. Вычислить вероятности $p_{ci} = P(C=i | x_j)$ того, что данные X_j были сформированы компонентом i . Согласно правилу Байеса, справедливо соотношение $P_{ij} = \alpha P\{X_j | C=i\} P(C=i)$. Терм $P\{X_j | C=i\}$ представляет собой вероятность значений данных x_j в i -м гауссовом распределении, а терм $P\{C=i\}$ — это параметр определения веса i -го гауссова распределения; по определению $P_i = \sum_j P_{ij}$.

2. M-шаг. Вычислить новые значения математического ожидания, ковариаций и весов компонента следующим образом: $\mu_i < \sum_j p_{ij} x_j / p_i$, $\Sigma_i < \sum_j p_{ij} (x_j - \mu_i)(x_j - \mu_i)^T / p_i$, $W_i < \sum_j p_{ij}$. E-шаг, или шаг ожидания (expectation), может рассматриваться как вычисление ожидаемых значений p_j скрытых индикаторных переменных Z_{ij} , где значение Z_{ij} равно 1, если данные x_j были сформированы i -м компонентом, и 0 — в противном случае. В M-шаге, или шаге максимизации (maximization), осуществляется поиск

новых значений параметров, которые максимизируют логарифмическое правдоподобие данных с учетом ожидаемых значений скрытых индикаторных переменных.

Окончательная модель, параметры которой определены в процессе обучения с помощью алгоритма EM, применяемого к данным, приведенным на рис. 20.8, я,

показана на рис. 20.8, в\ она практически не отличается от первоначальной модели, на основе которой были сформированы данные. На рис. 20.9, а показан график изменения логарифмического правдоподобия данных, соответствующих текущей модели, которое изменяется в процессе выполнения алгоритма ЕМ. На этом графике заслуживают внимания две особенности. Во-первых, логарифмическое правдоподобие модели, окончательно полученной в процессе обучения, немного превышает соответствующее значение для первоначальной модели, на основании которой были сформированы исходные данные. Это явление может на первый взгляд показаться удивительным, но оно просто отражает тот факт, что данные были сформированы случайным образом, поэтому существует вероятность того, что они не являются точным отражением самой базовой модели. Во-вторых, любопытно то, что «Зе в алгоритме ЕМ логарифмическое правдоподобие данных повышается после каждой итерации. Можно доказать, что такова общая особенность данного алгоритма. Кроме того, можно доказать, что при определенных условиях алгоритм ЕМ достигает локального максимума правдоподобия (а в редких случаях он может достичь точки перегиба или даже локального минимума). В этом смысле алгоритм ЕМ напоминает алгоритм восхождения к вершине с учетом градиента, но заслуживает внимания то, что в нем уже не применяется параметр со "ступенчатым изменением величины"!

20 40 60 80 100 120 Количество итераций б) Рис. 20.9. Графики, показывающие изменение логарифмического правдоподобия данных, L , как функции от количества итераций ЕМ. Горизонтальная линия соответствует логарифмическому правдоподобию истинной модели: график для модели смешанного гауссова распределения, показанной на рис. 20.8 (а); график для байесовской сети, приведенной на рис. 20.10, а (б) Но события не всегда развиваются так удачно, как можно было бы судить на основании рис. 20.9, а. Например, может случиться так, что один компонент гауссова распределения сузится настолько, что будет охватывать лишь единственную точку данных. В таком случае его дисперсия достигнет нуля, а правдоподобие увеличится до бесконечности! Еще одна проблема состоит в том, что может произойти "слияние" двух компонентов, в результате чего они примут одинаково $S \cdot e^{-200}$ Количество итераций а) J O) S п о O, O) о S •B- -1975 - 1980 -1985 -1990 -1995 -2000 -2005 -2010 -2015 S -2020 1 ? -2025

вые значения средних и дисперсий, а точки данных станут для них общими.

Вырожденные локальные максимумы такого рода представляют собой серьезную проблему, особенно в случае большого количества измерений. Одно из решений состоит в наложении распределений априорных вероятностей на параметры модели и применении версии MAP алгоритма ЕМ. Еще одно решение состоит в перезапуске вычислений для некоторого компонента с новыми случайно выбранными параметрами, если он становится слишком малым или слишком

близким к другому компоненту. Иногда имеет также смысл выбирать для инициализации параметров более обоснованные значения.

Обучение байесовских сетей со скрытыми переменными Чтобы определить в процессе обучения параметры байесовской сети со скрытыми переменными, можно применить такие же подходы, которые позволили добиться успеха в случае смешанных гауссовых распределений. На рис. 20.10 показана ситуация, в которой имеются два пакета конфет, смешанных друг с другом. Для описания конфет применяются три характеристики: кроме разновидности Flavor и обертки Wrapper, в некоторых конфетах находятся леденцы с отверстиями Hole в середине, а в некоторых — леденцы без отверстий. Распределение конфет в каждом пакете описано с помощью наивной байесовской модели: в каждом конкретном пакете характеристики являются независимыми, но распределение условных вероятностей каждой характеристики зависит от пакета. Применяются следующие параметры: θ — априорная вероятность того, что конфета взята из пакета Bag 1; θ_{F1} и θ_{F2} — вероятности того, что конфета относится к разновидности вишневых леденцов, при условии, что эта конфета взята из пакета Bag 1 и Bag 2 соответственно; θ_{w1} и θ_{w2} задают вероятности того, что обертка имеет красный цвет; а θ_{h1} и θ_{h2} — вероятности того, что леденец имеет отверстие. Обратите внимание на то, что вся эта модель в целом представляет собой модель смешанного распределения (в действительности это смешанное гауссово распределение можно также промоделировать в виде байесовской сети, как показано на рис. 20.10, б). На этом рисунке скрытая переменная соответствует пакету, поскольку после смешивания конфет мы больше не имеем возможности определить, из какого пакета взята каждая конфета. Можно ли в таком случае восстановить описание этих двух пакетов, наблюдая за характеристиками конфет, взятых из этой смеси?

Проведем одну итерацию алгоритма EM для решения этой задачи. Вначале рассмотрим данные. Сформировано 1000 выборок из модели, истинными параметрами которой являются следующие: $\theta = 0.5$, $\theta_{F1} = \theta_{F2} = 0.8$, $\theta_{w1} = \theta_{w2} = 0.3$ (0.7) Это означает, что равновероятно получение конфет из одного или другого пакета; в первом пакете в основном находятся вишневые леденцы в красных обертках и с отверстиями, а во втором — в основном лимонные леденцы в зеленых обертках и без отверстий. Количество восьми возможных разновидностей конфет определено в табл. 20.1.

$P(\text{Bag} = \backslash) \backslash \text{Bag}$ $P(F = \text{cherry} \backslash \text{Bag})$ ел On а) б) Рис. 20.10. Примеры применения модели смешанного гауссова распределения и байесовской сети: модель смешанного распределения для задачи с конфетами. Относительные количества различных разновидностей оберток и количества отверстий в леденцах зависят от пакета, который определен с помощью ненаблюдаемой переменной (а); байесовская сеть, соответствующая смешанному гауссову распределению. Среднее и ковариация

наблюдаемой переменной X зависит от компонента C (6) Таблица 20.1. Восемь возможных разновидностей конфет $F = \text{cherry}$ $F = \text{lime}$ $Y = 1$ $W = \text{red}$ $Y = 0$ $Y = 1$ $W = \text{green}$ $Y = 0$ Начнем с инициализации параметров. Для упрощения числовых расчетов выберем следующие значения параметров: $\theta_{10} = 0.6$, $\theta_{11} = \theta_{12} = \theta_{13} = 0.6$, $\theta_{14} = \theta_{15} = \theta_{16} = 0.4$ (20.8) Вначале проведем расчет для параметра 9. В полностью наблюдаемом случае можно было бы получить оценку для этого параметра непосредственно из наблюдаемых значений количества конфет, относящихся к пакетам 1 и 2. Но поскольку номер пакета — это скрытая переменная, рассчитаем вместо этого ожидаемые значения количества. Ожидаемое количество $N(\text{Bag}=1)$ представляет собой сумму вероятностей того, что конфета взята из пакета 1, по всем конфетам:

$N(e_A) = N(\text{Bag}=1) / N = \sum_j P(\text{Bag}=1 | \text{flavor}_j, \text{wrapper}^j, \text{holes}_j) / N$ Эти вероятности можно вычислить с помощью любого алгоритма вероятностного вывода для байесовских сетей. А применительно к наивной байесовской модели, по7 На практике лучше выбирать эти значения случайным образом для предотвращения появления локальных максимумов из-за симметрии.

добной той, что рассматривается в данном примере, этот вероятностный вывод можно выполнить "вручную", используя правило Байеса и применяя выражение для условной независимости:

$N(e_A) = \sum_i P(\text{flavor}_i | \text{Bag}=1) P(\text{wrapper}^i | \text{Bag}=1) P(\text{holes}^i | \text{Bag}=1) P(\text{Bag}=1) / \sum_i P(\text{flavor}_i | \text{Bag}=i) P(\text{wrapper}^i | \text{Bag}=i) P(\text{holes}^i | \text{Bag}=i) P(\text{Bag}=i)$ (Обратите внимание на то, что нормализующая константа также зависит от параметров.) Применяя эту формулу, например, к данным о 273 конфетах в красной обертке, среди которых находятся вишневые леденцы с отверстиями, определим, какой вклад они вносят в распределение вероятностей:

$977 A_{10} = \theta_{10}$ $\theta_{11} = 0.6684$, $\theta_{12} = 0.6483$, $\theta_{13} = 0.6558$ $\theta_{14} = 0.3887$, $\theta_{15} = 0.3817$, $\theta_{16} = 0.3827$ (20.9) Продолжая эти расчеты для семи других видов конфет, количество которых указано в табл. 20.1, получим, что $\theta_{10} = 0.6124$.

Теперь рассмотрим другие параметры, такие как θ_{11} . В полностью наблюдаемом случае это значение можно было бы оценить непосредственно на основе наблюдаемых значений количества вишневых и лимонных леденцов из пакета 1. Ожидаемое количество вишневых леденцов из пакета 1 задается с помощью следующего выражения: $z P(\text{Bag}=1 | \text{Flavor}^z = \text{cherry}, \text{wrapper}^z, \text{holes}_z)$ Эти вероятности также можно вычислить с помощью любого алгоритма для байесовской сети. Продолжая этот процесс, получим новые значения для всех параметров: $\theta_{10} = 0.6124$, $\theta_{11} = 0.6684$, $\theta_{12} = 0.6483$, $\theta_{13} = 0.6558$ $\theta_{14} = 0.3887$, $\theta_{15} = 0.3817$, $\theta_{16} = 0.3827$ (20.9) Логарифмическое правдоподобие данных возрастает от первоначального значения, примерно равного -2.044, приблизительно до -2.021

после первой итерации, как показано на рис. 20.9, б. Это означает, что в данном обновлении само значение правдоподобия улучшается примерно на коэффициент $e^{23} = 10^{10}$. К десятой итерации модель, полученная в процессе обучения, лучше согласуется с данными, чем первоначальная модель ($\ln \mathcal{L} = -1982.214$). Но дальнейший прогресс очень сильно замедляется. Такая ситуация в приложениях алгоритма EM встречается весьма часто, поэтому во многих практически применяемых системах алгоритм EM на последнем этапе обучения используется в сочетании с таким алгоритмом на основе градиента, как алгоритм Ньютона-Рафсона (см. главу 4).

Общий вывод, который может быть сделан на основании данного примера, состоит в том, что θ обновления параметров при обучении байесовской сети со скрытыми переменными являются непосредственно доступными из результатов вероятностного вывода по каждому примеру. Более того, для каждого параметра требуются только локальные апостериорные вероятности. Для общего случая, в котором в процессе обучения определяются параметры условной вероятности для каждой переменной x_i если даны ее роди-

тельские переменные (иначе говоря, $\theta_{ijk} = P(X_i = x_i | \text{Pa}_i = \text{pa}_i)$), обновление задается с помощью нормализованных ожидаемых количеств следующим образом:

$\theta_{ijk} \leftarrow \frac{\sum_{\mathbf{x}} \mathbf{1}\{X_i = x_i, \text{Pa}_i = \text{pa}_i\}}{N(\text{Pa}_i = \text{pa}_i)}$ Эти ожидаемые количества можно получить путем суммирования по всем примерам и вычисления вероятностей $P(X_i = x_i, \text{Pa}_i = \text{pa}_i)$ для каждого из них с использованием любого алгоритма вероятностного вывода в байесовской сети. Для использования в точных алгоритмах (включая алгоритмы удаления переменных) все эти вероятности могут быть получены непосредственно как побочный продукт стандартного вероятностного вывода, без необходимости применения дополнительных вычислений, характерных для обучения. Более того, информация, необходимая для обучения, доступна локально применительно к каждому параметру.

Обучение скрытых марковских моделей Последнее приложение алгоритма EM, рассматриваемой в данной главе, касается изучения вероятностей перехода в скрытых марковских моделях (Hidden Markov Model— HMM). Напомним, что, как было сказано в главе 15, скрытая марковская модель может быть представлена с помощью динамической байесовской сети с одной дискретной переменной состояния (рис. 20.11). Каждая точка данных состоит из последовательности наблюдений конечной длины, поэтому задача заключается в том, чтобы определить в процессе обучения вероятности перехода на основании множества последовательностей наблюдений (или, возможно, на основании только одной длинной последовательности). $\mathcal{L} = \sum_{\mathbf{y}} P(\mathbf{y} | \theta)$ (Ram) Umbrella \ шк h i \mathbb{C} \setminus \mathbb{S} \setminus \mathbb{R} \setminus \mathbb{f} \setminus \mathbb{C} \setminus \mathbb{E} \setminus \mathbb{I}?

Рис. 20.11. Развернутая динамическая байесовская сеть, которая представляет скрытую марковскую модель (повторение рис. 15.12) В данной главе уже было показано, как проводить обучение байесовских сетей, но в этом случае возникает одна сложность: в байесовских сетях каждый параметр является различным, а в скрытой марковской модели, с другой стороны, отдельные вероятности перехода из состояния i в состояние j во время t , $\theta_{ijt} = P(X_{t+1}=j | X_t=i)$, повторяются во времени, т.е. $\theta_{ijt} = \theta_{ij}$ для всех t . Чтобы оценить вероятность перехода из состояния i в состояние j , достаточно вычислить ожидаемую долю случаев, в которых система подвергается переходу в состояние j , находясь в состоянии i :

$$\theta_{ij} = \frac{1}{N} \sum_{t=1}^N I(X_t=i, X_{t+1}=j)$$

Опять-таки эти ожидаемые количества могут быть вычислены с помощью любого алгоритма вероятностного вывода для скрытой марковской модели. Для вычисления необходимых вероятностей можно очень легко модифицировать прямой—обратный алгоритм, приведенный в листинге 15.1. Заслуживает внимание одно важное замечание, что требуются вероятности, полученные путем сглаживания, а не фильтрации; это означает, что при оценке вероятности того, что произошел конкретный переход, необходимо учитывать полученное впоследствии свидетельство. Например, как было указано в главе 15, свидетельство в случае убийства обычно может быть получено только после этого преступления (т.е. после перехода из состояния i в состояние j).

Общая форма алгоритма EM Выше было описано несколько вариантов применения алгоритма EM. Каждый из них сводился к тому, что вычислялись ожидаемые значения скрытых переменных для каждого примера, а затем повторно вычислялись параметры с использованием ожидаемых значений так, как если бы они были наблюдаемыми значениями.

Допустим, что x — все наблюдаемые значения во всех примерах, и предположим, что z обозначает все скрытые переменные для всех примеров, а θ представляет собой все параметры для модели вероятности. В таком случае алгоритм EM можно представить с помощью следующего уравнения: $\theta^{(i+1)} = \arg\max_{\theta} \sum_{j=1}^J \sum_{i=1}^I \theta_{ij} \log \theta_{ij}$ Это уравнение составляет саму суть алгоритма EM. При этом E-шаг сводится к вычислению выражения для суммы, которая представляет собой ожидаемое значение логарифмического правдоподобия "дополненных" данных по отношению к распределению $P(z=z | x, \theta^{(i)})$, т.е. распределение апостериорных вероятностей по скрытым переменным при наличии полученных данных. С другой стороны, M-шаг представляет собой максимизацию этого ожидаемого логарифмического правдоподобия по отношению к параметрам. При использовании смешанного гауссова распределения скрытыми переменными являются z_{ij} где z^i равна 1, если пример j был сформирован компонентом i . При

использовании байесовских сетей скрытые переменные представляют собой значения ненаблюдаемых переменных для каждого примера. При использовании скрытых марковских моделей скрытые переменные являются переходами $i \rightarrow j$. Начиная с этой общей формы, можно вывести алгоритм EM для конкретного приложения, как только определены соответствующие скрытые переменные.

Поняв основную идею алгоритма EM, можно легко вывести все возможные его варианты и усовершенствования. Например, во многих случаях E-шаг (вычисление распределений апостериорных вероятностей по скрытым переменным) является трудновыполнимым, как при использовании больших байесовских сетей.

Оказалось, что в этом случае можно применить приближенный E-шаг и все еще получить эффективный алгоритм обучения. А если используется такой алгоритм формирования выборки, как MCMC (см. раздел 14.5), то процесс обучения становится полностью интуитивно понятным: каждое состояние (конфигурация скрытых и наблюдаемых переменных), посещенное алгоритмом MCMC, рассматривается точно так

же, как если бы оно было полным наблюдением. Поэтому параметры могут обновляться непосредственно после каждого перехода из состояния в состояние в алгоритме MCMC. Для определения с помощью обучения параметров очень больших сетей показали свою эффективность и другие формы приближенного вероятностного вывода, такие как вариационные и циклические методы.

Определение с помощью обучения структур байесовских сетей со скрытыми переменными В разделе 20.2 обсуждалась задача определения с помощью обучения структур байесовских сетей на основе полных данных. А если приходится учитывать скрытые переменные, то задача усложняется. В простейшем случае скрытые переменные могут быть включены в общий список наряду с наблюдаемыми переменными; хотя их значения не наблюдаются, алгоритм обучения получает сведения о том, что они существуют, и должен найти для них место в структуре сети. Например, с помощью алгоритма может быть предпринята попытка определить в процессе обучения структуру, показанную на рис. 20.7, а, на основе той информации, что в модель должна быть включена переменная HeartDisease (трехзначная переменная). Если же алгоритму обучения не предоставлена эта информация, то в процессе обучения возникают два варианта: либо исходить из того, что данные действительно являются полными (что вынуждает алгоритм определить в процессе обучения модель с гораздо большим количеством параметров, показанную на рис. 20.7, б), либо изобрести новые скрытые переменные для упрощения модели. Последний подход можно реализовать путем включения новых вариантов операций модификации в процедуру поиска структуры — предусмотреть в алгоритме возможность не только модифицировать связи, но и добавлять или удалять скрытые переменные либо менять их арность.

Безусловно, в таком алгоритме нельзя предусмотреть возможность назвать вновь изобретенную переменную, допустим HeartDisease, а также присваивать ее значениям осмысленные имена. Но, к счастью, вновь изобретенные скрытые переменные обычно связываются с ранее существовавшими переменными, поэтому человек— специалист в данной проблемной области часто может проверять локальные распределения условных вероятностей, касающиеся новой переменной, и устанавливать ее смысл.

Как и в случае полных данных, процесс определения структуры с помощью обучения с учетом максимального правдоподобия в чистом виде приводит к созданию полносвязной сети (более того, сети без скрытых переменных), поэтому требуется ввести какую-то форму штрафования за сложность. Для аппроксимации байесовского обучения можно также применить алгоритм МСМС. Например, можно предусмотреть определение в процессе обучения параметров смешанного гауссова распределения с неизвестным количеством компонентов, осуществляя выборки по такому количеству; приближенное распределение апостериорных вероятностей для количества заданных гауссовых распределений определяется частотами выборки в процессе применения алгоритма МСМС.

До сих пор рассматриваемый процесс обучения состоял из внешнего цикла, который представлял собой процесс поиска структуры, и внутреннего цикла, представляющего собой процесс оптимизации параметров. В случае полных данных внутренний цикл выполняется очень быстро, поскольку при этом достаточно лишь извлечь информацию об условных частотах из набора данных. Если же имеются скрытые переменные, то для выполнения внутреннего цикла может потребоваться применить много итераций алгоритма ЕМ или алгоритма на основе градиента, а каждая итерация потребует вычисления распределений апостериорных вероятностей в байесовской сети, что само представляет собой NP-трудную задачу. К настоящему времени доказано, что такой подход является практически не применимым для определения в процессе обучения сложных моделей. Одно из возможных усовершенствований состоит в использовании так называемого алгоритма $\wedge$ структурного ЕМ, который действует во многом таким же образом, как и обычный (параметрический) алгоритм ЕМ, за исключением того, что этот алгоритм может обновлять не только параметры, но и структуру. Так же как в обычном алгоритме ЕМ используются текущие параметры для вычисления ожидаемых количеств в Е-шаге, а затем эти количества применяются в М-шаге для выбора новых параметров, так и в алгоритме структурного ЕМ используется структура для вычисления ожидаемых количеств, после чего эти количества применяются в М-шаге для оценки правдоподобия потенциальных новых структур (в этом состоит отличие данного метода от метода внешнего цикла/внутреннего цикла, в котором вычисляются новые ожидаемые количества для каждой потенциальной структуры).

Таким образом, структурный алгоритм EM позволяет вносить в сеть несколько структурных изменений без каких-либо повторных вычислений ожидаемых количеств и обладает способностью определять в процессе обучения нетривиальные структуры байесовских сетей. Тем не менее необходимо проделать еще очень много работы, прежде чем можно будет утверждать, что задача определения структуры в процессе обучения окончательно решена.

20.4. Обучение на основе экземпляра До сих пор приведенное в данной главе описание статистического обучения сосредоточивалось в основном на задаче подгонки параметров ограниченного семейства вероятностных моделей к неограниченному набору данных. Например, в методе неконтролируемой кластеризации используются смешанные гауссовы распределения на основании того предположения, что структуру рассматриваемых данных можно объяснить, трактуя их как сумму постоянного количества гауссовых распределений. Авторы настоящей книги называют такие методы $\wedge$ параметрическим обучением. Методы параметрического обучения часто бывают простыми и эффективными, но предположение о том, что в данных воплощено конкретное ограниченное семейство моделей, часто слишком упрощает то, что происходит в реальном мире, из которого поступают эти данные. Верно, что при наличии очень малого объема данных нельзя надеяться определить в процессе обучения параметры сложной и подробной модели, но представляется неразумным по-прежнему придерживаться гипотезы с той же фиксированной сложностью, даже после того, как доступный набор данных становится очень большим!

В отличие от параметрического обучения, методы $\wedge$ непараметрического обучения позволяют увеличивать сложность гипотезы по мере роста объема данных. Чем больше данных поступает в распоряжение исследователя, тем более развитой может становиться гипотеза. В данном разделе рассматриваются два очень простых семейства методов непараметрического $\wedge$ обучения на основе экземпляра (или обучения на основе

содержимого памяти), получивших такое название потому, что они позволяют конструировать гипотезы непосредственно на основе самих обучающих экземпляров.

Модели ближайшего соседа Ключевая идея моделей $\wedge$ ближайшего соседа состоит в том, что свойства любой конкретной входной точки x , по-видимому, должны быть подобными свойствам точек, соседних по отношению к x . Например, если требуется выполнить оценку плотности (т.е. оценить значение неизвестной плотности вероятности в точке x), то можно просто измерить ту плотность, с какой расположены точки, рассеянные в окрестности x . Такая задача на первый взгляд может показаться очень простой, пока не станет очевидно, что нужно точно определить, что подразумевается под понятием "окрестность". Если окрестность

слишком мала, то не будет содержать ни одной точки данных, а если слишком велика, то может включить все точки данных, в результате чего будет получена всюду одинаковая оценка плотности. Одно из возможных решений состоит в том, чтобы определить окрестность как достаточно большую для включения к точек, где к достаточно велико для обеспечения получения осмысленной оценки. При постоянном значении к размеры окрестности изменяются — если данные являются разреженными, то окрестность велика, а если данные расположены плотно, то окрестность мала. На рис. 20.12, а показан пример данных, рассеянных в двух измерениях, а на рис. 20.13 приведены результаты оценки плотности по к ближайшим соседним точкам на основании этих данных при $k=3$, 10 и 40 соответственно. При $k=3$ оценка плотности в любой точке основана только на 3 соседних точках и весьма изменчива. При $k=10$ полученная оценка представляет собой хорошую реконструкцию истинной плотности, показанной на рис. 20.12, б. При $k=40$ окрестность становится слишком большой и информация о структуре данных полностью теряется. На практике хорошие результаты для большинства наборов данных с малым количеством размерностей можно получить с помощью значения k , находящегося примерно между 5 и 10. Подходящее значение для k можно также выбрать с использованием перекрестной проверки.

Для выявления соседних точек, ближайших к точке запроса, нужна метрика расстояний $D(x_1, x_2)$. В двухмерном примере, приведенном на рис. 20.12, используется евклидово расстояние. Но такая метрика становится неподходящей, если каждая размерность пространства измеряет что-то другое (например, рост и вес), поскольку изменение масштаба одной размерности приводит к изменению множества ближайших соседних точек. Одним из решений является стандартизация масштаба для каждой размерности. Для этого измеряется среднееквадратичное отклонение каждой характеристики по всему набору данных и значения характеристик выражаются как кратные среднееквадратичного отклонения для этой характеристики (это — частный случай χ^2 расстояния Махаланобиса, в котором также учитывается ковариация характеристик). Наконец, для дискретных характеристик можно использовать χ^2 расстояние Хемминга, в котором $D(x_1, x_2)$ определяется как количество характеристик, по которым отличаются точки x_1 и x_2 .

Оценки плотности, подобные приведенным на рис. 20.13, определяют совместные распределения по пространству входных данных. Но в отличие от байесовской сети, представление на основе экземпляра не может содержать скрытых перемен-

ных, а это означает, что может выполняться неконтролируемая кластеризация, как это было в случае с моделью смешанного гауссова распределения. Тем не менее оценка плотности все еще может использоваться для предсказания целевого значения y по входным значениям характеристики x путем вычисления вероятности

$P(y|x) = P\{y, x\} / P(x)$, при условии, что обучающие данные включают значения для соответствующей целевой характеристики.

0,8 0,6] 0,4 0,2 Плотность 0 0,2 0,4 0,6 0,8 1 а) б) Рис. 20.12. Пример применения оценки плотности с k ближайшими соседними точками: частичная выборка данных, показанных на рис. 20.8, а, состоящая из 128 точек, наряду с двумя точками запроса и их окрестностями, которые включают 10 ближайших соседних точек (а); трехмерный график смешанного гауссова распределения, на основании которого были получены эти данные (б) Плотность Плотность $|\wedge n \Gamma^{\wedge \wedge}$ Плотность 0 0,2 0 $\Gamma^{\wedge \wedge \wedge}$ $\wedge^{\wedge}$ $\wedge^{\wedge}$ 60,8 * о ' D0,8 $\wedge$ о а) б) в) Рис. 20.13. Результаты применения метода оценки плотности с использованием окрестностей, включающих k ближайших соседних точек, к данным, приведенным на рис. 20.12, а, при $k=3, 10$ и 40 соответственно. Идею оценки характеристик с помощью ближайшей соседней точки можно также непосредственно использовать в контролируемом обучении. Если имеется проверочный пример с входными данными x , то выходное значение $y=h(x)$ можно получить на основании значений y для k ближайших соседних точек точки x . В дискретном случае единственное предсказание можно получить с помощью мажоритарного голосования, а в непрерывном случае — предусмотреть усреднение по k значениям

или применить локальную линейную регрессию, подгоняя гиперплоскость к k точкам и предсказывая значение в точке x с помощью этой гиперплоскости.

Алгоритм обучения с помощью k ближайших соседних точек является очень простым для реализации, требует весьма небольшой настройки и часто показывает достаточно приемлемую производительность. Вполне целесообразно, столкнувшись с новой задачей обучения, вначале попытаться воспользоваться этим алгоритмом.

Но при наличии больших наборов данных требуется эффективный механизм поиска соседних точек, ближайших к точке запроса x , поскольку выполнение метода, в котором просто вычисляется расстояние до каждой точки, занимает слишком много времени. Было предложено много остроумных методов, позволяющих повысить эффективность этого этапа, которые основаны на предварительной обработке обучающих данных. Но, к сожалению, большинство из этих методов не достаточно хорошо масштабируются с увеличением количества размерностей пространства (т.е. количества характеристик).

При использовании многомерных пространств возникает еще одна проблема, связанная с тем, что ближайшие соседние точки в таких пространствах в действительности часто находятся очень далеко! Рассмотрим набор данных с размером N в d -мерном единичном гиперкубе и предположим, что нужно найти гиперкубическую окрестность с размером b и объемом b^d (те же рассуждения

относятся к гиперсферам, но формула объема гиперсферы является более сложной). Чтобы в нее вошло k точек, средняя окрестность должна занимать долю k/N всего объема гиперкуба, который равен 1. Поэтому $bd=k/N$, или $b=\{k/N\}/d$. До сих пор все шло хорошо.

А теперь допустим, что количество характеристик d равно 100, и предположим, что N равно 10, а AT равно 1 000 000. В таком случае получаем, что $b=0.89$, т.е. что окрестность должна охватывать почти все пространство входных данных! Эти расчеты говорят о том, что методам с ближайшими соседними точками нельзя доверять, когда дело касается многомерных данных, а в случае малого количества размерностей не возникает никаких проблем; при $d=2$ получаем $b=0.003$.

Ядерные модели В ядерной модели каждый обучающий экземпляр рассматривается как самостоятельно вырабатывающий небольшую функцию плотности (ядерную функцию). Оценка всей плотности в целом представляет собой нормализованную сумму всех небольших ядерных функций. Обучающий экземпляр в точке X_i вырабатывает ядерную функцию $i; C(x/x_i)$, которая присваивает определенную вероятность каждой точке x в пространстве. Поэтому оценка плотности принимает такой вид:

$$P(x) = \frac{1}{N} \sum_{i=1}^N C(x, X_i)$$
 Ядерная функция обычно зависит только от расстояния $D(x, X_i)$ между точкой x и экземпляром X_i . Наиболее широко применяемой ядерной функцией (безусловно) является гауссово распределение. Для упрощения предполагается использование сферического гауссова распределения со среднеквадратичным отклонением w вдоль каждой оси, т.е. следующего распределения:

$$K(x, x_i) = \frac{1}{(2\pi)^{d/2} w^d} \exp\left(-\frac{D(x, x_i)^2}{2w^2}\right)$$
 где d — количество размерностей в точке x . И при таком подходе все еще приходится сталкиваться с проблемой выбора подходящего значения для w , как и прежде, применение слишком малой окрестности приводит к получению оценки, состоящей из слишком большого количества пиков, как показано на рис. 20.14, а. На рис. 20.14, б видно, что промежуточное значение w позволяет получить очень хорошую реконструкцию первоначального распределения. А на рис. 20.14, в применение слишком большой окрестности приводит к полной потере структуры. Хорошее значение w может быть выбрано с использованием перекрестной проверки.

Рис. 20.14. Оценка плотности с помощью ядерной модели для данных, показанных на рис. 20.12, а, в которых используется ядерная функция гауссова распределения со значениями $w=0.02$, 0.07 и 0.20 . Контролируемое обучение с помощью ядерных функций осуществляется путем взятия взвешенных комбинаций всех предсказаний из обучающих экземпляров (сравните такой подход с предсказанием с помощью k ближайших соседних точек, в котором берется невзвешенная комбинация

ближайших к экземпляров). Вес i -го экземпляра для точки запроса x определяется по значению ядерной функции $K(x, X_i)$. Для предсказания в дискретном случае можно выполнить взвешенное голосование, а для предсказания в непрерывном случае — получить взвешенное среднее или применить метод взвешенной линейной регрессии. Следует отметить, что для получения предсказаний с помощью ядерной функции требуется анализ каждого обучающего экземпляра. Существует также возможность объединять ядерные функции со схемами индексации ближайших соседних точек для получения взвешенных предсказаний только с помощью самых близких экземпляров.

20.5. Нейронные сети Нейрон — это клетка мозга или нервной системы, основной функцией которой является сбор, обработка и распространение электрических сигналов. Схематическое изображение типичного нейрона приведено на рис. 1.1 (см. с. 48). Считается, что способность мозга к обработке информации в основном обусловлена функционированием сетей, состоящих из таких нейронов. По этой причине целью некоторых из самых ранних работ по искусственному интеллекту было создание искусственных ^ нейронных сетей (эта область научной деятельности упоминалась также под дру-

гими названиями, включая коннекционизм, параллельная распределенная обработка и нейронные вычисления). На рис. 20.15 показана простая математическая модель нейрона, предложенная Мак-Каллоком и Питтсом [1017]. Грубо говоря, нейрон "активизируется", когда линейная комбинация значений на его входах превышает некоторый порог. Начиная с 1943 года были разработаны гораздо более подробные и реалистичные модели как для нейронов, так и для более крупных систем в мозгу, что привело к созданию такой современной научной области, как ^ вычислительная неврология. С другой стороны, у исследователей в области искусственного интеллекта и статистики пробудился интерес к изучению более абстрактных свойств нейронных сетей, таких как способность выполнять распределенные вычисления, справляться с зашумленными входными данными и обеспечивать обучение. Хотя со временем стало ясно, что подобные возможности предоставляют и другие системы (включая байесовские сети), нейронные сети остаются одной из наиболее широко применяемых и эффективных форм систем обучения и сами по себе могут стать важным предметом для изучения.

Смещенный вес Входные Входная Функция пыхол Выходные связи функция активации связи Рис. 20.15. Простая математическая модель нейрона. Выходной активацией этого элемента является где a_j — выходная активация элемента j ; $W_{j,i}$ — вес связи от элемента j к данному элементу Элементы в нейронных сетях Нейронные сети состоят из узлов, или ^ элементов (см. рис. 20.15), соединенных направленными ^ связями. Связь от элемента j к элементу i служит для распространения ^ активации a_j от j к i . Кроме того, каждая связь имеет назначенный ей числовой ^ вес W_j , в

который определяет силу и знак связи. Каждый элемент i прежде всего вычисляет взвешенную сумму своих входных данных: $p_j = 0$. Затем он применяет к этой сумме $\wedge$ функцию активации d , чтобы определить, какими должны быть выходные данные:

($p_i a_i = g(i r_{ii}) = d \wedge \text{Шлс } B0.10) \forall j=0$ у Обратите внимание на то, что в эту формулу входит $\wedge$ смещенный вес w_{0ii} , относящийся к постоянному входному значению $a_0 = -1$. Роль, которую играет эта величина, будет описана немного позже.

Функция активации d предназначена для выполнения двух назначений. Во-первых, необходимо, чтобы элемент был "активным" (находился на уровне активации примерно $+1$) при наличии "правильных" входных данных и "неактивным" (с уровнем активации, близким к 0) при получении "неправильных" входных данных. Во-вторых, функция активации должна быть нелинейной, поскольку в противном случае произойдет сворачивание всех функций активации нейронной сети в простую линейную функцию (см. упр. 20.17). Два варианта формы функции активации d показаны на рис. 20.16 — $\wedge$ пороговая функция и $\wedge$ сигмоидальная функция (называемая также $\wedge$ логистической функцией). Преимуществом сигмоидальной функции является то, что она дифференцируема, а это, как показано ниже, — важное свойство для алгоритма обучения с учетом весов. Обратите внимание на то, что обе функции имеют пороговое значение (либо жесткое, либо мягкое) около нуля; смещенный вес $w_{0i}/0, \pm$ задает фактическое пороговое значение для данного элемента в том смысле, что элемент активизируется после того, как взвешенная сумма "реальных" входных данных (т.е. сумма, из которой исключен смещенный вход) превышает $w_{0ii} + 1$ $g(M_i)$ $g(i \gg i)$ а) Рис. 20.16. Основные виды функций активации: пороговая функция активации, которая выводит 1 , когда входные данные являются положительными, и 0 — в противном случае (иногда вместо нее используется знаковая функция, которая выводит ± 1 в зависимости от знака входного значения) (а); сигмоидальная функция $1 / A + e^{-x}$ (б) Представление о том, как работают отдельные элементы, можно получить, сравнив их с логическими элементами. Одной из первоначальных причин, по которым исследователи занялись проектированием отдельных элементов [1017], была способность элементов представлять основные булевы функции. На рис. 20.17 показано, как можно представить булевы функции AND, OR и NOT с помощью пороговых элементов, входам которых назначены подходящие веса. Такое свойство является

важным, поскольку оно означает, что эти элементы можно использовать для создания сети, обеспечивающей вычисление любой булевой функции от входных данных.

$Ж0=1.5 \wedge 0 = 0.5 W0 = -0.5 W2=\wedge W2 = I$ AND OR NOT Рис. 20.17. Примеры элементов с пороговой функцией активации, которые могут действовать как логические элементы, если заданы соответствующие веса простых и смещенных входов

Структуры сетей Существуют две основные категории структур нейронных сетей: ациклические сети, или $\wedge$ сети с прямым распространением, и циклические, или $\wedge$ рекуррентные, сети. Сеть с прямым распространением представляет определенную функцию ее текущих входных данных, поэтому не имеет внутреннего состояния, отличного от самих весов. Рекуррентная сеть, с другой стороны, подает свои выходные данные обратно на свои собственные входы. Это означает, что уровни активации сети образуют динамическую систему, которая может достигать устойчивого состояния, или переходить в колебательный режим, или даже проявлять хаотичное поведение. Более того, отклик сети на конкретные входные данные зависит от ее начального состояния, которое, в свою очередь, может зависеть от предыдущих входных данных.

Поэтому рекуррентные сети (в отличие от сетей с прямым распространением) могут моделировать кратковременную память. Это означает, что они являются более интересными объектами для использования в качестве моделей мозга, но вместе с тем являются более трудными для понимания. В данном разделе в основном рассматриваются сети с прямым распространением; некоторые указания на источники для дополнительного чтения по рекуррентным сетям приведены в конце данной главы.

Проанализируем более внимательно утверждение о том, что сеть с прямым распространением представляет функцию от ее входных данных. Рассмотрим простую сеть, показанную на рис. 20.18, которая состоит из входных элементов, двух $\wedge$ скрытых элементов и одного выходного элемента (чтобы упростить рассматриваемую схему, в данном примере удалены элементы, на которые подается смещение). Если задан вектор входных данных $x = (x_1, x_2)$, активации входных элементов принимают вид $(a_1, a_2) = (x_1, x_2)$, а сеть вычисляет следующее значение:
$$a_5 = g(W_3 a_1 + W_4 a_2) = g(W_3/5g(W_1 x_1 + W_2 x_2) + W_5 a_1 + W_6 a_2)$$
 (20.11) Таким образом, выразив выходное значение каждого скрытого элемента как функцию его входных значений, мы показали, что выход всей сети, a_5 , является функцией от ее входов. Кроме того, мы показали, что веса в сети действуют как параметры этой функции; если применить запись w для обозначения параметров, то можно утверждать, что сеть вычисляет функцию $hw(x)$. Корректируя веса, можно изменять функцию, представленную сетью. Именно так происходит обучение в нейронных сетях.

Рис. 20.18. Очень простая нейронная сеть с двумя входными элементами, одним скрытым слоем из двух элементов и одного выходного элемента Нейронная сеть может использоваться для классификации или регрессии. Для булевой классификации с непрерывными выходными данными (например, формируемыми с помощью сигмоидальных элементов) обычно принято применять один выходной элемент, в котором значение, превышающее 0.5, интерпретируется как

принадлежащее к одному классу, а значение ниже 0.5 — как принадлежащее к другому. Для J_c -сторонней классификации можно разделить диапазон одного выходного элемента на k частей, но чаще используется структура сети с k отдельными выходными элементами, притом что значение на каждом из них представляет относительное правдоподобие конкретного класса на основании текущих входных данных.

Сети с прямым распространением обычно размещаются по n слоям, таким, что каждый элемент получает входные данные только от элементов, относящихся к непосредственно предшествующему слою. В следующих двух подразделах рассматриваются однослойные сети, не имеющие скрытых элементов, и многослойные сети, которые имеют один или несколько слоев скрытых элементов.

Однослойные нейронные сети с прямым распространением (персептроны) Сеть, в которой все входные элементы соединены непосредственно с выходными элементами, называется n однослойной нейронной сетью, или сетью n персептрона.

Поскольку каждый выходной элемент является независимым от других (каждый вес влияет только на один из выходов), можно ограничиться в нашем исследовании рассмотрением персептронов с единственным выходным элементом, как показано на рис. 20.19, а.

Начнем с исследования пространства гипотез, которое может быть представлено с помощью персептрона. Если применяется пороговая функция активации, то персептрон может рассматриваться как представляющий некоторую булеву функцию.

Кроме элементарных булевых функций AND, OR и NOT (см. рис. 20.17), персептрон позволяет представлять очень компактно некоторые весьма "сложные" булевы функции. Например, мажоритарная функция, которая выводит 1, только если 1 присутствует больше чем на половине из ее n входов, может быть представлена с помощью персептрона, в котором каждое значение $w_i = 1$, а пороговое значение $w_0 = n/2$.

В дереве решений для представления этой функции потребовалось бы $O(2^n)$ узлов.

Входные Выходные элементы t_i элементы а) б) Рис. 20.19. Свойства сети персептрона: сеть персептрона, состоящая из трех выходных элементов персептрона, в которых совместно используются пять входных элементов (а).

Рассматривая конкретный выходной элемент (скажем, второй элемент, выделенный жирным контуром), можно обнаружить, что веса его входных связей не влияют на другие выходные элементы; графическое изображение выходных данных элемента персептрона с двумя входами, имеющего сигмоидальную функцию активации (б) К сожалению, существует также много булевых функций, которые не могут быть

представлены с помощью порогового персептрона. Рассматривая уравнение 20.10, можно обнаружить, что пороговый персептрон возвращает 1 тогда и только тогда, когда взвешенная сумма его входных данных (включая смещение) является положительной: $\sum_{j=0}^n W_j x_j > 0$ или $W \cdot x > 0$. Итак, $W \cdot x = 0$ определяет гиперплоскость в пространстве входных данных, поэтому персептрон возвращает 1 тогда и только тогда, когда входные данные находятся с одной стороны от этой гиперплоскости. По такой причине пороговый персептрон называют $\wedge$ линейным разделителем. На рис. 20.20, а, б показана такая гиперплоскость (которая в двух измерениях является линией) для представления с помощью персептрона функций AND и OR от двух входных значений. Черные кружки обозначают точки в пространстве входных данных, где значение этой функции равно 1, а белые кружки — точки, где это значение равно 0. Персептрон способен представить эти функции, поскольку есть какая-то линия, которая отделяет все белые кружки от всех черных кружков. Такие функции называются $\wedge$ линейно разделимыми. На рис. 20.20, в показан пример функции, которая не является линейно разделимой, — функции XOR. Безусловно, что пороговый персептрон не позволяет определить в процессе обучения эту функцию. Вообще говоря, $\wedge$ ЖЗ пороговые персептроны способны представить только линейно разделимые функции. Но такие функции составляют лишь небольшую часть всех функций; в упр. 20.14 предлагается определить величину этой части. Сигмоидальные персептроны характеризуются аналогичными ограничениями, с той поправкой, что они представляют только "мягкие" линейные разделители (см. рис. 20.19, б). ⁱ

1 . оО о 1 x2 а) X\ and X2 б)X\ or X2 в) X] хог X2 Рис. 20.20. Иллюстрация свойства линейной разделимости пороговых персептронов. Черные кружки показывают точки в пространстве входных данных, где значение функции равно 1, а белые кружки показывают точки, где это значение равно 0.

Персептрон возвращает 1 при получении данных из области, соответствующей незаштрихованной стороне линии. На рис. 20.20, в не существует такой линии, которая могла бы правильно классифицировать входные данные. Несмотря на их ограниченную выразительную мощь, пороговые персептроны имеют некоторые преимущества. В частности, Ф=* существует простой алгоритм обучения, позволяющий выполнить подгонку весов порогового персептрона к любому линейно разделимому обучающему множеству, но в данном разделе вместо описания этого алгоритма мы выведем тесно связанный с ним алгоритм для обучения с помощью сигмоидальных персептронов.

В основе этого алгоритма, а в действительности в основе большинства алгоритмов для обучения нейронных сетей, лежит такая идея, что в процессе обучения необходимо корректировать веса в сети для минимизации некоторого критерия измерения ошибок в обучающем множестве. Таким образом, задача обучения

формулируется как некоторая задача оптимизационного поиска в $\mathcal{W}$ пространстве весов.

"Классическим" критерием измерения ошибок является сумма квадратичных ошибок, которая использовалась в том числе в алгоритме линейной регрессии, приведенном на с. 1. Квадратичная ошибка для единственного обучающего примера с входными данными x и истинными выходными данными y определяется следующим образом:

$$E = \frac{1}{2} (y - h_w(x))^2$$
 где $h_w(x)$ — выходные данные персептрона при обработке рассматриваемого примера.

Для уменьшения квадратичной ошибки можно применить метод градиентного спуска, вычисляя частичную производную E по отношению к каждому весу. При этом может быть получено следующее соотношение:

8 Описание общих методов оптимизации, применимых к непрерывным пространствам, приведено в разделе 4.4.

$$\frac{\partial E}{\partial w_j} = -(y - h_w(x)) \cdot g'(x_j)$$
 где g' — производная функции активации⁹. В алгоритме градиентного спуска предусмотрено, что если требуется уменьшить E , то вес обновляется следующим образом:

$$w_j \leftarrow w_j + \alpha (y - h_w(x)) \cdot g'(x_j)$$
 где α — скорость обучения. Интуитивно ясно, что приведенное выше уравнение имеет большой смысл. Если ошибка $Err = y - h_w(x)$ является положительной, то выход сети слишком мал и поэтому веса увеличиваются при положительных входных данных и уменьшаются при отрицательных входных данных. А если ошибка является отрицательной, то происходит противоположное¹⁰.

Полный алгоритм приведен в листинге 20.1. Он предусматривает прогон обучающих примеров через сеть каждый раз по одному и небольшую корректировку весов после прогона каждого примера для уменьшения ошибки. Каждый цикл прогона примеров называется $\mathcal{W}$ эпохой. Эпохи повторяются до тех пор, пока не достигается некоторый критерий останова; как правило, такая ситуация достигается, когда изменения весов становятся очень небольшими. В других методах для всего обучающего набора вычисляется градиент путем сложения всех градиентных вкладов в уравнении 20.12 перед обновлением весов. А в методе $\mathcal{W}$ стохастического градиента примеры выбираются случайным образом из обучающего набора вместо их циклической обработки.

Листинг 20.1. Алгоритм обучения на основе градиентного спуска для персептронов, в котором предполагается использование дифференцируемой функции активации d_1 . Для пороговых персептронов коэффициент $d_1'(in)$ из обновления веса исключается. Функция `Neural-Net-Hypothesis` возвращает гипотезу, которая вычисляет выход сети для любого конкретного примера `function Perceptron-`

Learning(examples, network) returns гипотеза перцептрона inputs: examples, множество примеров, для каждого из которых заданы входные данные $x = x_1, \dots, x_n$ и выходные данные y network, перцептрон с весами W_j , где $j=0 \dots n$, и функцией активации g repeat for each e in examples do $S_n \leftarrow \sum_{j=0}^n W_j X_j[e]$ $Err \leftarrow |y[e] - g(S_n)|$ 9 Для сигмоидального перцептрона эта производная задается выражением $g' = g(A - g)$.

10 Для пороговых перцептронов, где производная g' (in) не определена, оригинальное правило обучения перцептрона, разработанное Розенблаттом [1302], идентично уравнению 20.12, однако в нем исключено значение $g'(in)$. Но поскольку значение g' (in) является одинаковым для всех весов, его исключение влияет только на величину, а не на направление общего обновления веса для каждого примера.

$W_j \leftarrow W_j + OC \cdot X_j[e] \cdot (y[e] - g(S_n))$ until не достигается некоторый критерий останова return Neural-Net-Hypothesis(network) На рис. 20.21 показаны кривые обучения перцептрона для двух разных задач.

Слева показана кривая для определения в процессе обучения мажоритарной функции с 11 булевыми входами (т.е. функция выводит 1, если на шести или больше входах присутствует 1). Как и следовало ожидать, перцептрон определяет эту функцию в процессе обучения очень быстро, поскольку мажоритарная функция является линейно разделимой. С другой стороны, обучающееся устройство, основанное на использовании дерева решений, не добивается существенного прогресса, поскольку мажоритарную функцию очень сложно (хотя и возможно) представить в виде дерева решений. Справа показана задача с рестораном. Решение этой задачи можно легко представить в виде дерева решений, но соответствующая функция не является линейно разделимой. Наилучшая гиперплоскость, разделяющая данные, позволяет правильно классифицировать только 65% примеров. li * ? 0,9] || 0,7 110,6 С О К О. \$?0,5 П х 0,4 0 10 20 30 40 50 60 70 80 90 Объем обучающего множества а) 0,4 .-х-* .х-х-х-х-к"

Х Перцептрон Дерево решений 100 0 10 20 30 40 50 60 70 80 90 100 Объем обучающего множества б) Рис. 20.21. Сравнение производительности перцептронов и деревьев решений: перцептроны показывают более высокую производительность при определении в процессе обучения мажоритарной функции от 11 входов (а); деревья решений показывают лучшую производительность при определении в процессе обучения предиката willWait в задаче с рестораном (б) До сих пор перцептроны рассматривались как средства реализации детерминированных функций, выходные данные которых могут содержать ошибки. Выход сигмоидального перцептрона можно также интерпретировать как вероятность, а именно вероятность того, что истинным выходом является 1, если заданы все входы. При такой интерпретации сигмоидальный перцептрон может использоваться как каноническое представление для условных распределений в байесовских сетях (см. раздел 14.3). Существует также возможность выполнять вероятностный вывод

правила обучения с использованием стандартного метода максимизации (условного) логарифмического правдоподобия данных, как было описано выше в данной главе.

Рассмотрим, как действует последний метод. Допустим, что рассматривается единственный обучающий пример с истинным выходным значением T , и предположим, что p — вероятность, возвращаемая персептроном для этого примера. Если $t=1$, то условная вероятность этих данных равна

p , а если $T=0$, то условная вероятность данных равна $A-p$). Теперь можно воспользоваться простым приемом, чтобы записать интересующее нас выражение для логарифмического правдоподобия в дифференцируемой форме. Этот прием состоит в том, что переменная со значениями 0/1 в экспоненте выражения рассматривается как T индикаторная переменная: $p^T(1-p)^{1-T}$, если $T=1$, и $1-p$ — в противном случае; аналогичным образом, $(1-p)^T p^{1-T}$, если $T=0$, и p — в противном случае. Поэтому можно записать выражение для логарифмического правдоподобия данных следующим образом:

$L = \log p^T(1-p)^{1-T} = T \log p + (1-T) \log (1-p)$ (20.13) В силу самих свойств сигмоидальной функции полученный градиент сводится к очень простой формуле (упр. 20.16):

$\frac{\partial L}{\partial p} = T - p$; Обратите внимание на то, что $\frac{\partial L}{\partial p}$ вектор обновления весов для обучения с максимальным правдоподобием в сигмоидальных персептронах по сути идентичен вектору обновления для минимизации квадратичной ошибки. Таким образом, можно утверждать, что персептроны имеют вероятностную интерпретацию, даже если правило их обучения выведено на основании детерминированного подхода.

Многослойные нейронные сети с прямым распространением Теперь рассмотрим сети со скрытыми элементами. Наиболее общий случай охватывает сети с одним скрытым слоем, как показано на рис. 20.23. Преимущество введения скрытых слоев состоит в том, что они приводят к расширению пространства гипотез, которые могут быть представлены сетью. Каждый скрытый элемент можно рассматривать как персептрон, который представляет мягкую пороговую функцию в пространстве входов (см. рис. 20.19, б). В таком случае любой выходной элемент должен рассматриваться как средство создания линейной комбинации нескольких таких функций с мягким порогом. Например, путем сложения двух противоположно направленных мягких пороговых функций и определения порогового значения результата можно получить функцию с графиком в виде "хребта"

(рис. 20.22, а). Соединение двух таких хребтов под прямыми углами друг к другу (например, комбинирование выходов четырех скрытых элементов) позволяет получить "выступ" (см. рис. 20.22, б).

11 Некоторые специалисты называют сеть с одним скрытым слоем трехслойной сетью, а другие — двухслойной сетью (поскольку считают, что входные элементы не являются "настоящими" элементами). Чтобы избежать путаницы, авторы будут называть такую сеть "сетью с одним скрытым слоем".

а) б) Рис. 20.22. Примеры применения мягких пороговых функций: результат комбинирования двух противоположно направленных мягких пороговых функций для создания хребта (а); результат комбинирования двух хребтов для создания выступа (б) При наличии большого количества скрытых элементов появляется возможность создавать еще больше выступов различных размеров, расположенных в разных местах. В действительности с помощью одного, достаточно большого скрытого слоя возможно представить любую непрерывную функцию от входных данных с произвольной точностью, а при наличии двух слоев можно представить даже функции с разрывами¹². К сожалению, применительно к любой конкретной сетевой структуре сложно точно охарактеризовать, какие функции могут быть представлены с ее помощью, а какие — нет.

Предположим, что необходимо сконструировать сеть со скрытым слоем для задачи с рестораном. Имеется 10 атрибутов, описывающих каждый пример, поэтому требуется 10 входных элементов. А сколько нужно скрытых элементов? На рис. 20.23 показана сеть с четырьмя скрытыми элементами. Как оказалось, такое количество скрытых элементов почти полностью подходит для решения данной задачи. Но проблема заблаговременного выбора подходящего количества скрытых элементов все еще полностью не исследована (с. 990).

Алгоритмы обучения для многослойных сетей подобны алгоритму обучения для персептронов, приведенному в листинге 20.1. Одно небольшое различие состоит в том, что может быть предусмотрено несколько выходов, поэтому должен применяться вектор выходов $hw(x)$, а не одно значение, и с каждым примером должен быть связан вектор выходов y . Между этими алгоритмами существует также важное различие, которое заключается в том, что ошибка $y - hw$ в выходном слое является очевидной, а ошибка в скрытых слоях кажется неуловимой, поскольку в обучающих данных отсутствует информация о том, какие значения должны иметь скрытые узлы.

Как оказалось, можно обеспечить $\wedge$ обратное распространение ошибки из выходного слоя в скрытые слои. Процесс обратного распространения может быть организован непосредственно на основе исследования общего градиента ошибки. Вначале мы опишем этот процесс на основе интуитивных представлений, а затем приведем его обоснование.

12 Доказательство этого утверждения является сложным, но основное следствие из него состоит в том, что необходимое количество скрытых элементов растет

экспоненциально в зависимости от количества входов. Например, чтобы закодировать все булевы функции от p входов, требуется $2^n/p$ скрытых элементов ($M \cdot p^2$)

Выходные элементы a { Скрытые элементы d . Входные элементы a_k Рис. 20.23. Многослойная нейронная сеть с одним скрытым слоем и 10 входами, применяемая для решения задачи с рестораном. Правило обновления весов, применяемое в выходном слое, идентично уравнению 20.12. Предусмотрено несколько выходных элементов, поэтому предположим, что E_{gg} является i -м компонентом вектора ошибки $y - h_w$. Авторы находят также удобным определить модифицированную ошибку $A_i = E_{gg} \cdot g'(h_{wi})$, с помощью которой правило обновления весов можно преобразовать следующим образом:

$W_{j,i} \leftarrow W_{j,i} + \alpha \sum_k x_{kj} \cdot A_i$ (20.14) Чтобы обновить веса связей между входными и скрытыми элементами, необходимо определить величину, аналогичную терму ошибки для выходных узлов.

Именно в этом и заключается суть метода обратного распространения ошибки. Идея его состоит в том, что скрытый узел j "отвечает" за некоторую долю ошибки A_i в каждом из выходных узлов, с которыми он соединен. Таким образом, значения A_i разделяются в соответствии с весом связи между скрытым узлом и выходным узлом и распространяются обратно, обеспечивая получение значений A_j для скрытого слоя.

Правило распространения для значений A состоит в следующем:

$A_j = g'(h_{ji}) \cdot \sum_i W_{ji} \cdot A_i$ (20.15) Теперь правило обновления весов между входными элементами и элементами скрытого слоя становится почти идентичным правилу обновления для выходного слоя:

$W_{s,j} \leftarrow W_{s,j} + \alpha \sum_k x_{ks} \cdot A_j$ Таким образом, процесс обратного распространения можно кратко описать, как показано ниже.

- Вычислить значения A для выходных элементов с использованием наблюдаемой ошибки.
- Начиная с выходного слоя, повторять следующие действия для каждого слоя в сети до тех пор, пока не будет достигнут самый первый скрытый слой:
- распространять значения A обратно к предыдущему слою;
- обновлять веса между этими двумя слоями.

Подробный алгоритм приведен в листинге 20.2. Листинг 20.2. Алгоритм обратного распространения для обучения в многослойных сетях

```
function Back-Prop-Learning(examples, network)
    returns нейронная сеть
    inputs: examples, множество примеров, для каждого из которых заданы входной вектор  $x$  и выходной вектор  $y$ 
    network, многослойная сеть с  $L$  слоями, весами  $W_{j,i}$  и функцией активации  $g$ 
    repeat
    for each  $e$  in examples do
    for each узел  $j$  in выходной слой do
     $a_j \leftarrow x_j[e]$ 
    for  $? = 2$  to  $L$  do
     $ini \leftarrow y$ 
    for each узел  $i$  in выходной слой do
     $A_i \leftarrow g'(h_{di}) \cdot \sum_j W_{ji} \cdot a_j$ 
```

- aL) for ? = L-1 to 1 do for each узел j in слой ? do $A_j \leftarrow -g'(in_j) \cdot W_{ji}$ Ai for each узел i in слой ?+1 do $W_{ji} \leftarrow W_{ji} + \alpha \cdot x_j \cdot A_i$ until не достигается некоторый критерий останова return Neural-Net-Hypothesis(network) Теперь исходя из основных принципов выведем уравнения обратного распространения к радости читателей, интересующихся математикой. Квадратичная ошибка на одном примере определяется следующим образом: $E = \frac{1}{2} \sum a_i^2$ где сумма вычисляется по всем узлам выходного слоя. Чтобы определить градиент по отношению к конкретному весу W_{ji} в выходном слое, достаточно развернуть только выражение для активации a_i поскольку все другие термы в этой операции суммирования не зависят от P_i/j , i.: $\frac{dE}{da_i} = g'(in_j) \cdot a_i$ $\frac{dE}{dW_{ji}} = \frac{dE}{da_i} \cdot \frac{da_i}{dW_{ji}} = g'(in_j) \cdot a_i \cdot x_j$ $\frac{dE}{dW_{ji}} = -a_j \cdot A_i$ где A_i определено, как указано выше. Чтобы получить градиент по отношению к весам w_{kj} связей, соединяющих входной слой со скрытым слоем, необходимо попрежнему вычислять всю сумму по i, поскольку изменение в значениях P_i/k могут повлиять на каждое выходное значение a_i . При этом придется также развертывать

выражение для активаций a_j . Ниже показан ход вывода формулы во всех подробностях, поскольку любопытно понаблюдать за тем, как оператор производной распространяется в обратном направлении через сеть: $\frac{dE}{dW_{kj}} = \frac{dE}{da_j} \cdot \frac{da_j}{dW_{kj}} = g'(in_j) \cdot a_j \cdot x_k$ $\frac{dE}{dW_{kj}} = -a_k \cdot A_j$ где A_j определено, как показано выше. Таким образом, получено то же правило обновления, которое было сформулировано выше на основании интуитивных соображений. Очевидно также, что этот процесс может быть продолжен применительно к сетям, имеющим больше одного скрытого слоя, и это соображение является обоснованием для общего алгоритма, приведенного в листинге 20.2.

Внимательно ознакомившись со всеми этими математическими выкладками (или пропустив их), рассмотрим, какую производительность показывает сеть с одним скрытым слоем при решении задачи с рестораном. На рис. 20.24 показаны две кривые. Первая из них представляет собой кривую обучения, которая характеризует изменение среднеквадратичной ошибки на заданном обучающем множестве из 100 примеров для задачи с рестораном в процессе обновления весов. Эта кривая демонстрирует, что параметры сети действительно сходятся, позволяя достичь идеального согласования с обучающими данными. Вторая кривая представляет собой стандартную кривую обучения для данных о ресторане. Нейронная сеть позволяет достичь в процессе обучения хороших результатов, хотя и не совсем с такой скоростью, как при обучении дерева решений; по-видимому, в этом нет ничего удивительного, хотя бы потому, что сами эти данные были сформированы с помощью простого дерева решений.

Безусловно, нейронные сети способны решать гораздо более сложные задачи обучения, хотя и следует отметить, что для создания подходящей сетевой структуры и достижения сходимости к величине, достаточно близкой к глобальному оптимуму в пространстве весов, необходимо приложить определенные усилия. Количество приложений нейронных сетей, описанных в опубликованной литературе, исчисляется буквально десятками тысяч. В разделе 20.7 одно из таких приложений рассматривается более подробно.

0 50 100 150 200 250 300 350 400 0 10 20 30 40 50 60 70 80 90 100 Количество эпох
 Объем обучающего множества а) б) Рис. 20.24. Результаты анализа производительности алгоритма обучения: кривая обучения, показывающая постепенное уменьшение ошибки по мере того, как в течение нескольких эпох происходит модификация весов для заданного множества примеров в задаче с рестораном (а); сопоставление кривых обучения, которое показывает, что обучение дерева решений обеспечивает немного лучшие результаты, чем обратное распространение в многослойной сети (б) Определение в процессе обучения структур нейронных сетей До сих пор в данной главе рассматривалась проблема определения в процессе обучения весов связей при наличии заданной структуры сети; но, как и в случае с байесовскими сетями, необходимо также знать, как найти наилучшую структуру сети. Если будет выбрана сеть, которая слишком велика, она будет способна запомнить все примеры благодаря формированию большой поисковой таблицы, но не обязательно обеспечит качественное обобщение применительно к входным данным, с которыми еще не сталкивалась¹³. Иными словами, как и все статистические модели, нейронные сети подвержены такому недостатку, как чрезмерно тщательная подгонка, когда количество параметров в модели слишком велико. В этом можно убедиться, рассматривая рис. 18.1, где показано, что модели с большим количеством параметров на рис. 18.1, б, в хорошо согласуются со всеми данными, но не обеспечивают такого же хорошего обобщения, как модели с малым количеством параметров, показанные на рис. 18.1, а и г.

Если мы остановимся исключительно на полносвязных сетях, то единственные изменения, которые могут быть внесены в структуру сети, относятся только к количеству скрытых слоев и их размерам. Обычно применяется подход, который предусматривает опробование нескольких вариантов и сохранение наилучшего. Если же решено придерживаться требования по предотвращению компрометации проверочного набора, то необходимо использовать методы перекрестной проверки, описанные в главе 18. Это означает, что выбор останавливается на такой архитектуре сети, ко¹³ Было замечено, что очень большие сети все же позволяют обеспечивать хорошее обобщение, при условии, что веса остаются небольшими. Это ограничение равносильно тому, что значения активации остаются в линейной

области сигмоидальной функции $d(x)$, где переменная x близка к нулю. Это, в свою очередь, означает, что сеть действует по аналогии с линейной функцией (упр.

20.17) с гораздо меньшим количеством параметров.

торая обеспечивает наивысшую точность прогнозирования при проверке с помощью испытательных наборов.

Если же решено прибегнуть к использованию сетей, которые не являются полносвязными, то необходимо найти какой-то эффективный метод поиска в очень большом пространстве возможных топологий связей. Алгоритм $\wedge$ оптимального повреждения мозга (optimal brain damage) начинает свою работу с полносвязной сети и удаляет из нее связи. После обучения сети в первый раз с помощью подхода на основе теории информации определяется оптимальный выбор связей, которые могут быть удалены.

После этого осуществляется повторное обучение сети, и если ее производительность не уменьшается, то этот процесс повторяется. Кроме удаления связей, возможно также удаление элементов, которые не вносят большого вклада в результат.

Было также предложено несколько алгоритмов для формирования большой сети из малой. Один из таких алгоритмов, называемый алгоритмом $\wedge$ заполнения мозаики (tiling), напоминает алгоритм обучения списков решений. Его идея состоит в том, что процесс начинается с одного элемента, который настраивается наилучшим образом для выработки правильных выходных данных для максимально возможного количества обучающих примеров. Для обработки примеров, которые не могли быть правильно обработаны первым элементом, вводятся дополнительные элементы.

Этот алгоритм предусматривает введение только такого количества элементов, которое требуется для охвата всех примеров.

20.6. Ядерные машины Приведенное выше описание нейронных сетей не дает ответа на одну дилемму.

Однослойные сети позволяют использовать простой и эффективный алгоритм обучения, но обладают лишь очень ограниченной выразительной мощностью, поскольку способны определять в процессе обучения только линейные границы между решениями в пространстве входов. Многослойные сети, с другой стороны, являются гораздо более выразительными (они способны представлять нелинейные функции общего вида), но задача их обучения становится очень сложной из-за большого количества локальных минимумов и высокой размерности пространства весов. В этом разделе рассматривается относительно новое семейство методов обучения, основанных на использовании $\wedge$ машин поддерживающих векторов

(Support Vector Machine— SVM), или, в более общем смысле, "ядерных машин (kernel machine).

Ядерные машины позволяют в определенной степени воспользоваться наилучшими свойствами и однослойных, и многослойных сетей. Это означает, что в методах, основанных на их использовании, предусмотрен эффективный алгоритм обучения, а сами они позволяют представить сложные, нелинейные функции.

Полное описание ядерных машин выходит за рамки данной книги, но мы можем проиллюстрировать их основную идею на примере. На рис. 20.25, а показано двумерное пространство входов, определяемое атрибутами $x = (x_1/x_2)$, в котором положительные примеры ($y = +1$) находятся внутри круга, а отрицательные примеры ($y = -1$) — вне его.

Очевидно, что для данной задачи не существует линейного разделителя. А теперь предположим, что входные данные выражены иначе, с помощью каких-то вычислимых характеристик, т.е. что каждый вектор входных данных x отображен на новый век-

тор значений характеристик, $F(x)$. В частности, предположим, что используются следующие три характеристики: $f_1 = x_1^2$, $f_2 = x_2^2$, $f_3 = x_1 x_2$ (рис. 20.25, б). Вскоре будет показано, как получены эти выражения, а пока просто рассмотрим, что происходит. На рис. 20.25, б показаны данные в этом новом, трехмерном пространстве, определенном тремя характеристиками; очевидно, что данные в этом пространстве являются линейно разделимыми! Такой подход действительно является достаточно общим: если данные отображаются на пространство с достаточно большим количеством размерностей, то они всегда могут быть преобразованы в линейно разделимую форму. В данном случае использовались только три размерности, но если бы количество точек данных было равно N , то, за исключением частных случаев, они всегда являются разделимыми в пространстве с $N-1$ размерностями или больше (упр. 20.21). Рис. 20.25. Пример применения преобразования: двумерная задача обучения с положительными примерами, показанными в виде черных кружков, и отрицательными примерами, обозначенными белыми кружками. Показана также истинная граница решений, $x_1^2 + x_2^2 < 1$ (а); те же данные после отображения на трехмерное пространство входов $(x_1^2, x_2^2, x_1 x_2)$.

Круглая граница решений, показанная на рис. 20.25, а, в трехмерном пространстве становится линейной границей решения (б). Итак, можно ли считать, что на этом проблема исчерпана? Достаточно ли просто подготовить целый ряд расчетных характеристик, а затем найти линейный разделитель в соответствующем многомерном пространстве? К сожалению, все не так просто. Напомним, что линейный разделитель в пространстве с d размерностями определяется уравнением

с d параметрами, поэтому возникает серьезная опасность 14 Читатель мог заметить, что достаточно было использовать только характеристики f_1 и f_2 , но трехмерное отображение позволяет лучше проиллюстрировать эту идею.

чрезмерно тщательной подгонки данных, если $d \sim N$, т.е. приблизительно равно количеству точек данных. (Такая ситуация аналогична чрезмерно тщательной подгонке к данным с помощью полинома высокой степени, о чем шла речь в главе 18.) По этой причине ядерные машины обычно находят оптимальный линейный разделитель, т.е. такой разделитель, который имеет наибольший γ край между ним и положительными примерами, с одной стороны, и отрицательными примерами, с другой (рис. 20.26). Можно показать, используя аргументы, основанные на теории вычислительного обучения (см. раздел 18.5), что такой разделитель обладает желаемыми свойствами с точки зрения возможности надежного обобщения новых примеров.

Рис. 20.26. Замыкание оптимального разделителя, показанного на рис. 20.25, 6, спроектированное на первые две размерности.

Разделитель показан в виде жирной линии, а ближайшие тонки (поддерживающие векторы) обозначены кружками. Край представляет собой разделение между положительными и отрицательными примерами. Но как найти такой разделитель? Оказалось, что эта задача представляет собой задачу оптимизации из области γ квадратичного программирования. Предположим, что имеются примеры X_i с классификациями $y_i = \pm 1$ и необходимо найти оптимальный разделитель в пространстве входов; в таком случае задача квадратичного программирования сводится к поиску значений параметров a_i которые максимизируют следующее выражение с учетом ограничений $a_i > 0$ и $\sum_i a_i y_i = 0$: $\sum_i \sum_j a_i a_j y_i y_j (X_i - X_j)^2$ (B0.17) i, j Хотя для понимания излагаемого материала не обязательно знакомиться с тем, как было выведено данное выражение, следует отметить, что оно имеет два важных свойства.

Во-первых, это выражение имеет единственный глобальный максимум, который может быть найден с помощью эффективных методов. Во-вторых, $\sum_i a_i y_i = 0$ данные входят в это выражение только в форме точенных произведений пар точек. Утверждение о существовании

такого второго свойства является также справедливым применительно к уравнению для самого разделителя; как только будут вычислены оптимальные значения a_i появляется возможность вычислить следующее выражение:

B0.18) $h(x) = \text{sign}[\sum_i a_i y_i (x - x_i)]$ Последнее важное свойство оптимального разделителя, определяемого этим уравнением, заключается в том, что веса a_i связанные с каждой точкой данных, являются нулевыми, кроме тех точек, которые являются ближайшими к разделителю; эти точки называются γ поддерживающими

векторами (они получили такое название потому, что на них как бы "опирается" разделяющая гиперплоскость).

Поскольку количество поддерживающих векторов обычно намного меньше, чем количество точек данных, результирующее количество параметров, определяющих оптимальный разделитель, также намного меньше, чем N . Итак, обычно не стоит рассчитывать на то, что удастся найти линейный разделитель в пространстве входов x , но легко показать, что можно найти линейные разделители в многомерном пространстве характеристик $F(x)$, просто заменяя точечное произведение $x^i x^j$ в уравнении 20.17 произведением $F(x_i) F(x_j)$. В этой операции, отдельно взятой, нет ничего необычного (поскольку требуемый эффект может быть достигнут путем замены x на $F(x)$ в любом алгоритме обучения), но точечное произведение обладает некоторыми особыми свойствами. Как оказалось, значение $F(x_i) F(x_j)$ часто можно вычислить без предварительного вычисления характеристики F для каждой точки. В рассматриваемом трехмерном пространстве, определяемом уравнением 20.16, с помощью несложных алгебраических преобразований можно показать, что справедливо следующее соотношение:

$F(x_i) \cdot F(x_j) = (x_i \cdot x_j)$ Выражение $(x_i \cdot x_j)$ называется ядерной функцией и обычно записывается как $\kappa(x_i/x_j)$. В контексте проблематики ядерных машин под этим подразумевается любая функция, которая может быть применена к парам выходных данных для вычисления точечных произведений в некотором соответствующем пространстве характеристик. Таким образом, утверждение, приведенное в начале данного абзаца, можно сформулировать следующим образом: линейные разделители в многомерном пространстве характеристик $F(x)$ можно найти, заменив выражение $x_i \cdot x_j$ в уравнении 20.17 ядерной функцией $\kappa(x_i/x_j)$. Таким образом, может быть организовано обучение в многомерном пространстве, но для этого придется вычислять только значения ядерных функций, а не значения полного списка характеристик для каждой точки данных.

Следующий этап, который теперь должен стать полностью очевидным, состоит в демонстрации того, что в ядерной функции $\kappa(x_i/x_j) = (x_i \cdot x_j)$ нет ничего особенного, просто она соответствует конкретному пространству характеристик с большим количеством размерностей, а другие ядерные функции соответствуют другим пространствам характеристик. Один из знаменитых результатов в математике, ^ теорема Мерсера

[1035], гласит, что любая "приемлемая" ядерная функция¹⁵ соответствует некоторому пространству характеристик. Эти пространства характеристик могут оказаться очень большими даже применительно к таким ядерным функциям, которые выглядят совершенно "невинно". Например, ^ полиномиальная ядерная функция, $\kappa(x_i/x_j) = A + x_i \cdot x_j$, соответствует пространству характеристик, количество размерностей которого определяется экспоненциальной зависимостью

от d . Поэтому использование ядерных функций, подобных приведенной в уравнении 20.17, ϕ обеспечивает эффективный поиск оптимальных линейных разделителей в пространствах характеристик с миллиардами (а в некоторых случаях с бесконечным количеством) размерностей. Полученные в результате линейные разделители после их обратного отображения на первоначальное пространство входов могут соответствовать произвольно сложным, нелинейным границам между положительными и отрицательными примерами.

Как было упомянуто в предыдущем разделе, ядерные машины превосходят все другие способы распознавания рукописных цифр; кроме того, они быстро находят применение и в других приложениях, особенно в тех, которые отличаются большим количеством входных характеристик. В составе этого процесса было разработано много новых ядерных функций, позволяющих работать со строками, деревьями и другими нечисловыми типами данных. Было также отмечено, что метод ядерных функций может применяться не только в алгоритмах обучения, которые находят оптимальные линейные разделители, но и в любых других алгоритмах, которые можно переформулировать применительно к использованию только точечных произведений пар точек данных, как в уравнениях 20.17 и 20.18. После выполнения такого преобразования точечное произведение заменяется ядерной функцией, что приводит к получению версии того же алгоритма, ϕ преобразованной в ядерную форму.

Кроме всего прочего, такое преобразование можно легко применить для обучения с ближайшими соседними точками и для обучения персептрона.

20.7. Практический пример: распознавание рукописных ЦИФР Задача распознавания рукописных цифр является важной во многих приложениях, включая автоматизированную сортировку почты по почтовому коду, автоматизированное чтение чеков и налоговых деклараций, а также ввод данных для портативных компьютеров. В этой области достигнут быстрый прогресс отчасти благодаря применению лучших алгоритмов обучения и отчасти благодаря наличию лучших обучающих наборов. В Национальном институте стандартов и технологии (National Institute of Standards and Technology — NIST) США был создан архив, представляющий собой базу данных из 60 000 рукописных цифр с расшифровками (с так называемой разметкой), каждая из которых задана в виде изображения, состоящего из $20 \times 20 = 400$ пикселей с восьмибитовой кодировкой уровня серого. Распознавание 15 Здесь под понятием "приемлемой" ядерной функции подразумевается, что матрица K $K(x_i, x_j)$ является положительной и определенной; см. приложение А.

цифр из этой базы данных стало одной из стандартных эталонных задач для сравнения новых алгоритмов обучения. Некоторые примеры цифр показаны на рис. 20.27. $\wedge$]/Мз1чйА \9\\0\7*b\B I\|\ 11ч1 °> Рис. 20.27. Примеры из базы данных

рукописных цифр института NIST. Верхний ряд: примеры цифр 0—9, которые несложно распознать; нижний ряд: более трудные примеры тех же цифр. Для решения этой задачи было опробовано много разных подходов к обучению.

Одним из первых и, по-видимому, самых простых стал классификатор по трем ближайшим соседним точкам, преимуществом которого является также то, что он не требует затрат времени на обучение. Но поскольку он представляет собой алгоритм, основанный на использовании памяти, то должен хранить все 60 000 изображений, поэтому показывает низкую производительность на этапе прогона. При использовании такого классификатора была достигнута частота ошибок при распознавании проверочного набора, равная 2,4%.

Кроме того, для решения этой задачи была разработана нейронная сеть с одним скрытым слоем, состоящая из 400 входных элементов (по одному на каждый пиксел) и 10 выходных элементов (по одному на каждый класс). С использованием перекрестной проверки было обнаружено, что наилучшую производительность обеспечивают примерно 300 скрытых элементов. Поскольку была предусмотрена полная связность между слоями, общее количество весов составляло 123300. Эта сеть позволила достичь уровня частоты ошибок 1,6%.

Чтобы можно было воспользоваться структурой самой задачи (тем, что вход состоит из пикселов в двумерном массиве и что небольшие изменения в положении или наклоне изображения не имеют значения), был разработан ряд специализированных нейронных сетей, называемых LeNet. Каждая сеть имела входной слой из 32×32 элементов, по которым 28×28 пикселов отцентровывались таким образом, чтобы каждый входной элемент был представлен с помощью локальной окрестности пикселов. За ними следовали три слоя скрытых элементов. Каждый слой состоял из нескольких гиперплоскостей из $p \times p$ массивов, где значение p было меньше, чем в предыдущем слое, с тем чтобы сеть сокращала объем выборки входных данных, и где веса каждого элемента в гиперплоскости ограничивались идентичными весами, с тем чтобы эта гиперплоскость действовала как детектор какой-то характеристики: она позволяла выделять некоторую характеристику, такую как длинная вертикальная линия или короткая полукруглая дуга. Выходной слой включал 10 элементов. Было опробовано много версий такой архитектуры; в одной из наиболее успешных версий применялись скрытые слои, состоящие соответственно из 768, 192 и 30 элементов. Обучающий набор был дополнен путем применения аффинных преобразований к фактическим входным данным — сдвиг, поворот на небольшой угол и масштабирование изображений (безусловно, эти преобразования должны были

быть небольшими, поскольку иначе цифра 6 могла бы превратиться в 9!).

Наилучшим значением частоты ошибок, достигнутым с помощью сетей LeNet, было 0,9%.

В усиленной нейронной сети комбинировались три копии архитектуры LeNet, притом что вторая обучалась на смеси образцов, которые первая распознавала с ошибками 50%, а третья обучалась на образцах, для которых не было достигнуто согласование с помощью первых двух копий. Во время проверки проводилось голосование между тремя сетями с назначенными с помощью их весами для каждой из десяти цифр, а полученные оценки складывались для определения победителя.

Частота ошибок при обработке проверочного набора составляла 0,7%.

Машина поддерживающих векторов (см. раздел 20.6) с 25000 поддерживающих векторов достигла частоты ошибок 1,1%. Это — замечательное достижение, поскольку метод SVM, как и простой подход с использованием ближайших соседних точек, почти не потребовал размышлений или неоднократных экспериментов со стороны разработчика, но вместе с тем позволил сразу же приблизиться к производительности сетей LeNet, на создание которых ушли годы интенсивных работ.

Разумеется, в машине поддерживающих векторов не используются данные о структуре задачи, поэтому она действовала бы с таким же успехом, если бы те же пиксели были представлены после применения к ним какой-то перестановки. ^ Виртуальная машина поддерживающих векторов начинает работу с обычной машины SVM, а затем совершенствует ее с помощью метода, позволяющего воспользоваться данными о структуре задачи. В этом подходе не разрешается использовать произведения всех пар пикселей — вместо этого в основном применяются ядерные функции, сформированные с помощью пар ближайших пикселей. В нем также была предусмотрена возможность дополнять обучающий набор преобразованными вариантами примеров, как и в проекте LeNet. Виртуальная машина SVM достигла наилучшего показателя частоты ошибок, зарегистрированного до настоящего времени, который равен 0,56%.

Согласование с формой — это метод из области машинного зрения, который используется для выравнивания соответствующих частей двух различных изображений объектов (см. главу 24). Идея этого метода состоит в том, что выбирается множество точек каждого из двух изображений, а затем для каждой точки из первого изображения с помощью вычислений определяется, какая точка соответствует ей во втором изображении. После этого на основании полученных данных о выравнивании вычисляется преобразование между изображениями, которое позволяет определить значение критерия расстояния между этими изображениями. Такой критерий расстояния является более обоснованным по сравнению с простым подсчетом количества различающихся пикселей, и, как

оказалось, очень высокую производительность показывает алгоритм с тремя ближайшими соседними точками, в котором используется этот критерий расстояния. После обучения только на 20000 из 60000 цифр и с использованием 100 выборочных точек в расчете на каждое изображение, выделенных с помощью детектора края Кэнни, классификатор с согласованием формы достиг частоты ошибок при обработке проверочного набора, равной 0,63%.

По некоторым оценкам, люди допускают ошибки при решении задачи распознавания рукописных цифр с частотой примерно 0,2%. Но этим данным не следует полностью доверять, поскольку отнюдь не проводилась такая исчерпывающая проверка способностей людей, как самих алгоритмов машинного обучения. На анало-

гичном наборе данных, состоящем из цифр, полученных из почтовой службы США, частота ошибок, допущенных людьми, составляла примерно 2,5%.

Цифры, приведенные ниже, представляют собой итоговые данные по частоте ошибок, производительности на этапе прогона, потребностям в памяти и продолжительности времени обучения для семи описанных здесь алгоритмов. К этим данным добавлен еще один критерий — процентное количество цифр, которые должны быть отброшены, чтобы можно было достичь частоты ошибок 0,5%. Например, если при использовании алгоритма SVM разрешено отбрасывать 1,8% входных данных (т.е. передавать их кому-то другому, чтобы он сделал окончательное заключение), то частота ошибок на оставшихся 98,2% входных данных сокращается с 1,1% до 0,5%.

Итоговые данные по частоте ошибок и некоторых других характеристиках семи методов, описанных в этом разделе, приведены в табл. 20.2.

Таблица 20.2. Итоговые данные о семи методах распознавания рукописных цифр

Метод	Частота ошибок (проценты)	Время выполнения (мс/цифра)	Потребность в памяти (Мбайт)	Время обучение (суток)	Процент отвергнутых примеров, позволяющий достичь ошибки 0,5%
С тремя ближайшими соседними точками	2,4	8,1	С 300 скрытых - элементов	1,6	0,49
LeNet	0,9	0,012	1,8	Усиленная LeNet	0,7
Виртуальная SVM	0,56	0,21	0,5	SVM	1,1
Согласование формы	0,63	20,8	Резюме	Разработан широкий перечень статистических методов обучения, начиная от простого вычисления средних и заканчивая построением сложных моделей, таких как байесовские и нейронные сети. Эти методы находят широкое применение в компьютерной науке, техническом проектировании, нейробиологии, психологии и физике. В данной главе представлены некоторые основные идеи из этой области и приведена часть математических выкладок. Основные темы, рассматриваемые в этой главе, перечислены ниже.	

• В методах байесовского обучения задача обучения формулируется как один из видов вероятностного вывода, в котором наблюдения

используются для обновления распределений априорных вероятностей по гипотезам. Такой подход представляет собой хороший способ реализации принципа бритвы Оккама, но быстро становится трудноосуществимым при возрастании сложности пространства гипотез.

- В обучении на основе максимальной апостериорной вероятности (Maximum A Posteriori — MAP) выбирается единственная гипотеза, наиболее вероятная согласно имеющимся данным. При этом все еще используется распределение априорных вероятностей гипотезы и сам этот метод часто является более легко осуществимым, чем полное байесовское обучение.
- В обучении с учетом максимального правдоподобия выбирается гипотеза, которая максимизирует правдоподобие данных; этот метод эквивалентен методу обучения MAP с равномерным распределением априорных вероятностей.

В простейших случаях, таких как линейная регрессия и полностью наблюдаемые байесовские сети, решения с учетом максимального правдоподобия можно легко найти в замкнутой форме. Особенно эффективным методом, который хорошо масштабируется, является наивное байесовское обучение.

- Если некоторые переменные скрыты, то решения с локальным максимальным правдоподобием можно найти с использованием алгоритма EM. В число приложений соответствующего метода входит кластеризация с помощью смешанных гауссовых распределений, обучение байесовских сетей и скрытых марковских моделей.
- Определение в процессе обучения структур байесовских сетей представляет собой пример метода выбора модели. В этом методе обычно предусматривается дискретный поиск в пространстве структур. При этом необходимо каким-то образом обеспечить поиск компромисса между сложностью модели и степенью ее соответствия данным.
- В моделях на основе экземпляра распределение представлено с использованием коллекции обучающих экземпляров. Таким образом, количество параметров растет с увеличением размеров обучающего множества. В методах ближайшей соседней точки осуществляется поиск экземпляров, ближайших к рассматриваемой точке, а в ядерных методах формируется комбинация всех экземпляров, взвешенная по расстоянию.
- Нейронные сети позволяют определять в процессе обучения сложные нелинейные функции со многими параметрами. Изучение этих параметров может осуществляться с применением зашумленных данных, а сами нейронные сети используются в тысячах приложений.
- Персептрон — это нейронная сеть с прямым распространением без скрытых элементов, которая способна представить только линейно разделимые функции. Если данные являются линейно разделимыми, то для обеспечения точного согласования с данными может использоваться простое правило обновления весов.
- Такие нейронные сети, как многослойные сети с прямым распространением, способны представлять любые функции при наличии

достаточного количества элементов. В алгоритме обратного распространения реализуется метод градиентного спуска в пространстве параметров для минимизации выходной ошибки.

Статистическое обучение продолжает оставаться очень активной областью исследования. Были достигнуты колоссальные успехи как в теории, так и в практике, и уровень знаний поднялся на такую высоту, что теперь существует возможность оп-ределить в процессе обучения параметры почти любой модели, для которой осуществим точный или приближенный вероятностный вывод.

Библиографические и исторические заметки В ранние годы развития искусственного интеллекта приложения статистических методов обучения были активной областью исследований [421], но отделились от основного направления искусственного интеллекта после того, как работы в этом направлении сосредоточились на символических методах. Тем не менее исследования статистических методов обучения продолжались в различных формах (часть которых явно относилась к вероятностным, а другая — нет) в таких областях, как распознавание образов [394] и информационный поиск [1347]. Возрождение всеобщего интереса к этой теме началось вскоре после появления моделей байесовских сетей в конце 1980-х годов; приблизительно в то же время начала формироваться статистическая научная трактовка процесса обучения нейронных сетей. В конце 1990-х годов произошло заметное пробуждение интереса к машинному обучению, статистике и нейронным сетям и значительные усилия были сосредоточены на разработке методов создания больших вероятностных моделей на основе данных.

Наивная байесовская модель представляет собой одну из самых старых и наиболее простых форм байесовской сети, которая была впервые описана в 1950-х годах.

О том, каково происхождение этой модели, упоминалось в заметках в конце главы 13; частичное описание этой темы приведено в [402]. Программа на основе усиленной формы наивного байесовского обучения стала победителем первого соревнования по интеллектуальному анализу данных на кубок KDD Сир [435]. В [641] приведено превосходное введение в общую проблему обучения байесовской сети.

Определение параметров байесовской сети с помощью распределений априорных вероятностей Дирихле для байесовских сетей рассматривалось в [1450]. Многие из этих идей были реализованы в программном пакете Bugs [555], который представляет собой очень мощное инструментальное средство для формулировки и обучения сложных вероятностных моделей. В первых алгоритмах для определения в процессе обучения структур байесовских сетей использовались проверки условной независимости [1191], [1193]. В [1452] разработан исчерпывающий подход и описан пакет Tetrad для обучения байесовских сетей на основе аналогичных идей. Достигнутые с тех пор усовершенствования алгоритмов стали причиной

убедительной победы метода обучения байесовской сети в соревновании по интеллектуальному анализу данных на кубок KDD Сир 2001 года [248]. (На этих соревнованиях рассматривалась конкретная задача из биоинформатики с 139351 характеристикой!) Подход к определению в процессе обучения структуры сети, основанный на учете максимального правдоподобия, был разработан Купером и Херсковицем [292] и усовершенствован Хекерманом и др. [642]. В [507] указано на то, какое влияние оказывает способ представления локальных распределений условных вероятностей на структуру, определяемую в процессе обучения.

Общая задача определения в процессе обучения параметров вероятностных моделей со скрытыми переменными и недостающими данными была решена с помощью алгоритма ЕМ, предложенного Демпстером [383], который представляет собой обобщение нескольких существующих методов, включая алгоритм Баума-Уэлша

для обучения скрытых марковских моделей [85]. (Сам Демпстер рассматривал ЕМ скорее как схему, а не как алгоритм, поскольку может потребоваться большой объем математической работы, прежде чем появится возможность применить подход на основе ЕМ к новому семейству распределений.) В настоящее время ЕМ представляет собой один из алгоритмов, наиболее широко используемых в науке, а Маклахлан и Кришнан посвятили этому алгоритму и его свойствам целую книгу [1030].

Конкретная задача определения в процессе обучения параметров моделей на основе смешанных распределений, включая смешанные гауссовы распределения, рассматривается в [1509]. В рамках искусственного интеллекта первой успешной системой, в которой использовался алгоритм ЕМ для моделирования смешанных распределений, была система Autoclass [245], [246]. Система Autoclass применялась для решения многих реальных задач научной классификации, включая открытие новых типов звезд на основе спектральных данных [567] и новых классов белков и интронов в базах данных последовательностей ДНК/белок [708].

Алгоритм ЕМ для обучения байесовских сетей со скрытыми переменными был разработан Лауритценом [892]. Наряду с этим свою эффективность при обучении байесовских сетей, а также динамических байесовских сетей показали методы на основе градиента [1326], [126]. Структурный алгоритм ЕМ был разработан Фридманом [506]. Способность к определению в процессе обучения структуры байесовских сетей тесно связана с проблемой извлечения причинной информации из данных.

Эта проблема сводится к поиску ответа на вопрос о том, существует ли возможность определять в процессе обучения структуру байесовских сетей таким образом, чтобы полученная структура сети демонстрировала реальные причинные связи? В течение многих лет статистики избегали анализа этого вопроса, считая, что данные самих

наблюдений (в отличие от данных, выработанных в результате экспериментальных попыток) могут предоставить только информацию о корреляции; в конце концов, любые две переменные, которые кажутся взаимосвязанными, могут в действительности испытывать влияние третьего, неизвестного причинного фактора, а не влиять друг на друга непосредственно. Перл [1192] представил убедительные доводы, опровергающие это мнение, и показал, что фактически возникает много ситуаций, в которых причинно-следственные связи можно подтвердить и выявить с помощью формальных средств [^] причинной сети для выражения причин и результатов вмешательства, а также обычных условных вероятностей.

Истоки моделей с использованием ближайших соседних точек прослеживаются по меньшей мере до работы Фикса и Ходжеса [474] и со времени ее появления такие модели считаются стандартным инструментом в статистике и распознавании образов. В искусственном интеллекте они нашли широкое применение под влиянием работы Стенфилла и Вальца [1457], которые исследовали методы адаптирования метрики расстояния к данным. Хасты и Тибширани [628] разработали способ локализации метрики применительно к каждой точке пространства в зависимости от распределения данных вокруг этой точки. Эффективные схемы индексации для поиска ближайших соседних точек исследовались в сообществе специалистов по алгоритмам (см., например, [715]). Оценки плотности ядра, называемые также оценками плотности окна Парцена, были первоначально исследованы Розенблаттом [1305] и Парценом [1178]. С тех пор было опубликовано огромное количество научных работ с результатами исследований свойств различных средств оценки.

Исчерпывающее введение в эту тему приведено в [393].

Объем литературы по нейронным сетям слишком велик (до настоящего времени опубликовано примерно 100 000 статей), чтобы всю ее можно было подробно рассмотреть в настоящем разделе. В [299], [300] приведен краткий обзор ранней истории этого направления, начиная с работы Мак-Каллока и Питтса [1017]. В сотрудничестве с Мак-Каллом и Питтсом работал Норберт Винер, основатель кибернетики и теории управления [1589], который оказал значительное влияние на дальнейшую деятельность многих молодых исследователей, включая Марвина Минского. По-видимому, именно Минский был первым, кто разработал действующую нейронную сеть на основе аппаратных средств; это произошло в 1951 году (см. [1055, с. ix-x]). Между тем в Великобритании У. Росс Эшби (также один из основателей кибернетики; см. [42]), Алан Тьюринг, Грей Уолтер и другие основали клуб Ratio (клуб Разума) для "тех, кто был носителем идей Винера еще до появления книги Винера". В книге Эшби Design for a Brain [43], [44] выдвинута идея, что интеллект можно создать с использованием гомеостатических устройств, реализующих соответствующие циклы обратной связи для достижения стабильного

адаптивного поведения. Тьюринг [1519] написал исследовательский отчет с заглавием *Intelligent Machinery*, который начинается со слов "Я предлагаю исследовать вопрос о том, может ли машина проявлять интеллектуальное поведение" и продолжается в виде описания архитектуры рекуррентной нейронной сети, названной Тьюрингом "неорганизованными машинами В-типа", и подхода к обучению этих машин. К сожалению, этот отчет оставался неопубликованным до 1969 года и его содержание почти полностью игнорировалось до недавнего времени.

Фрэнк Розенблатт [1302] изобрел современный "персептрон" и доказал теорему сходимости персептрона [1303], хотя его работы оставались в тени чисто математических исследований, выполненных вне контекста нейронных сетей [6], [1093].

Кроме того, некоторые ранние работы в области нейронных сетей были посвящены многослойным сетям, включая персептроны Гамба [517] и мадалины [1586]. В книге *Learning Machines* [1140] рассматриваются многие из этих ранних работ, а также другие интересные темы. В дальнейшем, в этот ранний период исследований персептронов, интерес к этой теме упал под влиянием книги *Perceptrons* [1054], авторы которой посетовали на отсутствие математической строгости в этой области (но сами авторы в последующем заявили, что они в своей книге просто объяснили причины этого падения интереса). В данной книге указано, что однослойные персептроны способны представить только линейно разделимые понятия, и отмечено отсутствие эффективных алгоритмов обучения для многослойных сетей.

Как свидетельство возрождения интереса к коннекционизму могут рассматриваться статьи в сборнике, выпущенном по материалам конференции в Сан-Диего в 1979 году [655]. Большое внимание исследователей привлекла двухтомная антология *PDP* (*Parallel Distributed Processing* — параллельная распределенная обработка) [1316] и короткая статья в журнале *Nature* [1317]; фактически количество статей по "нейронным сетям" за период между 1980—1984 и 1990—1994 гг. увеличилось в 200 раз. Анализ нейронных сетей с использованием физической теории магнитных спиновых стекол, приведенный в [26], упрочил связи между статистической механикой и теорией нейронных сетей, предоставляя последнему научному направлению не только полезные математические основы, но и научную респектабельность. Метод обратного распространения был изобретен довольно рано [201], но затем был забыт и снова открыт еще несколько раз [1175], [1579].

Машины поддерживающих векторов впервые были созданы в 1990-х годах [296], а теперь являются темой все более возрастающего количества литературных источников, включая такие учебники, как [309]. Было доказано, что эти машины могут стать очень широко применяемым и эффективным средством решения таких задач, как категоризация текста [738], исследования в области биоинформатики [194]

и обработка естественного языка, в частности распознавание рукописных цифр [374].

К примерам связанных с ними методов, в которых также используется "фокус с ядерными функциями" для неявного представления экспоненциального пространства характеристик, относится персептрон с голосованием [283].

Тема вероятностной интерпретации нейронных сетей рассматривалась в нескольких источниках, включая [84] и [185]. Роль сигмоидальной функции описана в [745]. Метод байесовского обучения параметрам для нейронных сетей был предложен Маккеем [965], а его дальнейшее исследование проведено Нилом [1118].

Способность нейронных сетей представлять функции была исследована Цыбенко [316], [317], который показал, что двух скрытых слоев достаточно для представления любой функции, а одного скрытого слоя достаточно для представления любой непрерывной функции. Метод "оптимального повреждения мозга", предназначенный для удаления бесполезных связей, изложен в [903], а в [1409] показано, как удалять ненужные элементы. Алгоритм заполнения мозаики, предназначенный для наращивания размеров структур, предложен в [1037]. В [904] приведен обзор целого ряда алгоритмов распознавания рукописных цифр. С тех пор были опубликованы сведения о достигнутых успехах в области уменьшения частоты ошибок в [98] с помощью алгоритма согласования с формой и в [374] — с помощью алгоритма для виртуальных поддерживающих векторов.

Проблемы сложности обучения нейронных сетей рассматривались исследователями, занимающимися теорией вычислительного обучения. Первые вычислительные результаты были получены Джаддом [753], который показал, что общая задача поиска множества весов, совместимых с множеством примеров, является NP-полной, даже при очень ограничительных предположениях. Некоторые из первых результатов, касающихся выборочной сложности, были получены Баумом и Хаусслером [82], которые показали, что количество примеров, требуемых для эффективного обучения, растет примерно пропорционально $W \log W$, где W — количество весов. С тех пор была разработана гораздо более совершенная теория [34], в том числе получен важный результат, показывающий, что репрезентативная способность сети зависит не только от количества весов, но и от их величины.

Наиболее широко применяемой разновидностью нейронных сетей из тех, которые не рассматривались в данной книге, является сеть $^{\wedge}$ с радиальной базисной функцией, или сокращенно RBF (Radial Basis Function). В радиальной базисной функции объединяется взвешенная коллекция ядерных функций (разумеется, обычно гауссовых распределений) для осуществления функциональной аппроксимации. Обучение сетей RBF может проводиться в два этапа: вначале с помощью подхода на основе неконтролируемой кластеризации происходит

определение в процессе обучения параметров гауссовых распределений (математических ожиданий 16 Этот результат примерно соответствует "правилу дяди Берни". Это правило получило название в честь Берни Видроу, который рекомендовал использовать примерно в десять раз больше примеров по сравнению с весами.

и дисперсий), как описано в разделе 20.3. На втором этапе определяются относительные веса гауссовых распределений. Они составляют систему линейных уравнений, которые, как известно, можно решить непосредственно. Поэтому два этапа обучения RBF предоставляют важное преимущество: первый этап является неконтролируемым, и поэтому для него не требуются размеченные обучающие данные, а второй этап, хотя и контролируемый, характеризуется высокой эффективностью.

Подробные сведения приведены в [133]. В этой главе упоминались, но подробно не рассматривались рекуррентные сети.

По-видимому, наиболее изученным классом рекуррентных сетей являются $^{\wedge}$ сети Хопфилда [674]. В них используются двунаправленные связи с симметричными весами (т.е. элементы с $f_i/\pm j = f_i^{\wedge}j,i$), все элементы являются одновременно входными и выходными, функция активации, d , представляет собой знаковую функцию, а уровни активации могут принимать только значения ± 1 . Сеть Хопфилда функционирует как $^{\wedge}$ ассоциативная память: после обучения сети на множестве примеров новый стимул вызывает установление в сети образа активации, соответствующего тому примеру в обучающем множестве, который наиболее близко напоминает этот новый стимул. Например, если обучающее множество состоит из набора фотографий и новым стимулом является небольшой фрагмент одной из фотографий, то уровни активации сети воспроизводят фотографию, из которой был взят этот фрагмент. Следует отметить, что оригинальные фотографии не хранятся отдельно в сети; каждый вес представляет собой результат частичного кодирования всех фотографий.

Одним из наиболее интересных теоретических результатов является то, что сети Хопфилда могут надежно хранить вплоть до $0,138 N$ обучающих примеров, где N — количество элементов в сети.

В $^{\wedge}$ машинах Больцмана [657], [658] также используются симметричные веса, но предусматриваются скрытые элементы. Кроме того, в них применяется стохастическая функция активации, такая что вероятность появления на выходе 1 определяется некоторой функцией от общего взвешенного входа. Поэтому машины Больцмана подвержены переходам между состояниями, которые напоминают поиск с эмуляцией отжига (см. главу 4), применительно к конфигурации, которая наилучшим образом аппроксимирует обучающее множество. Как оказалось,

машины Больцмана очень тесно связаны с частным случаем байесовских сетей, оценка параметров которых осуществляется с помощью алгоритма стохастического моделирования (см. раздел 14.5).

Первое приложение идей, лежащих в основе ядерных машин, было разработано Айзерманом и др. [11], но полная разработка теории этих машин под названием машин поддерживающих векторов была выполнена Владимиром Вапником и его коллегами [156], [1537]. Строгие введения в эту тематику приведены в [309] и [1364]; описание, более удобное для чтения, приведено в статье для журнала AI Magazine, написанной Кристианини и Шёлкопфом [308].

В материалах этой главы собраны вместе работы из области статистики, распознавания образов и нейронных сетей, поэтому излагаемые в ней сведения были освещены в литературе много раз многими способами. К хорошим учебникам по байесовской статистике относятся [101], [377] и [534]. В [629] приведено превосходное введение в методы статистического обучения. Классическим учебником по классификации образов в течение многих лет была книга [421], которая недавно была переиздана в обновленном виде [422]. Ведущими учебниками по нейронным сетям яв-

ляются [133] и [1290]. Область вычислительной неврологии рассматривается в [339].

Наиболее важной конференцией по нейронным сетям и относящимся к ним темам является ежегодная конференция NIPS (Neural Information Processing Conference), труды которой публикуются в виде серии Advances in Neural Information Processing Systems. Статьи по обучению байесовских сетей появляются также в трудах конференций Uncertainty in AI и Machine Learning, а также нескольких конференций по статистике. К числу журналов, посвященных нейронным сетям, относятся Neural Computation, Neural Networks и IEEE Transactions on Neural Networks.

Упражнения 20.1. Данные, которые использовались для графика, приведенного на рис. 20.1, можно рассматривать как сформированные с помощью гипотезы h_5 . Для каждой из остальных четырех гипотез сформируйте набор данных с длиной 100 и вычертите соответствующие графики для $P(h_i | \text{data}, D_m)$ и $P(h_i | \text{data}, D_{m+1})$.

Прокомментируйте полученные вами результаты. 20.2. Повторите упр. 20.1, но на этот раз нанесите на графики значения $P(h_i | \text{data}, D_m)$ и $P(h_i | \text{data}, D_{m+1})$.

20.3. Предположим, что для Анны полезности вишневого и лимонного леденцов равны s_A и t_A , а для Боба эти полезности равны s_B и t_B (но после того как Анна развертывает какую-то конфету, Боб эту конфету не покупает).

Предполагается, что если Боб любит лимонные леденцы гораздо больше чем Анна, то было бы разумным решением со стороны Анны продать Бобу свой пакет с конфетами после того, как она приобретет достаточную уверенность в наличии в

этом пакете большого количества лимонных леденцов. С другой стороны, если Анна в процессе анализа содержимого пакета разворачивает слишком много конфет, стоимость пакета уменьшается, поскольку Боб не платит за развернутые конфеты. Обсудите задачу определения оптимальной точки, в которой Анна должна продавать свой пакет. Определите ожидаемую полезность этой оптимальной процедуры с учетом распределения априорных вероятностей, описанного в разделе 20.1.

20.4. Два статистика попали на прием к врачу, который сообщил им одинаковый прогноз: с вероятностью 40% расстройство их здоровья вызвано смертельным заболеванием А, а с вероятностью 60% оно вызвано тяжелым заболеванием В.

К счастью, есть лекарства и от заболевания А, и от заболевания В, которые являются недорогими, эффективными на 100% и не вызывающими побочных эффектов. Этим статистикам предоставлена возможность выбрать для себя один из вариантов дальнейших действий — принимать одно из этих лекарств, оба эти лекарства или ни одного из этих лекарств. Как поступит первый статистик, который является убежденным сторонником байесовского подхода?

А как поступит второй статистик, который всегда использует гипотезу с максимальным правдоподобием?

Врач провел определенные исследования и обнаружил, что заболевание В фактически протекает в двух вариантах (правостороннее заболевание В и левостороннее заболевание В), которые являются равновероятными и одинаково хорошо излечиваются с помощью лекарства против заболевания В. Теперь количество гипотез стало равным трем; как поступят эти два статистика?

20.5. Объясните, как применить метод усиления, описанный в главе 18, для наивного байесовского обучения. Проверьте производительность результирующего алгоритма на задаче обучения с рестораном.

20.6. Рассмотрим t точек данных (x_j, y_j) , где координаты y_j вырабатываются на основании координат X_j в соответствии с моделью линейного гауссова распределения, приведенной в уравнении 20.5. Найдите значения θ и a , которые максимизируют условное логарифмическое правдоподобие этих данных.

20.7. Рассмотрим модель зашумленного ИЛИ для лихорадки, описанную в разделе 14.3. Объясните, как применить обучение с учетом максимального правдоподобия для согласования параметров такой модели с множеством полных данных. (Подсказка. Используйте цепное правило для частичных производных.) 20.8. В данном упражнении исследуются свойства бета-распределения, которое определено в уравнении 20.6. а) Выполнив интегрирование по отрезку $[0, 1]$, покажите, что константа нормализации для распределения $\text{beta}[a, b]$ задается

выражением $a = \Gamma(a+b) / \Gamma(a)\Gamma(b)$, где $\Gamma(x)$ — Γ -гамма-функция, определяемая выражением $\Gamma(x+1) = x\Gamma(x)$, а $\Gamma(1)=1$ (для целого числа x выражение $\Gamma(x+1)=x!$). б) Покажите, что математическое ожидание равно $a/(a+b)$. в) Найдите моду (моды) — наиболее вероятное значение (значения) θ . г) Опишите вид распределения $\text{beta}(\theta, \theta)$ для очень малого значения θ .

Что происходит при обновлении такого распределения?

20.9. Рассмотрим произвольную байесовскую сеть, полный набор данных для этой сети и правдоподобие этого набора данных согласно этой сети. Дайте простое доказательство того, что правдоподобие данных не может уменьшиться после добавления новой связи к сети и повторного вычисления значений параметров максимального правдоподобия.

20.10. Рассмотрим применение алгоритма EM для определения в процессе обучения параметров сети, приведенной на рис. 20.10, а, если даны истинные параметры в уравнении 20.7. а) Объясните, почему алгоритм EM не будет действовать, если в модели имеются только два атрибута, а не три. б) Покажите расчеты для первой итерации алгоритма EM, начиная с уравнения 20.8. в) Что происходит, если применение алгоритма начинается с присваивания всем параметрам одинакового значения p ? (Подсказка. Рекомендуется вначале провести эмпирическое исследование этого вопроса и только после этого выводить общий результат.) г) Запишите выражение для логарифмического правдоподобия табличных данных о конфетах, приведенных в табл. 20.1, с учетом параметров, рас-

считайте частичные производные по отношению к каждому параметру и исследуйте характер фиксированной точки, достигнутой при выполнении упр. 20.10, в.

20.11. Постройте вручную нейронную сеть, которая вычисляет функцию XOR от двух входов. Обязательно укажите, какого рода элементы вы используете.

20.12. Сконструируйте машину поддерживающих векторов, которая вычисляет функцию XOR. При этом для входов и выходов удобнее использовать значения 1 и -1 вместо 1 и 0. Поэтому некоторые примеры могут выглядеть как $([-1, 1], 1)$ или $([-1, -1, -1], -1)$. Обычно принято отображать входное значение x на пространство, состоящее из пяти размерностей: двух первоначальных размерностей, x_1 и x_2 , и трех комбинаций размерностей, x_1^2 , x_2^2 и x_1x_2 . Но для этого упражнения мы будем рассматривать только две размерности, x_1 и x_1x_2 . Нарисуйте в этом пространстве четыре входных точки и разделитель с максимальной шириной края. Каковым является этот край? А теперь снова преобразуйте разделительную линию и нарисуйте ее в первоначальном евклидовом пространстве входов.

20.13. Простой персептрон не способен представить функцию XOR (или, вообще говоря, функцию четности от его входов). Опишите, что происходит с весами в

четырёхходовом персептроне со ступенчатой функцией, начиная со всех весов, установленных равными 0.1, по мере поступления примеров функции четности.

20.14. Напомним, что, как было сказано в главе 18, существует 22 различных булевых функций от p входов. Какая часть функций из этого общего количества может быть представлена с помощью порогового персептрона?

20.15. (§1) Рассмотрим приведенное в табл. 20.3 множество примеров, каждый из которых имеет шесть входов и один целевой выход.

Таблица 20.3. Примеры для упр. 20.15 II ь Ъ Ъ и т а) Примените правило обучения персептрона к этим данным и покажите окончательные веса. б) Примените правило обучения дерева решений и покажите результирующее дерево решений. в) Прокомментируйте полученные вами результаты.

20.16. Начиная с уравнения 20.13, покажите, что $db/dW_j = Errx_j$.

20.17. Предположим, что имеется нейронная сеть с линейными функциями активации. Это означает, что выход каждого элемента определяется некоторой константой c , умноженной на взвешенную сумму его входов. а) Предположим, что эта сеть имеет один скрытый слой. Для данного присваивания весам W запишите уравнения для значений элементов в выходном слое как функции от w и значений элементов входного слоя I без какого-либо явного упоминания в этих выходных данных о скрытом слое.

Покажите, что существует сеть без скрытых элементов, которая вычисляет ту же функцию. б) Повторите вычисления, описанные в упр. 20.17, я, но на этот раз применительно к сети с любым количеством скрытых слоев. Какой можно сделать вывод в отношении линейных функций активации?

20.18. (Sid Реализуйте какую-то структуру данных для многослойных нейронных сетей с прямым распространением и не забудьте предусмотреть способ представления информации, необходимой как для прямого вычисления, так и для обратного распространения. Используя эту структуру данных, напишите функцию Neural-Network-Output, которая принимает на входе определения некоторого примера и сети, после чего вычисляет соответствующие выходные значения.

20.19. Предположим, что обучающее множество содержит только единственный пример, повторенный 100 раз. В 80 из 100 случаев единственным выходным значением является 1, а в остальных 20 случаях таковым является 0. Что предсказывает сеть с обратным распространением для данного примера, при условии, что по нему проведено обучение и достигнут глобальный оптимум?

(Подсказка. Чтобы найти глобальный оптимум, необходимо дифференцировать функцию ошибки и приравнять полученное выражение к нулю.) 20.20. Сеть, приведенная на рис. 20.23, имеет четыре скрытых узла. Это количество узлов было

выбрано фактически произвольно. Проведите систематические эксперименты, чтобы измерить кривые обучения для сетей с различным количеством скрытых узлов. Каковым является оптимальное количество? Было бы возможно использовать метод перекрестной проверки, чтобы найти наилучшую сеть еще до получения этих данных?

20.21. Рассмотрим задачу разделения N точек данных на положительные и отрицательные примеры с использованием линейного разделителя. Очевидно, что эту задачу всегда можно выполнить для количества точек $N=2$ на линии с размерностью $d=1$, независимо от того, как размечены эти точки или где они находятся (если только эти две точки не находятся в одном и том же месте). а) Покажите, что эту задачу всегда можно выполнить для количества точек $N=3$ на плоскости с размерностью $d=2$, если только эти точки не являются коллинеарными. б) Покажите, что эту задачу не всегда можно решить для количества точек $N=4$ на плоскости с размерностью $d=2$.

в) Покажите, что эту задачу не всегда можно решить для количества точек $N=4$ в пространстве с размерностью $d=3$, если только эти точки не являются копланарными. г) Покажите, что эту задачу не всегда можно решить для количества точек $N=5$ в пространстве с размерностью $d=3$. д) Амбициозный студент решил доказать, что произвольно расположенные точки (но не $N+1$ точка) являются линейно разделимыми в пространстве с размерностью $N-1$. Покажите, что из этого доказательства будет следовать, что VC-размерность (см. главу 18) линейных полупространств в пространствах с размерностью $N-1$ равна N .

Глава 21. Обучение с подкреплением

21 Обучение В этой главе рассматривается вопрос о том, как агент может учиться на своих успехах и неудачах, учитывая полученные вознаграждения и наказания.

21.1. Введение В главах 18 и 20 рассматривались методы обучения, позволяющие определять функции и вероятностные модели на основе примеров, а в этой главе будет описано, каким образом агенты могут определить в процессе обучения, что делать, особенно если нет учителя, сообщающего агенту, какое действие следует предпринять в тех или иных обстоятельствах.

Например, как известно, агент может обучиться игре в шахматы с помощью контролируемого обучения, в котором ему предъявляются примеры игровых ситуаций наряду с наилучшими ходами для этих ситуаций. Но если нет дружелюбного учителя, предоставляющего готовые примеры, то что может сделать агент? Опробуя случайно выбранные ходы, агент может в конечном итоге составить прогностическую модель своей среды, т.е. предсказать, как будет выглядеть доска после того, как он сделает данный конкретный ход, и даже как, скорее всего, ответит противник в такой ситуации. Но при этом возникает следующая проблема: ^ без

какой-либо обратной связи, говорящей о том, какой ход является хорошим и какой плохим, агент не будет иметь оснований для принятия решения о том, какой ход следует сделать. Агент должен знать, что его выигрыш — это благоприятный исход, а проигрыш — неблагоприятный. Обратная связь такого рода называется вознаграждением, или подкреплением. В играх, подобных шахматам, подкрепление дается только в конце игры.

В других вариантах среды вознаграждения могут поступать более часто. В настольном теннисе как вознаграждение может рассматриваться каждое выигранное очко, а при обучении новобранцев способам перемещения ползком достижением становится каждое движение вперед. В инфраструктуре для агентов, рассматриваемой в данной главе, вознаграждение считается частью результатов восприятия, но агент должен быть "настроен" на распознавание этой части как вознаграждения, а не про-

сто как еще одного вида сенсорных входных данных. Например, складывается впечатление, что животные настроены на распознавание боли и голода как отрицательных вознаграждений, а удовольствия и приема пищи — как положительных вознаграждений. Проблемы подкрепления тщательно исследовались специалистами в области психологии животных больше 60 лет.

Понятие вознаграждения было впервые представлено в главе 17, где оно использовалось для определения оптимальных стратегий в марковских процессах принятия решений (Markov Decision Process — MDP). Оптимальной является такая стратегия, которая максимизирует ожидаемое суммарное вознаграждение. Задача обучения с подкреплением состоит в том, чтобы обеспечить использование наблюдаемых вознаграждений для определения в процессе обучения оптимальной (или почти оптимальной) стратегии для данной среды. Но хотя агент, рассматриваемый в главе 17, имел полную модель среды и знал функцию вознаграждения, в данной главе предполагается отсутствие априорных знаний и о том и о другом. Представьте себе, что вы играете в новую игру, правил которой не знаете; примерно через сто ходов ваш противник объявляет: "Вы проиграли". В этом состоит вся суть обучения с подкреплением.

Во многих сложных проблемных областях обучение с подкреплением является единственным осуществимым способом, с помощью которого можно провести обучение некоторой программы, чтобы она могла действовать с высокой производительностью. Например, в случае ведения игр для человека является очень трудной задачей предоставление точных и согласованных оценок большого количества позиций, что требуется для определения в процессе обучения функций оценки непосредственно из примеров. Вместо этого программе можно сообщать, когда она выиграла или проиграла, а сама программа может использовать такую информацию для определения с помощью обучения такой функции оценки,

которая предоставляла бы достаточно точные оценки вероятности выигрыша из любой конкретной позиции.

Аналогичным образом, чрезвычайно трудно запрограммировать агента так, чтобы он научился вести вертолет; но, предоставляя соответствующие отрицательные вознаграждения за столкновение, болтанку или отклонение от заданного курса, можно дать агенту возможность научиться летать на вертолете самостоятельно.

Обучение с подкреплением может рассматриваться как задача, охватывающая всю тематику искусственного интеллекта: агента помещают в какую-то среду и обязывают его обучиться успешно действовать в ней. Поэтому, чтобы объем этой главы не вышел за пределы разумного, в ней будут рассматриваться только простые варианты среды и простые проекты агента. По большей части предполагается, что среда является полностью наблюдаемой, поэтому информация о текущем состоянии поступает с результатами каждого восприятия. С другой стороны, считается, что агент не знает, по каким принципам действует среда или какими являются результаты его действий, поэтому допускается наличие вероятностных результатов действий.

В этой главе речь пойдет о трех перечисленных ниже проектах агентов, которые были впервые представлены в главе 2. • Агент, действующий с учетом полезности, определяет с помощью обучения функцию полезности состояний и использует ее для выбора действий, которые максимизируют ожидаемую полезность результата. • Агент, действующий по принципу $^{\wedge}Q$ -обучения, определяет с помощью обучения функцию $^{\wedge}$ "действие—значение", или Q -функцию, получая сведения

об ожидаемой полезности выполнения данного конкретного действия в данном конкретном состоянии. • Рефлексный агент определяет с помощью обучения стратегию, которая непосредственно отображает состояния в действия.

Агент, действующий с учетом полезности, для принятия решений должен также иметь модель среды, поскольку он должен знать, в какие состояния приведут его выполненные им действия. Например, для того чтобы программа игры в нарды могла использовать функцию оценки для нард, она должна иметь информацию о том, каковыми являются допустимые ходы и как они влияют на позицию в игре. Это — единственный способ, позволяющий применить функцию полезности к результирующим состояниям. Агент, действующий по принципу Q -обучения, с другой стороны, может сравнивать значения, характеризующие доступные ему варианты действий, без необходимости знать их результаты, поэтому ему не требуется модель среды. Тем не менее агенты, действующие по принципу Q -обучения, не могут прогнозировать будущую ситуацию, поскольку не имеют информации о том, к чему приведут их действия; это может серьезно ограничить способность таких агентов к обучению, как будет описано ниже.

Изложение материала этой главы начинается в разделе 21.2 с описания $\wedge$ пассивного обучения, в котором стратегия агента остается неизменной, а задача состоит в том, чтобы определить с помощью обучения полезности состояний (или пар "состояниедействие"); для этого может также потребоваться определение с помощью обучения модели среды. В разделе 21.3 рассматривается $\wedge$ активное обучение, в ходе которого агент должен также определить, что следует делать. Принципиальной проблемой является $\wedge$ исследование среды: агент должен проводить в своей среде максимально возможное количество экспериментов, для того чтобы определить, как следует в ней действовать. В разделе 21.4 показано, что агент может использовать индуктивное обучение, чтобы как можно быстрее обучиться на своем опыте. В разделе 21.5 рассматриваются методы определения с помощью обучения непосредственных представлений стратегий в рефлексных агентах. Для освоения материала данной главы крайне важно понимание тематики марковских процессов принятия решений (см. главу 17).

21.2. Пассивное обучение с подкреплением Для того чтобы упростить изложение, начнем с описания случая пассивного обучающегося агента, в котором используется представление на основе состояний в полностью наблюдаемой среде. При пассивном обучении стратегия агента p является неизменной; это означает, что в состоянии s он всегда выполняет действие $p(s)$.

Цель агента состоит в том, чтобы определить с помощью обучения, насколько успешной является эта стратегия, т.е. определить с помощью обучения функцию полезности $U_f(s)$. В этом разделе в качестве примера будет использоваться мир 4×3 , представленный в главе 17. На рис. 21.1 для этого мира показаны стратегия и соответствующие полезности. Очевидно, что задача пассивного обучения аналогична задаче оценки стратегии, которая является частью алгоритма итерации по стратегиям, описанного в разделе 17.3. Основное различие состоит в том, что пассивный обу-

чающийся агент не знает модели перехода $T(s, a, s')$, которая определяет вероятность достижения состояния s' из состояния s после выполнения действия a ; он также не знает функцию вознаграждения $R(s)$, которая задает вознаграждение для каждого состояния.

1 11 t t ел ED 0,812 0,868 0,918 0.762 ИНИН 0.660 0,705 0,655 0,611 LIU I-i 1 0,388 а) б)
Рис. 21.1. Описание рассматриваемой среды: а) стратегия j для мира 4×3 ; оказалось, что эта стратегия является оптимальной при вознаграждениях $R(s) = -0.04$ в нетерминальных состояниях и при отсутствии обесценивания (α); полезности состояний в мире 4×3 с учетом стратегии j (б) Агент выполняет в данной среде ряд $\wedge$ попыток, используя свою стратегию p .

При осуществлении каждой попытки агент начинает с состояния A,1) и испытывает некоторую последовательность переходов между состояниями до тех пор, пока не достигнет одного из терминальных состояний, D,2) или D,3). В результатах восприятий ему сообщается и текущее состояние, и вознаграждение, полученное в этом состоянии. Типичные попытки могут выглядеть примерно так:

A,1)-.04->A,2)-.04->A,3)-.04->A,2)-.04->A,3)-.04->B#3)-.04->C#3)-.04->D,3)+1 AД)-.04->A,2)-.04->A,3)-.04->B#3)-.04->C,3)-.04->C,2)-.04->C#3)-.04->D,3)+1 A,1) -.04-> B ,1) -.04->C ,1) -.04->C , 2) -.04^ D , 2) -1 Обратите внимание на то, что результаты восприятия каждого состояния сопровождаются нижним индексом с указанием полученного вознаграждения. Цель состоит в том, чтобы использовать эту информацию о вознаграждении для определения с помощью обучения ожидаемой полезности $I_f(s)$, связанной с каждым нетерминальным состоянием s . Определяемая полезность должна представлять собой ожидаемую сумму (обесцениваемых) вознаграждений, полученных, если агент придерживается стратегии π . Как и в уравнении 17.3, это соотношение записывается следующим образом: $I_f(s) = \sum_{t=0}^{\infty} \gamma^t V_t(s)$ (1.1)

Мы будем включать коэффициент обесценивания γ во все уравнения, приведенные в данной главе, но для мира 4x3 принято использовать значение $\gamma=1$.

Непосредственная оценка полезности Простой метод ^ непосредственной оценки полезности был изобретен в конце 1950-х годов в области ^ адаптивной теории управления Видроу и Хоффом [1587].

Идея этого метода состоит в том, что полезностью данного конкретного состояния является ожидаемое суммарное вознаграждение, связанное с действиями, выполняемыми, начиная с этого состояния, а каждая попытка представляет собой выборку этого значения для каждого посещенного состояния. Например, первая попытка из трех приведенных выше предоставляет одну выборку с суммарным вознаграждением 0.72 для состояния A,1), две выборки со значениями 0.76 и 0.84 для состояния A,2), две выборки со значениями 0.80 и 0.88 для состояния A,3) и т.д. Таким образом, в конце каждой последовательности алгоритм вычисляет наблюдаемое будущее вознаграждение для каждого состояния и обновляет соответствующим образом оценку полезности для этого состояния путем ведения текущего среднего значения для каждого состояния в таблице. В пределе, после выполнения бесконечного количества попыток, среднее по выборкам сходится к значению истинного ожидания, приведенному в уравнении 21.1.

Очевидно, что непосредственная оценка полезности представляет собой один из видов контролируемого обучения, в котором каждый пример задает состояние в качестве входных данных, а наблюдаемое будущее вознаграждение — в качестве выходных.

Это означает, что данный метод позволяет свести обучение с подкреплением к стандартной задаче индуктивного обучения, которая рассматривалась в главе 18. В разделе 21.4 описано использование более мощных видов представлений для функции полезности, таких как нейронные сети. Методы обучения для этих представлений могут применяться непосредственно к наблюдаемым данным.

Метод непосредственной оценки полезности позволяет успешно свести задачу обучения с подкреплением к задаче индуктивного обучения, о которой уже многое известно. Но, к сожалению, этот метод не позволяет воспользоваться очень важным источником информации — в нем не учитывается тот факт, что полезности состояний не являются независимыми! Дело в том, что $U(s)$ полезность каждого состояния равна сумме его собственного вознаграждения и ожидаемой полезности его состояний-преемников. Это означает, что значения полезности подчиняются уравнениям Беллмана для данной конкретной стратегии (см. также уравнение 17.10): $U(s) = R(s) + \gamma \sum_{s'} T(s, a(s), s') U(s')$ (17.2). Поскольку в методе непосредственной оценки полезности игнорируются связи между состояниями, он не позволяет воспользоваться дополнительными возможностями для обучения. Например, во второй из трех попыток, приведенных выше, достигается состояние C,2), которое еще не было до сих пор посещено агентом.

После следующего перехода агент достигает состояния C,3), которое, как известно из первой попытки, имеет высокую полезность. Уравнение Беллмана позволяет сразу же определить, что состояние C,2) также, по-видимому, будет иметь высокую по-

лезность, поскольку оно ведет к состоянию C,3), но метод непосредственной оценки полезности не позволяет ничего определить с помощью обучения до конца этой попытки. В более широком контексте метод непосредственной оценки полезности можно рассматривать как поиск в пространстве гипотез для U , которое имеет размеры намного большие, чем необходимо, поскольку включает также много функций, которые нарушают уравнения Беллмана. По этой причине данный алгоритм часто сходится очень медленно.

Адаптивное динамическое программирование Для того чтобы можно было воспользоваться преимуществами наличия информации об ограничениях между состояниями, агент должен определить с помощью обучения, как связаны эти состояния. [^] Агент, действующий по принципу адаптивного динамического программирования (или сокращенно ADP— Adaptive Dynamic Programming), функционирует путем определения с помощью обучения модели перехода в этой среде по мере выполнения своих действий и находит решение в соответствующем марковском процессе принятия решений, используя метод динамического программирования. Для пассивного обучающегося агента такой подход означает, что он должен подставлять полученную с помощью обучения модель перехода

$T(s, \gamma_i(s), s')$ и наблюдаемые вознаграждения $R(s)$ в уравнения Беллмана 21.2 для вычисления полезностей состояний. Как было отмечено при обсуждении в главе 17 принципа итерации по стратегиям, эти уравнения являются линейными (для них не требуется максимизация), поэтому они могут быть решены с помощью любого пакета линейной алгебры. Еще один вариант состоит в том, что может быть принят подход, основанный на принципе модифицированной итерации по стратегиям (см. с. 831), в котором используется упрощенный процесс итерации по значениям для обновления оценок полезностей после каждого изменения в модели, определяемой с помощью обучения. Поскольку эта модель после каждого наблюдения подвергается только незначительным изменениям, в процессе итерации по значениям в качестве начальных значений могут использоваться предыдущие оценки полезностей, а сами вычисления с помощью этого метода должны сходиться очень быстро.

Сам процесс определения модели с помощью обучения осуществляется просто, поскольку среда полностью наблюдаема. Это означает, что данная задача обучения представляет собой задачу контролируемого обучения, в которой входными данными являются пары "состояние—действие", а выходными — результирующие состояния. В простейшем случае модель перехода можно представить как таблицу вероятностей и следить за тем, насколько часто возникает каждый результат действия¹, оценивая вероятность перехода $T(s, a, s')$ по данным о частоте, с которой достигается состояние s' при выполнении действия a в состоянии s . Например, в трех трассировках попыток, приведенных на с. 1013, действие Right в состоянии A,3) выполняется три раза, и двумя из трех результирующих состояний становится состояние B,3), поэтому вероятность перехода $T(A,3), \text{Right}, B,3)$ оценивается как $2/3$.

¹ Это — оценка максимального правдоподобия, которая рассматривается в главе 20. Способ байесовского обновления с использованием распределения априорных вероятностей Дирихле может оказаться лучше.

Полная программа агента для пассивного агента ADP приведена в листинге 21.1.

Производительность этой программы в мире 4x3 показана на рис. 21.2. Судя по тому, насколько быстро улучшаются полученные им оценки значений, агент ADP действует настолько успешно, насколько это возможно, учитывая его способность определять с помощью обучения модель перехода. В этом смысле данный проект агента может служить эталоном, с которым можно будет сравнивать производительность других алгоритмов обучения с подкреплением. Тем не менее в больших пространствах состояний такой подход становится не вполне осуществимым. Например, в нардях для его использования потребуется решить приблизительно 1050 уравнений с 1050 неизвестными.

Листинг 21.1. Алгоритм агента для пассивного обучения с подкреплением, основанный на адаптивном динамическом программировании. Для упрощения кода было принято предположение, что каждый результат восприятия можно разделить на информацию о состоянии и сигнал о вознаграждении

```
function Passive-ADP-Agent (percept) returns действие a
inputs: percept, результаты восприятия, обозначающие текущее состояние s' и сигнал вознаграждения r'
static: п, зафиксированная стратегия mdp, марковский процесс принятия решений с моделью T, вознаграждениями R, коэффициентом обесценивания  $\gamma$ , U, таблица значений полезностей, первоначально пустая iVSas, таблица частот пар "состояние-действие", первоначально содержащая нули iVSas', таблица частот троек "состояние-действие-состояние", первоначально содержащая нули s, a, предыдущие состояние и действие, первоначально пустые if состояние s1 является новым then do U[s] ] <rr' ; R[s] ] <- r* if состояние s не пусто then do увеличить значения Nsa[s, a] и iVSas' [s, a, s'] for each t, такого что значение iVSas' [s, a, t] является ненулевым do T[s, a, t] <- iVSas-[s, a, t] / Nsa[s, a] U <- Value-Determination ( $\gamma$ , U, mdp) if Terminal? [s'] then s, a <- пустые значения else s, a <- s', n[s]] return a
```

Обучение с учетом временной разницы Существует также возможность взять (почти) самое лучшее из обоих описанных выше подходов; это означает, что можно аппроксимировать приведенные выше уравнения ограничений, не решая их для всех возможных состояний, с[^] Ключом к созданию этого метода становится то, что можно использовать наблюдаемые переходы для корректировки значений наблюдаемых состояний так, чтобы они согласовывались с уравнениями ограничений. Рассмотрим, например, переход из состояния A,3) в состояние B,3) во второй попытке, показанной на с. 1013. Предположим, что в результате первой попытки были получены оценки полезности $I_f(l, 3) = 0.84$

и $I_f(rB, 3) = 0.92$. Итак, если будет постоянно происходить этот переход, то следует учитывать, что указанные полезности будут подчиняться следующему уравнению:

$$I_f(l, 3) = -0.04 + \gamma V(3) \leq 0,8 \quad \Leftrightarrow \quad I_f(l, 3) \leq 0,84$$

Количество попыток Количество попыток а) б) Рис. 21.2. Кривые пассивного обучения ADP для мира 4x3, полученные при оптимальной стратегии, которая показана на рис. 21.1: оценки полезности для избранного подмножества состояний, полученные как функции от количества попыток (а). Обратите внимание на то, что примерно при 78-й попытке происходят значительные изменения; именно тогда агент впервые попадает в терминальное состояние с полезностью -1, соответствующее квадрату D,2); среднеквадратичная ошибка в оценке для и A,1), усредненная по 20 прогонам, состоящим из 100 попыток каждый (б) поэтому значение $I_f(1, 3)$ будет равно 0.88. Таким образом, текущая оценка этой полезности, равная 0.84, может оказаться немного заниженной и должна быть увеличена. Более общий вывод состоит в том, что если происходит переход из состояния s в состояние s', то к значению полезности $I_f(s)$ применяется

следующее обновление: $I_f(s) \leftarrow I_f(s) + \alpha(R(s) + \gamma I_f(s') - I_f(s))$ В1.3) где α — параметр скорости обучения. Поскольку в этом правиле обновления используется разность между полезностями последовательных состояний, соответствующее уравнение часто называют уравнением Δ временной разности, или сокращенно TD (Temporal Difference).

Основная идея всех методов временной разности состоит в том, что вначале определяются условия, выполняемые локально, когда оценки полезностей являются правильными, а затем составляется уравнение обновления, в котором оценки переносятся в это идеальное уравнение "равновесия". В случае пассивного обучения равновесие задается уравнением 21.2. Теперь уравнение 21.3 по сути вынуждает агента достичь равновесия, заданного в уравнении 21.2, но с учетом некоторых нюансов. Прежде всего следует отметить, что данное обновление касается только наблюдаемого преемника s' , тогда как фактические условия равновесия касаются всех возможных следующих состояний. Можно было бы предположить, что это вызовет необоснованно большие изменения в значении $I_f(s)$ при возникновении очень редких переходов, но фактически, поскольку эти редкие переходы действительно случаются крайне редко, среднее значение $I_f(s)$ сходится к правильному значению.

U.0 1 D,3) C,3) A,3) A,1) C,2) 0,5 а и. ю а ? ° ^ а | о,4 | Цо,з. о. й 1 1 0,2 | <L> Си 0,1 \

Более того, если в качестве коэффициента α вместо фиксированного параметра будет применяться функция со значением, уменьшающимся по мере увеличения количества случаев посещения некоторого состояния, то само значение $I_f(s)$ будет сходиться к правильному значению². Эти рассуждения позволяют составить программу агента, приведенную в листинге 21.2. На рис. 21.3 показана производительность пассивного агента TD в мире 4x3. Обучение этого агента происходит менее быстро по сравнению с агентом ADP, и он показывает более значительную изменчивость, но сам алгоритм агента гораздо проще и требует меньше вычислений в расчете на каждое наблюдение. Обратите внимание на то, что с Δ алгоритм TD не требует применения модели для осуществления предусмотренных в нем обновлений.

Информацию о связях между соседними состояниями предоставляет сама среда в форме наблюдаемых переходов.

Листинг 21.2. Алгоритм агента для пассивного обучения с подкреплением, который позволяет определить с помощью обучения оценки полезностей на основе временных разностей

```
function Passive-TD-Agent(percept) returns действие
  inputs: percept, результаты восприятия, обозначающие текущее состояние  $s'$  и сигнал вознаграждения  $g'$ 
  static:  $U$ , зафиксированная стратегия
   $U$ , таблица значений полезностей, первоначально пустая
   $NSf$  таблица частот состояний, первоначально
```

содержащая нули s , a , g , предыдущее состояние, действие и вознаграждение, первоначально пустые

```

if состояние  $s'$  является новым then  $L[s'] \leftarrow r$ 
if состояние  $s$  не пусто then do
    увеличить значение  $U[s] \leftarrow r - U[s] + a(\{N[s]\}) (r + \gamma U[s'] - U[s])$ 
if Terminal? [ $s$ ] then  $s, a, r \leftarrow$  пустые значения else  $s, a, r \leftarrow s', \gamma U[s'] / r$ 
return  $a$ 

```

Подход на основе ADP и подход на основе TD фактически тесно связаны. В обоих этих алгоритмах предпринимаются попытки внести локальные корректировки в оценки полезностей, для того чтобы обеспечить "согласование" каждого состояния с его преемниками. Одно из различий между ними состоит в том, что в алгоритме TD состояние корректируется для согласования с его наблюдаемым преемником (уравнение 21.3), а в алгоритме ADP состояние корректируется для согласования со всеми преемниками, которые могут быть получены с учетом весов их вероятностей (уравнение 21.2). Это различие исчезает, когда результаты корректировок TD усредняются по большому количеству переходов, поскольку частота появления каждого преемника в множестве переходов приблизительно пропорциональна его вероятности. Более важное различие состоит в том, что в алгоритме TD выполняется отдельная корректировка в расчете на каждый наблюдаемый переход, а в алгоритме ADP выполняется столько корректировок, сколько требуется для восстановления соглашения. Формально требуется, чтобы соблюдались условия $\lim_{n \rightarrow \infty} a(n) = 0$ и $\sum_{n=0}^{\infty} a(n) = \infty$ (21.4). Этим условиям удовлетворяет функция затухания $a(n) = 1/n$, а на рис. 21.3 использовалась функция $a(n) = 60/(n+1)$.

выполняется столько корректировок, сколько требуется для восстановления согласованности между оценками полезностей и моделью среды T . Хотя наблюдаемый переход вносит в T только локальное изменение, его результаты могут потребовать распространения по всем полезностям. Таким образом, алгоритм TD может рассматриваться как грубое, но эффективное первое приближение алгоритма ADP.

100 200 300 400 500 Количество попыток 20 40 60 80 (Количество попыток а) б) Рис. 21.3. Кривые обучения с помощью алгоритма TD для мира 4x3: оценки полезности для избранного подмножества состояний, полученные как функции от количества попыток (а); среднеквадратичная ошибка в оценке для $U(A,1)$, усредненная по 20 прогонам, состоящим из 500 попыток каждый (б). Показаны результаты только для первых 100 попыток, чтобы можно было провести сравнение с рис. 21.2. Каждая корректировка, внесенная алгоритмом ADP, с точки зрения пользователя алгоритма TD может рассматриваться как результат "псевдоэксперимента", полученный путем моделирования в текущей модели среды. Подход, предусмотренный в алгоритме TD, можно дополнить в целях использования модели среды для выработки результатов нескольких псевдоэкспериментов, иначе говоря, переходов, возможность осуществления которых агент TD может допустить согласно его текущей модели. В ответ на каждый наблюдаемый переход агент TD может вырабатывать большое

количество воображаемых переходов. Благодаря этому результирующие оценки полезностей TD будут все больше и больше аппроксимировать соответствующие оценки ADP, разумеется, за счет увеличения продолжительности времени вычислений.

Аналогичным образом могут создаваться все более эффективные версии алгоритма ADP путем непосредственной аппроксимации алгоритмов для итерации по значениям или итерации по стратегиям. Напомним, что полная итерация по значениям может быть трудновыполнимой, когда количество состояний велико. Однако многие этапы корректировки являются чрезвычайно малыми. Одним из возможных подходов, применимых для быстрой выработки достаточно качественных ответов, является ограничение количества корректировок, внесенных после каждого наблюдаемого перехода. Можно также использовать какую-то эвристику для ранжирования возможных корректировок, с тем чтобы в дальнейшем осуществлять только наиболее значимые из них. Эвристика, предусматривающая $\wedge$ выметание с учетом приоритетов, позволяет предпочесть вариант с внесением корректировок в состояния, возможные преемники которых уже подвергались большим корректировкам

в их собственных оценках полезностей. Приближенные алгоритмы ADP, в которых используются подобные эвристики, обычно обеспечивают обучение примерно с таким же быстродействием, как и полные алгоритмы ADP, с точки зрения количества обучающих последовательностей, но могут оказаться на несколько порядков величины более эффективными с точки зрения объема вычислений (см. упр. 21.3.) Это дает возможность применять такие алгоритмы для обработки пространств состояний, намного превышающих по размерам те, которые являются приемлемыми для полного алгоритма ADP. Приближенные алгоритмы ADP имеют еще одно дополнительное преимущество: на ранних этапах обучения в новой среде модель среды t часто далека от правильной, поэтому нет смысла слишком точно вычислять функцию полезности для согласования с ней. Приближенный алгоритм позволяет использовать минимальную величину корректировки, которая уменьшается по мере того, как модель среды становится все более точной. Это позволяет устранить очень продолжительные итерации по значениям, которые могут возникать на ранних этапах обучения из-за больших изменений в модели.

21.3. Активное обучение с подкреплением Пассивный обучающийся агент руководствуется постоянно заданной стратегией, которая определяет его поведение, а активный агент должен сам принимать решение о том, какие действия следует предпринять. Начнем с описания агента, действующего с помощью адаптивного динамического программирования, и рассмотрим, какие изменения необходимо внести в его проект, чтобы он мог функционировать с учетом этой новой степени свободы.

Прежде всего агенту потребуется определить с помощью обучения полную модель с вероятностями результатов для всех действий, а не просто модель для заданной стратегии. Для этой цели превосходно подходит простой механизм обучения, используемый в алгоритме Passive-ADP-Agent. Затем необходимо принять в расчет тот факт, что агент должен осуществлять выбор из целого ряда действий.

Полезности, которые ему потребуются для обучения, определяются оптимальной стратегией; они подчиняются уравнениям Беллмана, приведенным на с. 824, которые мы еще раз приведем ниже для удобства.

$U(s) = R(s) + \gamma \max_{a'} \sum_{s'} T(s,a,s') U(s')$ B1.4) s' Эти уравнения могут быть решены для получения функции полезности и с помощью алгоритмов итерации по значениям или итерации по стратегиям, приведенных в главе 17. Последняя задача состоит в определении того, что делать на каждом этапе. Получив функцию полезности U , оптимальную для модели, определяемой с помощью обучения, агент может извлечь информацию об оптимальном действии, составляя одношаговый прогноз для максимизации ожидаемой полезности; еще один вариант состоит в том, что если используется итерация по стратегиям, то оптимальная стратегия уже известна, поэтому агент должен просто выполнить действие, рекомендуемое согласно оптимальной стратегии. Но действительно ли он должен выполнять именно это действие?

Исследование среды На рис. 21.4 показаны результаты одной последовательности попыток для агента ADP, который следует рекомендациям по выбору оптимальной стратегии для модели, определяемой с помощью обучения, на каждом этапе. Как оказалось, агент не находит с помощью обучения истинные полезности или истинную оптимальную стратегию! Вместо этого происходит то, что после 39-й попытки агент находит стратегию, позволяющую достичь вознаграждения +1 вдоль нижнего маршрута, проходящего через квадраты B,1), C,1), C,2) и C,3) (рис. 21.4). После проведения экспериментов с небольшими вариантами, начиная от 276-й попытки и дальше, агент постоянно придерживается этой стратегии, так и не определив с помощью обучения полезности других состояний и не найдя оптимальный маршрут через квадраты A,2), A,3) и B,3). Авторы называют такого агента, действующего с помощью жадного алгоритма, просто $\wedge$ жадным агентом. Повторные эксперименты показали, что поиски жадного агента очень редко сходятся в пределе к оптимальной стратегии для данной среды, а иногда сходятся к таким стратегиям, которые являются действительно устрашающими по своей неэффективности.

Как могло оказаться, что выбор оптимального действия приводит к неоптимальным результатам? Ответ состоит в том, что модель, определяемая с помощью обучения, не является такой же, как истинная среда; поэтому то, что оптимально в модели, определяемой с помощью обучения, может оказаться неоптимальным в истинной

среде. К сожалению, агент не имеет информации о том, какова истинная среда, поэтому не может вычислить оптимальное действие для истинной среды. Так что же делать?

В проекте жадного агента не учтено то, что действия не только предоставляют вознаграждения в соответствии с моделью, определяемой в настоящее время с помощью обучения, но и вносят вклад в определение с помощью обучения самой истинной модели, влияя на полученные результаты восприятия. Совершенствуя эту модель, агент сможет получать большие вознаграждения не сразу же, а в будущем.

Поэтому агент должен искать компромисс между $\wedge$ потреблением полученных результатов для максимизации своего вознаграждения (что отражается в его текущих оценках полезностей) и $\wedge$ исследованием среды для максимизации своего долгосрочного благосостояния. Занимаясь исключительно потреблением полученных благ, агент рискует застрять в одной колее, а занимаясь исключительно исследованием для повышения уровня своих знаний, агент не получит пользы, если так и не внедрит эти знания на практике. В реальном мире человеку постоянно приходится решать, стоит ли продолжать беззаботное существование или нужно окунуться в неизвестность в надежде найти новые и лучшие условия жизни. Но чем большими знаниями он обладает, тем меньше нуждается в дальнейших исследованиях.

Можно ли выработать более точные рекомендации по сравнению с этими общими рассуждениями? Существует ли оптимальный способ организации исследования среды? Как оказалось, эти вопросы глубоко изучались в той области статистической теории принятия решений, которая касается так называемых $\wedge$ задач с n -рукими бандитами, — так принято называть игровые автоматы, управляемые с помощью рукояток (см. врезку).

3 Обратите внимание на то, что здесь просматривается прямая аналогия с теорией ценности информации, описанной в главе 16.

ас Ю К к 5 &S | 0,5 и Среднеквадратичная ошибка - Убыточность стратегии - 0 50 100 150 200 250 300 350 400 450 500 Количество попыток а) б) $\S$ ц t t ED ED Pwc. 27.4 Производительность агента ADP, действующего с помощью жадного алгоритма, который осуществляет действие, рекомендованное согласно оптимальной стратегии для модели, определяемой с помощью обучения: среднеквадратичная ошибка в оценках полезностей, усредненная по девяти нетерминальным квадратам (а); неоптимальная стратегия, к которой в пределе сходится поиск стратегии, выполняемый агентом с помощью жадного алгоритма, в данной конкретной последовательности попыток (б) Исследование среды и n -рукие бандиты В Лас-Вегасе одноруким бандитом называют игровой автомат определенного типа, в который игрок может вложить монету, потянуть за рукоятку и забрать выигрыш

(если только таковой действительно появится). Существует также разновидность этого автомата с n ручьяками, называемая n -руким бандитом.

Игрок должен выбрать, какую ручьяку следует потянуть на себя после вкладывания каждой следующей монеты, — ту, которая когда-то дала наибольший выигрыш, а может быть, ту, которую он еще не пытался использовать?

Задача с n -руким бандитом — это формальная модель реальных задач во многих жизненно важных областях, таких как принятие решения о выделении годового бюджета на исследования и разработки по искусственному интеллекту. Каждая ручьяка соответствует определенному действию (такому как выделение 20 миллионов долларов на подготовку новых учебников по искусственному интеллекту), а вознаграждение за подтягивание к себе такой ручьяки соответствует прибыли, полученной от выполнения соответствующего действия (в данном случае — просто колоссальной). Исследование среды, будь то исследование перспектив нового научного направления или зондирование нового товарного рынка, является рискованным, дорогостоящим и связано с неопределенными вознаграждениями; с другой стороны, полный отказ от проведения исследований означает, что не удастся обнаружить новые сферы деятельности, которые могут оказаться прибыльными.

Чтобы правильно сформулировать задачу с n -руким бандитом, необходимо точно определить, что подразумевается под оптимальным поведением.

Большинство определений, приведенных в литературе, основано на предположении, что цель состоит в максимизации ожидаемого суммарного вознаграждения, полученного в течение всего срока существования агента. В этих опреде-

лениях требуется, чтобы ожидаемое вознаграждение оценивалось по всем возможным мирам, в которых может оказаться агент, а также по возможным результатам каждой последовательности действий в любом конкретном мире.

В данном случае "мир" определяется моделью перехода $T(s, a, s')$. Таким образом, для того чтобы действовать оптимальным образом, агент должен знать распределение априорных вероятностей по всем возможным моделям.

Возникающие в конечном итоге задачи оптимизации обычно являются крайне трудно разрешимыми.

В некоторых случаях (например, когда вознаграждение, получаемое от игры на каждом автомате, является независимым и используются обесцениваемые вознаграждения) существует возможность рассчитать индекс Гиттинса для каждого игрового автомата [561]. Этот индекс представляет собой функцию, параметрами которой являются только количество игр, проведенных на игровом автомате, и сумма полученного выигрыша. Индекс для каждого автомата показывает, насколько

оправданы дополнительные затраты денег на продолжение игры с учетом комбинации ожидаемой прибыли и ожидаемой стоимости информации. Выбор автомата с наибольшим значением индекса позволяет найти оптимальную стратегию исследования среды. К сожалению, до сих пор не было обнаружено ни одного метода, позволяющего распространить понятие индексов Гиттинса на проблематику задач последовательного принятия решений.

Теорию n -руких бандитов можно использовать для обоснования приемлемости стратегии отбора в генетических алгоритмах (см. главу 4). Если в задаче с n -руким бандитом каждая рукоятка рассматривается как возможная строка генов, а вкладывание монеты для подтягивания одной рукоятки — как воспроизведение этих генов, то генетические алгоритмы должны обеспечить оптимальное вложение монет в игорный автомат с учетом соответствующего множества предположений о независимости.

Хотя задачи с n -рукими бандитами чрезвычайно трудно поддаются точному решению для получения оптимального метода исследования среды, тем не менее возможно предложить приемлемую схему, которая в конечном итоге приводит к оптимальному поведению со стороны агента. Формально любая такая схема должна быть жадной в пределе бесконечного исследования; это свойство сокращенно обозначается как $^{\wedge}$ GLIE (Greedy in the Limit of Infinite Exploration). Схема GLIE должна предусматривать опробование каждого действия в каждом состоянии путем осуществления неограниченного количества попыток такого действия для предотвращения появления конечной вероятности того, что будет пропущено какое-то оптимальное действие из-за крайне неудачного стечения обстоятельств. Агент ADP, в котором используется такая схема, должен в конечном итоге определить с помощью обучения истинную модель среды. Кроме того, схема GLIE должна в конечном итоге стать жадной, для того чтобы действия агента стали оптимальными по отношению к определенной с помощью обучения (и поэтому истинной) модели.

Предложено несколько схем GLIE; в одной из простейших из этих схем предусматривается, что агент должен в течение $1/t$ части времени выбирать случайное действие, а в течение остального времени придерживаться жадной стратегии. Хотя такая схема в конечном итоге сходится к оптимальной стратегии, весь этот процесс

может оказаться чрезвычайно замедленным. Более обоснованный подход предусматривает присваивание определенных весов тем действиям, которые агент не опробовал очень часто, стараясь вместе с тем избегать действий, которые считаются имеющими низкую полезность. Такой подход может быть реализован путем модификации уравнения ограничения 21.4 таким образом, чтобы в нем присваивалась более высокая оценка полезности относительно мало

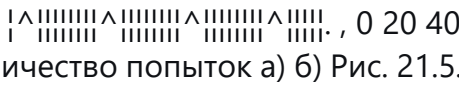
исследованным парам "состояние—действие". По сути это сводится к определению оптимистического распределения априорных вероятностей по возможным вариантам среды и вынуждает агента на первых порах вести себя так, как если бы повсеместно были разбросаны замечательные вознаграждения. Допустим, что для обозначения оптимистической оценки полезности состояния s (т.е. ожидаемого будущего вознаграждения) используется запись $if(s)$, и предположим, что $N(a|s)$ — количество попыток опробования действия a в состоянии s . Предположим также, что в обучающемся агенте ADP используется метод итерации по значениям; в таком случае необходимо переписать уравнение обновления (т.е. уравнение 17.6), чтобы включить в него оптимистическую оценку. Именно такая цель достигается с помощью следующего уравнения: $if(s) \leftarrow R(s) + \gamma \sum_{a'} T(s, a', s') N(a|s) f(u, n)$ где $f(u, n)$ называется "&". функцией исследования. Эта функция определяет компромисс между жадностью (предпочтениями, отданными высоким значениям u) и любопытством (предпочтениями, отданными низким значениям n , характеризующим действия, которые еще не были опробованы достаточно часто). Функция $f(u, n)$ должна увеличиваться в зависимости от u и уменьшаться в зависимости от n . Безусловно, что существует много возможных функций, которые соответствуют этим условиям. Одним из особенно простых определений такой функции является следующее:

$R_+ \text{ если } n < N_e$ и в противном случае где R_+ — оптимистическая оценка наилучшего возможного вознаграждения, которое может быть получено в любом состоянии; N_e — постоянный параметр. Результатом применения такой функции становится то, что агент пытается проверить каждую пару "состояние-действие" по меньшей мере N_e раз.

Тот факт, что в правой части уравнения 21.5 присутствует величина if , а не u , очень важен. Может вполне оказаться так, что по мере развития процесса исследования в большом количестве попыток будут опробованы состояния и действия, находящиеся недалеко от начального состояния. Если бы использовалась более пессимистическая оценка полезности, u , то агент вскоре стал бы не склонным проводить дальнейшее исследование среды. А применение оценки if означает, что выгоды от исследования среды распространяются в обратном направлении от границ неисследованных регионов, поэтому получают больший вес действия, ведущие к неисследованным регионам, а не просто действия, которые сами по себе остаются малоизученными.

Эффект использования такой исследовательской стратегии наглядно показан на рис. 21.5, который демонстрирует более быструю сходимость к оптимальной производительности в отличие от жадного подхода. Способ действий, очень близкий к оптималь-

ному, обнаруживается всего лишь после 18 попыток. Обратите внимание на то, что сами оценки полезности не сходятся так же быстро. Это связано с тем, что агент довольно рано прекращает исследование частей пространства состояний, не предоставляющих вознаграждения, и в дальнейшем посещает их только "по случаю". Но благодаря такому подходу агент приобретает идеальное понимание того, что не следует задумываться о точных значениях полезностей состояний, которые, как ему известно, являются нежелательными и которых можно избежать.

Среднеквадратичная ошибка Убыточность стратегии  , 0 20 40 60 80 100 "О 20 40 60 80 100 Количество попыток Количество попыток а) б) Рис. 21.5. Производительность агента ADP, проводящего исследование среды с использованием параметров $R+=2$ и $N_e=5$: оценки изменения полезностей для избранных состояний во времени (а); среднеквадратичная ошибка в значениях полезностей и связанная с ней убыточность стратегии (б) Определение функции "действие—стоимость" с помощью обучения Теперь, после разработки алгоритма активного агента ADP, рассмотрим, как сконструировать активного агента, обучающегося по методу временной разности.

Наиболее очевидным отличием от пассивного варианта является то, что агенту больше не предоставлена заданная стратегия, поэтому для определения с помощью обучения функции полезности и агенту потребуется определить с помощью обучения некоторую модель, чтобы иметь возможность выбрать с учетом значения [/некоторое действие, выполнив прогнозирование на один шаг вперед. Задача приобретения модели для агента TD идентична такой же задаче для агента ADP. А что можно сказать о самом правиле обновления TD? Возможно, что следующее утверждение на первый взгляд покажется удивительным, но правило обновления 21.3 остается неизменным. Такая ситуация может показаться странной по следующей причине: предположим, что агент делает шаг, который обычно приводит в приемлемое место назначения, но из-за наличия недетерминированности в самой среде в конечном итоге агент оказывается в катастрофическом состоянии. Правило обновления TD позволяет отнестись к этой ситуации так же серьезно, как если бы полученным итогом был нормальный результат действия, тогда как можно было предположить, что агент не должен слишком сильно об этом беспокоиться, поскольку такой итог возник лишь из-за стечения обстоятельств. Безусловно, такой маловероятный результат в большом множестве обучающих последовательностей может возникать 2 1 1,8 | 1,6 | 1,4-| 1,2 ' 1 \ 0,6A,1) A,2) A,3) B,3) C,2) C,3) D,3) 1,2 | s & 0,8 0,6 -I 0,4 J — |s 0,2

достаточно редко, поэтому можно надеяться, что в долговременной перспективе его последствия получают вес, пропорциональный их вероятности. Еще раз отметим следующее — можно доказать, что алгоритм TD сходится к тем же значениям, что и

ADP, по мере того как количество обучающих последовательностей стремится к бесконечности.

Есть также альтернативный метод TD, называемый Q-обучением, в котором предусматривается определение с помощью обучения некоторого представления "действие-стоимость" вместо определения полезностей. Для обозначения стоимости выполнения действия a в состоянии s будет использоваться запись $Q(a, s)$.

Отметим, что Q-значения непосредственно связаны со значениями полезностей следующим образом:

$U(s) = \max_a Q(a, s)$ В1.6) a На первый взгляд может показаться, что Q-функции представляют собой лишь еще один способ хранения информации о полезностях, но они обладают очень важным свойством: $\hat{U}$ = агенту TD, который определяет Q-функцию с помощью обучения, не требуется модель ни для обучения, ни для выбора действия. По этой причине Q-обучение называют $\hat{U}$ безмодельным методом. Как и в случае полезностей, можно записать следующее уравнение для ограничений, которое должно соблюдаться в точке равновесия, когда Q-значения являются правильными:

$Q(a, s) = R(s) + \gamma \sum_{s'} T(s' | a, s) \max_{a'} Q(a', s')$ В1.7) a', s' Как и в случае обучающегося агента ADP, это уравнение может непосредственно использоваться в качестве уравнения обновления для процесса итерации, в котором вычисляются точные Q-значения при наличии оцениваемой модели. Но для этого требуется, чтобы с помощью обучения осуществлялось также определение модели, поскольку в уравнении используется вероятность $T(s', a, s)$. С другой стороны, в подходе на основе временной разности модель не требуется. Уравнение обновления для Q-обучения по методу TD, которое вычисляется каждый раз, когда в состоянии s , ведущем к состоянию s' , выполняется действие a , является следующим:

$Q(a, s) \leftarrow Q(a, s) + \alpha (R(s) + \gamma \max_{a'} Q(a', s') - Q(a, s))$ В1.8) a' Полный проект агента для исследующего среду Q-обучающегося агента, в котором используется метод TD, приведен в листинге 21.3. Обратите внимание на то, что в нем используется точно такая же функция исследования f , которая была предусмотрена для исследующего среду агента ADP, поэтому возникает необходимость вести статистические данные о выполненных действиях (таблицу N). Если бы применялась более простая исследовательская стратегия (скажем, выбор действий случайным образом в некоторой части этапов, притом что эта часть уменьшается со временем), то можно было бы отказаться от ведения этих статистических данных.

Такой Q-обучающийся агент определяет с помощью обучения оптимальную стратегию для мира 4×3 , но достигает этой цели с гораздо меньшей скоростью по сравнению с агентом ADP. Это связано с тем, что метод TD не вынуждает агента

добиваться согласованности значений во всей модели. В связи с этим сравнением возникает общий вопрос:

что лучше — определять с помощью обучения модель и функцию полезности или функцию "действие—значение" без модели? Иными словами, в чем состоит наилучший способ представления функции агента? Это — фундаментальный вопрос искусственного интеллекта. Как было указано в главе 1, традиционно одной из ключевых характерных особенностей многих исследований по искусственному интеллекту была (часто не выраженная явно) приверженность подходу, основанному на знаниях. Такой подход сводится к предположению, что наилучший способ задания функции агента состоит в формировании представления некоторых аспектов среды, в которой находится агент.

Листинг 21.3. Проводящий исследование среды Q-обучающийся агент. Это — активный ученик, который определяет с помощью обучения значение $Q(a, s)$ каждого действия в каждой ситуации.

В нем используется такая же исследовательская функция $?$, как и в проводящем исследование среды агенте ADP, но исключается необходимость определять с помощью обучения модель перехода, поскольку Q -значение любого состояния может быть непосредственно связано с соответствующими значениями его соседних состояний `function Q-Learning-Agent(percept) returns действие a` `inputs: percept, результаты восприятия, обозначающие текущее состояние  $s'$  и сигнал вознаграждения  $r'$` `static: Q, таблица значений действий, индексированная по состояниям и действиям` `Nsa, таблица частот пар "состояние-действие"` `s, a, r, предыдущие состояние, действие и вознаграждение, первоначально пустые` `if состояние s не пусто then do` `увеличить значение  $Nsa[s, a]$` `$Q[a, s] < r - Q[a, s] + (X\{Nsa[s, a]\} (r + \gamma \max_{a'} Q[a, s'] - Q[a, s]))$` `$a' \text{ if Terminal? } [s'] \text{ then } s, a, r \leftarrow \text{пустое значение}$` `else` `$s, a, r \leftarrow s', \operatorname{argmax}_{a'} \{Q[a, s'], Nsa[s', a']\}, r'$` `return a` Некоторые исследователи, и принадлежащие, и не принадлежащие к сообществу специалистов по искусственному интеллекту, выступили с заявлениями, что доступность методов, не требующих применения модели, таких как Q-обучение, означает, что подход, основанный на знаниях, не является необходимым. Тем не менее пока нет почти никаких оснований, позволяющих судить об обоснованности этих заявлений, кроме интуиции. А интуиция авторов в размышлениях о том, какой подход является наиболее перспективным, подсказывает, что по мере усложнения среды преимущества подхода, основанного на знаниях, становятся все более очевидными. Это обнаруживается даже в играх, таких как шахматы, шашки и нарды (см. следующий раздел), где усилия по определению с помощью обучения функции оценки на основе модели увенчались большим успехом, чем методы Q-обучения.

21А Обобщение в обучении с подкреплением До сих пор в этой главе предполагалось, что функции полезности и Q-функции, определяемые агентами с помощью обучения, представлены в табличной форме

с одним выходным значением для каждого входного кортежа. Такой подход полностью себя оправдывает в небольших пространствах состояний, но время достижения сходимости и (в случае ADP) затраты времени на каждую итерацию быстро возрастают по мере увеличения размеров пространства. При использовании тщательно управляемых, приближенных методов ADP иногда удается справиться с задачами по обработке 10 000 или большего количества состояний. Этого достаточно для двухмерных вариантов среды, подобных лабиринтам, но более реальные миры далеко выходят за эти пределы. Шахматы и нарды представляют собой крошечные подмножества реального мира, но даже их пространства состояний содержат примерно от 10⁵ до 10¹² состояний. Даже само предположение о том, что нужно было бы посетить все эти состояния, для того чтобы узнать с помощью обучения, как играть в такую игру, является абсурдным!

Один из способов справиться с этими задачами состоит в использовании средств $\wedge$ функциональной аппроксимации; такая рекомендация просто означает, что для функции следует применять представления любого рода, отличные от таблиц. Такое представление рассматривается как аппроксимированное, поскольку может оказаться, что истинная функция полезности или Q-функция не может быть точно представлена в выбранной форме. Например, в главе 6 была описана функция оценки для шахмат, представленная в виде взвешенной линейной функции от множества характеристик (или $\wedge$ базисных функций) $flf..., fn$:

$UQ(s) = Q_1 f_1(s) + Q_2 f_2(s) + \dots + Q_n f_n(s)$ Алгоритм обучения с подкреплением позволяет определить с помощью обучения такие значения параметров $Q = Q_1, \dots, Q_n$, что функция оценки UQ аппроксимирует истинную функцию полезности. Вместо использования, скажем, 10¹² значений в таблице, такой аппроксиматор функции характеризуется, допустим, $l = 20$ параметрами, а это просто колоссальное сжатие. В частности, хотя никто не знает истинную функцию полезности для шахмат, никто и не считает, что ее можно точно представить с помощью 20 чисел. Но если эта аппроксимация является достаточно качественной, агент все равно приобретает возможность достичь поразительных успехов в шахматах⁴.

Функциональная аппроксимация может дать возможность представить функции полезности для очень больших пространств состояний, но ее основное преимущество состоит не в этом, с $\wedge$ Сжатие, достигнутое с помощью аппроксиматора функции, позволяет обучающемуся агенту делать обобщения, распространяющиеся с тех состояний, которые он уже посетил, на состояния, которые он еще не посетил. Это означает, что наиболее важным аспектом функциональной аппроксимации является не то, что она требует меньше

пространства, а то, что она обеспечивает индуктивное обобщение по входным состояниям. Чтобы дать читателю представление о возможностях 4 Известно, что такая точная функция полезности может быть представлена в виде одной-двух страниц кода на языке Lisp, Java или C++. Это означает, что она может быть представлена с помощью программы, которая находит точное решение для игры после каждого ее вызова. Но нас интересуют только аппроксиматоры функций, которые позволяют обойтись приемлемым объемом вычислений. В действительности может оказаться, что лучше найти с помощью обучения очень простой аппроксиматор функции и применять его в сочетании с некоторым объемом опережающего поиска. Но связанные с этим компромиссы в настоящее время еще недостаточно хорошо изучены.

этого подхода, укажем, что путем исследования только одного состояния из каждой группы по 1044 возможных состояний в игре в нарды можно определить с помощью обучения функцию полезности, позволяющую программе играть не хуже любого игрока-человека [1499].

Оборотной стороной этого подхода, безусловно, является то, что с ним связана такая проблема: невозможность найти в выбранном пространстве гипотез какую-либо функцию, аппроксимирующую истинную функцию полезности достаточно хорошо. Как и во всем научном направлении индуктивного обучения, необходимо найти компромисс между размером пространства гипотез и потребностью во времени для определения с помощью обучения требуемой функции. С увеличением пространства гипотез растет вероятность того, что может быть найдена хорошая аппроксимация, но это означает, что сходимость также, скорее всего, будет достигаться более медленно.

Начнем с простейшего случая, в котором предусматривается непосредственная оценка полезности (см. раздел 21.2). При использовании функциональной аппроксимации определение такой оценки представляет собой пример контролируемого обучения. Например, предположим, что полезности для мира 4×3 представлены с использованием простой линейной функции. Характеристиками квадратов являются их координаты x и y , поэтому получим следующее соотношение:

$L70(x,y) = G_0 + G_1 x + G_2 y$ (1.9) Таким образом, если $(80, 8X, 92) = @ . 5, 0 . 2, 0 . 1)$, то $UQ A,1) = 0 . 8$. По результатам некоторой совокупности попыток будет получено множество выборочных значений $UQ(x, y)$, после чего можно найти соответствие, наилучшее с точки зрения минимизации квадратичных ошибок, с помощью стандартной линейной регрессии (см. главу 20).

Применительно к обучению с подкреплением больше смысла имеет использование алгоритма оперативного обучения, который обновляет параметры после каждой попытки. Предположим, что осуществляется некоторая попытка, и суммарное

вознаграждение, полученное начиная с квадрата $A,1$), составляет 0.4. Такой результат наводит на мысль, что значение $UQ(A,1)$, составляющее в настоящее время 0.8, слишком велико и должно быть уменьшено. Как следует откорректировать параметры, чтобы добиться такого уменьшения? Как и в случае с обучением нейронной сети, запишем функцию ошибки и вычислим ее градиент по отношению к параметрам. Если $\delta_j(s)$ — наблюдаемое суммарное вознаграждение, полученное от состояния s и дальше, вплоть до j -й попытки, то ошибка определяется как (половина) квадратов разностей прогнозируемой общей суммы и фактической общей суммы:

$$E_j(s) = (UQ(s) - u_j(s))^2 / 2.$$
 Скорость изменения ошибки по отношению к каждому параметру θ_i равна $\partial E / \partial \theta_i$, поэтому, чтобы изменить значение параметра в направлении уменьшения ошибки, необходимо вычислить следующее выражение: $\delta E / \delta \theta_i(s) = dUe(s) / d\theta_i - \theta_i - a = \theta_i + a(u_j(s) - Ue(s))$ (Эе, В1.10) Это выражение называется δ правилом Видроу—Хоффа, или δ дельта-правилом для оперативного вычисления по методу наименьших квадратов. Применительно к линейному аппроксиматору функции $UQ(s)$ в уравнении 21.9 получаем три простых правила обновления:

$$\begin{aligned} G_0 &\leftarrow G_0 + a(u_j(s) - Uq(s)) \\ \theta_i &\leftarrow \theta_i + a(u_j(s) - r_0(s)) \\ G_2 &\leftarrow G_2 + a(u_j(s) - uQ(s)) \end{aligned}$$
 Эти правила можно применить к примеру, в котором $L7e(l/D)$ равно 0.8, а $\delta(A,1)$ равно 0.4. Все значения θ_0 , θ_2 и θ_3 уменьшаются на коэффициент 0.4а, который уменьшает ошибку, относящуюся к квадрату $A,1$). Обратите внимание на то, что изменение значений θ_i приводит также к изменению значений uQ для всех прочих состояний! Именно это подразумевалось в приведенном выше утверждении, что функциональная аппроксимация позволяет агенту, проводящему обучение с подкреплением, обобщать свой опыт.

Мы рассчитываем на то, что агент будет проводить обучение быстрее, если он использует какой-то аппроксиматор функции, при условии, что пространство гипотез не слишком велико, но включает некоторые функции, характеризующиеся достаточно приемлемым соответствием истинной функции полезности. В упр. 21.7 предлагается оценить производительность метода непосредственной оценки полезности, как с функциональной аппроксимацией, так и без нее. В мире 4×3 действительно достигается заметное, однако не столь существенное увеличение производительности, прежде всего потому, что это пространство состояний очень мало. Достигнутое увеличение производительности становится намного более значительным в мире 10×10 с вознаграждением +1 в квадрате $A0,10$). Этот мир хорошо приспособлен для линейной функции полезности, поскольку истинная функция полезности является гладкой и почти линейной (см. упр. 21.10). А если вознаграждение +1 будет помещено в квадрат $E,5$), то истинная функция полезности будет больше напоминать по своей форме пирамиду, и попытка применения аппроксиматора функции, приведенного в уравнении 21.9, окончится крахом. Но не

все потеряно! Напомним, что для линейной функциональной аппроксимации важно, чтобы функция линейно зависела от параметров. А сами характеристики могут представлять собой произвольные нелинейные функции от переменных состояния. Поэтому можно включить такой терм, как $\|x - x_g\|^2 + \|y - y_g\|^2$, измеряющий расстояние до цели.

Эти идеи можно применить столь же успешно к агентам, осуществляющим обучение по методу временной разности. Для этого достаточно откорректировать параметры, чтобы попытаться уменьшить временную разность между последовательными состояниями. Новые версии уравнений для метода TD и метода Q-обучения B1.3 и B1.8) приведены ниже. Уравнение для полезностей является следующим: $G_i \leftarrow G_i + \alpha [R(s) + u_Q(s') - u_Q(s)]$ (B1.11) А для Q-значений используется следующее уравнение: $Q(a, s) \leftarrow Q(a, s) + \alpha [R(s) + \max_{a'} Q(a', s') - Q(a, s)]$ (B1.12) Можно показать, что эти правила обновления сходятся к ближайшей возможной аппроксимации истинной функции, если аппроксиматор функции линейно зависит от параметров. Это определение расстояния между функциями полезности является довольно формальным; см. [1515].

функции, задаваемые с помощью нейронных сетей) больше ничего нельзя гарантировать. Параметры могут увеличиваться до бесконечности и в некоторых очень простых случаях, даже несмотря на то, что в пространстве гипотез существуют приемлемые решения. Разработаны более сложные алгоритмы, позволяющие избежать этих проблем, но в настоящее время вся область обучения с подкреплением на основе общих аппроксиматоров функций продолжает оставаться тонким искусством.

Функциональная аппроксимация может также оказаться очень полезной при определении с помощью обучения модели среды. Напомним, что задача определения с помощью обучения модели для наблюдаемой среды представляет собой задачу контролируемого обучения, поскольку каждый следующий результат восприятия предоставляет информацию о результирующем состоянии. При этом может использоваться любой из методов контролируемого обучения, описанный в главе 18, с соответствующими поправками с учетом того факта, что необходимо определить с помощью прогноза полное описание состояния, а не просто булеву классификацию или единственное реальное значение. Например, если состояние определяется с помощью p булевых переменных, то необходимо найти с помощью обучения p булевых функций для прогнозирования всех переменных. А в случае частично наблюдаемой среды задача обучения становится гораздо более сложной. Если известно, каковы скрытые переменные и какими причинными отношениями они связаны друг с другом и с наблюдаемыми переменными, то можно зафиксировать структуру динамической байесовской сети и воспользоваться алгоритмом EM для определения с помощью обучения ее параметров, как было

описано в главе 20. А задачи выявления скрытых переменных и определения структуры модели с помощью обучения все еще остаются открытыми.

Теперь обратимся к примерам крупномасштабных приложений обучения с подкреплением. На основании этих примеров будет показано, что в случае использования функции полезности (следовательно, некоторой модели) модель обычно считается заданной.

Например, при определении с помощью обучения функции оценки для нарда обычно предполагается, что допустимые ходы и их результаты известны заранее.

Приложения методов обучения к ведению игр Первое реальное приложение метода обучения с подкреплением оказалось также первой практически применимой программой из всех возможных типов таких программ; речь идет о программе игры в шашки, написанной Артуром Самюэлом [1349], [1350]. Самюэл впервые использовал для оценки позиций взвешенную линейную функцию, в которой одновременно применялось до 16 термов. В его программе обновление весов осуществлялось на основе некоторой версии уравнения 21.11. Тем не менее методы, применяемые в его программе, имели определенные существенные отличия от современных методов. Первое отличие состояло в том, что для обновления весов использовалась разность между значениями для текущего состояния и зарезервированным значением, сформированным путем полного опережающего просмотра в дереве поиска. Такой подход оказался очень успешным, поскольку он равносителен подходу, в котором пространство состояний рассматривается с разными степенями детализации. Второе отличие состояло в том, что в программе Самюэла не использовались какие-либо наблюдаемые вознаграждения! Это означает, что значения терминальных состояний игнорировались. Таким образом, сущест-

вовала реальная возможность, что вычисления в программе не будут сходиться, или по крайней мере сходиться к стратегии, позволяющей выиграть, а не проиграть.

Но автор программы сумел избежать подобного неблагоприятного развития событий, соблюдая такое требование, что вес материального преимущества должен всегда быть положительным. Замечательно то, что этого оказалось достаточно, чтобы всегда направлять программу в область пространства весов, соответствующую успешной игре в шашки.

Возможности методов обучения с подкреплением особенно наглядно показала система TD-Gammon Герри Тесауро [1499]. В одной из более ранних работ [1501] Тесауро попытался определить с помощью обучения некоторое представление функции $Q(a, s)$ в виде нейронной сети непосредственно на примерах ходов, которым были присвоены относительные значения экспертом-человеком. Но, как оказалось, этот подход требовал исключительно высоких трудозатрат со стороны

привлеченного эксперта. Применение такого подхода привело к созданию программы под названием Neurogammon, которая оказалась сильнее, чем другие программы игры в нарды, существовавшие в то время, но не выдерживала соревнования с опытными игроками в нарды. Проект TD-Gammon стал попыткой обучения программы исключительно в игре с самой собой. Единственный сигнал вознаграждения предоставлялся в конце каждой игры. Функция оценки была представлена в виде полносвязной нейронной сети с единственным скрытым слоем, содержащим 40 узлов.

Программа TD-Gammon сумела в результате обучения достичь значительно более высокого уровня игры по сравнению с Neurogammon путем повторного применения уравнения 21.11, даже несмотря на то, что входное представление содержало только грубые оценки позиции на доске, без вычисленных характеристик. Для обучения программы потребовалось проведение около 200 000 учебных игр и две недели машинного времени. Хотя такое количество игр может показаться очень большим, оно фактически составляет лишь исчезающе малую долю пространства состояний.

После того как к входному представлению были добавлены предварительно рассчитанные характеристики с применением сети, включающей 80 скрытых элементов, в результате 300 000 учебных игр удалось достичь уровня игры, сравнимого с уровнем трех ведущих людей-игроков во всем мире. Кит Вулси, ведущий игрок и аналитик, сказал: "У меня не было малейшего повода, чтобы усомниться в том, что эта программа оценивает позицию гораздо лучше по сравнению со мной".

Применение к управлению роботами Постановка знаменитой задачи поиска равновесия системы ^ тележка—шест, известной также под названием задачи ^ обратного маятника, показана на рис. 21.6.

Задача состоит в том, чтобы обеспечить управление положением тележки в направлении оси x таким образом, чтобы шест стоял примерно вертикально ($\theta \sim \pi/2$), а сама тележка оставалась в пределах длины пути, как показано на этом рисунке.

Такой с виду простой задаче посвящено свыше двух тысяч статей по обучению с подкреплением и теории управления. Задача с тележкой и шестом отличается от описанных выше задач тем, что переменные состояния x , $\dot{x}$, θ , $\dot{\theta}$ являются непрерывными. С другой стороны, действия обычно остаются дискретными: таковыми являются рывки тележки влево или вправо в так называемом режиме ^ релейного управления.

Л. 1 : Ж ш I Рис. 21.6. Постановка задачи уравнивания длинного шеста, установленного на движущейся тележке. Контроллер, измеряющий значения x , $\dot{x}$, θ , $\dot{\theta}$

и в, рывками двигает тележку влево или вправо. Одна из первых работ по обучению механической системы решению этой задачи была выполнена Мичи и Чамберсом [1046]. Разработанный ими алгоритм Voxes оказался способным поддерживать равновесие шеста больше одного часа после всего лишь 30 пробных попыток. Более того, в отличие от многих разработанных в дальнейшем систем, алгоритм Voxes был реализован с помощью реальной тележки и шеста, а не эмулятора.

На первых порах в этом алгоритме применялась дискретизация четырехмерного пространства состояний с разбивкой на гиперкубы, называемые разработчиками 'boxes', отсюда произошло название алгоритма. После ввода алгоритма в действие выполнялись учебные попытки, продолжавшиеся до тех пор, пока не падал шест или тележка не достигала конца пути. Отрицательное подкрепление связывалось с заключительным действием в конечном гиперкубе, а затем распространялось в обратном направлении через всю последовательность действий. В дальнейшем было обнаружено, что применяемый способ дискретизации становится причиной возникновения некоторых проблем, когда вся эта экспериментальная установка инициализировалась в позиции, отличной от той, которая применялась при обучении (что свидетельствовало о недостаточно качественном обобщении). Лучшего обобщения и более быстрого обучения можно добиться, используя алгоритм, который адаптивно фрагментирует пространство состояний согласно наблюдаемому изменению в вознаграждении. В наши дни считается тривиальной задачей уравнивание трех обратных маятников, что значительно превосходит возможности большинства людей.

21.5. Поиск СТРАТЕГИИ Последний подход к решению задач обучения с подкреплением, который рассматривается в этой главе, носит название λ поиска стратегии. В определенных отношениях метод, основанный на поиске стратегии, является самым простым из всех методов, рассматриваемых в данной главе; его идея состоит в том, что необходимо

продолжать корректировать стратегию до тех пор, пока связанная с ней производительность увеличивается, а после этого останавливаться.

Начнем с анализа самих стратегий. Напомним, что стратегией p называется функция, которая отображает состояния на действия. Нас прежде всего интересуют параметризованные представления p , которые имеют намного меньше параметров по сравнению с количеством состояний в пространстве состояний (как и в предыдущем разделе). Например, можно представить стратегию p в виде коллекции параметризованных Q -функций, по одной для каждого действия, и каждый раз предпринимать действие с наивысшим прогнозируемым значением:

$$p_t(s) = \max_a Q_t(s, a) \quad (21.13)$$
 а Каждая Q -функция может представлять собой линейную функцию от параметров θ , как показано в уравнении 21.9, или может быть и

нелинейной функцией, например, представленной в виде нейронной сети. В таком случае поиск стратегии сводится к корректировке параметров θ в целях улучшения стратегии. Следует отметить, что если стратегия представлена с помощью Q-функций, то поиск стратегии приводит к процессу определения Q-функций с помощью обучения. & Этот процесс не следует рассматривать как Q-обучение! При Q-обучении с помощью функциональной аппроксимации алгоритм находит такое значение θ , что функция Q_θ становится "близкой" к функции Q^* — оптимальной Q-функции. При поиске стратегии, с другой стороны, отыскивается значение θ , которое приводит к достижению высокой производительности; найденные значения могут отличаться весьма существенно. Еще одним примером, четко иллюстрирующим это различие, является случай, в котором $p(s)$ вычисляется, допустим, с помощью опережающего поиска на глубину 10 с приближенной функцией полезности u_Q . Значение θ , позволяющее обеспечить качественное ведение игры, может быть получено задолго до того, как удастся получить приближение $\pi^*/0$, напоминающее истинную функцию полезности.

Одним из недостатков представления стратегий в том виде, как показано в уравнении 21.13, является то, что в случае дискретных действий стратегия не является непрерывной функцией своих параметров⁷. Это означает, что существуют такие значения θ , при которых бесконечно малые изменения θ вызывают резкое переключение стратегии с одного действия на другое. Вследствие этого могут также происходить не являющиеся непрерывными изменения значения стратегии, в результате чего поиск на основе градиента становится затруднительным. По этой причине в методах поиска стратегии часто используется $\wedge$ стохастическое представление стратегии $\pi_\theta(s, a)$, которое задает вероятность выбора действия a в состоянии s . Одним из широко применяемых представлений такого типа является $\wedge$ функция softmax: $\pi_\theta(s, a) = \exp(\theta(s, a)) / \sum_a \exp(\theta(s, a))$.⁶ Стало тривиальным такое замечание, что приближенная Q-функция, определяемая соотношением $Q_\theta(a, s) = Q^*(a, s) / 10$, обеспечивает оптимальную производительность, даже несмотря на то, что она весьма далека от Q^* .

⁷ В случае непрерывного пространства действий стратегия может быть гладкой функцией от ее параметров.

Функция softmax становится почти детерминированной, если одно действие намного лучше по сравнению с другими, но она всегда позволяет получить дифференцируемую функцию от θ , поэтому ценность стратегии (которая связана непрерывной зависимостью с вероятностями выбора действий) определяется дифференцируемой функцией θ .

Теперь рассмотрим методы улучшения стратегии. Начнем с простейшего случая — детерминированной стратегии и детерминированной среды. В этом случае задача вычисления стратегии становится тривиальной — просто выполняется эта стратегия

и регистрируются накопленные вознаграждения; полученные данные определяют $\hat{V}_0 p(\theta)$ ценность стратегии $p(\theta)$. В таком случае улучшение стратегии представляет собой стандартную задачу оптимизации, как описано в главе 4. В частности, можно проследить за вектором $\hat{V}_0 p(\theta)$ градиента стратегии $V_0 p(\theta)$, при условии, что значение $p(\theta)$ является дифференцируемым. Другой вариант состоит в том, что можно проследовать за эмпирическим градиентом путем восхождения к вершине, т.е. вычисления изменений в стратегии в ответ на небольшие приращения в значениях каждого параметра. При соблюдении обычных предосторожностей такой процесс сходится к локальному оптимуму в пространстве стратегий.

Если же среда или стратегия является стохастической, задача становится более сложной. Предположим, что предпринимается попытка применить метод восхождения к вершине, для чего требуется сравнить $p(\theta)$ и $p(\theta + \Delta\theta)$ при некотором небольшом значении $\Delta\theta$. Проблема состоит в том, что суммарное вознаграждение при каждой попытке может существенно изменяться, поэтому оценки значения стратегии на основе небольшого количества попыток могут оказаться совершенно ненадежными, а еще более ненадежными становятся результаты, полученные при попытке сравнить две такие оценки. Одно из решений состоит в том, чтобы предпринять много попыток, измеряя дисперсию выборок и используя ее для определения того, достаточно ли много было сделано попыток, чтобы получить надежные данные о направлении улучшения для $p(\theta)$. К сожалению, такой подход является практически не применимым во многих реальных задачах, когда каждая попытка может оказаться дорогостоящей, требующей больших затрат времени, и, возможно, даже опасной.

В случае стохастической стратегии $p_Q(s, a)$ существует возможность получить несмещенную оценку для градиента θ , $V_0 p(\theta)$, соответствующего параметрам θ , непосредственно по результатам попыток, выполненных при таких значениях параметра θ . Для упрощения задачи выведем формулу такой оценки для простого случая непоследовательной среды, в которой вознаграждение предоставляется непосредственно после осуществления действия в начальном состоянии s_0 . В таком случае значение стратегии является просто ожидаемым значением вознаграждения, поэтому имеет место следующее:

$$V_{\theta} p(\theta) = V_{\theta} / \sum_{i=1}^{\infty} \gamma^{i-1} p_i(s_0, a) = \sum_{i=1}^{\infty} \gamma^{i-1} V_{\theta} p_i(s_0, a) p(a)$$
 а теперь можно применить простой пример, позволяющий аппроксимировать результаты этого суммирования с помощью выборок, сформированных на основании распределения вероятностей, определенного стратегией $p_Q(s_0, a)$. Предположим,

что общее количество попыток равно N , а действием, предпринятым в j -й попытке, является a_j . В таком случае получим следующее:

$N \text{ Vep}(9) = \sum_{j=1}^n \nabla_{\theta} V(s_0, a_j) = \sum_{j=1}^n \nabla_{\theta} \sum_{s'} P(s' | s, a_j) V(s')$ Поэтому истинный градиент значения стратегии аппроксимируется суммой термов, включающей градиент вероятности выбора действия при каждой попытке. Для последовательного случая это соотношение можно обобщить до такого соотношения для каждого посещенного состояния s : $\nabla_{\theta} V(s) = \sum_{j=1}^n \nabla_{\theta} \sum_{s'} P(s' | s, a_j) V(s')$ где a_j — действие, выполненное в состоянии s при j -й попытке; $R_j(s)$ — суммарное вознаграждение, полученное, начиная от состояния s и дальше, при j -й попытке.

Полученный в результате алгоритм называется Reinforce [1597]; обычно он является гораздо более эффективным по сравнению с восхождением к вершине, при котором используется большое количество попыток в расчете на каждое значение θ . Тем не менее он все еще действует гораздо медленнее, чем абсолютно необходимо.

Рассмотрим следующее задание: даны две программы игры в очко8; необходимо определить, какая из них лучше. Один из способов выполнения этого задания состоит в организации игры каждой программы против стандартной программы, "сдающей карты", в течение определенного количества раздач, с последующим сравнением выигрышей этих программ. Но этот подход связан с определенными проблемами, поскольку, как уже было сказано, выигрыш каждой программы изменяется в широких пределах в зависимости от того, получала ли она после каждой раздачи хорошие или плохие карты. Очевидным решением является заблаговременная выработка определенного количества раздач и $\sim 3 \times 10^6$ множества карт, выдаваемых на руки. Благодаря этому устраняется ошибка измерения, связанная с различиями в полученных картах. Именно эта идея лежит в основе алгоритма Pegasus [1134]. Такой алгоритм является применимым в таких проблемных областях, для которых предусмотрен эмулятор, позволяющий повторно вырабатывать "случайные" результаты действий. Алгоритм функционирует по принципу заблаговременного формирования N последовательностей случайных чисел, каждая из которых может использоваться для прогона одного варианта опробования любой стратегии. Поиск стратегии осуществляется путем оценки каждой потенциальной стратегии с помощью одного и того же множества случайных последовательностей для определения результатов действия.

Можно показать, что количество случайных последовательностей, требуемых для обеспечения качественной оценки значения любой стратегии, зависит только от сложности пространства стратегий, а не от сложности соответствующей проблемной области. Алгоритм Pegasus использовался для разработки эффективных стратегий в нескольких проблемных областях, включая автономный полет вертолета (рис. 21.7).

Эта игра известна также под названием двадцать одно или блек джек.

Рис. 21.7. Наложение изображений автономно управляемого вертолета, выполняющего очень сложный маневр инос в круге". Вертолет совершает полет под управлением стратегии, разработанной алгоритмом поиска стратегии Pegasus.

Модель эмулятора была разработана путем наблюдений за результатами различных управляющих манипуляций реального вертолета; после этого алгоритм в течение одной ночи выполнял прогон на этой модели эмулятора. Был разработан целый ряд контроллеров для различных маневров. Во всех случаях достигнутая производительность была намного выше по сравнению с производительностью опытного пилота-человека, использующего дистанционное управление (фотография приведена с разрешения Эндрю Энджи (Andrew Ng)) 21.6. Резюме В данной главе рассматривалась задача обучения с подкреплением, решение которой позволяет агенту добиться успеха в неизвестной среде, пользуясь только полученными им результатами восприятия, а изредка также вознаграждениями.

Обучение с подкреплением может рассматриваться как микроскопическая модель всей проблематики искусственного интеллекта, но для решения этой задачи применяется целый ряд упрощений, позволяющих быстрее добиться успеха. Основные идеи, изложенные в этой главе, перечислены ниже. • Характер информации, которая должна быть получена в результате обучения, зависит от общего проекта агента. В данной главе рассматривались три основных проекта: проект, основанный на модели, в котором используется модель T и функция полезности U ; проект без модели, в котором применяется функция "действие—значение", или Q -функция, и рефлексный проект, в котором используется стратегия тс. • Для определения полезностей с помощью обучения могут использоваться три перечисленных ниже подхода.

1. При непосредственной оценке полезности применяется суммарное прогнозируемое наблюдаемое вознаграждение для заданного состояния в качестве прямого свидетельства, позволяющего определить полезность данного состояния с помощью обучения.

2. В адаптивном динамическом программировании (Adaptive Dynamic Programming — ADP) с помощью обучения определяются модель и функция вознаграждения на основании наблюдений, а затем используется итерация по значениям или по стратегиям для получения полезностей или выявления оптимальной стратегии. В методе ADP обеспечивается оптимальное использование локальных ограничений, налагаемых на полезности состояний под влиянием структуры отношений соседства в рассматриваемой среде.

3. В методах временной разности (Temporal Difference — TD) обновляются оценки полезности для их согласования с состояниями-преемниками.

Подход, основанный на использовании этих методов, может рассматриваться как простая аппроксимация подхода ADP, в котором не требуется модель для процесса обучения. Однако применение определенной в процессе обучения модели для выработки псевдорезультатов опытов способствует ускорению обучения. • Определение с помощью обучения функций "действие—значение", или Q-функций, может быть организовано на основе подхода ADP или TD. При использовании метода TD в процессе Q-обучения не требуется модель ни на этапе обучения, ни на этапе выбора действия. Это позволяет упростить задачу обучения, но в принципе может привести к ограничению способности проводить обучение в сложных вариантах среды, поскольку агент не сможет моделировать результаты применения возможных стратегий. • Если от обучающегося агента требуется, чтобы он выбирал действия, пока еще продолжается обучение, то агенту приходится искать компромисс между оцениваемым значением этих действий и перспективами определения с помощью обучения новой полезной информации. Задача поиска точного решения такой проблемы исследования является неосуществимой, но некоторые простые эвристики позволяют добиться приемлемых результатов. • Если пространство состояний велико, то в алгоритмах обучения с подкреплением необходимо использовать приближенное функциональное представление для обобщения сведений о состояниях. Для обновления параметров таких представлений, как нейронные сети, можно непосредственно использовать информацию о временной разности. • Методы поиска стратегии применяются непосредственно к представлению стратегии в попытке улучшить ее с учетом наблюдаемой производительности.

Изменчивость производительности в стохастической проблемной области представляет собой серьезную проблему; в случае моделируемых проблемных областей эту сложность можно преодолеть, заранее формируя случайные выборки. • Обучение с подкреплением позволяет избавиться от необходимости разрабатывать вручную стратегии управления, поэтому продолжает оставаться одной из наиболее активных областей исследований машинного обучения.

Особенно ценными могут стать приложения этих подходов в робототехнике; для этого потребуются методы обеспечения действий в непрерывных, многомерных, частично наблюдаемых вариантах среды, в которых успешное поведение может складываться из тысяч или даже миллионов примитивных действий.

Библиографические и исторические заметки Подход, основанный на обучении с подкреплением, предложил Тьюринг [1519], [1520], но он не был убежден в его общей эффективности и писал, что "использование наказаний и вознаграждений в лучшем случае может составлять лишь часть процесса обучения". По-видимому, одним из первых успешных исследований по машинному обучению была работа Артура Самюэла [1349]. Хотя эта работа не была основана на теоретическом

фундаменте и имела целый ряд недостатков, она содержала большинство современных идей в области обучения с подкреплением, включая применение временной разности и функциональной аппроксимации.

Примерно в то же время исследователи в области теории адаптивного управления Видроу и Хофф [1587], опираясь на работу Хебба [638], занимались обучением простых сетей с помощью дельта-правила. (Долго существовавшие неправильные представления о том, что обучение с подкреплением является частью проблематики нейронных сетей, могло быть вызвано тем, что связь между этими научными областями установилась так рано.) Как метод обучения с подкреплением на основе аппроксиматора функции может также рассматриваться работа Мичи и Чамберса [1046], посвященная системе "тележка—шест". Психологические исследования в области обучения с подкреплением начались гораздо раньше; хороший обзор по этой теме приведен в [653]. Непосредственные свидетельства того, как осуществляется обучение с подкреплением у животных, были получены в исследованиях поведения пчел, связанного с добычей пищи; достоверно обнаружен нейронный аналог структуры передачи сигнала вознаграждения в виде крупного нейронного образования, связывающего рецепторы органов взятия нектара непосредственно с двигательной корой [1070]. Исследования, в которых используется регистрация активности отдельной клетки, показали, что допаминавая система в мозгу приматов реализует нечто напоминающее обучение на основе стоимостной функции [1369].

Связь между обучением с подкреплением и марковскими процессами принятия решений была впервые отмечена в [1580], но разработка методов обучения с подкреплением в рамках искусственного интеллекта началась с исследований, проводимых в Массачусеттском университете в начале 1980-х годов [76]. Хороший исторический обзор подготовлен Саттоном [1477]. Уравнение 21.3, приведенное в этой главе, представляет собой частный случай общего алгоритма TD(A) Саттона при $X=0$.

В алгоритме TD(A) обновляются значения всех состояний в последовательности, ведущей вплоть до каждого перехода, на величину, которая уменьшается в зависимости от X_t для состояний, отстоящих на t шагов в прошлое. Алгоритм TDA) идентичен правилу Видроу-Хоффа, или дельта-правилу. Бойян [162], опираясь на [170], доказал, что в алгоритме TD(A) и связанных с ним алгоритмах результаты, полученные опытным путем, используются неэффективно; по сути они представляют собой алгоритмы оперативной регрессии, которые сходятся гораздо медленнее, чем алгоритмы автономной регрессии. Предложенный Бойяном алгоритм LSTD(A) представляет собой оперативный алгоритм, позволяющий достичь таких же результатов, как и алгоритм автономной регрессии.

Применение сочетания метода обучения на основе временной разности с методом формирования моделируемых результатов опытов с помощью модели было предложено в архитектуре Саттона Dyna [1479]. Идея выметания с учетом приоритетов была предложена независимо Муром и Аتكесоном [1077], а также Пенгом и Уильямсом [1203]. Метод Q-обучения был разработан в докторской диссертации Уоткинса [1558].

Задачи с n -рукими бандитами, которые моделируют задачу исследования непоследовательных решений, были глубоко проанализированы в [116]. Оптимальные стратегии исследования для нескольких постановок задач могут быть получены с помощью метода, называемого индексами Гиттинса [561]. Целый ряд методов исследования, применимых для решения задач последовательного принятия решений, обсуждается в [74]. В [171] и [785] описаны алгоритмы, позволяющие исследовать неизвестные варианты среды и гарантирующие сходимость к стратегиям, близким к оптимальным, за полиномиальное время.

Истоки идей по применению функциональной аппроксимации в обучении с подкреплением можно найти в работах Самюэла, который использовал линейные и нелинейные функции оценки, а также методы выбора характеристик для уменьшения пространства характеристик. В дальнейшем были разработаны такие методы, как $\wedge$ CMAС (Cerebellar Model Articulation Controller) [12], который по сути сводится к использованию суммы перекрывающихся локальных ядерных функций, и ассоциативные нейронные сети [75]. В настоящее время в качестве аппроксиматоров функций наиболее широко используются нейронные сети. Наиболее известным приложением является программа TD-Gammon [1499], [1500], которая описывалась в данной главе. Одной из существенных проблем, возникающих при использовании обучающихся по методу TD агентов, основанных на нейронной сети, является то, что они, как правило, забывают полученные раньше результаты опытов, особенно касающиеся тех частей пространства состояний, которых они стали избегать после приобретения достаточной компетентности. Этот недостаток может приводить к катастрофическому отказу, если снова возникают подобные обстоятельства. Такую проблему позволяет устранить функциональная аппроксимация с помощью обучения на основе экземпляра (instance-based learning) [477], [1159].

Тема, касающаяся сходимости алгоритмов обучения с подкреплением, в которых применяется функциональная аппроксимация, требует чрезвычайно формального освещения. В случае использования аппроксиматоров линейных функций результаты обучения по методу TD неизменно улучшались [338], [1477], [1515], но было показано, что при использовании нелинейных функций обнаруживаются некоторые примеры отсутствия сходимости (для ознакомления с этой темой см. [1515]). В [1172] описан новый тип алгоритма обучения с подкреплением, который

сходится при использовании любой формы аппроксиматора функции, при условии, что для наблюдаемых данных может быть найдена аппроксимация с наилучшим соответствием.

Методы поиска стратегии вышли на передний план под влиянием исследований Уильямса [1597], который разработал семейство алгоритмов Reinforce. Дальнейшие исследования, описанные в [86], [981] и [1478], позволили усилить и обобщить результаты достижения сходимости в методе поиска стратегии. Алгоритм Pegasus был предложен Энджи и Джорданом [1134], но аналогичные методы изложены в докторской диссертации Ван Роя [1535]. Как упоминалось в этой главе, производительность стохастической стратегии представляет собой непрерывную функцию от ее

параметров, что способствует применению методов поиска с учетом градиента.

Это — не единственное преимущество указанных методов; в [721] доказано, что стохастические стратегии обычно функционируют в частично наблюдаемых вариантах среды лучше, чем детерминированные стратегии, если те и другие ограничиваются действиями, основанными на текущих результатах восприятия. (Одна из причин этого состоит в том, что стохастическая стратегия с меньшей вероятностью "заходит в тупик", встретив какое-то невидимое препятствие.) Кроме того, в главе 17 было указано, что оптимальные стратегии в частично наблюдаемых задачах MDP представляют собой детерминированные функции от доверительного состояния, а не от текущих результатов восприятия, поэтому можно рассчитывать на получение еще лучших результатов с помощью слежения за доверительным состоянием с использованием методов фильтрации, приведенных в главе 15. К сожалению, пространство доверительных состояний является многомерным и непрерывным, и еще не разработаны эффективные алгоритмы обучения с подкреплением на основе доверительных состояний.

Реальные варианты среды также характеризуются невероятной сложностью с точки зрения количества примитивных действий, требуемых для достижения значимых вознаграждений. Например, роботу, играющему в футбол, могут потребоваться сотни тысяч отдельных движений ног для того, чтобы забить мяч. Одним из широко применяемых методов, который первоначально использовался при обучении животных, является так называемый метод [^] формирования вознаграждения (reward shaping). Он предусматривает предоставление агенту дополнительных вознаграждений за то, что он "добивается успехов". Что касается футбола, то такие вознаграждения могут предоставляться за удар по мячу или за отправку мяча в сторону ворот. Подобные вознаграждения позволяют существенно повысить скорость обучения, а сам способ их предоставления является очень простым, но существует опасность того, что агент обучится максимизации таких псевдовознаграждений и не будет стремиться к истинным вознаграждениям;

например, многочисленных контактов с мячом можно добиться, стоя рядом с мячом и "совершая колебательные движения". В [1133] показано, что агент все равно будет искать с помощью обучения оптимальную стратегию, при условии, что псевдовознаграждение $F(s, a, s_1)$ удовлетворяет соотношению $F(s, a, s') = \gamma \Phi(s') - \Phi(s)$, где Φ — произвольная функция от состояния. Функцию Φ можно составить таким образом, чтобы она отражала все желательные аспекты состояния, такие как достижение подцелей или уменьшение расстояния до целевого состояния.

Выработку сложных вариантов поведения можно также обеспечить с помощью методов $\wedge$ иерархического обучения с подкреплением, которые представляют собой попытку решать задачи на нескольких уровнях абстракции, во многом аналогично методам планирования HTN, описанным в главе 12. Например, цель "забить мяч" можно разделить на подцели "овладеть мячом", "приближаясь к воротам, обвести противников" и "ударить по воротам", а каждая из этих подцелей может быть дополнительно разделена на двигательные варианты поведения еще более низкого уровня. Фундаментальный результат в этой области получен Форестьером и Варайя [481], которые доказали, что варианты поведения низкого уровня с произвольной сложностью можно рассматривать как примитивные действия (хотя и учитывая при этом то, что они могут потребовать разного количества времени) с точки зрения вариантов поведения более высокого уровня, в которых они вызываются. В современ-

ных подходах [32], [397], [1177], [1478] эти результаты используются для создания методов, позволяющих предоставить агенту $\wedge$ частичную программу, которая ограничивает поведение агента так, чтобы оно имело конкретную иерархическую структуру. После этого применяется обучение с подкреплением, чтобы агент определил наилучший вариант поведения, совместимый с этой частичной программой.

Сочетание методов функциональной аппроксимации, формирования вознаграждения и иерархического обучения с подкреплением может стать основой успешного подхода к решению крупномасштабных задач.

Хорошим исходным пунктом для дальнейшего изучения литературы по этой теме может служить обзор [759]. В книге Саттона и Барто [1480], двух основоположников этой области, изложение сосредоточено на архитектурах и алгоритмах, а также показано, как методы обучения с подкреплением подпитываются идеями в области обучения, планирования и осуществления действий. В немного более формальной работе [121] приведено строгое изложение основ теории динамического программирования и стохастической сходимости. Статьи по обучению с подкреплением часто публикуются в журналах Machine Learning и Journal of Machine Learning Research, а также в материалах конференций International Conferences on Machine Learning и семинаров Neural Information Processing Systems.

Упражнения 21.1. d§3 Реализуйте пассивного обучающегося агента, действующего в простой среде, такой как мир 4x3. Для случая первоначально неизвестной модели среды сравните производительность обучения с помощью алгоритмов непосредственной оценки полезности, TD и ADP. Проведите сравнение оптимальной стратегии и некоторых случайно выбранных стратегий. Применительно к какой из них быстрее всего сходятся оценки полезности? Что происходит при увеличении размеров среды? (Опробуйте варианты среды с препятствиями и без препятствий.) 21.2. В главе 17 была определена правильная стратегия для некоторой задачи MDP как стратегия, позволяющая достичь терминального состояния. Покажите, что для пассивного агента ADP возможна такая ситуация, что он найдет с помощью обучения модель перехода, для которой его стратегия является неправильной, даже если p правильна для истинной задачи MDP; при использовании таких моделей этап определения значения может закончиться неудачей, если $u=1$. Покажите, что такая проблема не может возникнуть, если этап определения значения применяется к модели, создаваемой с помощью обучения, только в конце каждой попытки.

21.3. Начиная с проекта пассивного агента ADP, модифицируйте его так, чтобы в нем использовался приближенный алгоритм ADP, как описано в настоящей главе. Выполните это задание за два описанных ниже этапа. а) Реализуйте приоритетную очередь для внесения корректировок в оценки полезностей. После корректировки данных для некоторого состояния все его предшественники также становятся кандидатами на проведение корректировки и должны быть внесены в очередь. Очередь инициализирует-

ся с учетом состояния, из которого произошел самый последний переход.

Предусмотрите возможность использования только фиксированного количества корректировок. б) Проведите эксперименты с использованием различных эвристик для упорядочения приоритетной очереди, исследуя их влияние на скорость обучения и продолжительность вычислений.

21.4. В методе непосредственной оценки полезности, описанном в разделе 21.2, используются различные терминальные состояния для обозначения конца каждой попытки. Как можно модифицировать этот метод для применения в вариантах среды с обесцениваемыми вознаграждениями и без терминальных состояний?

21.5. Как можно использовать алгоритм определения значения для вычисления ожидаемых потерь, испытываемых агентом, который применяет заданное множество оценок полезностей и и оцениваемую модель γ , по сравнению с агентом, использующим правильные значения?

21.6. Приспособьте мир пылесоса (см. главу 2) для обучения с подкреплением, включив в него вознаграждения за то, что пылесос собирает каждый фрагмент

мусора, отправляется в свой исходный квадрат и отключается. Сделайте этот мир доступным, предоставив агенту соответствующие результаты восприятия.

После этого проведите эксперименты с различными агентами, действующими на основе обучения с подкреплением. Является ли функциональная аппроксимация обязательным условием успеха? Какого рода аппроксиматор может применяться в этом приложении?

21.7. (Й) Реализуйте проект агента, исследующего свою среду и действующего на основе обучения с подкреплением, в котором используются непосредственные оценки полезностей. Подготовьте две версии — с табличным представлением неприменением аппроксиматора функции, показанного в уравнении 21.9.

Сравните их производительность в трех вариантах среды, описанных ниже. а) Мир 4x3, описанный в данной главе. б) Мир 10x10 без препятствий и с вознаграждением +1 в квадрате A0,10). в) Мир 10x10 без препятствий и с вознаграждением +1 в квадрате E,5).

21.8. Запишите уравнения обновления параметров для метода обучения TD со следующим условием: $v(x,y) = v_0 + V_1x + V_2y + V_3\ln(|x-x_d| + |y-y_d|)$ 21.9. Составьте список применимых характеристик для стохастических миров с решетками (обобщений мира 4x3), которые содержат множество препятствий и множество терминальных состояний с вознаграждением -1 или +1.

21.10. Вычислите истинную функцию полезности и наилучшую линейную аппроксимацию в точках x и y (как описано в уравнении 21.9) для перечисленных ниже вариантов среды. а) Мир 10x10 с единственным терминальным состоянием +1 в квадрате A0,10).

б) Как и в упр. 21.10, а, но с дополнительным терминальным состоянием -1 в квадрате A0,1). в) Как и в упр. 21.10, б, но с дополнительными препятствиями в 10 квадратах, выбранных случайным образом. г) Как и в упр. 21.10, б, но с размещением стены, простирающейся от квадрата E,2) до квадрата E,9). д) Как и в упр. 21.10, а, но с терминальным состоянием в квадрате E,5).

Действия представляют собой детерминированные движения в четырех направлениях. В каждом случае сравните результаты с использованием трехмерных графиков. Для каждой среды предложите дополнительные характеристики (кроме x и y), которые позволили бы улучшить эту аппроксимацию, и продемонстрируйте полученные результаты.

21.11. (И) Дополните стандартную среду ведения игры (см. главу 6) для включения в нее сигнала вознаграждения. Поместите в эту среду два агента, действующих на основе обучения с подкреплением (они, безусловно, могут иметь общую программу агента), и вынудите их играть друг против друга. Примените обобщенное правило

обновления TD (уравнение 21.11) для обновления функции оценки. Вам может потребоваться вначале применить простую функцию оценки с линейными весами и простую игру, такую как крестики-нолики.

21.12. d\$D Реализуйте алгоритмы Reinforce и Pegasus и примените их к миру 4x3, используя выбранное вами семейство стратегий. Прокомментируйте полученные результаты.

21.13. Ш Исследуйте проблему применения идей обучения с подкреплением для моделирования поведения людей и животных.

21.14. Ш Может ли обучение с подкреплением служить подходящей абстрактной моделью для эволюции? Какая связь существует (и существует ли она вообще) между жестко закрепленными сигналами вознаграждения и эволюционной пригодностью?